Java Source Code Compilation

Total Files: 852

---

1. `[1]Two Sum.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    int n = nums.length;

    for (int i = 0; i < n; i++) {
      int key = target - nums[i];

      if (map.containsKey(key)) {
        return new int[] {map.get(key), i};
      }

      map.put(nums[i], i);
    }

    return new int[] {-1, -1};
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

2. `[10]Regular Expression Matching.java`

```
10
        Regular Expression Matching
        2022-12-23 16:15:20

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isMatch(String s, String p) {
        char[] cs = s.toCharArray();
        char[] cp = p.toCharArray();
        int m = cs.length;
        int n = cp.length;
        boolean[][] dp = new boolean[m + 1][n + 1];
        dp[m][n] = true;
        for (int i = m; i >= 0; i--) {
            for (int j = n - 1; j >= 0; j--) {
                boolean firstMatch = (i < m && (cs[i] == cp[j] || cp[j] == '.'));
                if (j < n - 1 && cp[j + 1] == '*') {
                    dp[i][j] = dp[i][j + 2] || firstMatch && dp[i + 1][j];
                } else {
                    dp[i][j] = firstMatch && dp[i + 1][j + 1];
                }
            }
        }
        return dp[0][0];
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

3. `[11]Container With Most Water.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int maxArea(int[] height) {
    int result = 0;
    int left = 0;
    int right = height.length - 1;

    while (left < right) {
      int width = Math.min(height[left], height[right]);
      int length = right - left;
      int currentArea = width * length;
      result = Math.max(result, currentArea);

      while (left < right && height[left] <= width) {
        left++;
      }

      while (left < right && height[right] <= width) {
        right--;
      }
    }

    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

4. [12]Integer to Roman.java

```
12
        Integer to Roman
        2023-01-01 01:03:23

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String intToRoman(int num) {
        int[] values = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
        String[] symbols = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V",
"IV", "I"};
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < values.length; i++) {
            while (num >= values[i]) {
                num -= values[i];
                sb.append(symbols[i]);
            }
        }
        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

5. `[13]Roman to Integer.java`

```
13
        Roman to Integer
        2023-01-01 01:03:17

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int romanToInt(String s) {
        int res = 0;
        int num = 0;
        int temp = 0;
        for (int i = s.length() - 1; i >= 0; i--) {
            switch (s.charAt(i)) {
                case 'I' -> nums = 1;
                case 'V' -> num = 5;
                case 'X' -> num = 10;
                case 'L' -> num = 50;
                case 'C' -> num = 100;
                case 'D' -> num = 500;
                case 'M' -> num = 1000;
            }
            if (num < temp) {
                res -= num;
            } else {
                res += num;
            }
            temp = num;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

6. `[14]Longest Common Prefix.java`

```
14
Longest Common
Prefix
        2023-01-02 10:39:06

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String longestCommonPrefix(String[] strs) {
        String prefixString = strs[0];

        for (String str : strs) {
            while (str.indexOf(prefixString) != 0) {
                prefixString = prefixString.substring(0, prefixString.length() - 1);
            }
        }

        return prefixString;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

7. `[15]3Sum.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    return kSum(nums, 0, 3, 0);
  }

  private List<List<Integer>> kSum(int[] nums, int target, int k, int start) {
    List<List<Integer>> result = new ArrayList<>();
    int n = nums.length;

    if (k == 2) {
      int left = start;
      int right = n - 1;

      while (left < right) {
        int sum = nums[left] + nums[right];

        if (sum < target) {
          left++;
        } else if (sum > target) {
          right--;
        } else {
          result.add(Arrays.asList(nums[left], nums[right]));
          left++;
          right--;

          while (left < right && nums[left] == nums[left - 1]) {
            left++;
          }

          while (left < right && nums[right] == nums[right + 1]) {
            right--;
          }
        }
      }

      return result;
    }

    for (int i = start; i < n - (k - 1); i++) {
      if (i > start && nums[i] == nums[i - 1]) {
        continue;
      }

      if (nums[i] * k > target) {
        break;
      }
```

```
      if (nums[n - 1] * k < target) {
        break;
      }

      List<List<Integer>> subResult = kSum(nums, target - nums[i], k - 1, i + 1);

      for (List<Integer> subset : subResult) {
        List<Integer> tuple = new ArrayList<>();
        tuple.add(nums[i]);
        tuple.addAll(subset);
        result.add(tuple);
      }
    }

    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

8. `[16]3Sum Closest.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int threeSumClosest(int[] nums, int target) {
        Arrays.sort(nums);
        int n = nums.length;
        int res = nums[0] + nums[1] + nums[2];
        int diff = Math.abs(res - target);
        for (int i = 0; i < n - 2; i++) {
            int left = i + 1;
            int right = n - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (Math.abs(sum - target) < diff) {
                    res = sum;
                    diff = Math.abs(sum - target);
                }
                if (sum == target) {
                    return target;
                } else if (sum < target) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

9. `[17]Letter Combinations of a Phone Number.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  private static final Map<Character, String> DIGIT_TO_LETTERS = new HashMap<>();

  static {
    DIGIT_TO_LETTERS.put('2', "abc");
    DIGIT_TO_LETTERS.put('3', "def");
    DIGIT_TO_LETTERS.put('4', "ghi");
    DIGIT_TO_LETTERS.put('5', "jkl");
    DIGIT_TO_LETTERS.put('6', "mno");
    DIGIT_TO_LETTERS.put('7', "pqrs");
    DIGIT_TO_LETTERS.put('8', "tuv");
    DIGIT_TO_LETTERS.put('9', "wxyz");
  }

  public List<String> letterCombinations(String digits) {
    List<String> result = new ArrayList<>();

    if (digits == null || digits.length() == 0) {
      return result;
    }

    backtrack(digits, 0, new StringBuilder(), result);
    return result;
  }

  private void backtrack(String digits, int index, StringBuilder path, List<String>
result) {
    if (index == digits.length()) {
      result.add(path.toString());
      return;
    }

    String letters = DIGIT_TO_LETTERS.get(digits.charAt(index));

    for (char c : letters.toCharArray()) {
      path.append(c);
      backtrack(digits, index + 1, path, result);
      path.deleteCharAt(path.length() - 1);
    }
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

10. `[18]4Sum.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        List<List<Integer>> list = new ArrayList<>();
        Arrays.sort(nums);
        int n = nums.length;
        for (int i = 0; i < n - 3; i++) {
            if (i != 0 && nums[i] == nums[i - 1]) {
                continue;
            }
            if ((long) nums[i] + nums[i + 1] + nums[i + 2] + nums[i + 3] > target) {
                break;
            }
            if ((long) nums[i] + nums[n - 3] + nums[n - 2] + nums[n - 1] < target) {
                continue;
            }
            for (int j = i + 1; j < n - 2; j++) {
                if (j != i + 1 && nums[j] == nums[j - 1]) {
                    continue;
                }
                if ((long) nums[j] + nums[j + 1] + nums[j + 2] > target - nums[i]) {
                    break;
                }
                if ((long) nums[j] + nums[n - 2] + nums[n - 1] < target - nums[i]) {
                    continue;
                }
                int left = j + 1;
                int right = n - 1;
                while (left < right) {
                    int sum = nums[i] + nums[j] + nums[left] + nums[right];
                    if (sum == target) {
                        list.add(Arrays.asList(nums[i], nums[j], nums[left],
nums[right]));
                        left++;
                        right--;
                        while (left < right && nums[left] == nums[left - 1]) {
                            left++;
                        }
                        while (left < right && nums[right] == nums[right + 1]) {
                            right--;
                        }
                    } else if (sum < target) {
                        left++;
                    } else {
                        right--;
                    }
                }
            }
        }
        return list;
```

```
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

11. [19]Remove Nth Node From End of List.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode fast = dummy;
    ListNode slow = dummy;

    for (int i = 0; i < n; i++) {
      fast = fast.next;
    }

    while (fast.next != null) {
      fast = fast.next;
      slow = slow.next;
    }

    slow.next = slow.next.next;
    return dummy.next;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

12. `[20]Valid Parentheses.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public boolean isValid(String s) {
    if (s.length() % 2 != 0) {
      return false;
    }

    Map<Character, Character> map = Map.of(')', '(', ']', '[', '}', '{');
    Deque<Character> stack = new ArrayDeque<>();

    for (char c : s.toCharArray()) {
      if (map.containsKey(c)) {
        if (stack.isEmpty() || stack.pop() != map.get(c)) {
          return false;
        }
      } else {
        stack.push(c);
      }
    }

    return stack.isEmpty();
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

13. `[21]Merge Two Sorted Lists.java`

```
// leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
    if (list1 == null) {
      return list2;
    }

    if (list2 == null) {
      return list1;
    }

    if (list1.val <= list2.val) {
      list1.next = mergeTwoLists(list1.next, list2);
      return list1;
    } else {
      list2.next = mergeTwoLists(list1, list2.next);
      return list2;
    }
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

14. `[22]Generate Parentheses.java`

```
22
        Generate Parentheses
        2022-12-14 09:40:16

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    List<String> list = new ArrayList<>();

    public List<String> generateParenthesis(int n) {
        backtrack("", 0, 0, n);
        return list;
    }

    public void backtrack(String str, int left, int right, int n) {
        if (str.length() == 2 * n) {
            list.add(str);
            return;
        }
        if (left < n) {
            backtrack(str + "(", left + 1, right, n);
        }
        if (right < left) {
            backtrack(str + ")", left, right + 1, n);
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

15. `[23]Merge k Sorted Lists.java`

```
// leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null) {
      return null;
    }

    PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) ->
Integer.compare(a.val, b.val));

    for (ListNode list : lists) {
      if (list != null) {
        minHeap.offer(list);
      }
    }

    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;

    while (!minHeap.isEmpty()) {
      ListNode node = minHeap.poll();

      curr.next = node;
      curr = curr.next;

      if (node.next != null) {
        minHeap.offer(node.next);
      }
    }

    return dummy.next;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

16. `[24]Swap Nodes in Pairs.java`

```
// leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  public ListNode swapPairs(ListNode head) {
    if (head == null || head.next == null) {
      return head;
    }

    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prev = dummy;
    ListNode first = head;

    while (first != null && first.next != null) {
      ListNode second = first.next;

      prev.next = second;
      first.next = second.next;
      second.next = first;

      prev = first;
      first = first.next;
    }

    return dummy.next;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

17. `[25]Reverse Nodes in k-Group.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public ListNode reverseKGroup(ListNode head, int k) {
    if (head == null || k == 1) {
      return head;
    }

    ListNode dummy = new ListNode(0);
    dummy.next = head;

    ListNode groupPrev = dummy;

    while (true) {
      ListNode kth = groupPrev;
      for (int i = 0; i < k && kth != null; i++) {
        kth = kth.next;
      }
      if (kth == null) {
        break;
      }

      ListNode groupNext = kth.next;

      ListNode prev = groupNext;
      ListNode curr = groupPrev.next;
      while (curr != groupNext) {
        ListNode temp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = temp;
      }

      ListNode oldHead = groupPrev.next;
      groupPrev.next = prev;
      groupPrev = oldHead;
    }

    return dummy.next;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

18. [26]Remove Duplicates from Sorted Array.java

```
26
        Remove Duplicates from Sorted Array
        2022-11-29 10:38:45

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int removeDuplicates(int[] nums) {
        int count = 1;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[count - 1]) {
                nums[count] = nums[i];
                count++;
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

19. [27]Remove Element.java

```

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int removeElement(int[] nums, int val) {
        int count = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != val) {
                nums[count] = nums[i];
                count++;
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

20. [28]Find the Index of the First Occurrence in a String.java

```
28
        Find the Index of the First Occurrence in a String
        2023-03-03 14:07:24

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int strStr(String haystack, String needle) {
        return haystack.indexOf(needle);
    }
}

//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int strStr(String haystack, String needle) {
        return haystack.indexOf(needle);
    }
}
 */
```

21. [100]Same Tree.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null && q == null) {
            return true;
        }

        if (p == null || q == null || p.val != q.val) {
            return false;
        }

        return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

22. `[101]Symmetric Tree.java`

```
101
        Symmetric Tree
        2022-12-02 13:31:21
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public boolean isSymmetric(TreeNode root) {
        return dfs(root.left, root.right);
    }

    public boolean dfs(TreeNode node1, TreeNode node2) {
        if (node1 == null && node2 == null) {
            return true;
        }
        if (node1 == null || node2 == null) {
            return false;
        }
        if (node1.val != node2.val) {
            return false;
        }
        return dfs(node1.left, node2.right) && dfs(node1.right, node2.left);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

23. `[102]Binary Tree Level Order Traversal.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();

    if (root == null) {
      return result;
    }

    Queue<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
      int size = queue.size();
      List<Integer> subResult = new ArrayList<>();
      for (int i = 0; i < size; i++) {
        TreeNode curr = queue.poll();
        subResult.add(curr.val);

        if (curr.left != null) {
          queue.offer(curr.left);
        }

        if (curr.right != null) {
          queue.offer(curr.right);
        }
      }
      result.add(subResult);
    }
    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

24. [103]Binary Tree Zigzag Level Order Traversal.java

```
103
        Binary Tree Zigzag Level Order Traversal
        2022-12-03 16:22:19
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        if (root == null) {
            return new ArrayList<>();
        }
        List<List<Integer>> res = new ArrayList<>();
        Deque<TreeNode> dq = new ArrayDeque<>();
        dq.add(root);
        boolean isLeft = true;
        while (!dq.isEmpty()) {
            List<Integer> list = new ArrayList<>();
            int size = dq.size();
            for (int i = 0; i < size; i++) {
                TreeNode node = dq.poll();
                list.add(node.val);
                if (node.left != null) {
                    dq.add(node.left);
                }
                if (node.right != null) {
                    dq.add(node.right);
                }
            }
            if (!isLeft) {
                Collections.reverse(list);
            }
            res.add(list);
            isLeft = !isLeft;
        }
        return res;
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

25. [104]Maximum Depth of Binary Tree.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int maxDepth(TreeNode root) {
    if (root == null) {
      return 0;
    }

    Deque<TreeNode> queue = new ArrayDeque<>();
    int depth = 0;
    queue.offer(root);

    while (!queue.isEmpty()) {
      int size = queue.size();

      for (int i = 0; i < size; i++) {
        TreeNode node = queue.poll();

        if (node.left != null) {
          queue.offer(node.left);
        }

        if (node.right != null) {
          queue.offer(node.right);
        }
      }

      depth++;
    }

    return depth;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

26. `[105]Construct Binary Tree from Preorder and`

Inorder Traversal.java

```
105
        Construct Binary Tree from Preorder and Inorder Traversal
        2022-12-02 14:10:43
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    int preIndex;
    int inIndex;

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        return dfs(preorder, inorder, Integer.MAX_VALUE);
    }

    public TreeNode dfs(int[] preOrder, int[] inOrder, int stop) {
        if (inIndex >= inOrder.length || inOrder[inIndex] == stop) {
            return null;
        }
        TreeNode node = new TreeNode(preOrder[preIndex]);
        preIndex++;
        node.left = dfs(preOrder, inOrder, node.val);
        inIndex++;
        node.right = dfs(preOrder, inOrder, stop);
        return node;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    int preorderIndex;
    Map<Integer, Integer> inorderMap;
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        preorderIndex = 0;
        // build a hashmap to store value -> its index relations
        inorderMap = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) {
            inorderMap.put(inorder[i], i);
```

```
        }

        return arrayToTree(preorder, 0, preorder.length - 1);
    }

    private TreeNode arrayToTree(int[] preorder, int left, int right) {
        // if there are no elements to construct the tree
        if (left > right) return null;

        // select the preorder_index element as the root and increment it
        int rootValue = preorder[preorderIndex++];
        TreeNode root = new TreeNode(rootValue);

        // build left and right subtree
        // excluding inorderMap[rootValue] element because it's the root
        root.left = arrayToTree(preorder, left, inorderMap.get(rootValue) - 1);
        root.right = arrayToTree(preorder, inorderMap.get(rootValue) + 1, right);
        return root;
    }
}
 */
```

27. `[106]Construct Binary Tree from Inorder and`

Postorder Traversal.java

```
106
        Construct Binary Tree from Inorder and Postorder Traversal
        2022-12-02 13:52:51
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
/*
目前不會
 */
class Solution {
    int inIndex;
    int postIndex;

    public TreeNode buildTree(int[] inorder, int[] postorder) {
        inIndex = inorder.length - 1;
        postIndex = postorder.length - 1;
        return dfs(inorder, postorder, Integer.MAX_VALUE);
    }

    public TreeNode dfs(int[] inOrder, int[] postOrder, int stop) {
        if (inIndex < 0 || inOrder[inIndex] == stop) {
            return null;
        }
        TreeNode node = new TreeNode(postOrder[postIndex]);
        postIndex--;
        node.right = dfs(inOrder, postOrder, node.val);
        inIndex--;
        node.left = dfs(inOrder, postOrder, stop);
        return node;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
  int postIndex;
  int[] postorder;
  int[] inorder;
```

```
  HashMap<Integer, Integer> idx_map = new HashMap<Integer, Integer>();

  public TreeNode dfs(int in_left, int in_right) {
    // if there is no elements to construct subtrees
    if (in_left > in_right)
      return null;

    // pick up postIndex element as a root
    int root_val = postorder[postIndex];
    TreeNode root = new TreeNode(root_val);

    // root splits inorder list
    // into left and right subtrees
    int index = idx_map.get(root_val);

    // recursion
    postIndex--;
    // build right subtree
    root.right = dfs(index + 1, in_right);
    // build left subtree
    root.left = dfs(in_left, index - 1);
    return root;
  }

  public TreeNode buildTree(int[] inorder, int[] postorder) {
    this.postorder = postorder;
    this.inorder = inorder;
    // start from the last postorder element
    postIndex = postorder.length - 1;

    // build a hashmap value -> its index
    int idx = 0;
    for (Integer val : inorder)
      idx_map.put(val, idx++);
    return dfs(0, inorder.length - 1);
  }
}
 */
```

28. [107]Binary Tree Level Order Traversal II.java

```
107
        Binary Tree Level Order Traversal II
        2022-12-24 10:33:27
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public List<List<Integer>> levelOrderBottom(TreeNode root) {
        if (root == null) {
            return new ArrayList<>();
        }
        List<List<Integer>> res = new ArrayList<>();
        Deque<TreeNode> dq = new ArrayDeque<>();
        dq.add(root);
        while (!dq.isEmpty()) {
            int size = dq.size();
            List<Integer> list = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode curr = dq.poll();
                list.add(curr.val);
                if (curr.left != null) {
                    dq.add(curr.left);
                }
                if (curr.right != null) {
                    dq.add(curr.right);
                }
            }
            res.add(list);
        }
        Collections.reverse(res);
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

29. `[108]Convert Sorted Array to Binary Search`

Tree.java

```
/**
 * <p>Given an integer array <code>nums</code> where the elements are sorted in
<strong>ascending order</strong>, convert <em>it to a </em><span data-keyword="height-
balanced"><strong><em>height-balanced</em></strong></span> <em>binary search tree</em>.
</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2021/02/18/btree1.jpg"
style="width: 302px; height: 222px;" />
 * <pre>
 * <strong>Input:</strong> nums = [-10,-3,0,5,9]
 * <strong>Output:</strong> [0,-3,9,-10,null,5]
 * <strong>Explanation:</strong> [0,-10,5,null,-3,null,9] is also accepted:
 * <img alt="" src="https://assets.leetcode.com/uploads/2021/02/18/btree2.jpg"
style="width: 302px; height: 222px;" />
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2021/02/18/btree.jpg"
style="width: 342px; height: 142px;" />
 * <pre>
 * <strong>Input:</strong> nums = [1,3]
 * <strong>Output:</strong> [3,1]
 * <strong>Explanation:</strong> [1,null,3] and [3,1] are both height-balanced BSTs.
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>1 &lt;= nums.length &lt;= 10<sup>4</sup></code></li>
 * <li><code>-10<sup>4</sup> &lt;= nums[i] &lt;= 10<sup>4</sup></code></li>
 * <li><code>nums</code> is sorted in a <strong>strictly increasing</strong> order.</li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>Array</li><li>Divide and Conquer</li>
<li>Tree</li><li>Binary Search Tree</li><li>Binary Tree</li></div></div><br><div><li>👍
9471</li><li>👎 474</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
```

```
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    int[] nums;

    public TreeNode sortedArrayToBST(int[] nums) {
        this.nums = nums;
        return toBST(0, nums.length - 1);
    }

    public TreeNode toBST(int left, int right) {
        if (left > right) {
            return null;
        }
        int mid = left + (right - left) / 2;
        TreeNode node = new TreeNode(nums[mid]);
        node.left = toBST(left, mid - 1);
        node.right = toBST(mid + 1, right);
        return node;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

30. [109]Convert Sorted List to Binary Search Tree.java

```
/**
 * <p>Given the <code>head</code> of a singly linked list where elements are sorted in
<strong>ascending order</strong>, convert <em>it to a </em><span data-keyword="height-
balanced"><strong><em>height-balanced</em></strong></span> <em>binary search tree</em>.
</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/08/17/linked.jpg"
style="width: 500px; height: 388px;" />
 * <pre>
 * <strong>Input:</strong> head = [-10,-3,0,5,9]
 * <strong>Output:</strong> [0,-3,9,-10,null,5]
 * <strong>Explanation:</strong> One possible answer is [0,-3,9,-10,null,5], which
represents the shown height balanced BST.
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> head = []
 * <strong>Output:</strong> []
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li>The number of nodes in <code>head</code> is in the range <code>[0, 2 *
10<sup>4</sup>]</code>.</li>
 * <li><code>-10<sup>5</sup> &lt;= Node.val &lt;= 10<sup>5</sup></code></li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>Linked List</li><li>Divide and Conquer</li>
<li>Tree</li><li>Binary Search Tree</li><li>Binary Tree</li></div></div><br><div><li>👍
5664</li><li>👎 128</li></div>
 * <p>
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 * <p>
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
```

```
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 * <p>
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */

//leetcode submit region begin(Prohibit modification and deletion)
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode sortedListToBST(ListNode head) {
        if (head == null) {
            return null;
        }
        return toBST(head, null);
    }

    private TreeNode toBST(ListNode left, ListNode right) {
        if (left == right) {
            return null;
        }
        ListNode fast = left;
```

```
        ListNode slow = left;
        while (fast != right && fast.next != right) {
            fast = fast.next.next;
            slow = slow.next;
        }
        TreeNode node = new TreeNode(slow.val);
        node.left = toBST(left, slow);
        node.right = toBST(slow.next, right);
        return node;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

31. `[110]Balanced Binary Tree.java`

```
/**
 * <p>Given a binary tree, determine if it is <span data-keyword="height-balanced">
<strong>height-balanced</strong></span>.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/10/06/balance_1.jpg"
style="width: 342px; height: 221px;" />
 * <pre>
 * <strong>Input:</strong> root = [3,9,20,null,null,15,7]
 * <strong>Output:</strong> true
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/10/06/balance_2.jpg"
style="width: 452px; height: 301px;" />
 * <pre>
 * <strong>Input:</strong> root = [1,2,2,3,3,null,null,4,4]
 * <strong>Output:</strong> false
 * </pre>
 *
 * <p><strong class="example">Example 3:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> root = []
 * <strong>Output:</strong> true
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li>The number of nodes in the tree is in the range <code>[0, 5000]</code>.</li>
 * <li><code>-10<sup>4</sup> &lt;= Node.val &lt;= 10<sup>4</sup></code></li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>Tree</li><li>Depth-First Search</li><li>Binary
Tree</li></div></div><br><div><li>👍 8459</li><li>👎 476</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
```

```
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    boolean res = true;

    public boolean isBalanced(TreeNode root) {
        dfs(root);
        return res;
    }

    public int dfs(TreeNode node) {
        if (node == null) {
            return 0;
        }
        int left = dfs(node.left);
        int right = dfs(node.right);
        if (Math.abs(left - right) > 1) {
            res = false;
        }
        return Math.max(left, right) + 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

32. [111]Minimum Depth of Binary Tree.java

```
111
        Minimum Depth of Binary Tree
        2022-12-01 17:18:41
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public int minDepth(TreeNode root) {
        if (root == null) {
            return 0;
        }
        Deque<TreeNode> dq = new ArrayDeque<>();
        dq.add(root);
        int count = 0;
        while (!dq.isEmpty()) {
            int size = dq.size();
            count++;
            for (int i = 0; i < size; i++) {
                TreeNode curr = dq.poll();
                if (curr.left == null && curr.right == null) {
                    return count;
                }
                if (curr.left != null) {
                    dq.add(curr.left);
                }
                if (curr.right != null) {
                    dq.add(curr.right);
                }
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
 - 1ms
 class Solution {
```

```
    public int minDepth(TreeNode root) {
        if (root == null) {
            return 0;
        }
        Deque<TreeNode> dq = new ArrayDeque<>();
        dq.add(root);
        int count = 0;
        while (!dq.isEmpty()) {
            int size = dq.size();
            count++;
            for (int i = 0; i < size; i++) {
                TreeNode curr = dq.poll();
                if (curr.left == null && curr.right == null) {
                    return count;
                }
                if (curr.left != null) {
                    dq.add(curr.left);
                }
                if (curr.right != null) {
                    dq.add(curr.right);
                }
            }
        }
        return count;
    }
}

 - 8ms
class Solution {
    public int minDepth(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int left = minDepth(root.left);
        int right = minDepth(root.right);
        if (left == 0 || right == 0) {
            return left + right + 1;
        }
        return Math.min(left, right) + 1;
    }
}
 */
```

33. `[112]Path Sum.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  public boolean hasPathSum(TreeNode root, int targetSum) {
    if (root == null) {
      return false;
    }

    if (root.left == null && root.right == null) {
      return root.val == targetSum;
    }

    return hasPathSum(root.left, targetSum - root.val)
        || hasPathSum(root.right, targetSum - root.val);
  }
}

// leetcode submit region end(Prohibit modification and deletion)
```

34. `[113]Path Sum II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    List<List<Integer>> result = new ArrayList<>();

    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        checkPathSum(root, targetSum, new ArrayList<>());
        return result;
    }

    private void checkPathSum(TreeNode node, int targetSum, List<Integer> list) {
        if (node == null) {
            return;
        }

        list.add(node.val);

        if (node.left == null && node.right == null && node.val == targetSum) {
            result.add(new ArrayList<>(list));
        } else {
            checkPathSum(node.left, targetSum - node.val, list);
            checkPathSum(node.right, targetSum - node.val, list);
        }

        list.remove(list.size() - 1);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

35. `[114]Flatten Binary Tree to Linked List.java`

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public void flatten(TreeNode root) {
        if (root == null) {
            return;
        }

        TreeNode node = root;

        while (node != null) {
            if (node.left != null) {
                TreeNode rightMost = node.left;

                while (rightMost.right != null) {
                    rightMost = rightMost.right;
                }

                rightMost.right = node.right;
                node.right = node.left;
                node.left = null;
            }

            node = node.right;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

36. `[115]Distinct Subsequences.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numDistinct(String s, String t) {
        char[] c1 = s.toCharArray();
        char[] c2 = t.toCharArray();
        int m = c1.length;
        int n = c2.length;
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) {
            dp[i][0] = 1;
        }
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (c1[i - 1] == c2[j - 1]) {
                    dp[i][j] = dp[i - 1][j] + dp[i - 1][j - 1];
                } else {
                    dp[i][j] = dp[i - 1][j];
                }
            }
        }
        return dp[m][n];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
Top-Down dynamic programming - 3ms
class Solution {
    Integer[][] dp;

    public int numDistinct(String s, String t) {
        char[] c1 = s.toCharArray();
        char[] c2 = t.toCharArray();
        int m = c1.length;
        int n = c2.length;
        dp = new Integer[m][n];
        return dyP(c1, c2, 0, 0);
    }

    public int dyP(char[] c1, char[] c2, int i, int j) {
        int m = c1.length;
        int n = c2.length;
        if (i == m || j == n || m - i < n - j) {
            return j == n ? 1 : 0;
        }
        if (dp[i][j] != null) {
            return dp[i][j];
        }
        int res = dyP(c1, c2, i + 1, j);
        if (c1[i] == c2[j]) {
            res += dyP(c1, c2, i + 1, j + 1);
```

```
        }
        dp[i][j] = res;
        return res;
    }
}

Buttom-Up dyPnamic programming - 10 ms
class Solution {
    public int numDistinct(String s, String t) {
        char[] c1 = s.toCharArray();
        char[] c2 = t.toCharArray();
        int m = c1.length;
        int n = c2.length;
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) {
            dp[i][0] = 1;
        }
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (c1[i - 1] == c2[j - 1]) {
                    dp[i][j] = dp[i - 1][j] + dp[i - 1][j - 1];
                } else {
                    dp[i][j] = dp[i - 1][j];
                }
            }
        }
        return dp[m][n];
    }
}
 */
```

37. `[116]Populating Next Right Pointers in Each`

Node.java

```
116
        Populating Next Right Pointers in Each Node
        2022-12-02 18:26:26
//leetcode submit region begin(Prohibit modification and deletion)
/*
// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node next;

    public Node() {}

    public Node(int _val) {
        val = _val;
    }

    public Node(int _val, Node _left, Node _right, Node _next) {
        val = _val;
        left = _left;
        right = _right;
        next = _next;
    }
};
*/

class Solution {
    public Node connect(Node root) {
        if (root == null) {
            return null;
        }
        Deque<Node> dq = new ArrayDeque<>();
        dq.add(root);
        while (!dq.isEmpty()) {
            int size = dq.size();
            for (int i = 0; i < size; i++) {
                Node node = dq.poll();
                if (i < size - 1) {
                    node.next = dq.peek();
                }
                if (node.left != null) {
                    dq.add(node.left);
                }
                if (node.right != null) {
                    dq.add(node.right);
                }
            }
        }
        return root;
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

38. `[117]Populating Next Right Pointers in Each Node`

II.java

```
117
        Populating Next Right Pointers in Each Node II
        2022-12-02 18:26:28
//leetcode submit region begin(Prohibit modification and deletion)
/*
// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node next;

    public Node() {}

    public Node(int _val) {
        val = _val;
    }

    public Node(int _val, Node _left, Node _right, Node _next) {
        val = _val;
        left = _left;
        right = _right;
        next = _next;
    }
};
*/

class Solution {
    public Node connect(Node root) {
        if (root == null) {
            return null;
        }
        Deque<Node> dq = new ArrayDeque<>();
        dq.add(root);
        while (!dq.isEmpty()) {
            int size = dq.size();
            Node next = null;
            for (int i = 0; i < size; i++) {
                Node node = dq.poll();
                node.next = next;
                next = node;
                if (node.right != null) {
                    dq.add(node.right);
                }
                if (node.left != null) {
                    dq.add(node.left);
                }

            }
        }
        return root;
```

```
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

39. `[118]Pascal's Triangle.java`

```
118
        Pascal's Triangle
        2022-12-22 00:47:10

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i < numRows; i++) {
            List<Integer> list = new ArrayList<>();

            for (int j = 0; j <= i; j++) {
                if (j == 0 || j == i) {
                    list.add(1);
                } else {
                    list.add(result.get(i - 1).get(j - 1) + result.get(i - 1).get(j));
                }
            }

            result.add(list);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

40. `[119]Pascal's Triangle II.java`

```
119
        Pascal's Triangle II
        2022-12-22 00:47:08

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> getRow(int rowIndex) {
        List<Integer> result = new ArrayList<>();
        long num = 1;

        for (int i = 0; i <= rowIndex; i++) {
            result.add((int) num);
            num = num * (rowIndex - i) / (1 + i);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public List<Integer> getRow(int rowIndex) {
        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i <= rowIndex; i++) {
            List<Integer> list = new ArrayList<>();

            for (int j = 0; j <= i; j++) {
                if (j == 0 || j == i) {
                    list.add(1);
                } else {
                    list.add(result.get(i - 1).get(j - 1) + result.get(i - 1).get(j));
                }
            }
            result.add(list);
        }

        return result.get(rowIndex);
    }
}
 */
```

41. `[120]Triangle.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {
        int n = triangle.size();
        int[] dp = new int[n + 1];
        for (int i = n - 1; i >= 0; i--) {
            for (int j = 0; j <= i; j++) {
                dp[j] = Math.min(dp[j], dp[j + 1])+triangle.get(i).get(j);
            }
        }
        return dp[0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {
        int n = triangle.size();
        int[][] dp = new int[n + 1][n + 1];
        for (int i = n - 1; i >= 0; i--) {
            for (int j = 0; j <= i; j++) {
                dp[i][j] = Math.min(dp[i + 1][j], dp[i + 1][j + 1]) +
triangle.get(i).get(j);
            }
        }
        return dp[0][0];
    }
}
 */
```

42. `[121].java`

```
121
        Best Time to Buy and Sell Stock
        2023-03-06 17:44:29

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxProfit(int[] prices) {
        int currMax = 0;
        int max = 0;
        int n = prices.length;
        for (int i = 1; i < n; i++) {
            currMax = Math.max(0, currMax += prices[i] - prices[i - 1]);
            max = Math.max(max, currMax);
        }
        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

43. `[121]Best Time to Buy and Sell Stock.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE;
    int maxProfit = 0;

    for (int price : prices) {
      if (price < minPrice) {
        minPrice = price;
      } else {
        maxProfit = Math.max(maxProfit, price - minPrice);
      }
    }

    return maxProfit;
  }
}
//leetcode submit region end(Prohibit modification and deletion)
```

44. [122]Best Time to Buy and Sell Stock II.java

```
122
        Best Time to Buy and Sell Stock II
        2023-02-24 16:14:08

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxProfit(int[] prices) {
        int hold = Integer.MIN_VALUE;
        int notHold = 0;
        for (int price : prices) {
            int preHold = hold;
            int preNotHold = notHold;
            hold = Math.max(preHold, preNotHold - price);
            notHold = Math.max(preNotHold, preHold + price);
        }
        return notHold;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int maxProfit(int[] prices) {
        int hold = Integer.MIN_VALUE;
        int notHold = 0;
        for (int price : prices) {
            hold = Math.max(hold, notHold - price);
            notHold = Math.max(notHold, hold + price);
        }
        return notHold;
    }
}
 */
```

45. `[123].java`

```
123
        Best Time to Buy and Sell Stock III
        2023-03-06 17:56:18

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxProfit(int[] prices) {
        int k = 2;
        int[] cost = new int[k + 1];
        int[] profit = new int[k + 1];
        Arrays.fill(cost, Integer.MAX_VALUE);
        for (int price : prices) {
            for (int i = 1; i <= k; i++) {
                cost[i] = Math.min(cost[i], price - profit[i - 1]);
                profit[i] = Math.max(profit[i], price - cost[i]);
            }
        }
        return profit[k];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

46. `[123]Best Time to Buy and Sell Stock III.java`

```
123
        Best Time to Buy and Sell Stock III
        2023-02-24 17:44:08

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxProfit(int[] prices) {
        int[] cost = new int[3];
        int[] profit = new int[3];
        Arrays.fill(cost, Integer.MAX_VALUE);
        for (int price : prices) {
            for (int i = 1; i < 3; i++) {
                cost[i] = Math.min(cost[i], price - profit[i - 1]);
                profit[i] = Math.max(profit[i], price - cost[i]);
            }
        }
        return profit[2];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

47. [124]Binary Tree Maximum Path Sum.java

```
124
        Binary Tree Maximum Path Sum
        2022-12-12 00:33:53
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    int max = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
        dfs(root);
        return max;
    }

    public int dfs(TreeNode node) {
        if (node == null) {
            return 0;
        }
        int left = Math.max(0, dfs(node.left));
        int right = Math.max(0, dfs(node.right));
        max = Math.max(max, left + right + node.val);
        return Math.max(left, right) + node.val;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

48. `[125]Valid Palindrome.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public boolean isPalindrome(String s) {
    int left = 0;
    int right = s.length() - 1;

    while (left < right) {
      while (left < right && !Character.isLetterOrDigit(s.charAt(left))) {
        left++;
      }
      while (left < right && !Character.isLetterOrDigit(s.charAt(right))) {
        right--;
      }

      if (Character.toLowerCase(s.charAt(left)) !=
Character.toLowerCase(s.charAt(right))) {
        return false;
      }

      left++;
      right--;
    }

    return true;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

49. `[126]Word Ladder II.java`

```
126
        Word Ladder II
        2023-02-01 22:38:23
/*
還不會
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    List<List<String>> res;

    public List<List<String>> findLadders(String beginWord, String endWord, List<String>
wordList) {
        res = new ArrayList<>();
        Set<String> dict = new HashSet<>(wordList);
        Deque<String> dq = new ArrayDeque<>();
        dq.add(beginWord);
        Map<String, List<String>> map = new HashMap<>();
        boolean find = false;
        while (!dq.isEmpty()) {
            int size = dq.size();
            HashSet<String> set = new HashSet<>();
            while (size-- > 0) {
                String curr = dq.poll();
                if (curr.equals(endWord)) {
                    find = true;
                    continue;
                }
                List<String> nexts = convert(curr, dict);
                for (String next : nexts) {
                    if (set.add(next)) {
                        map.putIfAbsent(next, new ArrayList<>());
                        dq.add(next);
                    }
                    map.get(next).add(curr);
                }
            }
            dict.removeAll(set);
            if (find) {
                List<String> path = new ArrayList<>();
                path.add(endWord);
                dfs(map, path, endWord, beginWord);
                break;
            }
        }
        return res;

    }

    private void dfs(Map<String, List<String>> map, List<String> path, String curr,
String end) {
```

```
        if (curr.equals(end)) {
            List<String> temp = new ArrayList<>(path);
            Collections.reverse((temp));
            res.add(temp);
            return;
        }
        for (String next : map.get(curr)) {
            path.add(next);
            dfs(map, path, next, end);
            path.remove(path.size() - 1);
        }
    }

    private List<String> convert(String s, Set<String> dict) {
        List<String> res = new ArrayList<>();
        char[] chars = s.toCharArray();
        for (int i = 0; i < chars.length; i++) {
            char temp = chars[i];
            for (char c = 'a'; c <= 'z'; c++) {
                chars[i] = c;
                String str = String.valueOf(chars);
                if (c != temp && dict.contains(str)) {
                    res.add(str);
                }
            }
            chars[i] = temp;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

50. `[127]Word Ladder.java`

```
127
        Word Ladder
        2023-02-01 22:38:21

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> set = new HashSet<>(wordList);
        if (!set.contains(endWord)) {
            return 0;
        }
        Set<String> isVisited = new HashSet<>();
        Deque<String> dq = new ArrayDeque<>();
        dq.add(beginWord);
        isVisited.add(beginWord);
        int count = 1;
        while (!dq.isEmpty()) {
            int size = dq.size();
            for (int i = 0; i < size; i++) {
                String curr = dq.poll();
                if (curr.equals(endWord)) {
                    return count;
                }
                char[] c = curr.toCharArray();
                for (int j = 0; j < c.length; j++) {
                    char temp = c[j];
                    for (char k = 'a'; k <= 'z'; k++) {
                        c[j] = k;
                        String str = String.valueOf(c);
                        if (!set.contains(str)) {
                            continue;
                        }
                        if (!isVisited.contains(str)) {
                            dq.add(str);
                            isVisited.add(str);
                        }
                    }
                    c[j] = temp;
                }
            }
            count++;
        }
        return 0;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
13ms
class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> set = new HashSet<>(wordList);
```

```
        if (!set.contains(endWord)) {
            return 0;
        }
        Set<String> begin = new HashSet<>();
        Set<String> end = new HashSet<>();
        begin.add(beginWord);
        end.add(endWord);
        return search(begin, end, set, true, 1);
    }

    private int search(Set<String> begin, Set<String> end, Set<String> set, boolean
isForward, int curr) {
        if (begin.isEmpty() || end.isEmpty()) {
            return 0;
        }
        curr++;
        set.removeAll(begin);
        Set<String> next = new HashSet<>();
        for (String str : begin) {
            char[] c = str.toCharArray();
            for (int i = 0; i < c.length; i++) {
                char ch = c[i];
                for (char j = 'a'; j <= 'z'; j++) {
                    c[i] = j;
                    String temp = String.valueOf(c);
                    if (set.contains(temp)) {
                        if (end.contains(temp)) {
                            return curr;
                        }
                        next.add(temp);
                    }
                }
                c[i] = ch;
            }
        }
        return next.size() > end.size() ? search(end, next, set, false, curr) :
search(next, end, set, true, curr);
    }
}
 */
```

51. [128]Longest Consecutive Sequence.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> set = new HashSet<>();

        for (int num : nums) {
            set.add(num);
        }

        int streak = 0;

        for (int num : set) {
            if (!set.contains(num - 1)) {
                int currNum = num;
                int currStreak = 1;

                while (set.contains(currNum + 1)) {
                    currNum++;
                    currStreak++;
                }

                streak = Math.max(streak, currStreak);
            }
        }

        return streak;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

52. `[129]Sum Root to Leaf Numbers.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int sumNumbers(TreeNode root) {
        return dfs(root, 0);
    }

    private int dfs(TreeNode node, int num) {
        if (node == null) {
            return 0;
        }

        num = num * 10 + node.val;

        if (node.left == null && node.right == null) {
            return num;
        }

        return dfs(node.left, num) + dfs(node.right, num);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

53. [130]Surrounded Regions.java

```
130
        Surrounded Regions
        2023-02-18 09:39:08

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int m;
    int n;

    public void solve(char[][] board) {
        this.m = board.length;
        this.n = board[0].length;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (i * j == 0 || i == m - 1 || j == n - 1) {
                    if (board[i][j] == 'O') {
                        dfs(board, i, j);
                    }
                }
            }
        }
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == 'O') {
                    board[i][j] = 'X';
                } else if (board[i][j] == '@') {
                    board[i][j] = 'O';
                }
            }
        }
    }

    private void dfs(char[][] board, int i, int j) {
        if (i < 0 || i >= m || j < 0 || j >= n || board[i][j] != 'O') {
            return;
        }
        board[i][j] = '@';
        dfs(board, i + 1, j);
        dfs(board, i - 1, j);
        dfs(board, i, j + 1);
        dfs(board, i, j - 1);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

54. `[131]Palindrome Partitioning.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<String>> partition(String s) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

55. `[133]Clone Graph.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> neighbors;
    public Node() {
        val = 0;
        neighbors = new ArrayList<Node>();
    }
    public Node(int _val) {
        val = _val;
        neighbors = new ArrayList<Node>();
    }
    public Node(int _val, ArrayList<Node> _neighbors) {
        val = _val;
        neighbors = _neighbors;
    }
}
*/

class Solution {
  public Node cloneGraph(Node node) {
    if (node == null) {
      return null;
    }

    Map<Node, Node> map = new HashMap<>();
    Queue<Node> queue = new ArrayDeque<>();

    map.put(node, new Node(node.val));
    queue.offer(node);

    while (!queue.isEmpty()) {
      Node curr = queue.poll();

      for (Node nei : curr.neighbors) {
        if (!map.containsKey(nei)) {
          map.put(nei, new Node(nei.val));
          queue.offer(nei);
        }

        map.get(curr).neighbors.add(map.get(nei));
      }
    }

    return map.get(node);
  }
}
// leetcode submit region end(Prohibit modification and deletion)
/*
```

```
class Solution {
    public Node cloneGraph(Node node) {
        if (node == null) {
            return null;
        }
        Map<Node, Node> map = new HashMap<>();
        Deque<Node> dq = new ArrayDeque<>();
        map.put(node, new Node(node.val));
        dq.add(node);
        while (!dq.isEmpty()) {
            Node curr = dq.poll();
            for (Node neighbor : curr.neighbors) {
                if (!map.containsKey(neighbor)) {
                    map.put(neighbor, new Node(neighbor.val));
                    dq.add(neighbor);
                }
                map.get(curr).neighbors.add(map.get(neighbor));
            }
        }
        return map.get(node);
    }
}

class Solution {
    Map<Node, Node> map = new HashMap<>();

    public Node cloneGraph(Node node) {
        if (node == null) {
            return null;
        }
        if (map.containsKey(node)) {
            return map.get(node);
        }
        Node clone = new Node(node.val);
        map.put(node, clone);
        for (Node item : node.neighbors) {
            clone.neighbors.add(cloneGraph(item));
        }
        return clone;
    }
}
 */
```

56. `[134]Gas Station.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int canCompleteCircuit(int[] gas, int[] cost) {
    int n = gas.length;
    int totalTank = 0;
    int currTank = 0;
    int startStation = 0;

    for (int i = 0; i < n; i++) {
      int net = gas[i] - cost[i];
      totalTank += net;
      currTank += net;

      if (currTank < 0) {
        startStation = i + 1;
        currTank = 0;
      }
    }

    return (totalTank >= 0) ? startStation : -1;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

57. `[135]Candy.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int candy(int[] ratings) {
        int n = ratings.length;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);

        for (int i = 1; i < n; i++) {
            if (ratings[i] > ratings[i - 1]) {
                dp[i] = dp[i - 1] + 1;
            }
        }

        for (int i = n - 2; i >= 0; i--) {
            if (ratings[i] > ratings[i + 1]) {
                dp[i] = Math.max(dp[i], dp[i + 1] + 1);
            }
        }

        int sum = 0;

        for (int num : dp) {
            sum += num;
        }

        return sum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

58. `[136]Single Number.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int singleNumber(int[] nums) {
        int result = 0;

        for (int n : nums) {
            result ^= n;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

59. `[137]Single Number II.java`

```
137
        Single Number II
        2023-01-26 02:28:37

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int singleNumber(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }
        for (int key : map.keySet()) {
            if (map.get(key) == 1) {
                return key;
            }
        }
        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

60. [138]Copy List with Random Pointer.java

```
138
        Copy List with Random Pointer
        2023-02-01 22:21:31
//leetcode submit region begin(Prohibit modification and deletion)
/*
// Definition for a Node.
class Node {
    int val;
    Node next;
    Node random;

    public Node(int val) {
        this.val = val;
        this.next = null;
        this.random = null;
    }
}
*/

class Solution {
    public Node copyRandomList(Node head) {
        Map<Node, Node> map = new HashMap<>();
        Node node = head;
        while (node != null) {
            map.put(node, new Node(node.val));
            node = node.next;
        }
        node = head;
        while (node != null) {
            map.get(node).next = map.get(node.next);
            map.get(node).random = map.get(node.random);
            node = node.next;
        }
        return map.get(head);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

61. `[139]Word Break.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public boolean wordBreak(String s, List<String> wordDict) {
    Set<String> dict = new HashSet<>(wordDict);
    int n = s.length();
    boolean[] dp = new boolean[n + 1];
    dp[0] = true;

    for (int i = 1; i <= n; i++) {
      for (String word : wordDict) {
        int len = word.length();
        if (i >= len && dp[i - len] && s.regionMatches(i - len, word, 0, len)) {
          dp[i] = true;
          break;
        }
      }
    }
    return dp[n];
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

62. `[140]Word Break II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<String> wordBreak(String s, List<String> wordDict) {
        Map<Integer, List<String>> dp = new HashMap<>();
        int n = s.length();

        for (int i = n - 1; i >= 0; i--) {
            List<String> list = new ArrayList<>();

            for (int j = i + 1; j <= n; j++) {
                String word = s.substring(i, j);

                if (wordDict.contains(word)) {
                    addResult(word, list, dp.get(j));
                }
            }

            dp.put(i, list);
        }

        return dp.get(0);
    }

    private void addResult(String word, List<String> list, List<String> prevList) {
        if (prevList == null) {
            list.add(word);
        } else {
            for (String str : prevList) {
                list.add(new StringBuilder(word).append(" ").append(str).toString());
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

63. `[141]Linked List Cycle.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
  public boolean hasCycle(ListNode head) {
    ListNode fast = head;
    ListNode slow = head;

    while (fast != null && fast.next != null) {
      fast = fast.next.next;
      slow = slow.next;

      if (fast == slow) {
        return true;
      }
    }

    return false;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

64. `[142]Linked List Cycle II.java`

```
142
        Linked List Cycle II
        2023-01-02 22:53:26
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for singly-linked list.
 * class ListNode {
 * int val;
 * ListNode next;
 * ListNode(int x) {
 * val = x;
 * next = null;
 * }
 * }
 */
public class Solution {
    public ListNode detectCycle(ListNode head) {
        Set<ListNode> set = new HashSet<>();
        ListNode node = head;
        while (node != null) {
            if (!set.add(node)) {
                return node;
            }
            node = node.next;
        }
        return null;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

65. `[143]Reorder List.java`

```
// leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  public void reorderList(ListNode head) {
    if (head == null || head.next == null) {
      return;
    }

    ListNode fast = head;
    ListNode slow = head;

    while (fast != null && fast.next != null) {
      fast = fast.next.next;
      slow = slow.next;
    }

    ListNode secondHalf = reverseList(slow);
    ListNode firstHalf = head;

    while (secondHalf.next != null) {
      ListNode temp1 = firstHalf.next;
      ListNode temp2 = secondHalf.next;

      firstHalf.next = secondHalf;
      firstHalf = temp1;

      secondHalf.next = firstHalf;
      secondHalf = temp2;
    }
  }

  private ListNode reverseList(ListNode curr) {
    ListNode prev = null;

    while (curr != null) {
      ListNode next = curr.next;
      curr.next = prev;
      prev = curr;
      curr = next;
    }

    return prev;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

66. `[144]Binary Tree Preorder Traversal.java`

```
144
Binary Tree
Preorder Traversal
        2022-12-02 11:43:26
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> list = new ArrayList<>();
        preOrder(root, list);
        return list;
    }

    private void preOrder(TreeNode node, List<Integer> list) {
        if (node == null) {
            return;
        }

        list.add(node.val);
        preOrder(node.left, list);
        preOrder(node.right, list);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> list = new ArrayList<>();
        Stack<TreeNode> stack = new Stack<>();
        if (root != null)
            stack.push(root);
        TreeNode curr;
        while (!stack.isEmpty()) {
            curr = stack.pop();
            list.add(curr.val);
            if (curr.left != null)
                stack.push(curr.left);
            if (curr.right != null)
                stack.push(curr.right);
        }
        return list;
    }
}
 */
```

67. [145]Binary Tree Postorder Traversal.java

```
145
Binary Tree
Postorder Traversal
        2022-12-02 11:52:01

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> postorderTraversal(TreeNode root) {
        List<Integer> list = new ArrayList<>();
        postOrder(root, list);
        return list;
    }

    private void postOrder(TreeNode node, List<Integer> list) {
        if (node == null) {
            return;
        }

        list.add(node.val);
        postOrder(node.left, list);
        postOrder(node.right, list);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    List<Integer> list = new ArrayList<>();

    public List<Integer> postorderTraversal(TreeNode root) {
        dfs(root);
        return list;
    }

    private void dfs(TreeNode node) {
        if (node == null)
            return;
        dfs(node.left);
        dfs(node.right);
        list.add(node.val);
    }
}
 */
```

68. `[146].java`

```
146
        LRU Cache
        2023-03-04 13:55:14

//leetcode submit region begin(Prohibit modification and deletion)
class LRUCache extends LinkedHashMap<Integer, Integer> {
    int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75F, true);
        this.capacity = capacity;
    }

    public int get(int key) {
        return super.getOrDefault(key, -1);
    }

    public void put(int key, int value) {
        super.put(key, value);
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
        return size() > capacity;
    }
}
/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache obj = new LRUCache(capacity);
 * int param_1 = obj.get(key);
 * obj.put(key,value);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

69. `[146]LRU Cache.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Node {
    public int key;
    public int value;
    public Node prev;
    public Node next;

    Node(int key, int value) {
        this.key = key;
        this.value = value;
        this.prev = null;
        this.next = null;
    }
}

class LRUCache {
    private Map<Integer, Node> map;
    private Node head;
    private Node tail;
    private int capacity;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.map = new HashMap<>();
        this.head = new Node(-1, -1);
        this.tail = new Node(-1, -1);
        head.next = tail;
        tail.prev = head;
    }

    private void deleteNode(Node p) {
        Node pre = p.prev;
        Node nex = p.next;
        pre.next = nex;
        nex.prev = pre;
    }

    private void addNode(Node newNode) {
        Node temp = head.next;
        head.next = newNode;
        newNode.prev = head;
        newNode.next = temp;
        temp.prev = newNode;
    }

    public int get(int key) {
        if (!map.containsKey(key)) {
            return -1;
        }
        Node p = map.get(key);
        deleteNode(p);
```

```
        addNode(p);
        return p.value;
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            Node c = map.get(key);
            deleteNode(c);
            c.value = value;
            addNode(c);
        } else {
            if (map.size() == capacity) {
                Node leastRecentlyUsed = tail.prev;
                deleteNode(leastRecentlyUsed);
                map.remove(leastRecentlyUsed.key);
            }
            Node newNode = new Node(key, value);
            addNode(newNode);
            map.put(key, newNode);
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

70. [147]Insertion Sort List.java

```
147
        Insertion Sort List
        2022-12-16 16:05:32
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode() {}
 * ListNode(int val) { this.val = val; }
 * ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode insertionSortList(ListNode head) {
        ListNode dummy = new ListNode();
        ListNode curr = head;
        while (curr != null) {
            ListNode prev = dummy;
            while (prev.next != null && prev.next.val <= curr.val) {
                prev = prev.next;
            }
            ListNode next = curr.next;
            curr.next = prev.next;
            prev.next = curr;
            curr = next;
        }
        return dummy.next;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

71. `[148]Sort List.java`

```
148
        Sort List
        2022-12-16 16:24:22
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode() {}
 * ListNode(int val) { this.val = val; }
 * ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode sortList(ListNode head) {
        int count = 0;
        ListNode node = head;
        while (node != null) {
            count++;
            node = node.next;
        }
        int[] arr = new int[count];
        node = head;
        for (int i = 0; i < count; i++) {
            arr[i] = node.val;
            node = node.next;
        }
        Arrays.sort(arr);
        ListNode dummy = new ListNode();
        node = dummy;
        for (int num : arr) {
            node.next = new ListNode(num);
            node = node.next;
        }
        return dummy.next;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public ListNode sortList(ListNode head) {
        List<Integer> list = new ArrayList<>();
        ListNode node = head;
        while (node != null) {
            list.add(node.val);
            node = node.next;
        }
        Collections.sort(list);
        ListNode dummy = new ListNode();
```

```
        node = dummy;
        for (int num : list) {
            node.next = new ListNode(num);
            node = node.next;
        }
        return dummy.next;
    }
}
 */
```

72. [149]Max Points on a Line.java

```
149
        Max Points on a Line
        2023-01-08 09:52:13

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxPoints(int[][] points) {
        int n = points.length;
        if (n <= 2) {
            return n;
        }
        int res = 2;
        for (int i = 0; i < n; i++) {
            int[] pointA = points[i];
            for (int j = i + 1; j < n; j++) {
                int[] pointB = points[j];
                int count = 2;
                for (int k = j + 1; k < n; k++) {
                    int[] pointC = points[k];
                    int slope1 = (pointB[1] - pointA[1]) * (pointC[0] - pointB[0]);
                    int slope2 = (pointC[1] - pointB[1]) * (pointB[0] - pointA[0]);
                    if (slope1 == slope2) {
                        count++;
                    }
                }
                res = Math.max(res, count);
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

73. [150]Evaluate Reverse Polish Notation.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int evalRPN(String[] tokens) {
    Deque<Integer> deque = new ArrayDeque<>();
    int x = 0;
    int y = 0;

    for (String token : tokens) {
      switch (token) {
        case "+" -> {
          y = deque.pop();
          x = deque.pop();
          deque.push(x + y);
        }
        case "-" -> {
          y = deque.pop();
          x = deque.pop();
          deque.push(x - y);
        }
        case "*" -> {
          y = deque.pop();
          x = deque.pop();
          deque.push(x * y);
        }
        case "/" -> {
          y = deque.pop();
          x = deque.pop();
          deque.push(x / y);
        }
        default -> deque.push(Integer.parseInt(token));
      }
    }

    return deque.pop();
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

74. [151]Reverse Words in a String.java

```
151
        Reverse Words in a String
        2023-01-23 20:57:26

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String reverseWords(String s) {
        s = s.trim();
        List<String> list = Arrays.asList(s.split("\\s+"));
        Collections.reverse(list);
        return String.join(" ", list);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

75. [152]Maximum Product Subarray.java

```
152
        Maximum Product Subarray
        2023-02-02 16:53:43

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxProduct(int[] nums) {
        int min = nums[0];
        int max = nums[0];
        int res = nums[0];
        for (int i = 1; i < nums.length; i++) {
            int preMax = max;
            int preMin = min;
            int curr = nums[i];
            if (curr < 0) {
                max = Math.max(preMin * curr, curr);
                min = Math.min(preMax * curr, curr);
            } else {
                max = Math.max(preMax * curr, curr);
                min = Math.min(preMin * curr, curr);
            }
            res = Math.max(res, max);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

76. [153]Find Minimum in Rotated Sorted Array.java

```
153
        Find Minimum in Rotated Sorted Array
        2022-12-29 12:30:46

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findMin(int[] nums) {
        int left = 0;
        int right = nums.length - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (mid > 0 && nums[mid] < nums[mid - 1]) {
                return nums[mid];
            }
            if (nums[mid] >= nums[left] && nums[mid] > nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return nums[left];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

77. [154]Find Minimum in Rotated Sorted Array II.java

```

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findMin(int[] nums) {
        int left = 0;
        int right = nums.length - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (nums[mid] > nums[right]) {
                left = mid + 1;
            } else if (nums[mid] < nums[right]) {
                right = mid;
            } else {
                right--;
            }
        }
        return nums[left];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

78. `[155]Min Stack.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class MinStack {
    Deque<Integer> stack;
    Deque<Integer> minStack;

    public MinStack() {
        stack = new ArrayDeque<>();
        minStack = new ArrayDeque<>();
    }

    public void push(int val) {
        stack.push(val);
        if (minStack.isEmpty() || val <= minStack.peek()) {
            minStack.push(val);
        }
    }

    public void pop() {
        int removed = stack.pop();
        if (removed == minStack.peek()) {
            minStack.pop();
        }
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}

/**
 * Your MinStack object will be instantiated and called as such:
 * MinStack obj = new MinStack();
 * obj.push(val);
 * obj.pop();
 * int param_3 = obj.top();
 * int param_4 = obj.getMin();
 */
//leetcode submit region end(Prohibit modification and deletion)
```

79. `[156]Binary Tree Upside Down.java`

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode upsideDownBinaryTree(TreeNode root) {
        if (root == null || root.left == null) {
            return root;
        }

        TreeNode newRoot = upsideDownBinaryTree(root.left);
        root.left.left = root.right;
        root.left.right = root;

        root.left = null;
        root.right = null;

        return newRoot;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

80. `[157]Read N Characters Given Read4.java`

```
/**
 * The read4 API is defined in the parent class Reader4.
 * int read4(char[] buf4);
 */

public class Solution extends Reader4 {
    /**
     * @param buf Destination buffer
     * @param n   Number of characters to read
     * @return The number of actual characters read
     */
    public int read(char[] buf, int n) {
        boolean eof = false;
        int total = 0;
        char[] temp = new char[4];

        while (!eof && total < n) {
            int count = read4(temp);
            eof = count < 4;
            count = Math.min(count, n - total);

            for (int i = 0; i < count; i++) {
                buf[total++] = temp[i];
            }
        }

        return total;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

81. `[159]Longest Substring with At Most Two Distinct Characters.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int lengthOfLongestSubstringTwoDistinct(String s) {
        int[] arr = new int[256];
        int left = 0;
        int count = 0;
        int max = 0;

        for (int right = 0; right < s.length(); right++) {
            char rightChar = s.charAt(right);

            if (arr[rightChar] == 0) {
                count++;
            }

            arr[rightChar]++;

            while (count > 2) {
                char leftChar = s.charAt(left);
                arr[leftChar]--;

                if (arr[leftChar] == 0) {
                    count--;
                }

                left++;
            }

            max = Math.max(max, right - left + 1);
        }

        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

82. [160]Intersection of Two Linked Lists.java

```
160
        Intersection of Two Linked Lists
        2023-01-03 00:35:30
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode(int x) {
 * val = x;
 * next = null;
 * }
 * }
 */
public class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        ListNode nodeA = headA;
        ListNode nodeB = headB;
        while (nodeA != nodeB) {
            nodeA = nodeA == null ? headB : nodeA.next;
            nodeB = nodeB == null ? headA : nodeB.next;
        }
        return nodeA;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

83. `[161]One Edit Distance.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isOneEditDistance(String s, String t) {
        int m = s.length();
        int n = t.length();

        if (Math.abs(m - n) > 1) {
            return false;
        }

        for (int i = 0; i < Math.min(m, n); i++) {
            if (s.charAt(i) != t.charAt(i)) {
                if (m == n) {
                    return s.substring(i + 1).equals(t.substring(i + 1));
                } else if (m > n) {
                    return s.substring(i + 1).equals(t.substring(i));
                } else {
                    return s.substring(i).equals(t.substring(i + 1));
                }
            }
        }

        return Math.abs(m - n) == 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

84. `[162]Find Peak Element.java`

```
162
        Find Peak Element
        2022-12-29 00:53:05

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findPeakElement(int[] nums) {
        if (nums.length == 1) {
            return 0;
        }

        int left = 0;
        int right = nums.length - 1;

        while (left < right) {
            int mid = (left + right) >> 1;

            if (nums[mid] < nums[mid + 1]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }

        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

85. `[163]Missing Ranges.java`

```
163
        Missing Ranges
        2023-01-01 09:58:49

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> findMissingRanges(int[] nums, int lower, int upper) {
        int n = nums.length;
        List<List<Integer>> list = new ArrayList<>();
        if (n == 0) {
            list.add(Arrays.asList(lower, upper));
            return list;
        }
        if (lower < nums[0]) {
            list.add(Arrays.asList(lower, nums[0] - 1));
        }
        for (int i = 1; i < n; i++) {
            if (nums[i] - nums[i - 1] > 1) {
                list.add(Arrays.asList(nums[i - 1] + 1, nums[i] - 1));
            }
        }
        if (upper > nums[n - 1]) {
            list.add(Arrays.asList(nums[n - 1] + 1, upper));
        }
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

86. `[164]Maximum Gap.java`

```
164
        Maximum Gap
        2022-12-18 22:40:31

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumGap(int[] nums) {
        int n = nums.length;
        if (n < 2) {
            return 0;
        }
        Arrays.sort(nums);
        int max = Integer.MIN_VALUE;
        for (int i = 1; i < n; i++) {
            max = Math.max(max, nums[i] - nums[i - 1]);
        }
        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

87. `[165]Compare Version Numbers.java`

```


//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int compareVersion(String version1, String version2) {
        String[] v1 = version1.split("\\.");
        String[] v2 = version2.split("\\.");
        int length = Math.max(v1.length, v2.length);

        for (int i = 0; i < length; i++) {
            int index1 = i < v1.length ? Integer.parseInt(v1[i]) : 0;
            int index2 = i < v2.length ? Integer.parseInt(v2[i]) : 0;

            if (index1 != index2) {
                return index1 > index2 ? 1 : -1;
            }
        }

        return 0;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

88. [166]Fraction to Recurring Decimal.java

```
166
        sb to Recurring Decimal
        2023-01-13 13:53:18

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String fractionToDecimal(int numerator, int denominator) {
        if (numerator == 0) {
            return "0";
        }
        StringBuilder sb = new StringBuilder();
        if (numerator < 0 ^ denominator < 0) {
            sb.append("-");
        }
        long dividend = Math.abs(Long.valueOf(numerator));
        long divisor = Math.abs(Long.valueOf(denominator));
        sb.append(String.valueOf(dividend / divisor));
        long remainder = dividend % divisor;
        if (remainder == 0) {
            return sb.toString();
        }
        sb.append(".");
        Map<Long, Integer> map = new HashMap<>();
        while (remainder != 0) {
            if (map.containsKey(remainder)) {
                sb.insert(map.get(remainder), "(");
                sb.append(")");
                break;
            }
            map.put(remainder, sb.length());
            remainder *= 10;
            sb.append(String.valueOf(remainder / divisor));
            remainder %= divisor;
        }
        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

89. [167]Two Sum II - Input Array Is Sorted.java

```
167
        Two Sum II-Input Array Is Sorted
        2022-12-30 12:06:48

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int left = 0;
        int right = numbers.length - 1;
        while (left <= right) {
            int sum = numbers[left] + numbers[right];
            int mid = (left + right) >> 1;
            if (sum == target) {
                return new int[]{left + 1, right + 1};
            } else if (sum < target) {
                if (numbers[mid] + numbers[right] < target) {
                    left = mid;
                }
                left++;
            } else {
                if (numbers[left] + numbers[mid] > target) {
                    right = mid;
                }
                right--;
            }
        }
        return new int[]{-1, -1};
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

90. [168]Excel Sheet Column Title.java

```

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String convertToTitle(int columnNumber) {
        StringBuilder sb = new StringBuilder();
        while (columnNumber > 0) {
            columnNumber--;
            char c = (char) (columnNumber % 26 + 'A');
            sb.append(c);
            columnNumber /= 26;
        }
        return sb.reverse().toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

91. `[169]Majority Element.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int majorityElement(int[] nums) {
        int count = 0;
        int candidate = 0;

        for (int num : nums) {
            if (count == 0) {
                candidate = num;
            }

            count += (num == candidate) ? 1 : -1;
        }

        return candidate;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int majorityElement(int[] nums) {
        Arrays.sort(nums);
        return nums[nums.length / 2];
    }
}
 */
```

92. `[171]Excel Sheet Column Number.java`

```
/**
 * <p>Given a string <code>columnTitle</code> that represents the column title as appears
in an Excel sheet, return <em>its corresponding column number</em>.</p>
 *
 * <p>For example:</p>
 *
 * <pre>
 * A -&gt; 1
 * B -&gt; 2
 * C -&gt; 3
 * ...
 * Z -&gt; 26
 * AA -&gt; 27
 * AB -&gt; 28
 * ...
 * </pre>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> columnTitle = "A"
 * <strong>Output:</strong> 1
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> columnTitle = "AB"
 * <strong>Output:</strong> 28
 * </pre>
 *
 * <p><strong class="example">Example 3:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> columnTitle = "ZY"
 * <strong>Output:</strong> 701
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>1 &lt;= columnTitle.length &lt;= 7</code></li>
 * <li><code>columnTitle</code> consists only of uppercase English letters.</li>
 * <li><code>columnTitle</code> is in the range <code>["A", "FXSHRXW"]</code>.</li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>Math</li><li>String</li></div></div><br><div>
<li>👍 4130</li><li>👎 327</li></div>
 */
```

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int titleToNumber(String columnTitle) {
        int res = 0;
        for (char c : columnTitle.toCharArray()) {
            int num = c - 'A' + 1;
            res = res * 26 + num;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

93. [173]Binary Search Tree Iterator.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class BSTIterator {
    List<Integer> list = new ArrayList<>();
    int index = 0;

    public BSTIterator(TreeNode root) {

        inOrder(root);
    }

    private void inOrder(TreeNode node) {
        if (node == null) {
            return;
        }

        inOrder(node.left);
        list.add(node.val);
        inOrder(node.right);
    }

    public int next() {
        return list.get(index++);
    }

    public boolean hasNext() {
        return list.size() > index ? true : false;
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

94. `[179]Largest Number.java`

```
179
        Largest Number
        2023-01-30 16:49:00

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String largestNumber(int[] nums) {
        List<String> list = new ArrayList<>();
        for (int num : nums) {
            list.add(Integer.toString(num));
        }
        Collections.sort(list, (a, b) -> (b + a).compareTo(a + b));
        if (list.get(0).equals("0")) {
            return "0";
        }
        StringBuilder sb = new StringBuilder();
        for (String num : list) {
            sb.append(num);
        }
        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

95. [186]Reverse Words in a String II.java

```
186
        Reverse Words in a String II
        2023-01-23 20:57:33

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public void reverseWords(char[] s) {
        reverse(s, 0, s.length - 1);
        reverseEach(s);
    }

    public void reverse(char[] s, int left, int right) {
        while (left < right) {
            char temp = s[left];
            s[left] = s[right];
            s[right] = temp;
            left++;
            right--;
        }
    }

    public void reverseEach(char[] s) {
        int n = s.length;
        int left = 0;
        int right = 0;
        while (left < n) {
            while (right < n && s[right] != ' ') {
                right++;
            }
            reverse(s,left, right - 1);
            left = right + 1;
            right++;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

96. [188]Best Time to Buy and Sell Stock IV.java

```
188
        Best Time to Buy and Sell Stock IV
        2022-12-14 17:31:45

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxProfit(int k, int[] prices) {
        int[] cost = new int[k + 1];
        int[] profit = new int[k + 1];
        Arrays.fill(cost, Integer.MAX_VALUE);
        for (int price : prices) {
            for (int i = 1; i <= k; i++) {
                cost[i] = Math.min(cost[i], price - profit[i - 1]);
                profit[i] = Math.max(profit[i], price - cost[i]);
            }
        }
        return profit[k];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int maxProfit(int k, int[] prices) {
        int n = prices.length;
        if (k >= n / 2) {
            int maxProfit = 0;
            for (int i = 1; i < n; i++) {
                if (prices[i] > prices[i - 1]) {
                    maxProfit += prices[i] - prices[i - 1];
                }
            }
            return maxProfit;
        }
        int[][] dp = new int[k + 1][n];
        for (int i = 1; i <= k; i++) {
            int maxDiff = -prices[0];
            for (int j = 1; j < n; j++) {
                dp[i][j] = Math.max(dp[i][j - 1], maxDiff + prices[j]);
                maxDiff = Math.max(maxDiff, dp[i - 1][j] - prices[j]);
            }
        }
        return dp[k][n - 1];
    }
}

class Solution {
    public int maxProfit(int k, int[] prices) {
        int n = prices.length;
        int[][][] dp = new int[n + 1][k + 1][2];
        for (int i = n - 1; i >= 0; i--) {
            for (int j = 1; j <= k; j++) {
```

```
                for (int hold = 0; hold < 2; hold++) {
                    int notTrade = dp[i + 1][j][hold];
                    int doTrade;
                    if (hold == 1) {
                        doTrade = prices[i] + dp[i + 1][j - 1][0];
                    } else {
                        doTrade = -prices[i] + dp[i + 1][j][1];
                    }
                    dp[i][j][hold] = Math.max(doTrade, notTrade);
                }
            }
        }
        return dp[0][k][0];
    }
}
 */
```

97. [189]Rotate Array.java

```
189
        Rotate Array
        2022-12-30 18:41:35

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public void rotate(int[] nums, int k) {
        k = k % nums.length;
        int[] numToShift = new int[k];
        for (int i = 0; i < k; i++) {
            numToShift[i] = nums[nums.length - k + i];
        }
        for (int i = nums.length - 1; i >= 0; i--) {
            if (i >= k) {
                nums[i] = nums[i - k];
            } else {
                nums[i] = numToShift[i];
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

98. [190]Reverse Bits.java

```
190
        Reverse Bits
        2023-01-24 16:04:06

//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    // you need treat n as an unsigned value
    public int reverseBits(int n) {
        int res = 0;
        for (int i = 0; i < 32; i++) {
            res = res << 1;
            res = res + (n & 1);
            n = n >> 1;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

99. [191]Number of 1 Bits.java

```
191
        Number of 1Bits
        2023-01-24 15:10:04

//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    // you need to treat n as an unsigned value
    public int hammingWeight(int n) {
        int count = 0;

        for (int i = 0; i < 32; i++) {
            if ((n & 1) == 1) {
                count++;
            }

            n = n >> 1;
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

100. `[192]Word Frequency.java`

```
/**
<p>Write a bash script to calculate the <span data-keyword="frequency-
textfile">frequency</span> of each word in a text file <code>words.txt</code>.</p>

<p>For simplicity sake, you may assume:</p>

<ul>
 <li><code>words.txt</code> contains only lowercase characters and space <code>' '</code>
characters.</li>
 <li>Each word must consist of lowercase characters only.</li>
 <li>Words are separated by one or more whitespace characters.</li>
</ul>

<p><strong class="example">Example:</strong></p>

<p>Assume that <code>words.txt</code> has the following content:</p>

<pre>
the day is sunny the the
the sunny is is
</pre>

<p>Your script should output the following, sorted by descending frequency:</p>

<pre>
the 4
is 3
sunny 2
day 1
</pre>

<p><b>Note:</b></p>

<ul>
 <li>Don't worry about handling ties, it is guaranteed that each word's frequency count
is unique.</li>
 <li>Could you write it in one-line using <a href="http://tldp.org/HOWTO/Bash-Prog-Intro-
HOWTO-4.html">Unix pipes</a>?</li>
</ul>

<div><div>Related Topics</div><div><li>Shell</li></div></div><br><div><li>👍 474</li><li>
👎 284</li></div>
*/

//There is no code of Java type for this problem
```

101. `[198]House Robber.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;

        if (n == 1) {
            return nums[0];
        }

        int[] dp = new int[n];
        dp[0] = nums[0];
        dp[1] = Math.max(nums[0], nums[1]);

        for (int i = 2; i < n; i++) {
            dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);
        }

        return dp[n - 1];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

102. `[199]Binary Tree Right Side View.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        if (root == null) {
            return new ArrayList<>();
        }

        List<Integer> list = new ArrayList<>();
        Deque<TreeNode> dq = new ArrayDeque<>();
        dq.add(root);

        while (!dq.isEmpty()) {
            int size = dq.size();

            for (int i = 0; i < size; i++) {
                TreeNode curr = dq.poll();

                if (i == size - 1) {
                    list.add(curr.val);
                }

                if (curr.left != null) {
                    dq.add(curr.left);
                }

                if (curr.right != null) {
                    dq.add(curr.right);
                }
            }
        }

        return list;
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

103. `[200]Number of Islands.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numIslands(char[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int count = 0;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1') {
                    count++;
                    dfs(grid, i, j);
                }
            }
        }

        return count;
    }

    private void dfs(char[][] grid, int i, int j) {
        if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length || grid[i][j] ==
'0') {
            return;
        }

        grid[i][j] = '0';
        dfs(grid, i - 1, j);
        dfs(grid, i + 1, j);
        dfs(grid, i, j - 1);
        dfs(grid, i, j + 1);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
1.DFS
class Solution {
    char[][] grid;
    int m;
    int n;

    public int numIslands(char[][] grid) {
        this.grid = grid;
        m = grid.length;
        n = grid[0].length;
        int count = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1') {
                    count++;
                    dfs(i, j);
                }
```

```
            }
        }
        return count;
    }

    public void dfs(int i, int j) {
        if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == '0') {
            return;
        }
        grid[i][j] = '0';
        dfs(i + 1, j);
        dfs(i - 1, j);
        dfs(i, j + 1);
        dfs(i, j - 1);
    }
}

2.BFS
class Solution {
    public int numIslands(char[][] grid) {
        int[][] dirs = new int[][]{{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        Deque<int[]> dq = new ArrayDeque<>();
        int m = grid.length;
        int n = grid[0].length;
        int count = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1') {
                    count++;
                    grid[i][j] = '0';
                    dq.add(new int[]{i, j});
                    while (!dq.isEmpty()) {
                        int[] curr = dq.poll();
                        for (int[] dir : dirs) {
                            int x = curr[0] + dir[0];
                            int y = curr[1] + dir[1];
                            if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] == '0')
{
                                continue;
                            }
                            dq.add(new int[]{x, y});
                            grid[x][y] = '0';
                        }
                    }
                }
            }
        }
        return count;
    }
}

3.Union Find
class Solution {
    public int numIslands(char[][] grid) {
```

```
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int m = grid.length;
        int n = grid[0].length;
        UnionFind uf = new UnionFind(grid);
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1') {
                    for (int[] dir : dirs) {
                        int x = i + dir[0];
                        int y = j + dir[1];
                        if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] == '0') {
                            continue;
                        }
                        uf.union(i * n + j, x * n + y);
                    }
                }
            }
        }
        return uf.getCount();
    }
}

class UnionFind {
    int[] group;
    int count;

    public UnionFind(char[][] grid) {
        count = 0;
        int m = grid.length;
        int n = grid[0].length;
        group = new int[m * n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1') {
                    count++;
                    group[i * n + j] = i * n + j;
                }
            }
        }
    }

    public int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }

    public void union(int i, int j) {
        int x = find(i);
        int y = find(j);
        if (x != y) {
            group[x] = y;
            count--;
        }
    }
```

```
    public int getCount() {
        return count;
    }
}

 */
```

104. [201]Bitwise AND of Numbers Range.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int rangeBitwiseAnd(int left, int right) {
        int shift = 0;

        while (left != right) {
            left >>= 1;
            right >>= 1;
            shift++;
        }

        return left << shift;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

105. `[202]Happy Number.java`

```
202
        Happy Number
        2023-01-28 09:57:57

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isHappy(int n) {
        Set<Integer> set = new HashSet<>();
        while (!set.contains(n) && n != 1) {
            set.add(n);
            int sum = 0;
            while (n > 0) {
                int x = n % 10;
                sum += x * x;
                n /= 10;
            }
            n = sum;
        }
        return n == 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public boolean isHappy(int n) {
        List<Integer> list = new ArrayList<>();
        list.add(n);
        while (n != 1) {
            int temp = 0;
            while (n != 0) {
                temp += (n % 10) * (n % 10);
                n /= 10;
            }
            n = temp;
            if (list.contains(n)) {
                break;
            } else {
                list.add(n);
            }
        }
        return n == 1;
    }
}
 */
```

106. [203]Remove Linked List Elements.java

```
203
Remove Linked List Elements
        2023-01-02 14:33:25
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode() {}
 * ListNode(int val) { this.val = val; }
 * ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode dummy = new ListNode(0, head);
        ListNode prev = dummy;
        ListNode curr = head;

        while (curr != null) {
            if (curr.val == val) {
                prev.next = curr.next;
            } else {
                prev = curr;
            }
            curr = curr.next;
        }

        return dummy.next;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

107. `[204]Count Primes.java`

```
204
        Count Primes
        2022-12-31 22:22:23

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countPrimes(int n) {
        if (n <= 2) {
            return 0;
        }
        boolean[] nums = new boolean[n];
        for (int i = 2; i <= (int) Math.sqrt(n); i++) {
            if (nums[i] == false) {
                for (int j = 2 * i; j < n; j += i) {
                    nums[j] = true;
                }
            }
        }
        int count = 0;
        for (int i = 2; i < n; i++) {
            if (nums[i] == false) {
                count++;
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

108. `[205]Isomorphic Strings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isIsomorphic(String s, String t) {
        Map<Character, Character> map1 = new HashMap<>();
        Map<Character, Character> map2 = new HashMap<>();

        for (int i = 0; i < s.length(); i++) {
            char sc = s.charAt(i);
            char tc = t.charAt(i);
            Character mappedToT = map1.get(sc);
            Character mappedToS = map2.get(tc);

            if (mappedToT == null) {
                map1.put(sc, tc);
            } else if (!mappedToT.equals(tc)) {
                return false;
            }

            if (mappedToS == null) {
                map2.put(tc, sc);
            } else if (!mappedToS.equals(sc)) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

109. `[206]Reverse Linked List.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null;
        ListNode curr = head;
        ListNode next = null;

        while (curr != null) {
            next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }

        return prev;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

110. `[207]Course Schedule.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) {
      adj.add(new ArrayList<>());
    }

    int[] inDegree = new int[numCourses];
    for (int[] pre : prerequisites) {
      int course = pre[0];
      int before = pre[1];
      adj.get(before).add(course);
      inDegree[course]++;
    }

    Queue<Integer> queue = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++) {
      if (inDegree[i] == 0) {
        queue.offer(i);
      }
    }

    int count = 0;
    while (!queue.isEmpty()) {
      int u = queue.poll();
      count++;
      for (int v : adj.get(u)) {
        inDegree[v]--;
        if (inDegree[v] == 0) {
          queue.offer(v);
        }
      }
    }

    return count == numCourses;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

111. `[208].java`

```
208
        Implement Trie(Prefix Tree)
        2023-03-07 11:07:16

//leetcode submit region begin(Prohibit modification and deletion)
class Trie {
    TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children.get(c) == null) {
                node.children.put(c, new TrieNode());
            }
            node = node.children.get(c);
        }
        node.isWord = true;
    }

    public boolean search(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children.get(c) == null) {
                return false;
            }
            node = node.children.get(c);
        }
        return node.isWord;
    }

    public boolean startsWith(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toCharArray()) {
            if (node.children.get(c) == null) {
                return false;
            }
            node = node.children.get(c);
        }
        return true;
    }
}

class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    boolean isWord;
}
```

```
/**
 * Your Trie object will be instantiated and called as such:
 * Trie obj = new Trie();
 * obj.insert(word);
 * boolean param_2 = obj.search(word);
 * boolean param_3 = obj.startsWith(prefix);
 * <p>
 * Your Trie object will be instantiated and called as such:
 * Trie obj = new Trie();
 * obj.insert(word);
 * boolean param_2 = obj.search(word);
 * boolean param_3 = obj.startsWith(prefix);
 * <p>
 * Your Trie object will be instantiated and called as such:
 * Trie obj = new Trie();
 * obj.insert(word);
 * boolean param_2 = obj.search(word);
 * boolean param_3 = obj.startsWith(prefix);
 * <p>
 * Your Trie object will be instantiated and called as such:
 * Trie obj = new Trie();
 * obj.insert(word);
 * boolean param_2 = obj.search(word);
 * boolean param_3 = obj.startsWith(prefix);
 */

/**
 * Your Trie object will be instantiated and called as such:
 * Trie obj = new Trie();
 * obj.insert(word);
 * boolean param_2 = obj.search(word);
 * boolean param_3 = obj.startsWith(prefix);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

112. [208]Implement Trie (Prefix Tree).java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Trie {
  private static class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEndofWord = false;
  }

  private final TrieNode root;

  public Trie() {
    root = new TrieNode();
  }

  public void insert(String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
      int index = c - 'a';
      if (node.children[index] == null) {
        node.children[index] = new TrieNode();
      }
      node = node.children[index];
    }
    node.isEndofWord = true;
  }

  public boolean search(String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
      int index = c - 'a';
      if (node.children[index] == null) {
        return false;
      }
      node = node.children[index];
    }
    return node.isEndofWord;
  }

  public boolean startsWith(String prefix) {
    TrieNode node = root;
    for (char c : prefix.toCharArray()) {
      int index = c - 'a';
      if (node.children[index] == null) {
        return false;
      }
      node = node.children[index];
    }
    return true;
  }
}

/**
```

```
 * Your Trie object will be instantiated and called as such:
 * Trie obj = new Trie();
 * obj.insert(word);
 * boolean param_2 = obj.search(word);
 * boolean param_3 = obj.startsWith(prefix);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

113. [209]Minimum Size Subarray Sum.java

```
209
        Minimum Size Subarray Sum
        2023-02-26 17:31:02

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int res = Integer.MAX_VALUE;
        int left = 0;
        int sum = 0;
        for (int right = 0; right < nums.length; right++) {
            sum += nums[right];
            while (sum >= target) {
                res = Math.min(res, right - left + 1);
                sum -= nums[left++];
            }
        }
        return res == Integer.MAX_VALUE ? 0 : res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

114. `[210]Course Schedule II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> list = new ArrayList<>();

        for (int i = 0; i < numCourses; i++) {
            list.add(new ArrayList<>());
        }

        int[] inDegree = new int[numCourses];

        for (int[] pre : prerequisites) {
            list.get(pre[1]).add(pre[0]);
            inDegree[pre[0]]++;
        }

        Queue<Integer> queue = new ArrayDeque<>();

        for (int i = 0; i < numCourses; i++) {
            if (inDegree[i] == 0) {
                queue.offer(i);
            }
        }

        int count = 0;
        int[] result = new int[numCourses];

        while (!queue.isEmpty()) {
            int curr = queue.poll();
            result[count] = curr;
            count++;

            for (int num : list.get(curr)) {
                inDegree[num]--;

                if (inDegree[num] == 0) {
                    queue.offer(num);
                }
            }
        }

        if (count == numCourses) {
            return result;
        } else {
            return new int[0];
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

115. `[211]Design Add and Search Words Data`

Structure.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    boolean isWord = false;
}

class WordDictionary {
    TrieNode trie;
    int max;

    public WordDictionary() {
        trie = new TrieNode();
        max = 0;
    }

    public void addWord(String word) {
        TrieNode node = trie;
        for (char c : word.toCharArray()) {
            if (node.children.get(c) == null) {
                node.children.put(c, new TrieNode());
            }
            node = node.children.get(c);
        }
        node.isWord = true;
        max = Math.max(max, word.length());
    }

    public boolean searchInNode(String word, TrieNode node, int index) {
        if (index >= word.length()) {
            return node.isWord;
        }
        char c = word.charAt(index);
        if (c == '.') {
            for (char key : node.children.keySet()) {
                TrieNode child = node.children.get(key);
                if (child == null) {
                    continue;
                }
                if (searchInNode(word, child, index + 1)) {
                    return true;
                }
            }
            return false;
        } else {
            if (node.children.get(c) == null) {
                return false;
            }
            return searchInNode(word, node.children.get(c), index + 1);
        }
    }
```

```
    public boolean search(String word) {
        if (word.length() > max) {
            return false;
        }
        return searchInNode(word, trie, 0);
    }
}

/**
 * Your WordDictionary object will be instantiated and called as such:
 * WordDictionary obj = new WordDictionary();
 * obj.addWord(word);
 * boolean param_2 = obj.search(word);
 * <p>
 * Your WordDictionary object will be instantiated and called as such:
 * WordDictionary obj = new WordDictionary();
 * obj.addWord(word);
 * boolean param_2 = obj.search(word);
 * <p>
 * Your WordDictionary object will be instantiated and called as such:
 * WordDictionary obj = new WordDictionary();
 * obj.addWord(word);
 * boolean param_2 = obj.search(word);
 * <p>
 * Your WordDictionary object will be instantiated and called as such:
 * WordDictionary obj = new WordDictionary();
 * obj.addWord(word);
 * boolean param_2 = obj.search(word);
 * <p>
 * Your WordDictionary object will be instantiated and called as such:
 * WordDictionary obj = new WordDictionary();
 * obj.addWord(word);
 * boolean param_2 = obj.search(word);
 */

/**
 * Your WordDictionary object will be instantiated and called as such:
 * WordDictionary obj = new WordDictionary();
 * obj.addWord(word);
 * boolean param_2 = obj.search(word);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

116. `[212]Word Search II.java`

```
/*
還不會
 */
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    char[][] board;
    boolean[][] isVisited;
    int[][] dirs = new int[][]{{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    List<String> list = new ArrayList<>();
    int m;
    int n;

    public List<String> findWords(char[][] board, String[] words) {
        this.board = board;
        m = board.length;
        n = board[0].length;
        isVisited = new boolean[m][n];
        TrieNode root = new TrieNode();
        for (String word : words) {
            TrieNode node = root;
            for (char c : word.toCharArray()) {
                if (!node.children.containsKey(c)) {
                    node.children.put(c, new TrieNode());
                }
                node = node.children.get(c);
            }
            node.word = word;
        }
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (root.children.containsKey(board[i][j])) {
                    isVisited[i][j] = true;
                    backtrack(i, j, root);
                    isVisited[i][j] = false;
                }
            }
        }
        return list;
    }

    public void backtrack(int i, int j, TrieNode root) {
        char c = board[i][j];
        TrieNode node = root.children.get(c);
        if (node.word != null) {
            list.add(node.word);
            node.word = null;
        }
        for (int[] dir : dirs) {
            int x = i + dir[0];
            int y = j + dir[1];
            if (x < 0 || x >= m || y < 0 || y >= n || isVisited[x][y] == true) {
```

```
                continue;
            }
            if (node.children.containsKey(board[x][y])) {
                isVisited[x][y] = true;
                backtrack(x, y, node);
                isVisited[x][y] = false;
            }
        }
        if (node.children.isEmpty()) {
            root.children.remove(c);
        }
    }
}

class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    String word = null;
}

//leetcode submit region end(Prohibit modification and deletion)
```

117. [215]Kth Largest Element in an Array.java

```
215
        Kth Largest Element in an Array
        2022-12-09 14:55:35

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findKthLargest(int[] nums, int k) {
        Arrays.sort(nums);
        return nums[nums.length - k];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

118. [216]Combination Sum III.java

```
216
        Combination Sum III
        2022-12-14 10:42:42

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    List<List<Integer>> res = new ArrayList<>();

    public List<List<Integer>> combinationSum3(int k, int n) {
        backtreck(k, n, new ArrayList<Integer>(), 1);
        return res;
    }

    private void backtreck(int k, int n, List<Integer> list, int index) {
        if (list.size() == k && n == 0) {
            res.add(new ArrayList<>(list));
            return;
        }
        for (int i = index; i <= 9; i++) {
            list.add(i);
            backtreck(k, n - i, list, i + 1);
            list.remove(list.size() - 1);
        }
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

119. [217]Contains Duplicate.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public boolean containsDuplicate(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int num : nums) {
      if (!set.add(num)) {
        return true;
      }
    }
    return false;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

120. `[218]The Skyline Problem.java`

```
218
        The Skyline Problem
        2022-12-26 13:38:58

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> getSkyline(int[][] buildings) {
        PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> buildings[b][2] -
buildings[a][2]);
        List<List<Integer>> list = new ArrayList<>();
        int n = buildings.length;
        Arrays.sort(buildings, (a, b) -> a[0] == b[0] ? b[2] - a[2] : a[0] - b[0]);
        for (int i = 0; i < n; i++) {
            while (!pq.isEmpty() && buildings[pq.peek()][1] < buildings[i][0]) {
                pq.poll();
            }
            if (pq.isEmpty() || buildings[i][2] > buildings[pq.peek()][2]) {
                list.add(Arrays.asList(buildings[i][0], buildings[i][2]));
            }
            pq.add(i);
        }
        pq.clear();
        Arrays.sort(buildings, (a, b) -> a[1] == b[1] ? a[2] - b[2] : a[1] - b[1]);
        for (int i = n - 1; i >= 0; i--) {
            while (!pq.isEmpty() && buildings[pq.peek()][0] > buildings[i][1]) {
                pq.poll();
            }
            if (pq.isEmpty()) {
                list.add(Arrays.asList(buildings[i][1], 0));
            }
            if (!pq.isEmpty() && buildings[i][2] > buildings[pq.peek()][2]) {
                list.add(Arrays.asList(buildings[i][1], buildings[pq.peek()][2]));
            }
            pq.add(i);
        }
        Collections.sort(list, (a, b) -> a.get(0) - b.get(0));
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

121. `[219]Contains Duplicate II.java`

```
219
        Contains Duplicate II
        2023-01-28 17:42:56

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean containsNearbyDuplicate(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            if (map.containsKey(nums[i])) {
                if (i - map.get(nums[i]) <= k) {
                    return true;
                }
            }
            map.put(nums[i], i);
        }
        return false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

122. `[220]Contains Duplicate III.java`

```
220
Contains Duplicate III
2023-01-28 17:44:47
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean containsNearbyAlmostDuplicate(int[] nums, int indexDiff, int
valueDiff) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

123. [221]Maximal Square.java

```
221Maximal Square 2023-02-21 10:14:41

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximalSquare(char[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        int[][] dp = new int[m + 1][n + 1];
        int max = 0;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (matrix[i - 1][j - 1] == '1') {
                    dp[i][j] = Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp[i][j
- 1])) + 1;
                    max = Math.max(max, dp[i][j]);
                }
            }
        }
        return max * max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

124. [223]Rectangle Area.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int computeArea(int ax1, int ay1, int ax2, int ay2, int bx1, int by1, int bx2,
int by2) {
        int area = Math.abs(ax1 - ax2) * Math.abs(ay1 - ay2) + Math.abs(bx1 - bx2) *
Math.abs(by1 - by2);
        int x = Math.min(ax2, bx2) - Math.max(ax1, bx1);
        int y = Math.min(ay2, by2) - Math.max(ay1, by1);
        int repeat = 0;
        if (x > 0 && y > 0)
            repeat = x * y;
        return area - repeat;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

125. `[224]Basic Calculator.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int calculate(String s) {
        Deque<Character> dq = new ArrayDeque<>();

        for (char c : s.toCharArray()) {
            dq.add(c);
        }

        return calc(dq);
    }

    private int calc(Queue<Character> dq) {
        int num = 0;
        Deque<Integer> stack = new ArrayDeque<>();
        char prevOp = '+';

        while (!dq.isEmpty()) {
            char c = dq.poll();

            if (Character.isDigit(c)) {
                num = num * 10 + (c - '0');
            } else if (c == '(') {
                num = calc(dq);
            } else if ("+-".indexOf(c) != -1) {
                eval(stack, num, prevOp);
                num = 0;
                prevOp = c;
            } else if (c == ')') {
                break;
            }
        }

        eval(stack, num, prevOp);
        int res = 0;

        while (!stack.isEmpty()) {
            res += stack.pop();
        }

        return res;
    }

    public void eval(Deque<Integer> stack, int num, char op) {
        switch (op) {
            case '+' -> stack.push(num);
            case '-' -> stack.push(-num);
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

126. `[225]Implement Stack using Queues.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
/** Stack implementation using a single queue. */
public class MyStack {
  private final Queue<Integer> queue;

  /** Initialize your data structure here. */
  public MyStack() {
    queue = new LinkedList<>();
  }

  /**
   * Push element x onto stack.
   *
   * @param x the element to push
   */
  public void push(int x) {
    queue.offer(x);
    // Rotate the queue to move the new element to the front
    int sz = queue.size();
    for (int i = 0; i < sz - 1; i++) {
      queue.offer(queue.poll());
    }
  }

  /**
   * Removes the element on top of the stack and returns that element.
   *
   * @return the top element
   */
  public int pop() {
    return queue.isEmpty() ? -1 : queue.poll();
  }

  /**
   * Get the top element.
   *
   * @return the top element without removing it
   */
  public int top() {
    return queue.isEmpty() ? -1 : queue.peek();
  }

  /**
   * Returns whether the stack is empty.
   *
   * @return true if empty, false otherwise
   */
  public boolean empty() {
    return queue.isEmpty();
  }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

127. [226]Invert Binary Tree.java

```
// leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  public TreeNode invertTree(TreeNode root) {
    if (root == null) {
      return null;
    }

    TreeNode leftSubTree = invertTree(root.left);
    TreeNode rightSubTree = invertTree(root.right);

    root.left = rightSubTree;
    root.right = leftSubTree;

    return root;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

128. `[227]Basic Calculator II.java`

```
227
        Basic Calculator II
        2023-01-31 14:18:14

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int calculate(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        int num = 0;
        char prevOp = '+';
        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) {
                num = num * 10 + (c - '0');
            } else if ("+-*/".indexOf(c) != -1) {
                eval(stack, num, prevOp);
                num = 0;
                prevOp = c;
            }
        }
        eval(stack, num, prevOp);
        int res = 0;
        while (!stack.isEmpty()) {
            res += stack.pop();
        }
        return res;
    }
    public void eval(Deque<Integer> stack, int num, char op) {
        switch (op) {
            case '+' -> stack.push(num);
            case '-' -> stack.push(-num);
            case '*' -> stack.push(stack.pop() * num);
            case '/' -> stack.push(stack.pop() / num);
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

129. `[228]Summary Ranges.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<String> summaryRanges(int[] nums) {
        List<String> list = new ArrayList<>();
        int n = nums.length;
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            int left = nums[i];
            while (i + 1 < n && nums[i] + 1 == nums[i + 1]) {
                i++;
            }
            if (left != nums[i]) {
                sb.append(left).append("->").append(nums[i]);
                list.add(sb.toString());
                sb.setLength(0);
            } else {
                list.add(String.valueOf(left));
            }
        }
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

130. `[229]Majority Element II.java`

```
229
        Majority Element II
        2023-01-14 00:04:32

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> majorityElement(int[] nums) {
        int n = nums.length;
        List<Integer> list = new ArrayList<>();
        int bar = n / 3;
        int count = 1;

        Arrays.sort(nums);

        for (int i = 1; i < n; i++) {
            if (nums[i] == nums[i - 1]) {
                count++;
                continue;
            }

            if (count > bar) {
                list.add(nums[i - 1]);
            }

            count = 1;
        }

        if (count > bar) {
            list.add(nums[n - 1]);
        }

        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public List<Integer> majorityElement(int[] nums) {
        int bar = nums.length / 3;
        List<Integer> list = new ArrayList<>();
        int count = 1;
        Arrays.sort(nums);
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] == nums[i - 1]) {
                count++;
                continue;
            }
            if (count > bar) {
                list.add(nums[i - 1]);
            }
            count = 1;
```

```
        }
        if (count > bar) {
            list.add(nums[nums.length - 1]);
        }
        return list;
    }
}

class Solution {
    public List<Integer> majorityElement(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        List<Integer> list = new ArrayList<>();
        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }
        int bar = nums.length / 3;
        for (int key : map.keySet()) {
            if (map.get(key) > bar) {
                list.add(key);
            }
        }
        return list;
    }
}
 */
```

131. [230]Kth Smallest Element in a BST.java

```
/**
 * <p>Given the <code>root</code> of a binary search tree, and an integer <code>k</code>,
return <em>the</em> <code>k<sup>th</sup></code> <em>smallest value (<strong>1-
indexed</strong>) of all the values of the nodes in the tree</em>.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2021/01/28/kthtree1.jpg"
style="width: 212px; height: 301px;" />
 * <pre>
 * <strong>Input:</strong> root = [3,1,4,null,2], k = 1
 * <strong>Output:</strong> 1
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2021/01/28/kthtree2.jpg"
style="width: 382px; height: 302px;" />
 * <pre>
 * <strong>Input:</strong> root = [5,3,6,2,4,null,null,1], k = 3
 * <strong>Output:</strong> 3
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li>The number of nodes in the tree is <code>n</code>.</li>
 * <li><code>1 &lt;= k &lt;= n &lt;= 10<sup>4</sup></code></li>
 * <li><code>0 &lt;= Node.val &lt;= 10<sup>4</sup></code></li>
 * </ul>
 *
 * <p> </p>
 * <p><strong>Follow up:</strong> If the BST is modified often (i.e., we can do insert
and delete operations) and you need to find the kth smallest frequently, how would you
optimize?</p>
 *
 * <div><div>Related Topics</div><div><li>Tree</li><li>Depth-First Search</li><li>Binary
Search Tree</li><li>Binary Tree</li></div></div><br><div><li>👍 9605</li><li>👎 173</li>
</div>
 */

//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
```

```
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    List<Integer> list = new ArrayList<>();

    public int kthSmallest(TreeNode root, int k) {
        List<Integer> res = inOrder(root);
        return res.get(k - 1);
    }

    public List<Integer> inOrder(TreeNode node) {
        if (node == null) {
            return list;
        }
        inOrder(node.left);
        list.add(node.val);
        inOrder(node.right);
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

132. `[231]Power of Two.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isPowerOfTwo(int n) {
        return n > 0 && (n & (n - 1)) == 0;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

133. `[232]Implement Queue using Stacks.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
/** Queue implementation using two stacks with lazy transfer. All operations are
amortized O(1). */
public class MyQueue {
  private final Stack<Integer> inbox;
  private final Stack<Integer> outbox;

  /** Initialize your data structure here. */
  public MyQueue() {
    inbox = new Stack<>();
    outbox = new Stack<>();
  }

  /**
   * Pushes element x to the back of the queue.
   *
   * @param x the element to push
   */
  public void push(int x) {
    inbox.push(x);
  }

  /**
   * Removes the element from the front of the queue and returns it. Transfers elements
only if
   * outbox is empty.
   *
   * @return the front element
   */
  public int pop() {
    moveIfNeeded();
    return outbox.pop();
  }

  /**
   * Gets the front element.
   *
   * @return the front element without removing it
   */
  public int peek() {
    moveIfNeeded();
    return outbox.peek();
  }

  /**
   * Returns whether the queue is empty.
   *
   * @return true if empty, false otherwise
   */
  public boolean empty() {
    return inbox.isEmpty() && outbox.isEmpty();
```

```
  }

  /** Helper to transfer only when outbox is empty */
  private void moveIfNeeded() {
    if (outbox.isEmpty()) {
      while (!inbox.isEmpty()) {
        outbox.push(inbox.pop());
      }
    }
  }
}
//leetcode submit region end(Prohibit modification and deletion)
```

134. `[234]Palindrome Linked List.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
  public boolean isPalindrome(ListNode head) {
    if (head == null || head.next == null) {
      return true;
    }

    ListNode fast = head;
    ListNode slow = head;

    while (fast != null && fast.next != null) {
      fast = fast.next.next;
      slow = slow.next;
    }

    ListNode secondHalf = reverseList(slow);
    ListNode firstHalf = head;

    while (secondHalf != null) {
      if (firstHalf.val != secondHalf.val) {
        return false;
      }

      firstHalf = firstHalf.next;
      secondHalf = secondHalf.next;
    }

    return true;
  }

  private ListNode reverseList(ListNode curr) {
    ListNode prev = null;

    while (curr != null) {
      ListNode next = curr.next;
      curr.next = prev;
      prev = curr;
      curr = next;
    }

    return prev;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

135. [235]Lowest Common Ancestor of a Binary Search Tree.java

```
235
        Lowest Common Ancestor of a Binary Search Tree
        2022-12-27 16:53:14
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode(int x) { val = x; }
 * }
 */

class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode node = dfs(root, p, q);
        return node;
    }

    public TreeNode dfs(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) {
            return root;
        }
        TreeNode left = dfs(root.left, p, q);
        TreeNode right = dfs(root.right, p, q);
        if (left != null && right != null) {
            return root;
        }
        return left == null ? right : left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

136. [236]Lowest Common Ancestor of a Binary Tree.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null) {
      return null;
    }

    if (root == p || root == q) {
      return root;
    }

    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    if (left != null && right != null) {
      return root;
    }

    return left != null ? left : right;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

137. [237]Delete Node in a Linked List.java

```
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode(int x) { val = x; }
 * }
 */
class Solution {
    public void deleteNode(ListNode node) {
        node.val = node.next.val;
        node.next = node.next.next;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

138. `[238]Product of Array Except Self.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] answer = new int[n];
    answer[0] = 1;

    for (int i = 1; i < n; i++) {
      answer[i] = answer[i - 1] * nums[i - 1];
    }

    int postfix = 1;
    for (int i = n - 1; i >= 0; i--) {
      answer[i] *= postfix;
      postfix *= nums[i];
    }

    return answer;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

139. `[239]Sliding Window Maximum.java`

```
239
        Sliding Window Maximum
        2022-11-30 09:57:38

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        if (k == 1) {
            return nums;
        }
        int[] left = new int[n];
        left[0] = nums[0];
        int[] right = new int[n];
        right[n - 1] = nums[n - 1];
        for (int i = 1; i < n; i++) {
            if (i % k == 0) {
                left[i] = nums[i];
            } else {
                left[i] = Math.max(left[i - 1], nums[i]);
            }
            int j = n - i - 1;
            if ((j + 1) % k == 0) {
                right[j] = nums[j];
            } else {
                right[j] = Math.max(right[j + 1], nums[j]);
            }
        }
        int[] arr = new int[n - k + 1];
        for (int i = 0; i < n - k + 1; i++) {
            arr[i] = Math.max(left[i + k - 1], right[i]);
        }
        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
/**
    将链表 切分成大小为 k 的 chunk
    然后 left 记录在 chunk 内从左往右的最大值,
         right 记录 chunk 内从右往左的最大值
    滑动窗口只需要找 right[i] 和 left[i + k - 1]的最大值

            1  3  -1  * -3  5  3  *  6  7
    left    1  3  3     -3  5  5     6  7
    right   3  3  -1     5  5  3     7  7

    TC: O(N)    SC: O(N) 16ms

*/
/*
```

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int N = nums.length;
        int[] res = new int[N - k + 1];
        int[] left = new int[N];
        int[] right = new int[N];
        for (int i = 0; i < N; i += k) {
            int max = Integer.MIN_VALUE;
            int rightBound = Math.min(i + k - 1, N - 1);
            // from left to right -->
            for (int j = i; j <= rightBound; j++) {
                if (nums[j] > max)
                    max = nums[j];
                left[j] = max;
            }
            // from right to left <--
            max = Integer.MIN_VALUE;
            for (int j = rightBound; j >= i; j--) {
                if (nums[j] > max)
                    max = nums[j];
                right[j] = max;
            }
        }
        for (int i = 0; i < N - k + 1; i++) {
            int j = i + k - 1;
            res[i] = Math.max(right[i], left[j]);
        }
        return res;
    }
}

/**
 维护一个 单调递减 的 Monotonic Queue, 尾进头出
 TC: O(n)    SC: O(k)
 36ms
 */
/*
class Solution1 {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int N  = nums.length;
        int[] res = new int[N - k + 1];
        Deque<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < N; i++) {
            // 保证窗口内 从大到小, 如果前面的数小则需要一次弹出, 直到满足要求
            while (!queue.isEmpty() && nums[i] >= nums[queue.peekLast()]) {
                queue.pollLast();
            }
            queue.addLast(i);
            // 判断队首的值是否在 k 的滑动窗口内
            if (queue.peekFirst() <= i - k) {
                queue.poll();
            }
            // 当增长到窗口长度为k时 保存当前窗口中最大值
            if (i + 1 - k >= 0) {
```

```
                res[i + 1 - k] = nums[queue.peekFirst()];
            }
        }
        return res;
    }
}
 */
```

140. [240]Search a 2D Matrix II.java

```
240
        Search a 2DMatrix II
        2022-12-23 15:00:53

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length;
        int n = matrix[0].length;
        int row = 0;
        int col = n - 1;
        while (row < m && col >= 0) {
            if (matrix[row][col] > target) {
                col--;
            } else if (matrix[row][col] < target) {
                row++;
            } else {
                return true;
            }
        }
        return false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

141. `[242]Valid Anagram.java`

```
242
Valid Anagram
        2023-01-01 22:43:23

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isAnagram(String s, String t) {
        int[] arr = new int[26];
        countLetters(s, arr, 1);
        countLetters(t, arr, -1);

        return checkEqual(arr);
    }

    private void countLetters(String str, int[] arr, int i) {
        for (char c : str.toCharArray()) {
            arr[c - 'a'] += i;
        }
    }

    private boolean checkEqual(int[] arr) {
        for (int num : arr) {
            if (num != 0) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

142. `[243]Shortest Word Distance.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int shortestDistance(String[] wordsDict, String word1, String word2) {
        int index = -1;
        int min = Integer.MAX_VALUE;

        for (int i = 0; i < wordsDict.length; i++) {
            if (wordsDict[i].equals(word1) || wordsDict[i].equals(word2)) {
                if (index != -1 && !wordsDict[index].equals(wordsDict[i])) {
                    min = Math.min(min, i - index);
                }

                index = i;
            }
        }

        return min;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

143. `[244]Shortest Word Distance II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class WordDistance {
    Map<String, List<Integer>> map = new HashMap<>();

    public WordDistance(String[] wordsDict) {
        for (int i = 0; i < wordsDict.length; i++) {
            String key = wordsDict[i];

            if (map.containsKey(key)) {
                map.get(key).add(i);
            } else {
                List<Integer> list = new ArrayList<>();
                list.add(i);
                map.put(key, list);
            }
        }
    }

    public int shortest(String word1, String word2) {
        List<Integer> list1 = map.get(word1);
        List<Integer> list2 = map.get(word2);
        int i = 0;
        int j = 0;
        int min = Integer.MAX_VALUE;

        while (i < list1.size() && j < list2.size()) {
            int index1 = list1.get(i);
            int index2 = list2.get(j);

            if (index1 < index2) {
                min = Math.min(min, index2 - index1);
                i++;
            } else {
                min = Math.min(min, index1 - index2);
                j++;
            }
        }

        return min;
    }
}

/**
 * Your WordDistance object will be instantiated and called as such:
 * WordDistance obj = new WordDistance(wordsDict);
 * int param_1 = obj.shortest(word1,word2);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

144. [245]Shortest Word Distance III.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int shortestWordDistance(String[] wordsDict, String word1, String word2) {
        boolean isSame = word1.equals(word2);
        int index = -1;
        int min = Integer.MAX_VALUE;
        for (int i = 0; i < wordsDict.length; i++) {
            if (wordsDict[i].equals(word1) || wordsDict[i].equals(word2)) {
                if (index != -1 && (!wordsDict[index].equals(wordsDict[i]) || isSame)) {
                    min = Math.min(min, i - index);
                }
                index = i;
            }
        }
        return min;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

145. [246]Strobogrammatic Number.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isStrobogrammatic(String num) {
        int[] arr = new int[]{0, 1, -1, -1, -1, -1, 9, -1, 8, 6};
        char[] c = num.toCharArray();

        for (int left = 0, right = c.length - 1; left <= right; left++, right--) {
            if (arr[c[left] - '0'] != c[right] - '0') {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

146. `[247]Strobogrammatic Number II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public char[][] pairs = {{'0', '0'}, {'1', '1'}, {'6', '9'}, {'8', '8'}, {'9', '6'}};

    public List<String> findStrobogrammatic(int n) {
        Deque<String> dq = new ArrayDeque<>();
        int currLen;
        if (n % 2 == 0) {
            currLen = 0;
            dq.add("");
        } else {
            currLen = 1;
            dq.add("0");
            dq.add("1");
            dq.add("8");
        }
        while (currLen < n) {
            currLen += 2;
            for (int i = dq.size(); i > 0; --i) {
                String number = dq.poll();
                for (char[] pair : pairs) {
                    if (currLen != n || pair[0] != '0') {
                        dq.add(pair[0] + number + pair[1]);
                    }
                }
            }
        }
        List<String> list = new ArrayList<>();
        while (!dq.isEmpty()) {
            list.add(dq.poll());
        }
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

147. [248]Strobogrammatic Number III.java

```
/**
<p>Given two strings low and high that represent two integers <code>low</code> and
<code>high</code> where <code>low &lt;= high</code>, return <em>the number of
<strong>strobogrammatic numbers</strong> in the range</em> <code>[low, high]</code>.</p>

<p>A <strong>strobogrammatic number</strong> is a number that looks the same when rotated
<code>180</code> degrees (looked at upside down).</p>

<p> </p>
<p><strong class="example">Example 1:</strong></p>
<pre><strong>Input:</strong> low = "50", high = "100"
<strong>Output:</strong> 3
</pre>
<p><strong class="example">Example 2:</strong></p>
<pre><strong>Input:</strong> low = "0", high = "0"
<strong>Output:</strong> 1
</pre>
<p> </p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>1 &lt;= low.length, high.length &lt;= 15</code></li>
 <li><code>low</code> and <code>high</code> consist of only digits.</li>
 <li><code>low &lt;= high</code></li>
 <li><code>low</code> and <code>high</code> do not contain any leading zeros except for
zero itself.</li>
</ul>

<div><div>Related Topics</div><div><li>Array</li><li>String</li><li>Recursion</li></div>
</div><br><div><li>👍 293</li><li>👎 183</li></div>
*/

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int strobogrammaticInRange(String low, String high) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

148. [250]Count Univalue Subtrees.java

```
250
        Count Univalue Subtrees
        2022-12-02 13:39:38
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    int count = 0;

    public int countUnivalSubtrees(TreeNode root) {
        dfs(root, 0);
        return count;
    }

    private boolean dfs(TreeNode node, int val) {
        if (node == null) {
            return true;
        }
        if (!dfs(node.left, node.val) | !dfs(node.right, node.val)) {
            return false;
        }
        count++;
        return node.val == val;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

149. [252]Meeting Rooms.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        if (intervals == null || intervals.length <= 1) {
            return true;
        }

        Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));

        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] < intervals[i - 1][1]) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

150. [253]Meeting Rooms II.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]);
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.add(intervals[0][1]);

        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] >= pq.peek()) {
                pq.poll();
            }

            pq.add(intervals[i][1]);
        }

        return pq.size();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

151. `[254]Factor Combinations.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> getFactors(int n) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(n, 2, new ArrayList<>(), res);
        return res;
    }

    private void backtrack(int n, int start, List<Integer> current, List<List<Integer>>
res) {
        for (int i = start; i * i <= n; i++) {
            if (n % i == 0) {
                current.add(i);
                current.add(n / i);
                res.add(new ArrayList<>(current));
                current.remove(current.size() - 1);
                backtrack(n / i, i, current, res);
                current.remove(current.size() - 1);
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

152. `[255]Verify Preorder Sequence in Binary Search Tree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean verifyPreorder(int[] preorder) {
        Deque<Integer> deque = new ArrayDeque<>();
        int lower = Integer.MIN_VALUE;

        for (int value : preorder) {
            if (value < lower) {
                return false;
            }

            while (!deque.isEmpty() && value > deque.peek()) {
                lower = deque.pop();
            }

            deque.push(value);
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

153. `[256]Paint House.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minCost(int[][] costs) {
        int n = costs.length;

        for (int i = 1; i < n; i++) {
            costs[i][0] += Math.min(costs[i - 1][1], costs[i - 1][2]);
            costs[i][1] += Math.min(costs[i - 1][0], costs[i - 1][2]);
            costs[i][2] += Math.min(costs[i - 1][0], costs[i - 1][1]);
        }

        return Math.min(costs[n - 1][0], Math.min(costs[n - 1][1], costs[n - 1][2]));
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

154. `[257]Binary Tree Paths.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    List<String> list = new ArrayList<>();

    public List<String> binaryTreePaths(TreeNode root) {
        dfs(root, "");
        return list;
    }

    public void dfs(TreeNode node, String str) {
        if (node == null) {
            return;
        }
        str += Integer.toString(node.val);
        if (node.left == null && node.right == null) {
            list.add(str);
        }
        str += "->";
        dfs(node.left, str);
        dfs(node.right, str);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

155. `[258]Add Digits.java`

```
/**
 * <p>Given an integer <code>num</code>, repeatedly add all its digits until the result
has only one digit, and return it.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> num = 38
 * <strong>Output:</strong> 2
 * <strong>Explanation:</strong> The process is
 * 38 --&gt; 3 + 8 --&gt; 11
 * 11 --&gt; 1 + 1 --&gt; 2
 * Since 2 has only one digit, return it.
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> num = 0
 * <strong>Output:</strong> 0
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>0 &lt;= num &lt;= 2<sup>31</sup> - 1</code></li>
 * </ul>
 *
 * <p> </p>
 * <p><strong>Follow up:</strong> Could you do it without any loop/recursion in
<code>O(1)</code> runtime?</p>
 *
 * <div><div>Related Topics</div><div><li>Math</li><li>Simulation</li><li>Number
Theory</li></div></div><br><div><li>👍 3347</li><li>👎 1770</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int addDigits(int num) {
        while (num > 9) {
            int sum = 0;
            while (num > 0) {
                int temp = num % 10;
                sum += temp;
                num/=10;
            }
            num = sum;
        }
        return num;
```

```
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int addDigits(int num) {
        return num == 0 ? 0 : 1 + (num - 1) % 9;
    }
}
 */
```

156. `[259]3Sum Smaller.java`

```
/**
 * <p>Given an array of <code>n</code> integers <code>nums</code> and an integer 
<code>target</code>, find the number of index triplets <code>i</code>, <code>j</code>,
<code>k</code> with <code>0 &lt;= i &lt; j &lt; k &lt; n</code> that satisfy the
condition <code>nums[i] + nums[j] + nums[k] &lt; target</code>.</p>
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [-2,0,1,3], target = 2
 * <strong>Output:</strong> 2
 * <strong>Explanation:</strong> Because there are two triplets which sums are less than
2:
 * [-2,0,1]
 * [-2,0,3]
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [], target = 0
 * <strong>Output:</strong> 0
 * </pre>
 *
 * <p><strong class="example">Example 3:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [0], target = 0
 * <strong>Output:</strong> 0
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>n == nums.length</code></li>
 * <li><code>0 &lt;= n &lt;= 3500</code></li>
 * <li><code>-100 &lt;= nums[i] &lt;= 100</code></li>
 * <li><code>-100 &lt;= target &lt;= 100</code></li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>Array</li><li>Two Pointers</li><li>Binary
Search</li><li>Sorting</li></div></div><br><div><li>👍 1431</li><li>👎 138</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int threeSumSmaller(int[] nums, int target) {
        Arrays.sort(nums);
        int n = nums.length;
        int count = 0;
```

```
        for (int i = 0; i < n - 2 && nums[i] * 3 < target; i++) {
            int left = i + 1;
            int right = n - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum < target) {
                    count += right - left;
                    left++;
                } else {
                    right--;
                }
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

157. [260]Single Number III.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] singleNumber(int[] nums) {
        int xor = 0;

        for (int num : nums) {
            xor ^= num;
        }

        int bit =xor &-xor;
        int[] result = new int[2];

        for(int num : nums) {
            if((num & bit) == 0) {
                result[0] ^= num;
            }else{
                result[1] ^= num;
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int[] singleNumber(int[] nums) {
        int bitmask = 0;
        for (int num : nums) {
            bitmask ^= num;
        }
        int diff = bitmask & (-bitmask);
        int x = 0;
        for (int num : nums) {
            if ((num & diff) != 0) {
                x ^= num;
            }
        }
        return new int[]{x, bitmask ^ x};
    }
}
 */
```

158. `[261]Graph Valid Tree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean validTree(int n, int[][] edges) {
        if (edges.length != n - 1) {
            return false;
        }

        UnionFind uf = new UnionFind(n);

        for (int[] edge : edges) {
            if (!uf.union(edge[0], edge[1])) {
                return false;
            }
        }

        return true;
    }
}

class UnionFind {
    private int[] parent;
    private int[] rank;

    public UnionFind(int size) {
        parent = new int[size];
        rank = new int[size];

        for (int i = 0; i < size; i++) {
            parent[i] = i;
            rank[i] = 0;
        }
    }

    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }

        return parent[x];
    }

    public boolean union(int i, int j) {
        int x = find(i);
        int y = find(j);

        if (x == y) {
            return false;
        }

        if (rank[x] > rank[y]) {
            parent[y] = x;
```

```
        } else if (rank[x] < rank[y]) {
            parent[x] = y;
        } else {
            parent[y] = x;
            rank[x]++;
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

159. [264]Ugly Number II.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int nthUglyNumber(int n) {
        int[] dp = new int[n];
        dp[0] = 1;
        int i2 = 0;
        int i3 = 0;
        int i5 = 0;
        int next2 = 2;
        int next3 = 3;
        int next5 = 5;

        for (int i = 1; i < n; i++) {
            int nextUgly = Math.min(next2, Math.min(next3, next5));
            dp[i] = nextUgly;

            if (nextUgly == next2) {
                i2++;
                next2 = dp[i2] * 2;
            }

            if (nextUgly == next3) {
                i3++;
                next3 = dp[i3] * 3;
            }

            if (nextUgly == next5) {
                i5++;
                next5 = dp[i5] * 5;
            }
        }

        return dp[n - 1];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

160. `[265]Paint House II.java`

```
265
        Paint House II 2023-02-25 16:47:19

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minCostII(int[][] costs) {
        int m = costs.length;
        int n = costs[0].length;
        int[][] dp = new int[m][n];
        dp[0] = costs[0].clone();
        for (int i = 1; i < m; i++) {
            for (int j = 0; j < n; j++) {
                int min = Integer.MAX_VALUE;
                for (int k = 0; k < n; k++) {
                    if (j != k) {
                        min = Math.min(min, dp[i - 1][k]);
                    }
                }
                dp[i][j] = min + costs[i][j];
            }
        }
        return Arrays.stream(dp[m - 1]).min().getAsInt();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

161. `[266]Palindrome Permutation.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canPermutePalindrome(String s) {
        boolean isSingle = false;
        int[] arr = new int[26];
        for (char c : s.toCharArray()) {
            arr[c - 'a']++;
        }
        for (int i = 0; i < 26; i++) {
            if (arr[i] % 2 != 0) {
                if (isSingle) {
                    return false;
                } else {
                    isSingle = true;
                }
            }
        }
        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

162. [267]Palindrome Permutation II.java

```
/**
<p>Given a string s, return <em>all the palindromic permutations (without duplicates) of it</em>.</p>

<p>You may return the answer in <strong>any order</strong>. If <code>s</code> has no palindromic permutation, return an empty list.</p>

<p> </p>
<p><strong class="example">Example 1:</strong></p>
<pre><strong>Input:</strong> s = "aabb"
<strong>Output:</strong> ["abba","baab"]
</pre>
<p><strong class="example">Example 2:</strong></p>
<pre><strong>Input:</strong> s = "abc"
<strong>Output:</strong> []
</pre>
<p> </p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>1 &lt;= s.length &lt;= 16</code></li>
 <li><code>s</code> consists of only lowercase English letters.</li>
</ul>

<div><div>Related Topics</div><div><li>Hash Table</li><li>String</li>
<li>Backtracking</li></div></div><br><div><li>👍 813</li><li>👎 92</li></div>
*/

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<String> generatePalindromes(String s) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

163. `[268]Missing Number.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int result = (n) * (n + 1) / 2;

        for (int num : nums) {
            result -= num;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

164. `[269]Alien Dictionary.java`

```
269
        Alien Dictionary
        2023-02-16 11:43:41

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String alienOrder(String[] words) {
        List<List<Character>> list = new ArrayList<>();
        final int N = 26;
        int[] inDegree = new int[N];
        for (int i = 0; i < N; i++) {
            list.add(new ArrayList<>());
            inDegree[i] = -1;
        }
        int count = 0;
        for (String word : words) {
            for (char c : word.toCharArray()) {
                if (inDegree[c - 'a'] == -1) {
                    inDegree[c - 'a'] = 0;
                    count++;
                }
            }
        }
        for (int i = 0; i < words.length - 1; i++) {
            String s1 = words[i];
            String s2 = words[i + 1];
            if (s1.length() > s2.length() && s1.startsWith(s2)) {
                return "";
            }
            int n = Math.min(s1.length(), s2.length());
            for (int j = 0; j < n; j++) {
                if (s1.charAt(j) != s2.charAt(j)) {
                    list.get(s1.charAt(j) - 'a').add(s2.charAt(j));
                    inDegree[s2.charAt(j) - 'a']++;
                    break;
                }
            }
        }
        StringBuilder sb = new StringBuilder();
        Deque<Character> dq = new ArrayDeque<>();
        for (char c = 'a'; c <= 'z'; c++) {
            if (inDegree[c - 'a'] == 0) {
                dq.add(c);
            }
        }
        while (!dq.isEmpty()) {
            char c = dq.poll();
            sb.append(c);
            for (char ch : list.get(c - 'a')) {
                inDegree[ch - 'a']--;
                if (inDegree[ch - 'a'] == 0) {
```

```
                    dq.add(ch);
                }
            }
        }
        return sb.length() == count ? sb.toString() : "";
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

165. [270]Closest Binary Search Tree Value.java

```
// leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  public int closestValue(TreeNode root, double target) {
    int result = root.val;

    while (root != null) {
      double diffRoot = Math.abs(target - root.val);
      double diffResult = Math.abs(target - result);

      if (diffRoot < diffResult || (diffRoot == diffResult && root.val < result)) {
        result = root.val;
      }

      root = target < root.val ? root.left : root.right;
    }

    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

166. `[271]Encode and Decode Strings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Codec {
    // Encodes a list of strings to a single string.
    public String encode(List<String> strs) {
        StringBuilder sb = new StringBuilder();
        for (String str : strs) {
            sb.append(str.length()).append("-").append(str);
        }
        return sb.toString();
    }

    // Decodes a single string to a list of strings.
    public List<String> decode(String s) {
        List<String> list = new ArrayList<>();
        int index = 0;
        while (index < s.length()) {
            int dashPos = s.indexOf('-', index);
            String str = s.substring(index, dashPos);
            int start = index + str.length() + 1;
            int end = start + Integer.valueOf(str);
            list.add(s.substring(start, end));
            index = end;
        }
        return list;
    }
}

// Your Codec object will be instantiated and called as such:
// Codec codec = new Codec();
// codec.decode(codec.encode(strs));
//leetcode submit region end(Prohibit modification and deletion)
```

167. [272]Closest Binary Search Tree Value II.java

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<Integer> closestKValues(TreeNode root, double target, int k) {
        Deque<Integer> deque = new ArrayDeque<>();
        inOrderTraversal(root, target, k, deque);

        return new ArrayList<>(deque);
    }

    private void inOrderTraversal(TreeNode node, double target, int k, Deque<Integer>
deque) {
        if (node == null) {
            return;
        }

        inOrderTraversal(node.left, target, k, deque);

        if (deque.size() < k) {
            deque.addLast(node.val);
        } else {
            if (Math.abs(deque.peekFirst() - target) > Math.abs(node.val - target)) {
                deque.pollFirst();
                deque.addLast(node.val);
            } else {
                return;
            }
        }

        inOrderTraversal(node.right, target, k, deque);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

168. [273]Integer to English Words.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private final String[] LESS_THAN_20 = {"", "One", "Two", "Three", "Four", "Five",
"Six", "Seven", "Eight", "Nine", "Ten",
            "Eleven", "Twelve", "Thirteen", "Fourteen", "Fifteen", "Sixteen",
"Seventeen", "Eighteen", "Nineteen"};
    private final String[] TENS = {"", "", "Twenty", "Thirty", "Forty", "Fifty", "Sixty",
"Seventy", "Eighty", "Ninety"};
    private final String[] THOUSANDS = {"", "Thousand", "Million", "Billion"};

    public String numberToWords(int num) {
        if (num == 0) {
            return "Zero";
        }

        int i = 0;
        StringBuilder words = new StringBuilder();

        while (num > 0) {
            if (num % 1000 != 0) {
                words.insert(0, helper(num % 1000).append(THOUSANDS[i]).append(" "));
            }

            num /= 1000;
            i++;
        }

        return words.toString().trim();
    }

    private StringBuilder helper(int num) {
        StringBuilder sb = new StringBuilder();

        if (num == 0) {
            return sb;
        } else if (num < 20) {
            return sb.append(LESS_THAN_20[num]).append(" ");
        } else if (num < 100) {
            return sb.append(TENS[num / 10]).append(" ").append(helper(num % 10));
        } else {
            return sb.append(LESS_THAN_20[num / 100]).append(" Hundred
").append(helper(num % 100));
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

169. `[274]H-Index.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int hIndex(int[] citations) {
        int n = citations.length;
        int[] arr = new int[n + 1];
        for (int c : citations) {
            arr[Math.min(c, n)]++;
        }
        int count = 0;
        for (int i = n; i >= 0; i--) {
            count += arr[i];
            if (count >= i) {
                return i;
            }
        }
        return 0;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int hIndex(int[] citations) {
        Arrays.sort(citations);
        int count = 0;
        for (int i = citations.length - 1; i >= 0; i--) {
            if (citations[i] >= count + 1) {
                count++;
            } else {
                return count;
            }
        }
        return count;
    }
}
 */
```

170. `[275]H-Index II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int hIndex(int[] citations) {
        int n = citations.length;
        int left = 0;
        int right = n - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (citations[mid] == n - mid) {
                return citations[mid];
            } else if (citations[mid] < n - mid) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return n - left;
    }
}

//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int hIndex(int[] citations) {
        int count = 0;
        for (int i = citations.length - 1; i >= 0; i--) {
            if (citations[i] >= count + 1) {
                count++;
            }else{
                return count;
            }
        }
        return count;
    }
}
 */
```

171. `[276]Paint Fence.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    public int numWays(int n, int k) {
        if (n == 1) {
            return k;
        }

        int same = k;
        int diff = k * (k - 1);

        for (int i = 2; i < n; i++) {
            int temp = diff;
            diff = (same + diff) * (k - 1);
            same = temp;
        }

        return same + diff;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

172. `[277]Find the Celebrity.java`

```
277
        Find the Celebrity
        2023-01-14 23:14:24
//leetcode submit region begin(Prohibit modification and deletion)
/* The knows API is defined in the parent class Relation.
      boolean knows(int a, int b); */

public class Solution extends Relation {
    public int findCelebrity(int n) {
        int candidate = 0;
        for (int i = 1; i < n; i++) {
            if (knows(candidate, i)) {
                candidate = i;
            }
        }
        for (int i = 0; i < candidate; i++) {
            if (knows(candidate, i)) {
                return -1;
            }
        }
        for (int i = 0; i < n; i++) {
            if (i != candidate && !knows(i, candidate)) {
                return -1;
            }
        }
        return candidate;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

173. `[278]First Bad Version.java`

```
278
        First Bad Version
        2022-12-29 00:37:10
//leetcode submit region begin(Prohibit modification and deletion)
/* The isBadVersion API is defined in the parent class VersionControl.
      boolean isBadVersion(int version); */

public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int left = 1;
        int right = n;
        while (left <= right) {
            int mid = (left + right) >>> 1;
            if (!isBadVersion(mid)) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

174. `[279]Perfect Squares.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numSquares(int n) {
        while (n % 4 == 0) {
            n /= 4;
        }

        if (n % 8 == 7) {
            return 4;
        }

        if (this.isSquare(n)) {
            return 1;
        }

        for (int i = 1; i * i <= n; ++i) {
            if (this.isSquare(n - i * i)) {
                return 2;
            }
        }

        return 3;
    }

    private boolean isSquare(int n) {
        int sq = (int) Math.sqrt(n);
        return n == sq * sq;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int numSquares(int n) {
        // four-square and three-square theorems.
        while (n % 4 == 0) {
            n /= 4;
        }

        if (n % 8 == 7) {
            return 4;
        }

        if (this.isSquare(n)) {
            return 1;
        }
        // enumeration to check if the number can be decomposed into sum of two squares.
        for (int i = 1; i * i <= n; ++i) {
            if (this.isSquare(n - i * i)) {
                return 2;
            }
```

```
        }
        // bottom case of three-square theorem.
        return 3;
    }

    private boolean isSquare(int n) {
        int sq = (int) Math.sqrt(n);
        return n == sq * sq;
    }
}

class Solution {
    public int numSquares(int n) {
        int dp[] = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j * j <= i; j++) {
                dp[i] = Math.min(dp[i], dp[i - j * j] + 1);
            }
        }

        return dp[n];
    }
}
 */
```

175. `[280]Wiggle Sort.java`

```
280
        Wiggle Sort
        2023-02-01 22:41:03

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public void wiggleSort(int[] nums) {
        int n = nums.length;
        int[] arr = nums.clone();
        Arrays.sort(arr);
        int index = n - 1;
        for (int i = 1; i < n; i += 2) {
            nums[i] = arr[index];
            index--;
        }
        for (int i = 0; i < n; i += 2) {
            nums[i] = arr[index];
            index--;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

176. `[282]Expression Add Operators.java`

```
/**
<p>Given a string <code>num</code> that contains only digits and an integer
<code>target</code>, return <em><strong>all possibilities</strong> to insert the binary
operators </em><code>'+'</code><em>, </em><code>'-'</code><em>, and/or </em>
<code>'*'</code><em> between the digits of </em><code>num</code><em> so that the
resultant expression evaluates to the </em><code>target</code><em> value</em>.</p>

<p>Note that operands in the returned expressions <strong>should not</strong> contain
leading zeros.</p>

<p> </p>
<p><strong class="example">Example 1:</strong></p>

<pre>
<strong>Input:</strong> num = "123", target = 6
<strong>Output:</strong> ["1*2*3","1+2+3"]
<strong>Explanation:</strong> Both "1*2*3" and "1+2+3" evaluate to 6.
</pre>

<p><strong class="example">Example 2:</strong></p>

<pre>
<strong>Input:</strong> num = "232", target = 8
<strong>Output:</strong> ["2*3+2","2+3*2"]
<strong>Explanation:</strong> Both "2*3+2" and "2+3*2" evaluate to 8.
</pre>

<p><strong class="example">Example 3:</strong></p>

<pre>
<strong>Input:</strong> num = "3456237490", target = 9191
<strong>Output:</strong> []
<strong>Explanation:</strong> There are no expressions that can be created from
"3456237490" to evaluate to 9191.
</pre>

<p> </p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>1 &lt;= num.length &lt;= 10</code></li>
 <li><code>num</code> consists of only digits.</li>
 <li><code>-2<sup>31</sup> &lt;= target &lt;= 2<sup>31</sup> - 1</code></li>
</ul>

<div><div>Related Topics</div><div><li>Math</li><li>String</li><li>Backtracking</li>
</div></div><br><div><li>👍 3068</li><li>👎 542</li></div>
*/

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
```

```
    public List<String> addOperators(String num, int target) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

177. [283]Move Zeroes.java

```
283
        Move Zeroes
        2022-12-30 22:39:14

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public void moveZeroes(int[] nums) {
        int n = nums.length;
        int index = 0;
        for (int i = 0; i < n; i++) {
            if (nums[i] != 0) {
                nums[index] = nums[i];
                index++;
            }
        }
        for (int i = index; i < n; i++) {
            nums[i] = 0;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
//int[] arr = nums.clone();
//int[] arr = Arrays.copyOf(nums, n);
//int[] arr = Arrays.copyOfRange(nums, 0, n);
```

178. `[284]Peeking Iterator.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
// Java Iterator interface reference:
// https://docs.oracle.com/javase/8/docs/api/java/util/Iterator.html

class PeekingIterator implements Iterator<Integer> {
    Iterator<Integer> iter;
    Integer next = null;

    public PeekingIterator(Iterator<Integer> iterator) {
        // initialize any member here.
        iter = iterator;
        if (iter.hasNext()) {
            next = iter.next();
        }
    }

    // Returns the next element in the iteration without advancing the iterator.
    public Integer peek() {
        return next;
    }

    // hasNext() and next() should behave the same as in the Iterator interface.
    // Override them if needed.
    @Override
    public Integer next() {
        Integer curr = next;
        if (iter.hasNext()) {
            next = iter.next();
        } else {
            next = null;
        }
        return curr;
    }

    @Override
    public boolean hasNext() {
        return next != null;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

179. [286]Walls and Gates.java

```
286
        Walls and Gates
        2022-12-20 10:57:04

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private static final int EMPTY = Integer.MAX_VALUE;
    private static final int GATE = 0;
    private static final int[][] DIRECTIONS = new int[][]{{1, 0}, {-1, 0}, {0, 1}, {0,
-1}};

    public void wallsAndGates(int[][] rooms) {
        int m = rooms.length;
        int n = rooms[0].length;
        Queue<int[]> queue = new LinkedList<>();
        for (int row = 0; row < m; row++) {
            for (int col = 0; col < n; col++) {
                if (rooms[row][col] == GATE) {
                    queue.add(new int[]{row, col});
                }
            }
        }
        while (!queue.isEmpty()) {
            int[] point = queue.poll();
            int row = point[0];
            int col = point[1];
            for (int[] direction : DIRECTIONS) {
                int nextRow = row + direction[0];
                int nextCol = col + direction[1];
                if (nextRow < 0 || nextCol < 0 || nextRow >= m || nextCol >= n ||
rooms[nextRow][nextCol] != EMPTY) {
                    continue;
                }
                rooms[nextRow][nextCol] = rooms[row][col] + 1;
                queue.add(new int[]{nextRow, nextCol});
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

180. `[287]Find the Duplicate Number.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findDuplicate(int[] nums) {
        boolean[] isDupicated = new boolean[nums.length];

        for (int num : nums) {
            if (isDupicated[num]) {
                return num;
            }

            isDupicated[num] = true;
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

181. `[288]Unique Word Abbreviation.java`

```
288
        Unique Word Abbreviation
        2023-01-30 09:48:08

//leetcode submit region begin(Prohibit modification and deletion)
class ValidWordAbbr {
    Map<String, Set<String>> map;

    public ValidWordAbbr(String[] dictionary) {
        map = new HashMap<>();
        for (String word : dictionary) {
            String str = abbr(word);
            map.putIfAbsent(str, new HashSet<>());
            map.get(str).add(word);
        }
    }

    public boolean isUnique(String word) {
        String str = abbr(word);
        return !map.containsKey(str) || (map.get(str).contains(word) &&
map.get(str).size() == 1);
    }

    String abbr(String word) {
        String str = word;
        if (word.length() > 2) {
            StringBuilder sb = new StringBuilder();
            sb.append(word.charAt(0)).append(word.length() -
2).append(word.charAt(word.length() - 1));
            str = sb.toString();
        }
        return str;
    }
}

/**
 * Your ValidWordAbbr object will be instantiated and called as such:
 * ValidWordAbbr obj = new ValidWordAbbr(dictionary);
 * boolean param_1 = obj.isUnique(word);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

182. `[290]Word Pattern.java`

```
290
        Word Pattern
        2023-01-01 08:56:44

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean wordPattern(String pattern, String s) {
        Map<Character, String> map = new HashMap<>();
        String[] words = s.split(" ");
        if (words.length != pattern.length()) {
            return false;
        }
        for (int i = 0; i < pattern.length(); i++) {
            char c = pattern.charAt(i);
            if (map.containsKey(c)) {
                if (!map.get(c).equals(words[i])) {
                    return false;
                }
            } else {
                if (map.values().contains(words[i])) {
                    return false;
                }
                map.put(c, words[i]);
            }
        }
        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

183. `[291]Word Pattern II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean wordPatternMatch(String pattern, String s) {
        return backtrack(pattern, 0, s, 0, new HashMap<>(), new HashSet<>());
    }

    private boolean backtrack(String pattern, int pIndex, String s, int sIndex,
Map<Character, String> map, Set<String> set) {
        if (pIndex == pattern.length() && sIndex == s.length()) {
            return true;
        }

        if (pIndex == pattern.length() || sIndex == s.length()) {
            return false;
        }

        char c = pattern.charAt(pIndex);

        if (map.containsKey(c)) {
            String mappedStr = map.get(c);

            if (!s.startsWith(mappedStr, sIndex)) {
                return false;
            }

            return backtrack(pattern, pIndex + 1, s, sIndex + mappedStr.length(), map,
set);
        }

        for (int i = sIndex; i < s.length(); i++) {
            String candidate = s.substring(sIndex, i + 1);

            if (set.contains(candidate)) {
                continue;
            }

            map.put(c, candidate);
            set.add(candidate);

            if (backtrack(pattern, pIndex + 1, s, sIndex + candidate.length(), map, set))
{
                return true;
            }

            map.remove(c);
            set.remove(candidate);
        }

        return false;
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

184. [292]Nim Game.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canWinNim(int n) {
        return n % 4 != 0;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

185. [293]Flip Game.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<String> generatePossibleNextMoves(String currentState) {
        List<String> list = new ArrayList<>();
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < currentState.length() - 1; i++) {
            if (currentState.charAt(i) == '+' && currentState.charAt(i + 1) == '+') {
                sb.append(currentState.substring(0, i))
                        .append("--")
                        .append(currentState.substring(i + 2));
                list.add(sb.toString());
                sb.setLength(0);
            }
        }

        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

186. [1000]Minimum Cost to Merge Stones.java

```
/*
還不會
 */
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int mergeStones(int[] stones, int k) {
        int n = stones.length;
        if ((n - 1) % (k - 1) > 0) {
            return -1;
        }
        int[] prefix = new int[n + 1];
        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] + stones[i];
        }
        int[][] dp = new int[n][n];
        for (int m = k; m <= n; m++) {
            for (int i = 0; i + m <= n; i++) {
                int j = i + m - 1;
                dp[i][j] = Integer.MAX_VALUE;
                for (int mid = i; mid < j; mid += k - 1) {
                    dp[i][j] = Math.min(dp[i][j], dp[i][mid] + dp[mid + 1][j]);
                }
                if ((j - i) % (k - 1) == 0) {
                    dp[i][j] += prefix[j + 1] - prefix[i];
                }
            }
        }
        return dp[0][n - 1];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

187. `[1002]Find Common Characters.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<String> commonChars(String[] words) {
        int[] minFreq = new int[26];
        Arrays.fill(minFreq, Integer.MAX_VALUE);

        for (String word : words) {
            int[] charCount = count(word);

            for (int i = 0; i < 26; i++) {
                minFreq[i] = Math.min(minFreq[i], charCount[i]);
            }
        }

        List<String> result = new ArrayList<>();

        for (int i = 0; i < 26; i++) {
            while (minFreq[i] > 0) {
                result.add(String.valueOf((char) (i + 'a')));
                minFreq[i]--;
            }
        }

        return result;
    }

    private int[] count(String word) {
        int[] charCount = new int[26];

        for (char c : word.toCharArray()) {
            charCount[c - 'a']++;
        }

        return charCount;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

188. [1004]Max Consecutive Ones III.java

```
1004
        Max Consecutive Ones III
        2022-11-24 14:19:35

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestOnes(int[] nums, int k) {
        int zeros = 0;
        int res = 0;
        for (int left = 0, right = 0; right < nums.length; right++) {
            if (nums[right] == 0) {
                zeros++;
            }
            while (zeros > k) {
                if (nums[left] == 0) {
                    zeros--;
                }
                left++;
            }
            res = Math.max(res, right - left + 1);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

189. [1009]Complement of Base 10 Integer.java

```

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int bitwiseComplement(int n) {
        if (n == 0) {
            return 1;
        }

        int bitmast = (Integer.highestOneBit(n) << 1) - 1;
        return n ^ bitmast;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

190. [1010]Pairs of Songs With Total Durations Divisible by 60.java

```
1010
        Pairs of Songs With Total Durations Divisible by 60
        2023-03-05 22:39:07

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numPairsDivisibleBy60(int[] time) {
        int[] arr = new int[60];
        int count = 0;
        for (int item : time) {
            int num = item % 60;
            if (num == 0) {
                count += arr[0];
            } else {
                count += arr[60 - num];
            }
            arr[num]++;
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

191. [1011]Capacity To Ship Packages Within D Days.java

```
1011
        Capacity To Ship Packages Within D Days
        2023-02-22 09:35:20

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int left = 0;
        int right = 0;
        for (int num : weights) {
            left = Math.max(num, left);
            right += num;
        }
        while (left <= right) {
            int mid = (left + right) >>> 1;
            if (isCheck(weights, mid, days)) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }

    private boolean isCheck(int[] weights, int mid, int days) {
        int count = 1;
        int carry = 0;
        for (int num : weights) {
            carry += num;
            if (carry > mid) {
                count++;
                if (count > days) {
                    return true;
                }
                carry = num;
            }
        }
        return false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

192. [1014]Best Sightseeing Pair.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxScoreSightseeingPair(int[] values) {
        int result = 0;
        int curr = 0;

        for (int value : values) {
            result = Math.max(result, curr + value);
            curr = Math.max(curr, value) - 1;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

193. [1017]Convert to Base -2.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String baseNeg2(int n) {
        StringBuilder sb = new StringBuilder();
        while (n != 0) {
            sb.append(n & 1);
            n = -(n >> 1);
        }
        return sb.length() > 0 ? sb.reverse().toString() : "0";
    }
}

//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public String base2(int n) {
        StringBuilder sb = new StringBuilder();
        while (n != 0) {
            sb.append(n & 1);
            n = (n >> 1);
        }
        return sb.length() > 0 ? sb.reverse().toString() : "0";
    }
}
 */
```

194. `[1018]Binary Prefix Divisible By 5.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Boolean> prefixesDivBy5(int[] nums) {
        List<Boolean> list = new ArrayList<>();
        int n = 0;
        for (int num : nums) {
            n = (n * 2 + num) % 5;
            list.add(n == 0);
        }
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
- 2ms
class Solution {
    public List<Boolean> prefixesDivBy5(int[] nums) {
        int n = nums.length;
        Boolean[] arr = new Boolean[n];
        int index = 0;
        int temp = 0;
        for (int num : nums) {
            temp = (temp * 2 + num) % 5;
            arr[index++] = (temp == 0);
        }
        return Arrays.asList(arr);
    }
}
 */
```

195. `[1020]Number of Enclaves.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[][] grid;
    int m;
    int n;

    public int numEnclaves(int[][] grid) {
        this.grid = grid;
        m = grid.length;
        n = grid[0].length;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if ((i == 0 || j == 0 || i == m - 1 || j == n - 1) && grid[i][j] == 1) {
                    dfs(i, j);
                }
            }
        }
        int count = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                count += grid[i][j];
            }
        }
        return count;
    }

    public void dfs(int x, int y) {
        if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] == 0) {
            return;
        }
        grid[x][y] = 0;
        dfs(x + 1, y);
        dfs(x - 1, y);
        dfs(x, y + 1);
        dfs(x, y - 1);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

196. [1021]Remove Outermost Parentheses.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String removeOuterParentheses(String s) {
        StringBuilder sb = new StringBuilder();
        int opened = 0;

        for (char c : s.toCharArray()) {
            if (c == '(' && opened++ > 0) {
                sb.append(c);
            }

            if (c == ')' && opened-- > 1) {
                sb.append(c);
            }
        }

        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

197. [1026]Maximum Difference Between Node and Ancestor.java

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
    public int maxAncestorDiff(TreeNode root) {
        return dfs(root, root.val, root.val);
    }

    private int dfs(TreeNode node, int max, int min) {
        if (node == null) {
            return max - min;
        }

        int currMax = Math.max(max, node.val);
        int currMin = Math.min(min, node.val);
        int left = dfs(node.left, currMax, currMin);
        int right = dfs(node.right, currMax, currMin);
        return Math.max(left, right);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

198. `[1027]Longest Arithmetic Subsequence.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestArithSeqLength(int[] nums) {
        int n = nums.length;
        Map<Integer, Integer>[] dp = new HashMap[n];
        int res = 0;
        for (int i = 0; i < n; i++) {
            dp[i] = new HashMap<>();
            for (int j = 0; j < i; j++) {
                int diff = nums[i] - nums[j];
                dp[i].put(diff, dp[j].getOrDefault(diff, 1) + 1);
                res = Math.max(res, dp[i].get(diff));
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
- 52ms
class Solution {
    public int longestArithSeqLength(int[] nums) {
        int max = Integer.MIN_VALUE;
        int min = Integer.MAX_VALUE;
        for (int num : nums) {
            max = Math.max(max, num);
            min = Math.min(min, num);
        }
        int diff = max - min;
        int n = nums.length;
        int[][] dp = new int[n][diff * 2 + 1];
        int res = 2;
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                int temp = nums[i] - nums[j] + diff;
                dp[i][temp] = dp[j][temp] == 0 ? 2 : dp[j][temp] + 1;
                res = Math.max(res, dp[i][temp]);
            }
        }
        return res;
    }
}

- 621ms
class Solution {
    public int longestArithSeqLength(int[] nums) {
        int n = nums.length;
        Map<Integer, Integer>[] dp = new HashMap[n];
        int res = 0;
        for (int i = 0; i < n; i++) {
```

```
            dp[i] = new HashMap<>();
            for (int j = 0; j < i; j++) {
                int diff = nums[i] - nums[j];
                dp[i].put(diff, dp[j].getOrDefault(diff, 1) + 1);
                res = Math.max(res, dp[i].get(diff));
            }
        }
        return res;
    }
}
 */
```

199. `[1028]Recover a Tree From Preorder Traversal.java`

```
// leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  public TreeNode recoverFromPreorder(String traversal) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    int i = 0;
    int n = traversal.length();
    while (i < n) {
      int level = 0;
      while (i < n && traversal.charAt(i) == '-') {
        level++;
        i++;
      }

      int start = i;

      while (i < n && Character.isDigit(traversal.charAt(i))) {
        i++;
      }

      int value = Integer.parseInt(traversal.substring(start, i));
      TreeNode node = new TreeNode(value);

      while (stack.size() > level) {
        stack.pop();
      }

      if (!stack.isEmpty()) {
        TreeNode parent = stack.peek();

        if (parent.left == null) {
          parent.left = node;
        } else {
          parent.right = node;
        }
      }

      stack.push(node);
    }

    while (stack.size() > 1) {
      stack.pop();
    }

    return stack.peek();
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

200. [1035]Uncrossed Lines.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxUncrossedLines(int[] nums1, int[] nums2) {
        int m = nums1.length;
        int n = nums2.length;
        int[][] dp = new int[m + 1][n + 1];
        for (int i = m - 1; i >= 0; i--) {
            for (int j = n - 1; j >= 0; j--) {
                if (nums1[i] == nums2[j]) {
                    dp[i][j] = 1 + dp[i + 1][j + 1];
                } else {
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j + 1]);
                }
            }
        }
        return dp[0][0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

201. [1038]Binary Search Tree to Greater Sum Tree.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int sum = 0;

    public TreeNode bstToGst(TreeNode root) {
        if (root != null) {
            bstToGst(root.right);
            sum += root.val;
            root.val = sum;
            bstToGst(root.left);
        }

        return root;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

202. `[1041]Robot Bounded In Circle.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isRobotBounded(String instructions) {
        int[][] dirs = new int[][]{{0, 1}, {1, 0}, {0, -1}, {-1, 0}};   //  order matters
        int x = 0;
        int y = 0;
        int index = 0;
        for (char c : instructions.toCharArray()) {
            if (c == 'L') {
                index = (index + 3) % 4;
            } else if (c == 'R') {
                index = (index + 1) % 4;
            } else {
                x += dirs[index][0];
                y += dirs[index][1];
            }
        }
        return (x == 0 && y == 0) || (index != 0);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

203. `[1043]Partition Array for Maximum Sum.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxSumAfterPartitioning(int[] arr, int k) {
        int n = arr.length;
        int[] dp = new int[n + 1];

        for (int i = 1; i <= n; i++) {
            dp[i] = getMaxSum(arr, k, dp, i);
        }

        return dp[n];
    }

    private int getMaxSum(int[] arr, int k, int[] dp, int i) {
        int max = 0;
        int maxSum = 0;

        for (int j = 1; j <= k && i - j >= 0; j++) {
            max = Math.max(max, arr[i - j]);
            maxSum = Math.max(maxSum, dp[i - j] + max * j);
        }

        return maxSum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

204. `[1046]Last Stone Weight.java`

```
1046
        Last Stone Weight
        2022-12-09 09:52:02

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> b - a);
        for (int stone : stones) {
            pq.add(stone);
        }
        while (pq.size() > 1) {
            int first = pq.poll();
            int second = pq.poll();
            if (first == second) {
                continue;
            }
            pq.add(first - second);
        }
        return pq.isEmpty() ? 0 : pq.poll();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

205. `[1048]Longest String Chain.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestStrChain(String[] words) {
        Map<String, Integer> dp = new HashMap<>();
        int maxLength = 1;

        Arrays.sort(words, (a, b) -> a.length() - b.length());

        for (String word : words) {
            int currLength = 1;

            for (int i = 0; i < word.length(); i++) {
                StringBuilder sb = new StringBuilder(word);
                sb.deleteCharAt(i);
                String newWord = sb.toString();
                currLength = Math.max(currLength, dp.getOrDefault(newWord, 0) + 1);
            }

            dp.put(word, currLength);
            maxLength = Math.max(maxLength, currLength);
        }

        return maxLength;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    Map<String, Integer> memo;
    Set<String> wordSet;

    public int longestStrChain(String[] words) {
        memo = new HashMap<>();
        wordSet = new HashSet<>();
        int result = 0;

        Collections.addAll(wordSet, words);

        for (String word : words) {
            result = Math.max(result, dfs(word));
        }

        return result;
    }

    private int dfs(String word) {
        if (memo.containsKey(word)) {
            return memo.get(word);
        }
```

```
        int maxLength = 1;
        StringBuilder sb = new StringBuilder(word);

        for (int i = 0; i < word.length(); i++) {
            sb.deleteCharAt(i);
            String newWord = sb.toString();

            if (wordSet.contains(newWord)) {
                int currLength = 1 + dfs(newWord);
                maxLength = Math.max(maxLength, currLength);
            }

            sb.insert(i, word.charAt(i));
        }

        memo.put(word, maxLength);
        return maxLength;
    }
}

class Solution {
    public int longestStrChain(String[] words) {
        Map<String, Integer> dp = new HashMap<>();
        int maxLength = 1;

        Arrays.sort(words, (a, b) -> a.length() - b.length());

        for (String word : words) {
            int currLength = 1;

            for (int i = 0; i < word.length(); i++) {
                StringBuilder sb = new StringBuilder(word);
                sb.deleteCharAt(i);
                String newWord = sb.toString();
                int prevLength = dp.getOrDefault(newWord, 0);
                currLength = Math.max(currLength, prevLength + 1);
            }

            dp.put(word, currLength);
            maxLength = Math.max(maxLength, currLength);
        }

        return maxLength;
    }
}
 */
```

206. `[1051]Height Checker.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int heightChecker(int[] heights) {
        int[] arr = heights.clone();
        int count = 0;
        int n = arr.length;
        Arrays.parallelSort(arr);

        for (int i = 0; i < n; i++) {
            if (arr[i] != heights[i]) {
                count++;
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

207. `[1052]Grumpy Bookstore Owner.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxSatisfied(int[] customers, int[] grumpy, int minutes) {
        int n = customers.length;
        int baseSatisfied = IntStream.range(0, n)
                .filter(i -> grumpy[i] == 0)
                .map(i -> customers[i])
                .sum();
        int additionalSatisfied = 0;
        int maxAdditionalSatisfied = 0;

        for (int i = 0; i < n; i++) {
            if (grumpy[i] == 1) {
                additionalSatisfied += customers[i];
            }

            if (i >= minutes && grumpy[i - minutes] == 1) {
                additionalSatisfied -= customers[i - minutes];
            }

            maxAdditionalSatisfied = Math.max(maxAdditionalSatisfied,
additionalSatisfied);
        }

        return baseSatisfied + maxAdditionalSatisfied;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

208. [1055]Shortest Way to Form String.java

```
1055
        Shortest Way to Form String
        2023-03-08 11:55:39

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int shortestWay(String source, String target) {
        char[]chTarget = target.toCharArray();
        int count = 1;
        String str = source;
        int i = 0;
        while (i < chTarget.length) {
            char curr = chTarget[i];
            int index = str.indexOf(curr);
            if (str.equals(source) && index == -1) {
                return -1;
            }
            if (index != -1) {
                str = str.substring(index + 1);
                i++;
            } else {
                count++;
                str = source;
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

209. `[1056]Confusing Number.java`

```
1056
        Confusing Number
        2023-01-02 09:18:05

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean confusingNumber(int n) {
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, 0);
        map.put(1, 1);
        map.put(6, 9);
        map.put(8, 8);
        map.put(9, 6);
        int num = n;
        int revNum = 0;
        while (num > 0) {
            int key = num % 10;
            if (!map.containsKey(key)) {
                return false;
            }
            revNum = revNum * 10 + map.get(key);
            num /= 10;
        }
        return revNum != n;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

210. `[1057]Campus Bikes.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] assignBikes(int[][] workers, int[][] bikes) {
        int n = workers.length;
        int m = bikes.length;
        int[] result = new int[n];
        boolean[] workerAssigned = new boolean[n];
        boolean[] bikeAssigned = new boolean[m];

        List<int[]>[] list = new List[2001];

        for (int i = 0; i < 2001; i++) {
            list[i] = new ArrayList<>();
        }

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                int dist = Math.abs(workers[i][0] - bikes[j][0]) + Math.abs(workers[i][1]
- bikes[j][1]);
                list[dist].add(new int[]{i, j});
            }
        }

        for (List<int[]> pairs : list) {
            for (int[] pair : pairs) {
                int workerIndex = pair[0];
                int bikeIndex = pair[1];

                if (!workerAssigned[workerIndex] && !bikeAssigned[bikeIndex]) {
                    result[workerIndex] = bikeIndex;
                    workerAssigned[workerIndex] = true;
                    bikeAssigned[bikeIndex] = true;
                }
            }
        }

        return result;
    }
}


//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int[] assignBikes(int[][] workers, int[][] bikes) {
        int w = workers.length;
        int b = bikes.length;
        PriorityQueue<int[]> pq = new PriorityQueue<>((o1, o2) -> {
            if (o1[0] != o2[0]) {
                return o1[0] - o2[0];
```

```
            }

            if (o1[1] != o2[1]) {
                return o1[1] - o2[1];
            }

            return o1[2] - o2[2];
        });

        for (int i = 0; i < w; i++) {
            int[] worker = workers[i];

            for (int j = 0; j < b; j++) {
                int[] bike = bikes[j];
                int dist = Math.abs(worker[0] - bike[0]) + Math.abs(worker[1] - bike[1]);
                pq.offer(new int[]{dist, i, j});
            }
        }

        int[] result = new int[w];
        boolean[] workerAssigned = new boolean[w];
        boolean[] bikeAssigned = new boolean[b];

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();

            if (!workerAssigned[curr[1]] && !bikeAssigned[curr[2]]) {
                result[curr[1]] = curr[2];
                workerAssigned[curr[1]] = true;
                bikeAssigned[curr[2]] = true;
            }
        }

        return result;
    }
}
 */
```

211. [1059]All Paths from Source Lead to Destination.java

```
1059
        All Paths from Source Lead to Destination
        2023-02-04 11:50:07

//leetcode submit region begin(Prohibit modification and deletion)
/*
還不會
 */
class Solution {
    enum Color {GRAY, BLACK}

    public boolean leadsToDestination(int n, int[][] edges, int source, int destination)
{
        List<Integer>[] graph = buildDigraph(n, edges);
        return leadsToDest(graph, source, destination, new Color[n]);
    }

    private boolean leadsToDest(List<Integer>[] graph, int node, int dest, Color[]
states) {
        if (states[node] != null) {
            return states[node] == Color.BLACK;
        }
        if (graph[node].isEmpty()) {
            return node == dest;
        }
        states[node] = Color.GRAY;
        for (int next : graph[node]) {
            if (!leadsToDest(graph, next, dest, states)) {
                return false;
            }
        }
        states[node] = Color.BLACK;
        return true;
    }

    private List<Integer>[] buildDigraph(int n, int[][] edges) {
        List<Integer>[] graph = new List[n];
        for (int i = 0; i < n; i++) {
            graph[i] = new ArrayList<>();
        }
        for (int[] edge : edges) {
            graph[edge[0]].add(edge[1]);
        }
        return graph;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

212. `[1060]Missing Element in Sorted Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int missingElement(int[] nums, int k) {
        int n = nums.length;
        int left = 0;
        int right = n - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (nums[mid] - nums[0] - mid < k) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return nums[0] + k + left - 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

213. `[1061]Lexicographically Smallest Equivalent`

String.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public String smallestEquivalentString(String s1, String s2, String baseStr) {
    char[] c1 = s1.toCharArray();
    char[] c2 = s2.toCharArray();
    char[] cb = baseStr.toCharArray();
    UnionFind uf = new UnionFind(26);

    for (int i = 0; i < c1.length; i++) {
      uf.union(c1[i] - 'a', c2[i] - 'a');
    }

    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < cb.length; i++) {
      int p = uf.find(cb[i] - 'a');
      sb.append(uf.getChar(p));
    }

    return sb.toString();
  }
}

class UnionFind {
  private final int[] parent;

  public UnionFind(int n) {
    parent = new int[n];
    for (int i = 0; i < n; i++) {
      parent[i] = i;
    }
  }

  public int find(int x) {
    if (parent[x] != x) {
      parent[x] = find(parent[x]);
    }
    return parent[x];
  }

  public void union(int x, int y) {
    int rootX = find(x);
    int rootY = find(y);
    if (rootX == rootY) {
      return;
    }
    if (rootX < rootY) {
      parent[rootY] = rootX;
    } else {
      parent[rootX] = rootY;
    }
  }
```

```
  public char getChar(int x) {
    return (char) (x + 'a');
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

214. [1062]Longest Repeating Substring.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestRepeatingSubstring(String s) {
        int left = 0;
        int right = s.length();
        int mid;
        int result = 0;

        while (left <= right) {
            mid = left + (right - left) / 2;

            if (hasRepeatingSubstring(s, mid)) {
                result = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return result;
    }

    private boolean hasRepeatingSubstring(String s, int length) {
        Set<String> seen = new HashSet<>();

        for (int i = 0; i <= s.length() - length; i++) {
            String substr = s.substring(i, i + length);

            if (!seen.add(substr)) {
                return true;
            }
        }

        return false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

215. `[1063]Number of Valid Subarrays.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int validSubarrays(int[] nums) {
        int result = 0;
        Deque<Integer> dq = new ArrayDeque<>();

        for (int i = 0; i < nums.length; i++) {
            while (!dq.isEmpty() && nums[i] < nums[dq.peek()]) {
                result += (i - dq.peek());
                dq.pop();
            }
            dq.push(i);
        }

        while (!dq.isEmpty()) {
            result += (nums.length - dq.peek());
            dq.pop();
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

216. `[1065]Index Pairs of a String.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[][] indexPairs(String text, String[] words) {
        List<int[]> list = new ArrayList<>();
        for (String word : words) {
            int index = text.indexOf(word);
            while (index != -1) {
                list.add(new int[]{index, index + word.length() - 1});
                index = text.indexOf(word, index + 1);
            }
        }
        Collections.sort(list, (a, b) -> a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]);
        return list.toArray(new int[list.size()][2]);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

217. `[1066]Campus Bikes II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

import java.util.Arrays;

class Solution {
    private static final int MAX = Integer.MAX_VALUE;

    private int countSetBits(int mask) {
        return Integer.bitCount(mask);
    }

    private int computeDistance(int[] worker, int[] bike) {
        return Math.abs(worker[0] - bike[0]) + Math.abs(worker[1] - bike[1]);
    }

    private int computeMinimumDistance(int[][] workers, int[][] bikes, int[] memo) {
        int numOfBikes = bikes.length;
        int numOfWorkers = workers.length;
        int minDistanceSum = MAX;
        Arrays.fill(memo, MAX);
        memo[0] = 0;

        for (int mask = 0; mask < (1 << numOfBikes); mask++) {
            int nextWorker = countSetBits(mask);

            if (nextWorker >= numOfWorkers) {
                minDistanceSum = Math.min(minDistanceSum, memo[mask]);
                continue;
            }

            for (int bikeIndex = 0; bikeIndex < numOfBikes; bikeIndex++) {
                if ((mask & (1 << bikeIndex)) == 0) {
                    int newMask = mask | (1 << bikeIndex);
                    int newDistance = memo[mask] + computeDistance(workers[nextWorker],
bikes[bikeIndex]);
                    memo[newMask] = Math.min(memo[newMask], newDistance);
                }
            }
        }

        return minDistanceSum;
    }

    public int assignBikes(int[][] workers, int[][] bikes) {
        int[] memo = new int[1 << bikes.length];
        return computeMinimumDistance(workers, bikes, memo);
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

218. [1071]Greatest Common Divisor of Strings.java

```
1071
        Greatest Common Divisor of Strings
        2023-02-01 09:29:16

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String gcdOfStrings(String str1, String str2) {
        int n = str1.length();
        int m = str2.length();
        if (!(str1 + str2).equals(str2 + str1)) {
            return "";
        }
        int max = gcd(n, m);
        return str1.substring(0, max);
    }

    private int gcd(int x, int y) {
        return y == 0 ? x : gcd(y, x % y);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

219. `[1072]Flip Columns For Maximum Number of Equal Rows.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxEqualRowsAfterFlips(int[][] matrix) {
        Map<String, Integer> patternCount = new HashMap<>();

        for (int[] row : matrix) {
            StringBuilder sb = new StringBuilder();
            StringBuilder sbInverse = new StringBuilder();

            for (int num : row) {
                sb.append(num);
                sb.append(',');
                sbInverse.append(num == 0 ? '1' : '0');
                sbInverse.append(',');
            }

            String pattern = sb.toString();
            String inversePattern = sbInverse.toString();
            String key = pattern.compareTo(inversePattern) < 0 ? pattern :
inversePattern;
            patternCount.put(key, patternCount.getOrDefault(key, 0) + 1);
        }

        int maxCount = 0;

        for (int count : patternCount.values()) {
            if (count > maxCount) {
                maxCount = count;
            }
        }

        return maxCount;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

220. `[1073]Adding Two Negabinary Numbers.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] addNegabinary(int[] arr1, int[] arr2) {
        List<Integer> list = new ArrayList<>();
        int m = arr1.length - 1;
        int n = arr2.length - 1;
        int carry = 0;
        while (m >= 0 || n >= 0 || carry != 0) {
            int sum = carry;
            if (m >= 0) {
                sum += arr1[m--];
            }
            if (n >= 0) {
                sum += arr2[n--];
            }
            list.add(0, sum & 1);
            carry = -(sum >> 1);
        }
        while (list.size() > 1 && list.get(0) == 0) {
            list.remove(0);
        }
        int[] res = new int[list.size()];
        for (int i = 0; i < list.size(); i++) {
            res[i] = list.get(i);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

221. `[1074]Number of Submatrices That Sum to Target.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    public int numSubmatrixSumTarget(int[][] matrix, int target) {
        int m = matrix.length;
        int n = matrix[0].length;
        int result = 0;

        for (int i = 0; i < m; i++) {
            for (int j = 1; j < n; j++) {
                matrix[i][j] += matrix[i][j - 1];
            }
        }

        Map<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                map.clear();
                map.put(0, 1);
                int sum = 0;

                for (int k = 0; k < m; k++) {
                    sum += matrix[k][j] - (i > 0 ? matrix[k][i - 1] : 0);
                    result += map.getOrDefault(sum - target, 0);
                    map.put(sum, map.getOrDefault(sum, 0) + 1);
                }
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

222. [1079]Letter Tile Possibilities.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int numTilePossibilities(String tiles) {
    int[] count = new int[26];

    for (char c : tiles.toCharArray()) {
      count[c - 'A']++;
    }

    return dfs(count);
  }

  private int dfs(int[] count) {
    int sum = 0;

    for (int i = 0; i < 26; i++) {
      if (count[i] == 0) {
        continue;
      }

      sum++;
      count[i]--;
      sum += dfs(count);
      count[i]++;
    }

    return sum;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

223. [1088]Confusing Number II.java

```
1088
        Confusing Number II
        2023-01-02 09:18:07

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    final char[] digits = new char[]{'0', '1', '6', '8', '9'};

    public int confusingNumberII(int n) {
        return dfs(String.valueOf(n), false, new StringBuilder());
    }

    private int dfs(String s, boolean smaller, StringBuilder num) {
        if (num.length() == s.length()) {
            int zeros = 0;
            while (zeros < num.length() && num.charAt(zeros) == '0') {
                zeros++;
            }
            final String temp = num.substring(zeros);
            for (int i = 0, j = temp.length() - 1; i <= j; i++, j--) {
                if (temp.charAt(i) != other(temp.charAt(j))) {
                    return 1;
                }
            }
            return 0;
        }
        int ans = 0;
        for (char c : digits) {
            if (!smaller && c > s.charAt(num.length())) {
                break;
            }
            num.append(c);
            ans += dfs(s, smaller || c < s.charAt(num.length() - 1), num);
            num.setLength(num.length() - 1);
        }
        return ans;
    }

    private char other(char c) {
        switch (c) {
            case '0':
            case '1':
            case '8':
                return c;
            case '6':
                return '9';
            case '9':
                return '6';
            default:
                return '?';
        }
```

```
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

224. [1089]Duplicate Zeros.java

```
1089
        Duplicate Zeros
        2023-01-16 23:39:35

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public void duplicateZeros(int[] arr) {
        int n = arr.length;
        int[] res = arr.clone();
        int indexArr = 0;
        int indexRes = 0;
        while (indexArr < n) {
            if (res[indexRes] == 0 && indexArr <= n - 2) {
                arr[indexArr] = 0;
                arr[indexArr + 1] = 0;
                indexRes++;
                indexArr += 2;
            } else {
                arr[indexArr] = res[indexRes];
                indexRes++;
                indexArr++;
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

225. [1091]Shortest Path in Binary Matrix.java

```
1091
        Shortest Path in Binary Matrix
        2022-12-03 00:01:30

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {

    public int shortestPathBinaryMatrix(int[][] grid) {
        int n = grid.length;
        if (grid[0][0] == 1 || grid[n - 1][n - 1] == 1) {
            return -1;
        }
        int[][] dirs = new int[][]{{1, 0}, {-1, 0}, {0, 1}, {0, -1}, {1, 1}, {1, -1},
{-1, 1}, {-1, -1}};
        boolean[][] isVisited = new boolean[n][n];
        Deque<int[]> dq = new ArrayDeque<>();
        dq.add(new int[]{0, 0});
        isVisited[0][0] = true;
        int count = 1;
        while (!dq.isEmpty()) {
            int size = dq.size();
            for (int i = 0; i < size; i++) {
                int[] curr = dq.poll();
                if (curr[0] == n - 1 && curr[1] == n - 1) {
                    return count;
                }
                for (int[] dir : dirs) {
                    int x = curr[0] + dir[0];
                    int y = curr[1] + dir[1];
                    if (x < 0 || x >= n || y < 0 || y >= n || grid[x][y] == 1 ||
isVisited[x][y]) {
                        continue;
                    }
                    dq.add(new int[]{x, y});
                    isVisited[x][y] = true;
                }
            }
            count++;
        }
        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

226. [1092]Shortest Common Supersequence .java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String shortestCommonSupersequence(String str1, String str2) {
        int m = str1.length();
        int n = str2.length();
        int[][] dp = new int[m+1][n+1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (str1.charAt(i-1) == str2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                }
            }
        }

        StringBuilder lcs = new StringBuilder();
        int i = m, j = n;

        while (i > 0 && j > 0) {
            if (str1.charAt(i-1) == str2.charAt(j-1)) {
                lcs.append(str1.charAt(i-1));
                i--; j--;
            } else if (dp[i-1][j] >= dp[i][j-1]) {
                i--;
            } else {
                j--;
            }
        }

        lcs.reverse();
        String lcsStr = lcs.toString();

        StringBuilder scs = new StringBuilder();
        i = 0;
        j = 0;

        for (char c : lcsStr.toCharArray()) {
            while (i < str1.length() && str1.charAt(i) != c) {
                scs.append(str1.charAt(i++));
            }

            while (j < str2.length() && str2.charAt(j) != c) {
                scs.append(str2.charAt(j++));
            }

            scs.append(c);
            i++;
            j++;
```

```
        }

        scs.append(str1.substring(i));
        scs.append(str2.substring(j));

        return scs.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

227. `[1095]Find in Mountain Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * // This is MountainArray's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface MountainArray {
 * public int get(int index) {}
 * public int length() {}
 * }
 */

class Solution {
    MountainArray mountainArray;

    public int findInMountainArray(int target, MountainArray mountainArr) {
        this.mountainArray = mountainArr;
        int length = mountainArr.length();
        int peakIndex = binarySearch(1, length - 2, target, 1);
        int left = binarySearch(0, peakIndex, target, 2);

        if (mountainArr.get(left) == target) {
            return left;
        }

        left = binarySearch(peakIndex + 1, length - 1, target, 3);

        if (mountainArr.get(left) == target) {
            return left;
        }

        return -1;
    }

    private int binarySearch(int left, int right, int target, int type) {
        while (left != right) {
            int mid = (left + right) >> 1;
            int value = mountainArray.get(mid);

            switch (type) {
                case 1:
                    int nextValue = mountainArray.get(mid + 1);
                    if (value < nextValue) {
                        left = mid + 1;
                    } else {
                        right = mid;
                    }
                    break;
                case 2:
                    if (value < target) {
                        left = mid + 1;
```

```
                    } else {
                        right = mid;
                    }
                    break;
                case 3:
                    if (value > target) {
                        left = mid + 1;
                    } else {
                        right = mid;
                    }
                    break;
            }
        }

        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
Binary find peak in the mountain.
852. Peak Index in a Mountain Array
Binary find the target in strict increasing array
Binary find the target in strict decreasing array
Tip
Personally,
If I want find the index, I always use while (left < right)
If I may return the index during the search, I'll use while (left <= right)


    int findInMountainArray(int target, MountainArray A) {
        int n = A.length(), l, r, m, peak = 0;
        // find index of peak
        l  = 0;
        r = n - 1;
        while (l < r) {
            m = (l + r) / 2;
            if (A.get(m) < A.get(m + 1))
                l = peak = m + 1;
            else
                r = m;
        }
        // find target in the left of peak
        l = 0;
        r = peak;
        while (l <= r) {
            m = (l + r) / 2;
            if (A.get(m) < target)
                l = m + 1;
            else if (A.get(m) > target)
                r = m - 1;
            else
                return m;
        }
        // find target in the right of peak
```

```
        l = peak;
        r = n - 1;
        while (l <= r) {
            m = (l + r) / 2;
            if (A.get(m) > target)
                l = m + 1;
            else if (A.get(m) < target)
                r = m - 1;
            else
                return m;
        }
        return -1;
    }
 */
```

228. [1099]Two Sum Less Than K.java

```
1099
        Two Sum Less Than K
        2022-11-28 10:53:42

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int twoSumLessThanK(int[] nums, int k) {
        Arrays.sort(nums);
        int left = 0;
        int right = nums.length - 1;
        int res = -1;

        while (left < right) {
            int sum = nums[left] + nums[right];

            if (sum < k) {
                res = Math.max(res, sum);
                left++;
            } else {
                right--;
            }
        }

        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

229. `[1101]The Earliest Moment When Everyone Become`

Friends.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int earliestAcq(int[][] logs, int n) {
        Arrays.sort(logs, (a, b) -> a[0] - b[0]);
        UnionFind uf = new UnionFind(n);

        for (int[] log : logs) {
            int timestamp = log[0];
            int x = log[1];
            int y = log[2];
            uf.union(x, y);

            if (uf.getCount() == 1) {
                return timestamp;
            }
        }

        return -1;
    }
}

class UnionFind {
    private int[] parent;
    private int[] rank;
    private int count;

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        count = n;

        for (int i = 0; i < n; i++) {
            parent[i] = i;
            rank[i] = 1;
        }
    }

    public int find(int x) {
        if (parent[x] == x) {
            return x;
        }

        return parent[x] = find(parent[x]);
    }

    public void union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX == rootY) {
            return;
```

```
        }

        if (rank[rootX] > rank[rootY]) {
            parent[rootY] = rootX;
        } else if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }

        count--;
    }

    public int getCount() {
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

230. [1102]Path With Maximum Minimum Value.java

```
1102
        Path With Maximum Minimum Value
        2023-02-14 15:43:08

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumMinimumPath(int[][] grid) {
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int m = grid.length;
        int n = grid[0].length;
        boolean[][] isVisited = new boolean[m][n];
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> grid[b[0]][b[1]] -
grid[a[0]][a[1]]);
        pq.add(new int[]{0, 0});
        int res = grid[0][0];
        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            res = Math.min(res, grid[curr[0]][curr[1]]);
            if (curr[0] == m - 1 && curr[1] == n - 1) {
                break;
            }
            for (int[] dir : dirs) {
                int x = dir[0] + curr[0];
                int y = dir[1] + curr[1];
                if (x < 0 || x >= m || y < 0 || y >= n || isVisited[x][y]) {
                    continue;
                }
                pq.add(new int[]{x, y});
                isVisited[x][y] = true;
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

231. `[1103]Distribute Candies to People.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] distributeCandies(int candies, int num_people) {
        int[] arr = new int[num_people];
        int count = 1;
        int index = 0;
        while (candies > count) {
            arr[index % num_people] += count;
            candies -= count;
            count++;
            index++;
        }

        arr[index % num_people] += candies;
        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

232. `[1105]Filling Bookcase Shelves.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minHeightShelves(int[][] books, int shelfWidth) {
        int n = books.length;
        int[] dp = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;

        for (int i = 1; i <= n; i++) {
            int width = 0;
            int height = 0;

            for (int j = i; j > 0; j--) {
                width += books[j - 1][0];

                if (width > shelfWidth) {
                    break;
                }

                height = Math.max(height, books[j - 1][1]);
                dp[i] = Math.min(dp[i], dp[j - 1] + height);
            }
        }

        return dp[n];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

233. `[1106]Parsing A Boolean Expression.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private int idx = 0;

    public boolean parseBoolExpr(final String expression) {
        this.idx = 0;

        if(expression.length() == 1) {
            return expression.charAt(0) == 't';
        }

        return this.helper(expression);
    }

    private boolean helper(final String s) {
        final char operator = s.charAt(this.idx);
        this.idx += 2;
        char c = s.charAt(this.idx);
        boolean result = false;

        if(c == 't') {
            result = true;
            this.idx++;
        } else if(c == 'f') {
            result = false;
            this.idx++;
        } else {
            result = this.helper(s);
        }

        c = s.charAt(this.idx);

        while(c != ')') {
            if(c == ',') {
                c = s.charAt(++this.idx);
                continue;
            }

            boolean curr = false;

            if(c == 't') {
                curr = true;
                this.idx++;
            } else if(c == 'f') {
                curr = false;
                this.idx++;
            } else {
                curr = helper(s);
            }
```

```
            if(operator == '&') {
                result &= curr;
            }
            else if(operator == '|') {
                result |= curr;
            }

            c = s.charAt(this.idx);
        }

        this.idx++;
        return operator == '!' ? !result : result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

234. `[1110]Delete Nodes And Return Forest.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<TreeNode> delNodes(TreeNode root, int[] to_delete) {
        Set<Integer> set = new HashSet<>();

        for (int val : to_delete) {
            set.add(val);
        }

        List<TreeNode> result = new ArrayList<>();

        if (!set.contains(root.val)) {
            result.add(root);
        }

        deleteNode(root, set, result);
        return result;
    }

    private TreeNode deleteNode(TreeNode node, Set<Integer> set, List<TreeNode> list) {
        if (node == null) {
            return null;
        }

        node.left = deleteNode(node.left, set, list);
        node.right = deleteNode(node.right, set, list);

        if (set.contains(node.val)) {
            if (node.left != null) {
                list.add(node.left);
            }

            if (node.right != null) {
                list.add(node.right);
            }

            return null;
        }

        return node;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

235. `[1114]Print in Order.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Foo {
    private Semaphore firstJob;
    private Semaphore secondJob;


    public Foo() {
        firstJob = new Semaphore(0);
        secondJob = new Semaphore(0);
    }

    public void first(Runnable printFirst) throws InterruptedException {
        // printFirst.run() outputs "firstJob". Do not change or remove this line.
        printFirst.run();
        firstJob.release();
    }

    public void second(Runnable printSecond) throws InterruptedException {
        firstJob.acquire();
        // printSecond.run() outputs "second". Do not change or remove this line.
        printSecond.run();
        secondJob.release();
    }

    public void third(Runnable printThird) throws InterruptedException {
        secondJob.acquire();
        // printThird.run() outputs "third". Do not change or remove this line.
        printThird.run();

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

236. `[1115]Print FooBar Alternately.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class FooBar {
    private int n;
    private Semaphore fooJob;
    private Semaphore barJob;

    public FooBar(int n) {
        this.n = n;
        fooJob = new Semaphore(1);
        barJob = new Semaphore(0);
    }

    public void foo(Runnable printFoo) throws InterruptedException {

        for (int i = 0; i < n; i++) {
            fooJob.acquire();
            // printFoo.run() outputs "foo". Do not change or remove this line.
            printFoo.run();
            barJob.release();
        }
    }

    public void bar(Runnable printBar) throws InterruptedException {

        for (int i = 0; i < n; i++) {
            barJob.acquire();
            // printBar.run() outputs "bar". Do not change or remove this line.
            printBar.run();
            fooJob.release();
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

237. `[1116]Print Zero Even Odd.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class ZeroEvenOdd {
    private int n;
    private Semaphore zeroJob;
    private Semaphore oddJob;
    private Semaphore evenJob;

    public ZeroEvenOdd(int n) {
        this.n = n;
        zeroJob = new Semaphore(1);
        oddJob = new Semaphore(0);
        evenJob = new Semaphore(0);
    }

    // printNumber.accept(x) outputs "x", where x is an integer.
    public void zero(IntConsumer printNumber) throws InterruptedException {
        for (int i = 1; i <= n; i++) {
            zeroJob.acquire();
            printNumber.accept(0);
            if (i % 2 == 1) {
                oddJob.release();
            } else {
                evenJob.release();
            }
        }
    }

    public void even(IntConsumer printNumber) throws InterruptedException {
        for (int i = 2; i <= n; i += 2) {
            evenJob.acquire();
            printNumber.accept(i);
            zeroJob.release();
        }
    }

    public void odd(IntConsumer printNumber) throws InterruptedException {
        for (int i = 1; i <= n; i += 2) {
            oddJob.acquire();
            printNumber.accept(i);
            zeroJob.release();
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

238. [1117]Building H2O.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class H2O {
    Semaphore hydrogenJob;
    Semaphore oxygenJob;

    public H2O() {
        hydrogenJob = new Semaphore(2);
        oxygenJob = new Semaphore(0);
    }

    public void hydrogen(Runnable releaseHydrogen) throws InterruptedException {
        hydrogenJob.acquire();
        // releaseHydrogen.run() outputs "H". Do not change or remove this line.
        releaseHydrogen.run();
        oxygenJob.release();
    }

    public void oxygen(Runnable releaseOxygen) throws InterruptedException {
        oxygenJob.acquire(2);
        // releaseOxygen.run() outputs "O". Do not change or remove this line.
        releaseOxygen.run();
        hydrogenJob.release(2);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

239. [1119]Remove Vowels from a String.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String removeVowels(String s) {
        StringBuilder sb = new StringBuilder();

        for (char c : s.toCharArray()) {
            if (!isVowel(c)) {
                sb.append(c);
            }
        }

        return sb.toString();
    }

    private boolean isVowel(char c) {
        return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

240. `[1120]Maximum Average Subtree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    double max = 0;

    public double maximumAverageSubtree(TreeNode root) {
        postOrder(root);
        return max;
    }

    private int[] postOrder(TreeNode root) {
        if (root == null) {
            return new int[2];
        }

        int[] left = postOrder(root.left);
        int[] right = postOrder(root.right);

        int count = left[0] + right[0] + 1;
        int sum = left[1] + right[1] + root.val;
        max = Math.max(max, (double) sum / count);

        return new int[]{count, sum};
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

241. `[1122]Relative Sort Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] relativeSortArray(int[] arr1, int[] arr2) {
        Map<Integer, Integer> map = Arrays.stream(arr1)
                .boxed()
                .collect(Collectors.toMap(
                        num -> num,
                        num -> 1,
                        Integer::sum,
                        TreeMap::new
                ));

        int[] arr = new int[arr1.length];
        int index = 0;

        for (int num : arr2) {
            int count = map.get(num);

            while (count-- > 0) {
                arr[index++] = num;
            }

            map.put(num, 0);
        }

        for (int key : map.keySet()) {
            int count = map.get(key);

            while (count-- > 0) {
                arr[index++] = key;
            }
        }

        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

242. [1123]Lowest Common Ancestor of Deepest Leaves.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public TreeNode lcaDeepestLeaves(TreeNode root) {
    return dfs(root).lca;
  }

  private class Result {
    TreeNode lca;
    int depth;

    Result(TreeNode lca, int depth) {
      this.lca = lca;
      this.depth = depth;
    }
  }

  private Result dfs(TreeNode node) {
    if (node == null) {
      return new Result(null, 0);
    }

    Result left = dfs(node.left);
    Result right = dfs(node.right);

    if (left.depth == right.depth) {
      return new Result(node, left.depth + 1);
    } else if (left.depth > right.depth) {
      return new Result(left.lca, left.depth + 1);
    } else {
      return new Result(right.lca, right.depth + 1);
    }
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

243. `[1125]Smallest Sufficient Team.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] smallestSufficientTeam(String[] req_skills, List<List<String>> people) {
        int n = req_skills.length;
        int m = people.size();
        Map<String, Integer> map = new HashMap<>();
        for (int i = 0; i < n; i++) {
            map.put(req_skills[i], i);
        }
        List<Integer>[] dp = new List[1 << n];
        dp[0] = new ArrayList<>();
        for (int i = 0; i < m; i++) {
            int curr = 0;
            for (String str : people.get(i)) {
                curr |= 1 << map.get(str);
            }
            for (int prev = 0; prev < dp.length; prev++) {
                if (dp[prev] == null) {
                    continue;
                }
                int comb = prev | curr;
                if (dp[comb] == null || dp[prev].size() + 1 < dp[comb].size()) {
                    dp[comb] = new ArrayList<>(dp[prev]);
                    dp[comb].add(i);
                }
            }
        }
        return dp[(1 << n) - 1].stream().mapToInt(i -> i).toArray();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

244. `[1128]Number of Equivalent Domino Pairs.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int numEquivDominoPairs(int[][] dominoes) {
    int[][] counts = new int[10][10];
    int result = 0;

    for (int[] domino : dominoes) {
      int a = domino[0];
      int b = domino[1];
      int x = Math.min(a, b);
      int y = Math.max(a, b);

      result += counts[x][y];
      counts[x][y]++;
    }

    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

245. [1129]Shortest Path with Alternating Colors.java

```
1129
        Shortest Path with Alternating Colors
        2023-02-11 11:59:21

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] shortestAlternatingPaths(int n, int[][] redEdges, int[][] blueEdges) {
        List<List<int[]>> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (int[] edge : redEdges) {
            list.get(edge[0]).add(new int[]{edge[1], 0});
        }
        for (int[] edge : blueEdges) {
            list.get(edge[0]).add(new int[]{edge[1], 1});
        }
        int[][] dist = new int[n][2];
        for (int[] arr : dist) {
            Arrays.fill(arr, -1);
        }
        dist[0] = new int[]{0, 0};
        Deque<int[]> dq = new ArrayDeque<>();
        dq.add(new int[]{0, 0});
        dq.add(new int[]{0, 1});
        while (!dq.isEmpty()) {
            int[] curr = dq.poll();
            for (int[] arr : list.get(curr[0])) {
                if (curr[1] == arr[1]) {
                    continue;
                }
                if (dist[arr[0]][arr[1]] == -1) {
                    dist[arr[0]][arr[1]] = dist[curr[0]][curr[1]] + 1;
                    dq.add(new int[]{arr[0], arr[1], dist[arr[0]][arr[1]]});
                }
            }
        }
        int[] res = new int[n];
        for (int i = 0; i < n; i++) {
            if (dist[i][0] == -1 || dist[i][1] == -1) {
                res[i] = Math.max(dist[i][0], dist[i][1]);
            } else {
                res[i] = Math.min(dist[i][0], dist[i][1]);
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

246. [1130]Minimum Cost Tree From Leaf Values.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int mctFromLeafValues(int[] arr) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

247. [1133]Largest Unique Number.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int largestUniqueNumber(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        int result = -1;

        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            if (entry.getValue() == 1 && entry.getKey() > result) {
                result = entry.getKey();
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

248. [1135]Connecting Cities With Minimum Cost.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumCost(int n, int[][] connections) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[2] - b[2]);
        for (int[] conn : connections) {
            pq.add(conn);
        }
        UnionFind uf = new UnionFind(n);
        int count = 1;
        int res = 0;
        while (!pq.isEmpty() && count < n) {
            int[] curr = pq.poll();
            if (uf.union(curr[0], curr[1])) {
                res += curr[2];
                count++;
            }
        }
        return count == n ? res : -1;
    }
}

class UnionFind {
    private int[] group;

    public UnionFind(int n) {
        group = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            group[i] = i;
        }
    }

    private int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }

    public boolean union(int i, int j) {
        int x = find(i);
        int y = find(j);
        if (x == y) {
            return false;
        }
        group[x] = y;
        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

249. `[1136]Parallel Courses.java`

```
1136
        Parallel Courses
        2023-02-17 10:46:51

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumSemesters(int n, int[][] relations) {
        List<List<Integer>> graph = new ArrayList<>();
        int[] inDegree = new int[n + 1];

        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int[] relation : relations) {
            int prev = relation[0];
            int next = relation[1];
            graph.get(prev).add(next);
            inDegree[next]++;
        }

        int count = 0;
        Deque<Integer> dq = new ArrayDeque<>();

        for (int i = 1; i <= n; i++) {
            if (inDegree[i] == 0) {
                dq.add(i);
                count++;
            }
        }

        int res = 0;

        while (!dq.isEmpty()) {
            int size = dq.size();
            res++;

            for (int i = 0; i < size; i++) {
                int index = dq.poll();

                for (int num : graph.get(index)) {
                    inDegree[num]--;

                    if (inDegree[num] == 0) {
                        dq.add(num);
                        count++;
                    }
                }
            }
        }
```

```
        return n == count ? res : -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

250. `[1137]N-th Tribonacci Number.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int tribonacci(int n) {
        if (n <= 2) {
            return n == 0 ? 0 : 1;
        }

        int t0 = 0;
        int t1 = 1;
        int t2 = 1;
        int curr = 0;

        for (int i = 3; i <= n; i++) {
            curr = t0 + t1 + t2;
            t0 = t1;
            t1 = t2;
            t2 = curr;
        }

        return curr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

251. `[1140]Stone Game II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[][] dp;
    int[] preSum;

    public int stoneGameII(int[] piles) {
        int n = piles.length;
        dp = new int[n][n];
        preSum = piles.clone();
        for (int i = n - 2; i >= 0; i--) {
            preSum[i] += preSum[i + 1];
        }
        return dyP(1, 0);
    }

    private int dyP(int m, int p) {
        if (p + 2 * m >= preSum.length) {
            return preSum[p];
        }
        if (dp[p][m] > 0) {
            return dp[p][m];
        }
        int res = 0;
        int take = 0;
        for (int i = 1; i <= 2 * m; i++) {
            take = preSum[p] - preSum[p + i];
            res = Math.max(res, take + preSum[p + i] - dyP(Math.max(i, m), p + i));
        }
        dp[p][m] = res;
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
DP - Bottom-Up: -7ms
class Solution {
    public int stoneGameII(int[] piles) {
        int n = piles.length;
        int[][] dp = new int[n][n + 1];
        int[] preSum = piles.clone();
        for (int i = n - 2; i >= 0; i--) {
            preSum[i] += preSum[i + 1];
        }
        for (int i = n - 1; i >= 0; i--) {
            for (int m = 1; m <= n; m++) {
                if (i + 2 * m >= n) {
                    dp[i][m] = preSum[i];
                } else {
                    for (int x = 1; x <= 2 * m; x++) {
                        int opponentScore = dp[i + x][Math.max(x, m)];
```

```
                        int score = preSum[i] - opponentScore;
                        dp[i][m] = Math.max(dp[i][m], score);
                    }
                }
            }
        }

        return dp[0][1];
    }
}

DP - Top-Down: -3ms
class Solution {
    int[][] dp;
    int[] preSum;

    public int stoneGameII(int[] piles) {
        int n = piles.length;
        dp = new int[n][n];
        preSum = piles.clone();
        for (int i = n - 2; i >= 0; i--) {
            preSum[i] += preSum[i + 1];
        }
        return dyP(1, 0);
    }

    private int dyP(int m, int p) {
        if (p + 2 * m >= preSum.length) {
            return preSum[p];
        }
        if (dp[p][m] > 0) {
            return dp[p][m];
        }
        int res = 0;
        int take = 0;
        for (int i = 1; i <= 2 * m; i++) {
            take = preSum[p] - preSum[p + i];
            res = Math.max(res, take + preSum[p + i] - dyP(Math.max(i, m), p + i));
        }
        dp[p][m] = res;
        return res;
    }
}
 */
```

252. `[1143]Longest Common Subsequence.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int[] dp = new int[text2.length() + 1];

        for (int i = 0; i < text1.length(); i++) {
            getCommonSub(text1, text2, i, dp);
        }

        return dp[text2.length()];
    }

    private void getCommonSub(String text1, String text2, int i, int[] dp) {
        int prev = 0;

        for (int j = 0; j < text2.length(); j++) {
            int temp = dp[j + 1];

            if (text1.charAt(i) == text2.charAt(j)) {
                dp[j + 1] = prev + 1;
            } else {
                dp[j + 1] = Math.max(dp[j + 1], dp[j]);
            }

            prev = temp;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length();
        int n = text2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (text1.charAt(i) == text2.charAt(j)) {
                    dp[i + 1][j + 1] = dp[i][j] + 1;
                } else {
                    dp[i + 1][j + 1] = Math.max(dp[i + 1][j], dp[i][j + 1]);
                }
            }
        }

        return dp[m][n];
    }
}

class Solution {
```

```
    int[][] memo;

    public int longestCommonSubsequence(String text1, String text2) {
        memo = new int[text1.length() + 1][text2.length() + 1];

        for (int[] arr : memo) {
            Arrays.fill(arr, -1);
        }

        return getSequence(0, 0, text1, text2);
    }

    public int getSequence(int i, int j, String s1, String s2) {
        if (i >= s1.length() || j >= s2.length()) {
            return 0;
        }

        if (memo[i][j] != -1) {
            return memo[i][j];
        }

        if (s1.charAt(i) == s2.charAt(j)) {
            memo[i][j] = getSequence(i + 1, j + 1, s1, s2) + 1;
        } else {
            memo[i][j] = Math.max(getSequence(i + 1, j, s1, s2), getSequence(i, j + 1,
s1, s2));
        }

        return memo[i][j];
    }
}
 */
```

253. `[1146]Snapshot Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class SnapshotArray {
    int snapId = 0;
    TreeMap<Integer, Integer>[] records;

    public SnapshotArray(int length) {
        records = new TreeMap[length];
        for (int i = 0; i < length; i++) {
            records[i] = new TreeMap<>();
            records[i].put(0, 0);
        }
    }

    public void set(int index, int val) {
        records[index].put(snapId, val);
    }

    public int snap() {
        return snapId++;
    }

    public int get(int index, int snap_id) {
        return records[index].floorEntry(snap_id).getValue();
    }
}

/**
 * Your SnapshotArray object will be instantiated and called as such:
 * SnapshotArray obj = new SnapshotArray(length);
 * obj.set(index,val);
 * int param_2 = obj.snap();
 * int param_3 = obj.get(index,snap_id);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

254. [1150]Check If a Number Is Majority Element in a Sorted Array.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isMajorityElement(int[] nums, int target) {
        int count = 0;

        for (int num : nums) {
            if (num == target) {
                count++;
            }
        }

        return count > (nums.length / 2);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

255. [1151]Minimum Swaps to Group All 1's Together.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minSwaps(int[] data) {
        int totalOnes = Arrays.stream(data).sum();
        int left = 0;
        int maxOnes = 0;
        int currOnes = 0;

        for (int right = 0; right < data.length; right++) {
            currOnes += data[right];

            if (right - left + 1 > totalOnes) {
                currOnes -= data[left];
                left++;
            }

            maxOnes = Math.max(maxOnes, currOnes);
        }

        return totalOnes - maxOnes;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

256. [1152]Analyze User Website Visit Pattern.java

```
/*
還不會
 */
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {

    public List<String> mostVisitedPattern(String[] username, int[] timestamp, String[]
website) {
        List<String> res = new ArrayList<>();
        TrieNode head = new TrieNode();
        List<Entry> list = new ArrayList<>();
        for (int i = 0; i < username.length; i++) {
            list.add(new Entry(username[i], timestamp[i], website[i]));
        }
        list.sort(Comparator.comparingInt(entry -> entry.time));
        for (Entry entry : list) {
            if (!head.users.contains(entry.username)) {
                head.users.add(entry.username);
            }
            head.addChild(entry.username, entry.website);
        }

        while (!head.children.isEmpty()) {
            head = getWeb(head, res);
        }
        return res;
    }

    TrieNode getWeb(TrieNode head, List<String> res) {
        String[] webArr = head.children.keySet().toArray(new String[0]);
        Arrays.sort(webArr, (s1, s2) -> head.children.get(s2).count -
head.children.get(s1).count == 0 ? s1.compareTo(s2) : head.children.get(s2).count -
head.children.get(s1).count);
        res.add(webArr[0]);
        return head.children.get(webArr[0]);
    }

    class Entry {
        String username;
        int time;
        String website;

        Entry(String username, int time, String website) {
            this.username = username;
            this.time = time;
            this.website = website;
        }
    }

    class TrieNode {
        int level = 0;
```

```
        String website = null;
        int count = 0;
        Map<String, TrieNode> children = new HashMap<>();
        Set<String> users = new HashSet<>();

        int addChild(String username, String website) {
            if (!users.contains(username)) {
                return 0;
            }
            if (level < 2) {
                for (String childWeb : children.keySet()) {
                    count = Math.max(count, children.get(childWeb).addChild(username,
website));
                }
            }
            TrieNode childNode;
            if (!children.containsKey(website)) {
                childNode = new TrieNode();
                childNode.website = website;
                childNode.level = level + 1;
                children.put(website, childNode);
            }
            childNode = children.get(website);
            if (!childNode.users.contains(username)) {
                if (childNode.level == 3) {
                    childNode.count++;
                }
                childNode.users.add(username);
            }
            if (childNode.level == 3) {
                count = Math.max(count, childNode.count);
            }
            return count;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

257. [1155]Number of Dice Rolls With Target Sum.java

```
1155
Number of
Dice Rolls
With Target
Sum
        2023-02-27 13:43:03

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    final int MOD = (int) 1e9 + 7;

    public int numRollsToTarget(int n, int k, int target) {
        Integer[][] memo = new Integer[n + 1][target + 1];
        return getCount(n, k, target, memo);
    }

    private int getCount(int n, int k, int target, Integer[][] memo) {
        if (n == 0 && target == 0) {
            return 1;
        }

        if (n == 0 || target == 0) {
            return 0;
        }

        if (memo[n][target] != null) {
            return memo[n][target];
        }

        int count = 0;

        for (int i = 1; i <= k; i++) {
            if (target >= i) {
                count = (count + getCount(n - 1, k, target - i, memo)) % MOD;
            }
        }

        memo[n][target] = count;
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    int MOD = (int) 1e9 + 7;

    public int numRollsToTarget(int n, int k, int target) {
        int[][] dp = new int[n + 1][target + 1];
        dp[0][0] = 1;

        for (int i = 1; i <= n; i++) {
```

```
            int sum=0;
            for (int j = 1; j <= target; j++) {
                for (int l = 1; l <= k; l++) {
                    if (j - l >= 0) {
                        dp[i][j] = (dp[i][j] + dp[i - 1][j - l]) % MOD;
                    }
                }
            }
        }

        return dp[n][target];
    }
}
 */
```

258. `[1160]Find Words That Can Be Formed by Characters.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countCharacters(String[] words, String chars) {
        int[] charCount = new int[26];
        int sum = 0;

        for (char c : chars.toCharArray()) {
            charCount[c - 'a']++;
        }

        for (String word : words) {
            sum += checkWord(word, charCount);
        }

        return sum;
    }

    private int checkWord(String word, int[] charCount) {
        int[] wordCount = new int[26];

        for (int i = 0; i < word.length(); i++) {
            int x = word.charAt(i) - 'a';
            wordCount[x]++;

            if (wordCount[x] > charCount[x]) {
                return 0;
            }
        }

        return word.length();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

259. [1161]Maximum Level Sum of a Binary Tree.java

```
1161
        Maximum Level Sum of a Binary Tree
        2022-12-24 10:33:48
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public int maxLevelSum(TreeNode root) {
        Deque<TreeNode> dq = new ArrayDeque<>();
        List<Integer> list = new ArrayList<>();
        dq.add(root);
        int max = Integer.MIN_VALUE;
        int res = 0;
        while (!dq.isEmpty()) {
            int size = dq.size();
            int count = 0;
            for (int i = 0; i < size; i++) {
                TreeNode curr = dq.poll();
                count += curr.val;
                if (curr.left != null) {
                    dq.add(curr.left);
                }
                if (curr.right != null) {
                    dq.add(curr.right);
                }
            }
            list.add(count);
        }
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i) > max) {
                max = list.get(i);
                res = i + 1;
            }
        }
        return res;
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

260. [1162]As Far from Land as Possible.java

```
1162
        As Far from Land as Possible
        2023-02-10 09:26:21

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxDistance(int[][] grid) {
        int[][] dirs = new int[][]{{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int n = grid.length;
        Deque<int[]> dq = new ArrayDeque<>();
        boolean[][] isVisited = new boolean[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == 1) {
                    dq.add(new int[]{i, j});
                    isVisited[i][j] = true;
                }
            }
        }
        int count = 0;
        while (!dq.isEmpty()) {
            count++;
            int size = dq.size();
            for (int i = 0; i < size; i++) {
                int[] curr = dq.poll();
                for (int[] dir : dirs) {
                    int x = curr[0] + dir[0];
                    int y = curr[1] + dir[1];
                    if (x < 0 || x >= n || y < 0 || y >= n || isVisited[x][y] == true) {
                        continue;
                    }
                    dq.add(new int[]{x, y});
                    isVisited[x][y] = true;
                }
            }
        }
        return count == 1 ? -1 : count - 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

261. `[1165]Single-Row Keyboard.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int calculateTime(String keyboard, String word) {
        final int LEN = 26;
        int[] arr = new int[LEN];

        for (int i = 0; i < LEN; i++) {
            arr[keyboard.charAt(i) - 'a'] = i;
        }

        int time = 0;
        int prevIndex = 0;

        for (char c : word.toCharArray()) {
            int currIndex = arr[c - 'a'];
            time += Math.abs(currIndex - prevIndex);
            prevIndex = currIndex;
        }

        return time;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

262. `[1167]Minimum Cost to Connect Sticks.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int connectSticks(int[] sticks) {
        PriorityQueue<Integer>pq=new PriorityQueue<>();

        for (int stick : sticks) {
            pq.offer(stick);
        }

        int result=0;

        while (pq.size()>1){
            int first = pq.poll();
            int second = pq.poll();
            int sum=first+second;
            result+=sum;
            pq.offer(sum);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

263. `[1168]Optimize Water Distribution in a Village.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minCostToSupplyWater(int n, int[] wells, int[][] pipes) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[2] - b[2]);
        for (int i = 0; i < wells.length; i++) {
            pq.add(new int[]{0, i + 1, wells[i]});
        }
        for (int[] pipe : pipes) {
            pq.add(pipe);
        }
        UnionFind uf = new UnionFind(n + 1);
        int count = 1;
        int res = 0;
        while (count < n + 1) {
            int[] curr = pq.poll();
            if (uf.union(curr[0], curr[1])) {
                count++;
                res += curr[2];
            }
        }
        return res;
    }
}

class UnionFind {
    private int[] group;

    public UnionFind(int n) {
        group = new int[n];
        for (int i = 0; i < n; i++) {
            group[i] = i;
        }
    }

    private int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }

    public boolean union(int i, int j) {
        int x = find(i);
        int y = find(j);
        if (x == y) {
            return false;
        }
        group[x] = y;
        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

264. [1171]Remove Zero Sum Consecutive Nodes from Linked List.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public ListNode removeZeroSumSublists(ListNode head) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        Map<Integer, ListNode> map = new HashMap<>();
        int sum = 0;

        for (ListNode node = dummy; node != null; node = node.next) {
            sum += node.val;
            map.put(sum, node);
        }

        sum = 0;

        for (ListNode node = dummy; node != null; node = node.next) {
            sum += node.val;
            node.next = map.get(sum).next;
        }

        return dummy.next;
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

265. `[1183]Maximum Number of Ones.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumNumberOfOnes(int width, int height, int sideLength, int maxOnes) {
        int n = sideLength * sideLength;
        int[] arr = new int[n];
        for (int i = 0; i < height; i++) {
            int row = i % sideLength;
            for (int j = 0; j < width; j++) {
                int col = j % sideLength;
                arr[(row * sideLength) + col]++;
            }
        }
        Arrays.sort(arr);
        int res = 0;
        for (int i = 0; i < maxOnes; i++) {
            res += arr[n - 1 - i];
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

266. [1187]Make Array Strictly Increasing.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int makeArrayIncreasing(int[] arr1, int[] arr2) {
        Map<Integer, Integer> dp = new HashMap<>();
        dp.put(-1, 0);
        Arrays.sort(arr2);
        for (int num : arr1) {
            Map<Integer, Integer> nextDp = new HashMap<>();
            for (int key : dp.keySet()) {
                int steps = dp.get(key);
                if (num > key) {
                    nextDp.put(num, Math.min(nextDp.getOrDefault(num, Integer.MAX_VALUE),
steps));
                }
                int i = getNum(arr2, key);
                if (i < arr2.length) {
                    nextDp.put(arr2[i], Math.min(nextDp.getOrDefault(arr2[i],
Integer.MAX_VALUE), steps + 1));
                }
            }
            if (nextDp.isEmpty()) {
                return -1;
            }
            dp = nextDp;
        }
        return Collections.min(dp.values());
    }

    private int getNum(int[] arr, int value) {
        int left = 0;
        int right = arr.length - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (arr[mid] <= value) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int makeArrayIncreasing(int[] arr1, int[] arr2) {
        Map<Integer, Integer> dp = new HashMap<>();
        dp.put(-1, 0);
        Arrays.sort(arr2);
        for ( int a : arr1) {
```

```
            Map<Integer, Integer> nextDp = new HashMap<>();
            for ( int val : dp.keySet()) {
                 int steps = dp.get(val);
                if (a > val)
                    nextDp.put(a, Math.min(nextDp.getOrDefault(a, Integer.MAX_VALUE),
steps));
                 int i = firstGreater(arr2, val);
                if (i < arr2.length)
                    nextDp.put(arr2[i], Math.min(nextDp.getOrDefault(arr2[i],
Integer.MAX_VALUE), steps + 1));
            }
            if (nextDp.isEmpty())
                return -1;
            dp = nextDp;
        }
        return Collections.min(dp.values());
    }
    private int firstGreater(int[] A, int val) {
         int i = Arrays.binarySearch(A, val + 1);
        return i < 0 ? -(i + 1) : i;
    }
}
 */
```

267. [1188]Design Bounded Blocking Queue.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class BoundedBlockingQueue {
    private Deque<Integer> deque;
    private int capacity;
    private ReentrantLock lock;
    private Condition notEmpty;
    private Condition notFull;

    public BoundedBlockingQueue(int capacity) {
        this.deque = new LinkedList<>();
        this.capacity = capacity;
        this.lock = new ReentrantLock();
        this.notEmpty = lock.newCondition();
        this.notFull = lock.newCondition();
    }

    public void enqueue(int element) throws InterruptedException {
        lock.lock();

        try {
            while (deque.size() == capacity) {
                notFull.await();
            }

            deque.offer(element);
            notEmpty.signal();
        } finally {
            lock.unlock();
        }
    }

    public int dequeue() throws InterruptedException {
        lock.lock();

        try {
            while (deque.isEmpty()) {
                notEmpty.await();
            }

            int item = deque.poll();
            notFull.signal();
            return item;
        } finally {
            lock.unlock();
        }
    }

    public int size() {
        lock.lock();
```

```
        try {
            return deque.size();
        } finally {
            lock.unlock();
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

268. [1189]Maximum Number of Balloons.java

```
1189
        Maximum Number of Balloons
        2022-11-27 23:02:21

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxNumberOfBalloons(String text) {
        int b = 0;
        int a = 0;
        int l = 0;
        int o = 0;
        int n = 0;
        for (char ch : text.toCharArray()) {
            switch (ch) {
                case 'b':
                    b++;
                    break;
                case 'a':
                    a++;
                    break;
                case 'l':
                    l++;
                    break;
                case 'o':
                    o++;
                    break;
                case 'n':
                    n++;
                    break;
            }
        }
        l /= 2;
        o /= 2;
        int min = b;
        min = Math.min(Math.min(Math.min(Math.min(min, n), o), l), a);
        return min;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

269. `[1190]Reverse Substrings Between Each Pair of Parentheses.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String reverseParentheses(String s) {
        Deque<StringBuilder> stack = new ArrayDeque<>();
        StringBuilder sb = new StringBuilder();

        for (char c : s.toCharArray()) {
            if (c == '(') {
                stack.push(sb);
                sb = new StringBuilder();
            } else if (c == ')') {
                sb.reverse();

                if (!stack.isEmpty()) {
                    stack.peek().append(sb);
                    sb = stack.pop();
                }
            } else {
                sb.append(c);
            }
        }

        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

270. [1192]Critical Connections in a Network.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[] disc;
    int[] low;
    int time = 1;
    List<List<Integer>> res = new ArrayList<>();
    List<List<Integer>> list = new ArrayList<>();

    public List<List<Integer>> criticalConnections(int n, List<List<Integer>>
connections) {
        disc = new int[n];
        low = new int[n];
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (List<Integer> conn : connections) {
            list.get(conn.get(0)).add(conn.get(1));
            list.get(conn.get(1)).add(conn.get(0));
        }
        dfs(0, -1);
        return res;
    }

    public void dfs(int curr, int prev) {
        disc[curr] = low[curr] = time++;
        for (int num : list.get(curr)) {
            if (disc[num] == 0) {
                dfs(num, curr);
                low[curr] = Math.min(low[curr], low[num]);
            } else if (num != prev) {
                low[curr] = Math.min(low[curr], disc[num]);
            }
            if (low[num] > disc[curr]) {
                res.add(Arrays.asList(curr, num));
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

271. `[1195]Fizz Buzz Multithreaded.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

import java.util.concurrent.Semaphore;
import java.util.function.IntConsumer;

class FizzBuzz {
    private int n;
    private Semaphore fizz = new Semaphore(0);
    private Semaphore buzz = new Semaphore(0);
    private Semaphore fizzBuzz = new Semaphore(0);
    private Semaphore number = new Semaphore(1);

    public FizzBuzz(int n) {
        this.n = n;
    }

    public void fizz(Runnable printFizz) throws InterruptedException {
        for (int i = 3; i <= n; i += 3) {
            if (i % 5 != 0) {
                fizz.acquire();
                printFizz.run();
                number.release();
            }
        }
    }

    public void buzz(Runnable printBuzz) throws InterruptedException {
        for (int i = 5; i <= n; i += 5) {
            if (i % 3 != 0) {
                buzz.acquire();
                printBuzz.run();
                number.release();
            }
        }
    }

    public void fizzbuzz(Runnable printFizzBuzz) throws InterruptedException {
        for (int i = 15; i <= n; i += 15) {
            fizzBuzz.acquire();
            printFizzBuzz.run();
            number.release();
        }
    }

    public void number(IntConsumer printNumber) throws InterruptedException {
        for (int count = 1; count <= n; count++) {
            number.acquire();
            if (count % 3 == 0 && count % 5 == 0) {
                fizzBuzz.release();
            } else if (count % 3 == 0) {
```

```
                fizz.release();
            } else if (count % 5 == 0) {
                buzz.release();
            } else {
                printNumber.accept(count);
                number.release();
            }
        }
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

272. [1196]How Many Apples Can You Put into the Basket.java

```
1196
        How Many Apples Can You Put into the Basket
        2022-12-11 01:44:20

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxNumberOfApples(int[] weight) {
        Arrays.sort(weight);
        int basket = 5000;
        int res = 0;
        for (int num : weight) {
            basket -= num;
            if (basket >= 0)
                res++;
            else
                break;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

273. [1198]Find Smallest Common Element in All Rows.java

```
1198
        Find Smallest Common Element in All Rows
        2022-11-28 11:04:16

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int smallestCommonElement(int[][] mat) {
        Map<Integer, Integer> map = new HashMap<>();
        int ans = Integer.MAX_VALUE;
        for (int[] nums : mat) {
            for (int num : nums) {
                map.put(num, map.getOrDefault(num, 0) + 1);
            }
        }
        for (Integer key : map.keySet()) {
            if (map.get(key) == mat.length)
                ans = Math.min(ans, key);
        }
        return ans == Integer.MAX_VALUE ? -1 : ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

274. [1199]Minimum Time to Build Blocks.java

```


//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    public int minBuildTime(int[] blocks, int split) {
        PriorityQueue<Integer> heap = new PriorityQueue<>();
        for (int block : blocks) {
            heap.offer(block);
        }
        while (heap.size() > 1) {
            int block1 = heap.poll();
            int block2 = heap.poll();
            heap.offer(block2 + split);
        }
        return heap.poll();
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

275. `[1200]Minimum Absolute Difference.java`

```
1200
        Minimum Absolute Difference
        2022-12-17 10:01:04

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> minimumAbsDifference(int[] arr) {
        Arrays.sort(arr);
        int minDiff = Integer.MAX_VALUE;
        List<List<Integer>> res = new ArrayList<>();
        for (int i = 0; i < arr.length - 1; i++) {
            int currDiff = arr[i + 1] - arr[i];
            if (currDiff == minDiff) {
                res.add(Arrays.asList(arr[i], arr[i + 1]));
            } else if (currDiff < minDiff) {
                res.clear();
                res.add(Arrays.asList(arr[i], arr[i + 1]));
                minDiff = currDiff;
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

276. [1202]Smallest String With Swaps.java

```
1202
        Smallest String With Swaps
        2023-02-03 16:53:41

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String smallestStringWithSwaps(String s, List<List<Integer>> pairs) {
        int n = s.length();
        UnionFind uf = new UnionFind(n);
        for (List<Integer> pair : pairs) {
            uf.union(pair.get(0), pair.get(1));
        }
        Map<Integer, List<Integer>> map = new HashMap<>();
        for (int i = 0; i < n; i++) {
            int root = uf.find(i);
            map.putIfAbsent(root, new ArrayList<>());
            map.get(root).add(i);
        }
        char[] result = new char[n];
        for (List<Integer> indices : map.values()) {
            List<Character> ch = new ArrayList<>();
            for (int index : indices) {
                ch.add(s.charAt(index));
            }
            Collections.sort(ch);
            for (int index = 0; index < indices.size(); index++) {
                result[indices.get(index)] = ch.get(index);
            }
        }
        return new String(result);
    }
}

class UnionFind {
    private int[] root;

    public UnionFind(int size) {
        root = new int[size];
        for (int i = 0; i < size; i++) {
            root[i] = i;
        }
    }

    public int find(int x) {
        return root[x] == x ? x : (root[x] = find(root[x]));
    }

    public void union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);
        if (rootX != rootY) {
```

```
            root[rootX] = rootY;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

277. `[1203]Sort Items by Groups Respecting`

Dependencies.java

```
/*
還不會
 */
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] sortItems(int n, int m, int[] group, List<List<Integer>> beforeItems) {
        List<List<Integer>> groupGraph = new ArrayList<>();
        List<List<Integer>> itemGraph = new ArrayList<>();
        for (int i = 0; i < group.length; i++) {
            if (group[i] == -1) {
                group[i] = m++;
            }
        }
        int[] indegreeItem = new int[n];
        int[] indegreeGroup = new int[m];
        for (int i = 0; i < m; i++) {
            groupGraph.add(new ArrayList<>());
        }
        for (int i = 0; i < n; i++) {
            itemGraph.add(new ArrayList<>());
        }
        for (int i = 0; i < n; i++) {
            for (int beforeItem : beforeItems.get(i)) {
                itemGraph.get(beforeItem).add(i);
                indegreeItem[i]++;
                if (group[i] != group[beforeItem]) {
                    indegreeGroup[group[i]]++;
                    groupGraph.get(group[beforeItem]).add(group[i]);
                }
            }
        }
        int[] sortedGroups = topologicalSort(m, groupGraph, indegreeGroup);
        int[] sortedItems = topologicalSort(n, itemGraph, indegreeItem);
        if (sortedGroups.length == 0 || sortedItems.length == 0) {
            return new int[0];
        }
        List<Integer>[] tmp = new List[m];
        for (int i = 0; i < m; i++) {
            tmp[i] = new ArrayList<>();
        }
        for (int i = 0; i < n; i++) {
            tmp[group[sortedItems[i]]].add(sortedItems[i]);
        }
        int[] result = new int[n];
        int count = 0;
        for (int i = 0; i < m; i++) {
            for (int j : tmp[sortedGroups[i]]) {
                result[count++] = j;
            }
        }
        return result;
```

```
    }

    private int[] topologicalSort(int n, List<List<Integer>> graph, int[] indegree) {
        boolean[] visited = new boolean[n];
        Deque<Integer> queue = new ArrayDeque<>();
        int[] sortedArray = new int[n];
        for (int i = 0; i < n; i++) {
            if (indegree[i] == 0) {
                queue.add(i);
            }
        }
        int count = 0;
        while (!queue.isEmpty()) {
            int node = queue.remove();
            sortedArray[count++] = node;
            for (int nei : graph.get(node)) {
                if (--indegree[nei] == 0) {
                    queue.add(nei);
                }
            }
        }
        return count == n ? sortedArray : new int[0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

278. `[1207]Unique Number of Occurrences.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean uniqueOccurrences(int[] arr) {
        Map<Integer, Integer> map = new HashMap<>();

        for (int num : arr) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        Set<Integer> set = new HashSet<>(map.values());
        return map.size() == set.size();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

279. [1208]Get Equal Substrings Within Budget.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int equalSubstring(String s, String t, int maxCost) {
        int start = 0;
        int maxLength = 0;
        int currCost = 0;

        for (int end = 0; end < s.length(); end++) {
            currCost += Math.abs(s.charAt(end) - t.charAt(end));

            while (currCost > maxCost) {
                currCost -= Math.abs(s.charAt(start) - t.charAt(start));
                start++;
            }

            maxLength = Math.max(maxLength, end - start + 1);
        }

        return maxLength;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

280. `[1213]Intersection of Three Sorted Arrays.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> arraysIntersection(int[] arr1, int[] arr2, int[] arr3) {
        List<Integer> result = new ArrayList<>();
        int i = 0;
        int j = 0;
        int k = 0;

        while (i < arr1.length && j < arr2.length && k < arr3.length) {
            if (arr1[i] == arr2[j] && arr2[j] == arr3[k]) {
                result.add(arr1[i]);
                i++;
                j++;
                k++;
            } else {
                int min = Math.min(arr1[i], Math.min(arr2[j], arr3[k]));

                if (arr1[i] == min) {
                    i++;
                }

                if (arr2[j] == min) {
                    j++;
                }

                if (arr3[k] == min) {
                    k++;
                }
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

281. `[1214]Two Sum BSTs.java`

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public boolean twoSumBSTs(TreeNode root1, TreeNode root2, int target) {
        return dfs(root1, root2, target);
    }

    private boolean dfs(TreeNode root, TreeNode root2, int target) {
        if (root == null) {
            return false;
        }
        if (contains(root2, target - root.val)) {
            return true;
        }
        return dfs(root.left, root2, target) || dfs(root.right, root2, target);
    }

    private boolean contains(TreeNode root, int target) {
        if (root == null) {
            return false;
        }
        if (root.val == target) {
            return true;
        } else if (root.val < target) {
            return contains(root.right, target);
        } else {
            return contains(root.left, target);
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

282. `[1216]Valid Palindrome III.java`

```
1216
        Valid Palindrome III
        2023-01-01 22:52:30
/*
還不會
跟 1682 很像
 */
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isValidPalindrome(String s, int k) {
        char[] c = s.toCharArray();
        int n = c.length;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        for (int i = 0; i < n; i++) {
            int max = 0;
            int tmp = 0;
            for (int j = i - 1; j >= 0; j--) {
                tmp = dp[j];
                if (c[i] == c[j]) {
                    dp[j] = Math.max(dp[j], max + 2);
                }
                if (tmp > max) {
                    max = tmp;
                }
                if (n - dp[j] <= k) {
                    return true;
                }
            }
        }
        return false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

283. `[1218]Longest Arithmetic Subsequence of Given`

Difference.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestSubsequence(int[] arr, int difference) {
        Map<Integer, Integer> dp = new HashMap<>();
        int res = 0;
        for (int num : arr) {
            int count = dp.getOrDefault(num - difference, 0) + 1;
            dp.put(num, count);
            res = Math.max(res, count);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
- 5ms
class Solution {
    public int longestSubsequence(int[] arr, int difference) {
        int max = Integer.MIN_VALUE;
        int min = Integer.MAX_VALUE;
        for (int num : arr) {
            max = Math.max(max, num);
            min = Math.min(min, num);
        }
        int[] dp = new int[max - min + 1];
        int res = 1;
        for (int i = 0; i < arr.length; i++) {
            int curr = arr[i] - min;
            int prev = curr - difference;
            if (prev >= 0 && prev < dp.length) {
                dp[curr] = dp[prev] + 1;
            } else {
                dp[curr] = 1;
            }
            res = Math.max(res, dp[curr]);
        }
        return res;
    }
}

- 41ms
class Solution {
    public int longestSubsequence(int[] arr, int difference) {
        Map<Integer, Integer> dp = new HashMap<>();
        int res = 0;
        for (int num : arr) {
            int count = dp.getOrDefault(num - difference, 0) + 1;
            dp.put(num, count);
            res = Math.max(res, count);
        }
```

```
        return res;
    }
}
 */
```

284. `[1219]Path with Maximum Gold.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private static final int[][] directions = {{0, -1}, {-1, 0}, {0, 1}, {1, 0}};

    public int getMaximumGold(int[][] grid) {
        int maxGold = 0;
        int m = grid.length;
        int n = grid[0].length;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                maxGold = Math.max(maxGold, findMaxGold(grid, i, j));
            }
        }

        return maxGold;
    }

    private int findMaxGold(int[][] grid, int i, int j) {
        if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length || grid[i][j] == 0)
{
            return 0;
        }

        int current = grid[i][j];
        grid[i][j] = 0;
        int maxGold = 0;

        for (int[] dir : directions) {
            int x = i + dir[0];
            int y = j + dir[1];
            maxGold = Math.max(maxGold, findMaxGold(grid, x, y));
        }

        grid[i][j] = current;
        return maxGold + current;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

285. `[1220]Count Vowels Permutation.java`

```
1220
        Count Vowels Permutation
        2023-02-27 10:09:14

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countVowelPermutation(int n) {
        final int MOD = (int) 1e9 + 7;
        long aCount = 1, eCount = 1, iCount = 1, oCount = 1, uCount = 1;

        for (int i = 1; i < n; i++) {
            long prevA = aCount, prevE = eCount, prevI = iCount, prevO = oCount, prevU =
uCount;

            aCount = (prevE + prevI + prevU) % MOD;
            eCount = (prevA + prevI) % MOD;
            iCount = (prevE + prevO) % MOD;
            oCount = prevI % MOD;
            uCount = (prevI + prevO) % MOD;
        }

        return (int) ((aCount + eCount + iCount + oCount + uCount) % MOD);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

286. `[1226]The Dining Philosophers.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

class DiningPhilosophers {
    public DiningPhilosophers() {

    }

    // call the run() method of any runnable to execute its code
    public void wantsToEat(int philosopher,
                           Runnable pickLeftFork,
                           Runnable pickRightFork,
                           Runnable eat,
                           Runnable putLeftFork,
                           Runnable putRightFork) throws InterruptedException {

        synchronized (this) {
            pickLeftFork.run();
            pickRightFork.run();
            eat.run();
            putLeftFork.run();
            putRightFork.run();
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class DiningPhilosophers {

    private Semaphore[] forks = new Semaphore[5];
    private Lock eaters;

    public DiningPhilosophers() {
        for (int i = 0; i < 5; i++) {
            forks[i] = new Semaphore(1);
        }

        eaters = new ReentrantLock();
    }

    // call the run() method of any runnable to execute its code
    public void wantsToEat(int philosopher,
                           Runnable pickLeftFork,
                           Runnable pickRightFork,
                           Runnable eat,
                           Runnable putLeftFork,
                           Runnable putRightFork) throws InterruptedException {

        eaters.lock();
        forks[philosopher].acquire();
        forks[(philosopher + 1) % 5].acquire();
```

```
        pickLeftFork.run();
        pickRightFork.run();

        eat.run();

        putLeftFork.run();
        putRightFork.run();

        forks[philosopher].release();
        forks[(philosopher + 1) % 5].release();
        eaters.unlock();
    }
}
 */
```

287. `[1228]Missing Number In Arithmetic`

Progression.java

```
/**
 * <p>In some array <code>arr</code>, the values were in arithmetic progression: the
values <code>arr[i + 1] - arr[i]</code> are all equal for every <code>0 &lt;= i &lt;
arr.length - 1</code>.</p>
 *
 * <p>A value from <code>arr</code> was removed that <strong>was not the first or last
value in the array</strong>.</p>
 *
 * <p>Given <code>arr</code>, return <em>the removed value</em>.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> arr = [5,7,11,13]
 * <strong>Output:</strong> 9
 * <strong>Explanation:</strong> The previous array was [5,7,<strong>9</strong>,11,13].
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> arr = [15,13,12]
 * <strong>Output:</strong> 14
 * <strong>Explanation:</strong> The previous array was [15,<strong>14</strong>,13,12].
</pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>3 &lt;= arr.length &lt;= 1000</code></li>
 * <li><code>0 &lt;= arr[i] &lt;= 10<sup>5</sup></code></li>
 * <li>The given array is <strong>guaranteed</strong> to be a valid array.</li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>Array</li><li>Math</li></div></div><br><div>
<li>👍 281</li><li>👎 38</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int missingNumber(int[] arr) {
        int n = arr.length;
        int diff = (arr[n - 1] - arr[0]) / n;
        int left = 0;
        int right = n - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (arr[mid] == arr[0] + mid * diff) {
                left = mid + 1;
```

```
            } else {
                right = mid - 1;
            }
        }
        return arr[0] + left * diff;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

288. `[1229]Meeting Scheduler.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> minAvailableDuration(int[][] slots1, int[][] slots2, int
duration) {
        PriorityQueue<int[]> queue = new PriorityQueue<>((a, b) -> Integer.compare(a[0],
b[0]));

        for (int[] slot1 : slots1) {
            if (slot1[1] - slot1[0] >= duration) {
                queue.offer(slot1);
            }
        }

        for (int[] slot2 : slots2) {
            if (slot2[1] - slot2[0] >= duration) {
                queue.offer(slot2);
            }
        }

        while (queue.size() > 1) {
            int[] first = queue.poll();
            int[] second = queue.peek();

            if (first[1] >= second[0] + duration) {
                return Arrays.asList(second[0], second[0] + duration);
            }
        }

        return new ArrayList<>();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

289. `[1230]Toss Strange Coins.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public double probabilityOfHeads(double[] prob, int target) {
        int n = prob.length;
        double[][] dp = new double[n + 1][target + 1];
        dp[0][0] = 1;
        for (int i = 1; i <= n; i++) {
            dp[i][0] = dp[i - 1][0] * (1 - prob[i - 1]);
            for (int j = Math.min(i + 1, target); j > 0; j--) {
                dp[i][j] = dp[i - 1][j - 1] * prob[i - 1] + dp[i - 1][j] * (1 - prob[i -
1]);
            }
        }
        return dp[n][target];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
DP - Top-Down - 63ms
class Solution {
    double[][] dp;

    public double probabilityOfHeads(double[] prob, int target) {
        int n = prob.length;
        dp = new double[n][target + 1];
        for (double[] arr : dp) {
            Arrays.fill(arr, -1);
        }
        return dyP(prob, target, 0, n);
    }

    public double dyP(double[] prob, int target, int index, int n) {
        if (target < 0) {
            return 0;
        }
        if (index == n) {
            return target == 0 ? 1 : 0;
        }
        if (dp[index][target] != -1) {
            return dp[index][target];
        }
        dp[index][target] = dyP(prob, target - 1, index + 1, n) * prob[index] + dyP(prob,
target, index + 1, n) * (1 - prob[index]);
        return dp[index][target];
    }
}

DP - Bottom-up  -12ms
class Solution {
    public double probabilityOfHeads(double[] prob, int target) {
```

```
        int n = prob.length;
        double[][] dp = new double[n + 1][target + 1];
        dp[0][0] = 1;
        for (int i = 1; i <= n; i++) {
            dp[i][0] = dp[i - 1][0] * (1 - prob[i - 1]);
             for (int j = Math.min(i + 1, target); j > 0; j--) {
                dp[i][j] = dp[i - 1][j - 1] * prob[i - 1] + dp[i - 1][j] * (1 - prob[i -
1]);
            }
        }
        return dp[n][target];
    }
}

DP - Bottom-up  -8ms
class Solution {
    public double probabilityOfHeads(double[] prob, int target) {
        double[] dp = new double[target + 1];
        dp[0] = 1.0;
        for (int i = 0; i < prob.length; i++) {
            for (int j = Math.min(i + 1, target); j > 0; j--) {
                dp[j] = prob[i] * dp[j - 1] + (1 - prob[i]) * dp[j];
            }
            dp[0] = (1 - prob[i]) * dp[0];
        }
        return dp[target];
    }
}
 */
```

290. `[1231]Divide Chocolate.java`

```
1231
        Divide Chocolate
        2022-12-13 10:03:43

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximizeSweetness(int[] sweetness, int k) {
        int people = k + 1;
        int left = 0;
        int right = 0;
        for (int num : sweetness) {
            right += num;
        }
        right /= people;
        while (left <= right) {
            int mid = (left + right) >>> 1;
            if (isCheck(sweetness, mid, people)) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left - 1;
    }

    private boolean isCheck(int[] sweetness, int mid, int people) {
        int count = 0;
        int sum = 0;
        for (int num : sweetness) {
            sum += num;
            if (sum >= mid) {
                count++;
                sum = 0;
            }
        }
        return count >= people;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

291. [1232]Check If It Is a Straight Line.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean checkStraightLine(int[][] coordinates) {
        int n = coordinates.length;
        if (n == 2) {
            return true;
        }
        int dx = coordinates[1][0] - coordinates[0][0];
        int dy = coordinates[1][1] - coordinates[0][1];
        for (int i = 2; i < n; i++) {
            int currDx = coordinates[i][0] - coordinates[0][0];
            int currDy = coordinates[i][1] - coordinates[0][1];
            if (dx * currDy != dy * currDx) {
                return false;
            }
        }
        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

292. [1233]Remove Sub-Folders from the Filesystem.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<String> removeSubfolders(String[] folder) {
        Arrays.sort(folder);
        List<String> result = new ArrayList<>();

        for (String currentFolder : folder) {
            if (result.isEmpty() || !currentFolder.startsWith(result.get(result.size() -
1) + "/")) {
                result.add(currentFolder);
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

293. [1235]Maximum Profit in Job Scheduling.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int jobScheduling(int[] startTime, int[] endTime, int[] profit) {
        int n = startTime.length;
        Job[] jobs = new Job[n];

        for (int i = 0; i < n; i++) {
            jobs[i] = new Job(startTime[i], endTime[i], profit[i]);
        }

        Arrays.sort(jobs, (a, b) -> a.end - b.end);
        int[] dp = new int[n];
        dp[0] = jobs[0].profit;

        for (int i = 1; i < n; i++) {
            int includedProfit = jobs[i].profit;
            int index = binarySearch(jobs, i);

            if (index != -1) {
                includedProfit += dp[index];
            }

            dp[i] = Math.max(dp[i - 1], includedProfit);
        }

        return dp[n - 1];
    }

    private int binarySearch(Job[] jobs, int index) {
        int left = 0;
        int right = jobs.length - 1;

        while (left <= right) {
            int mid = (left + right) >> 1;

            if (jobs[mid].end <= jobs[index].start) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return right;
    }
}

class Job {
    int start;
    int end;
    int profit;
```

```
    public Job(int start, int end, int profit) {
        this.start = start;
        this.end = end;
        this.profit = profit;
    }
}

//leetcode submit region end(Prohibit modification and deletion)
/*
https://leetcode.com/problems/maximum-profit-in-job-scheduling/solutions/1358899/maximum-profit-in-job-scheduling/

- 55ms
class Solution {
    public int jobScheduling(int[] startTime, int[] endTime, int[] profit) {
        int n = profit.length;
        int[][] jobs = new int[n][3];
        for (int i = 0; i < n; i++) {
            jobs[i] = new int[]{startTime[i], endTime[i], profit[i]};
        }
        Arrays.sort(jobs, (a, b) -> a[1] - b[1]);
        TreeMap<Integer, Integer> dp = new TreeMap<>();
        dp.put(0, 0);
        for (int[] job : jobs) {
            int curr = dp.floorEntry(job[0]).getValue() + job[2];
            if (curr > dp.lastEntry().getValue()) {
                dp.put(job[1], curr);
            }
        }
        return dp.lastEntry().getValue();
    }
}
 */
```

294. `[1236]Web Crawler.java`

```
/**
<p>Given a url <code>startUrl</code> and an interface <code>HtmlParser</code>, implement
a web&amp;nbsp;crawler to crawl all links that are under the&amp;nbsp;<strong>same
hostname</strong> as&amp;nbsp;<code>startUrl</code>.&amp;nbsp;</p>

<p>Return&amp;nbsp;all urls obtained by your web crawler in <strong>any</strong> order.</p>

<p>Your crawler should:</p>

<ul>
 <li>Start from the page: <code>startUrl</code></li>
 <li>Call <code>HtmlParser.getUrls(url)</code> to get all urls from a webpage of given
url.</li>
 <li>Do not crawl the same link twice.</li>
 <li>Explore only the links that are under the <strong>same hostname</strong> as
<code>startUrl</code>.</li>
</ul>

<p><img alt="" src="https://assets.leetcode.com/uploads/2019/08/13/urlhostname.png"
style="width: 600px; height: 164px;" /></p>

<p>As shown in the example url above, the hostname is <code>example.org</code>. For
simplicity sake, you may assume all&amp;nbsp;urls use <strong>http protocol</strong> without
any&amp;nbsp;<strong>port</strong> specified. For example, the urls&amp;nbsp;
<code>http://leetcode.com/problems</code> and&amp;nbsp;
<code>http://leetcode.com/contest</code> are under the same hostname, while urls
<code>http://example.org/test</code> and <code>http://example.com/abc</code> are not
under the same hostname.</p>

<p>The <code>HtmlParser</code> interface is defined as such:&amp;nbsp;</p>

<pre>
interface HtmlParser {
  // Return a list of all urls from a webpage of given <em>url</em>.
  public List&amp;lt;String&amp;gt; getUrls(String url);
}</pre>

<p>Below&amp;nbsp;are two examples explaining the functionality of the problem, for custom
testing purposes you'll have three&amp;nbsp;variables&amp;nbsp;<code data-stringify-
type="code">urls</code>,&amp;nbsp;<code data-stringify-
type="code">edges</code>&amp;nbsp;and&amp;nbsp;<code data-stringify-type="code">startUrl</code>.
Notice that you will only have access to&amp;nbsp;<code data-stringify-
type="code">startUrl</code>&amp;nbsp;in your code, while&amp;nbsp;<code data-stringify-
type="code">urls</code>&amp;nbsp;and&amp;nbsp;<code data-stringify-
type="code">edges</code>&amp;nbsp;are not directly accessible to you in code.</p>

<p>Note: Consider the same URL with the trailing slash "/" as a different URL. For
example, "http://news.yahoo.com", and "http://news.yahoo.com/" are different urls.</p>

<p>&amp;nbsp;</p>
<p><strong class="example">Example 1:</strong></p>
```

<img alt="" src="https://assets.leetcode.com/uploads/2019/10/23/sample_2_1497.png"
style="width: 610px; height: 300px;" />

<pre>
Input:
urls = [
  "http://news.yahoo.com",
  "http://news.yahoo.com/news",
  "http://news.yahoo.com/news/topics/",
  "http://news.google.com",
  "http://news.yahoo.com/us"
]
edges = [[2,0],[2,1],[3,2],[3,1],[0,4]]
startUrl = "http://news.yahoo.com/news/topics/"
Output: [
  "http://news.yahoo.com",
  "http://news.yahoo.com/news",
  "http://news.yahoo.com/news/topics/",
  "http://news.yahoo.com/us"
]
</pre>

Example 2:

<img alt=""
src="https://assets.leetcode.com/uploads/2019/10/23/sample_3_1497.png" style="width:
540px; height: 270px;" />

<pre>
Input:
urls = [
  "http://news.yahoo.com",
  "http://news.yahoo.com/news",
  "http://news.yahoo.com/news/topics/",
  "http://news.google.com"
]
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
startUrl = "http://news.google.com"
Output: ["http://news.google.com"]
Explanation: The startUrl links to all other pages that do not share the
same hostname.</pre>

Constraints:

 <code>1 <= urls.length <= 1000</code>
 <code>1 <= urls[i].length <= 300</code>
 <code>startUrl</code> is one of the <code>urls</code>.
 Hostname label must be from 1 to 63 characters long, including the dots, may contain
only the ASCII letters from 'a' to 'z', digits from '0' to '9' and
the hyphen-minus character ('-').
 The hostname may not start or end with the hyphen-minus character ('-').

```
</li>
 <li>See:  <a
href="https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames">https://en.
wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames</a></li>
 <li>You may assume there're no duplicates in url library.</li>
</ul>

<div><div>Related Topics</div><div><li>String</li><li>Depth-First Search</li><li>Breadth-
First Search</li><li>Interactive</li></div></div><br><div><li>👍 224</li><li>👎 268</li>
</div>
*/

//leetcode submit region begin(Prohibit modification and deletion)
/**
 * // This is the HtmlParser's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface HtmlParser {
 *     public List<String> getUrls(String url) {}
 * }
 */

class Solution {
    Set<String> isVisited;
    public List<String> crawl(String startUrl, HtmlParser htmlParser) {
        isVisited = new HashSet<>();
        isVisited.add(startUrl);
        dfs(hostname(startUrl), startUrl, htmlParser);
        List<String> url = new ArrayList<>(isVisited);
        return url;
    }
    private String hostname(String url) {
        return url.split("/")[2];
    }
    private void dfs(String hostname, String url, HtmlParser htmlParser) {
        List<String> list = htmlParser.getUrls(url);
        for (String currUrl: list) {
            if (!isVisited.contains(currUrl) && currUrl.contains(hostname)) {
                isVisited.add(currUrl);
                dfs(hostname, currUrl, htmlParser);
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

295. [1239]Maximum Length of a Concatenated String with Unique Characters.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxLength(List<String> arr) {
        return backtrack(arr, "", 0);
    }

    private int backtrack(List<String> arr, String curr, int index) {
        if (index == arr.size()) {
            return curr.length();
        }

        int maxLength = curr.length();

        for (int i = index; i < arr.size(); i++) {
            String next = arr.get(i);

            if (isUnique(curr + next)) {
                maxLength = Math.max(maxLength, backtrack(arr, curr + next, i + 1));
            }
        }

        return maxLength;
    }

    private boolean isUnique(String str) {
        Set<Character> set = new HashSet<>();

        for (char c : str.toCharArray()) {
            if (set.contains(c)) {
                return false;
            }

            set.add(c);
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

296. `[1242]Web Crawler Multithreaded.java`

```
/**
 * // This is the HtmlParser's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface HtmlParser {
 * public List<String> getUrls(String url) {}
 * }
 */

/**
 * // This is the HtmlParser's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface HtmlParser {
 * public List<String> getUrls(String url) {}
 * }
 */
class Solution {
    static int working = 0;

    public List<String> crawl(String startUrl, HtmlParser htmlParser) {
        final Lock lock = new ReentrantLock();
        final Condition cv = lock.newCondition();
        Deque<String> deque = new LinkedList<>();
        Set<String> visited = new HashSet<>();
        visited.add(startUrl);
        deque.offer(startUrl);

        List<Thread> threads = new ArrayList<>();

        for (int i = 0; i < 8; i++) {
            Thread producer = new Thread(() -> {
                while (true) {
                    lock.lock();
                    while (deque.isEmpty() && working != 0) {
                        try {
                            cv.await();
                        } catch (Exception e) {
                        }
                    }

                    if (deque.isEmpty()) {
                        lock.unlock();
                        return;
                    }

                    String url = deque.poll();
                    working++;
                    lock.unlock();
                    List<String> newUrls = htmlParser.getUrls(url);
                    lock.lock();
                    for (String newUrl : newUrls) {
```

```
                        if (this.checkHostname(startUrl, newUrl) &&
!visited.contains(newUrl)) {
                            visited.add(newUrl);
                            deque.offer(newUrl);
                        }
                    }
                    working--;
                    cv.signalAll();
                    lock.unlock();
                }
            });
            producer.start();
            threads.add(producer);
        }

        for (Thread thread : threads) {
            try {
                thread.join();
            } catch (Exception e) {
            }
        }

        return new ArrayList<String>(visited);
    }

    private boolean checkHostname(String startUrl, String newUrl) {
        startUrl = startUrl.substring(7).split("/")[0];
        newUrl = newUrl.substring(7).split("/")[0];
        return startUrl.equals(newUrl);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

297. `[1245]Tree Diameter.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    List<List<Integer>> graph = new ArrayList<>();
    int result = 0;

    public int treeDiameter(int[][] edges) {
        int n = edges.length + 1;
        boolean[] isVisited = new boolean[n];

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int[] edge : edges) {
            graph.get(edge[0]).add(edge[1]);
            graph.get(edge[1]).add(edge[0]);
        }

        dfs(0, isVisited);
        return result;
    }

    public int dfs(int curr, boolean[] isVisited) {
        int first = 0;
        int second = 0;
        isVisited[curr] = true;

        for (Integer item : graph.get(curr)) {
            if (isVisited[item]) {
                continue;
            }

            int len = dfs(item, isVisited);

            if (len > first) {
                second = first;
                first = len;
            } else if (len > second) {
                second = len;
            }

            result = Math.max(result, first + second);
        }

        return first + 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

298. [1248]Count Number of Nice Subarrays.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfSubarrays(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);
        int count = 0;
        int result = 0;

        for (int num : nums) {
            if (num % 2 != 0) {
                count++;
            }

            if (map.containsKey(count - k)) {
                result += map.get(count - k);
            }

            map.put(count, map.getOrDefault(count, 0) + 1);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int numberOfSubarrays(int[] nums, int k) {
        return getCount(nums, k) - getCount(nums, k - 1);
    }

    private int getCount(int[] nums, int k) {
        int n = nums.length;
        int result = 0;
        int count = 0;

        for (int left = 0, right = 0; right < n; right++) {
            count += nums[right] % 2;

            while (count > k) {
                count -= nums[left] % 2;
                left++;
            }

            result += right - left + 1;
        }

        return result;
    }
}
 */
```

299. `[1249]Minimum Remove to Make Valid Parentheses.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String minRemoveToMakeValid(String s) {
        StringBuilder sb = new StringBuilder(s);
        Deque<Integer> dq = new ArrayDeque<>();

        for (int i = 0; i < sb.length(); i++) {
            if (sb.charAt(i) == '(') {
                dq.push(i);
            } else if (sb.charAt(i) == ')') {
                if (!dq.isEmpty()) {
                    dq.pop();
                } else {
                    sb.setCharAt(i, '*');
                }
            }
        }

        while (!dq.isEmpty()) {
            sb.setCharAt(dq.pop(), '*');
        }

        return sb.toString().replaceAll("\\*", "");
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

300. `[1254]Number of Closed Islands.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[][] grid;
    int m;
    int n;

    public int closedIsland(int[][] grid) {
        this.grid = grid;
        m = grid.length;
        n = grid[0].length;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if ((i == 0 || i == m - 1 || j == 0 || j == n - 1) && grid[i][j] == 0) {
                    dfs(i, j);
                }
            }
        }
        int count = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == 0) {
                    count++;
                    dfs(i, j);
                }
            }
        }
        return count;
    }

    public void dfs(int x, int y) {
        if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] == 1) {
            return;
        }
        grid[x][y] = 1;
        dfs(x + 1, y);
        dfs(x - 1, y);
        dfs(x, y + 1);
        dfs(x, y - 1);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

301. [1255]Maximum Score Words Formed by Letters.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxScoreWords(String[] words, char[] letters, int[] score) {
        int[] letter_count = new int[26];

        for (char letter : letters) {
            letter_count[letter - 'a']++;
        }

        int[] wordScores = new int[words.length];

        for (int i = 0; i < words.length; i++) {
            for (char c : words[i].toCharArray()) {
                wordScores[i] += score[c - 'a'];
            }
        }

        return backtrack(words, wordScores, letter_count, 0);
    }

    private int backtrack(String[] words, int[] wordScores, int[] letter_count, int
index) {
        if (index == words.length) {
            return 0;
        }

        int maxScore = backtrack(words, wordScores, letter_count, index + 1);
        boolean canInclude = true;
        int[] temp_count = letter_count.clone();

        for (char c : words[index].toCharArray()) {
            if (--temp_count[c - 'a'] < 0) {
                canInclude = false;
                break;
            }
        }

        if (canInclude) {
            maxScore = Math.max(maxScore, wordScores[index] + backtrack(words,
wordScores, temp_count, index + 1));
        }

        return maxScore;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

302. `[1259]Handshakes That Don't Cross.java`

```
1259
        Handshakes That Don't Cross
        2023-02-22 12:03:16

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfWays(int numPeople) {
        long mod = (long) 1e9 + 7;
        long[] dp = new long[numPeople / 2 + 1];
        dp[0] = 1;
        for (int i = 1; i <= numPeople / 2; i++) {
            for (int j = 0; j < i; j++) {
                dp[i] = (dp[i] + dp[j] * dp[i - 1 - j]) % mod;
            }
        }
        return (int) dp[numPeople / 2];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int numberOfWays(int numPeople) {
        long[] arr = new long[numPeople / 2 + 2];
        arr[1] = 1;
        long mod = (long) 1e9 + 7;
        long res = 1;
        for (int i = 2; i < numPeople / 2 + 2; ++i) {
            arr[i] = mod - mod / i * arr[(int) mod % i] % mod;
        }
        for (int i = 1; i <= numPeople / 2; ++i) {
            res = res * (i + numPeople / 2) % mod;
            res = res * arr[i] % mod;
        }
        return (int) (res * arr[numPeople / 2 + 1] % mod);
    }
}
 */
```

303. `[1261]Find Elements in a Contaminated Binary Tree.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
/**
 * Definition for a binary tree node. public class TreeNode { int val; TreeNode left;
TreeNode
 * right; TreeNode() {} TreeNode(int val) { this.val = val; } TreeNode(int val, TreeNode
left,
 * TreeNode right) { this.val = val; this.left = left; this.right = right; } }
 */
class FindElements {
  private Set<Integer> recoveredValues;

  public FindElements(TreeNode root) {
    recoveredValues = new HashSet<>();

    if (root != null) {
      root.val = 0;
      dfs(root);
    }
  }

  private void dfs(TreeNode node) {
    if (node == null){
      return;
    }

    recoveredValues.add(node.val);

    if (node.left != null) {
      node.left.val = 2 * node.val + 1;
      dfs(node.left);
    }

    if (node.right != null) {
      node.right.val = 2 * node.val + 2;
      dfs(node.right);
    }
  }

  public boolean find(int target) {
    return recoveredValues.contains(target);
  }
}

/**
 * Your FindElements object will be instantiated and called as such: FindElements obj =
new
 * FindElements(root); boolean param_1 = obj.find(target);
 */
// leetcode submit region end(Prohibit modification and deletion)
```

304. [1265]Print Immutable Linked List in Reverse.java

```
//leetcode submit region begin(Prohibit modification and deletion)
/**
 * // This is the ImmutableListNode's API interface.
 * // You should not implement it, or speculate about its implementation.
 * interface ImmutableListNode {
 *     public void printValue(); // print the value of this node.
 *     public ImmutableListNode getNext(); // return the next node.
 * };
 */

class Solution {
    public void printLinkedListInReverse(ImmutableListNode head) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

305. [1266]Minimum Time Visiting All Points.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minTimeToVisitAllPoints(int[][] points) {
        int totalTime = 0;

        for (int i = 1; i < points.length; i++) {
            totalTime += getTime(points[i - 1], points[i]);
        }

        //  System.gc();     用了 2 ms，少了1.5MB
        return totalTime;
    }

    private int getTime(int[] curr, int[] next) {
        int deltaX = Math.abs(next[0] - curr[0]);
        int deltaY = Math.abs(next[1] - curr[1]);
        return Math.max(deltaX, deltaY);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

306. [1269]Number of Ways to Stay in the Same Place

After Some Steps.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    Integer[][] memo;
    final int MOD = (int) 1e9 + 7;
    int arrLen;

    public int numWays(int steps, int arrLen) {
        arrLen = Math.min(arrLen, steps / 2 + 1);
        this.arrLen = arrLen;
        memo = new Integer[arrLen][steps + 1];

        return dp(0, steps);
    }

    private int dp(int curr, int remain) {
        if (remain == 0) {
            return curr == 0 ? 1 : 0;
        }

        if (memo[curr][remain] != null) {
            return memo[curr][remain];
        }

        int result = dp(curr, remain - 1);

        if (curr > 0) {
            result = (result + dp(curr - 1, remain - 1)) % MOD;
        }

        if (curr < arrLen - 1) {
            result = (result + dp(curr + 1, remain - 1)) % MOD;
        }

        memo[curr][remain] = result;
        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    Integer[][] memo;
    final int MOD = (int) 1e9 + 7;
    int arrLen;

    public int numWays(int steps, int arrLen) {
        arrLen = Math.min(arrLen, steps);
        this.arrLen = arrLen;
        memo = new Integer[arrLen][steps + 1];

        return dp(0, steps);
```

```
    }

    private int dp(int curr, int remain) {
        if (remain == 0) {
            return curr == 0 ? 1 : 0;
        }

        if (memo[curr][remain] != null) {
            return memo[curr][remain];
        }

        int result = dp(curr, remain - 1);

        if (curr > 0) {
            result = (result + dp(curr - 1, remain - 1)) % MOD;
        }

        if (curr < arrLen - 1) {
            result = (result + dp(curr + 1, remain - 1)) % MOD;
        }

        memo[curr][remain] = result;
        return result;
    }
}

class Solution {
    public int numWays(int steps, int arrLen) {
        final int MOD = (int) 1e9 + 7;
        arrLen = Math.min(arrLen, steps / 2 + 1);
        int[][] dp = new int[arrLen][steps + 1];

        dp[0][0] = 1;

        for (int remain = 1; remain <= steps; remain++) {
            for (int curr = arrLen - 1; curr >= 0; curr--) {
                int result = dp[curr][remain - 1];

                if (curr > 0) {
                    result = (result + dp[curr - 1][remain - 1]) % MOD;
                }

                if (curr < arrLen - 1) {
                    result = (result + dp[curr + 1][remain - 1]) % MOD;
                }

                dp[curr][remain] = result;
            }
        }

        return dp[0][steps];
    }
}
 */
```

307. `[1272]Remove Interval.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> removeInterval(int[][] intervals, int[] toBeRemoved) {
        List<List<Integer>> result = new ArrayList<>();
        int removeStart = toBeRemoved[0];
        int removeEnd = toBeRemoved[1];

        for (int[] interval : intervals) {
            int start = interval[0];
            int end = interval[1];

            if (end <= removeStart || start >= removeEnd) {
                result.add(List.of(start, end));
            } else {
                if (start < removeStart) {
                    result.add(List.of(start, removeStart));
                }

                if (end > removeEnd) {
                    result.add(List.of(removeEnd, end));
                }
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

308. [1275]Find Winner on a Tic Tac Toe Game.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String tictactoe(int[][] moves) {
        int n = moves.length;
        int[][] arr = new int[3][3];
        for (int i = 0; i < moves.length; i++) {
            if (i % 2 == 0) {
                arr[moves[i][0]][moves[i][1]] = 1;
            } else {
                arr[moves[i][0]][moves[i][1]] = -1;
            }
        }
        for (int i = 0; i < 3; i++) {
            if (arr[i][0] == arr[i][1] && arr[i][1] == arr[i][2]) {
                if (arr[i][0] == 1) {
                    return "A";
                } else if (arr[i][0] == -1) {
                    return "B";
                }
            }
            if (arr[0][i] == arr[1][i] && arr[1][i] == arr[2][i]) {
                if (arr[0][i] == 1) {
                    return "A";
                } else if (arr[0][i] == -1) {
                    return "B";
                }
            }
        }
        if (arr[0][0] == arr[1][1] && arr[1][1] == arr[2][2]) {
            if (arr[0][0] == 1) {
                return "A";
            } else if (arr[0][0] == -1) {
                return "B";
            }
        }
        if (arr[2][0] == arr[1][1] && arr[1][1] == arr[0][2]) {
            if (arr[2][0] == 1) {
                return "A";
            } else if (arr[2][0] == -1) {
                return "B";
            }
        }
        if (n < 9) {
            return "Pending";
        }
        return "Draw";
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

309. `[1277]Count Square Submatrices with All Ones.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countSquares(int[][] matrix) {
        int rows = matrix.length;
        int cols = matrix[0].length;
        int count = 0;

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (matrix[i][j] == 1 && i > 0 && j > 0) {
                    matrix[i][j] = Math.min(Math.min(matrix[i][j - 1], matrix[i - 1][j]),
matrix[i - 1][j - 1]) + 1;
                }

                count += matrix[i][j];
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

310. `[1279]Traffic Light Controlled Intersection.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class TrafficLight {
    int
    private int light;

    public TrafficLight() {
        light = 1;
    }

    public void carArrived(
            int carId,           // ID of the car
            int roadId,          // ID of the road the car travels on. Can be 1 (road A)
or 2 (road B)
            int direction,       // Direction of the car
            Runnable turnGreen,  // Use turnGreen.run() to turn light to green on current
road
            Runnable crossCar    // Use crossCar.run() to make car cross the intersection
    ) throws InterruptedException {
        synchronized (this) {
            if (light != roadId) {
                turnGreen.run();
                light = roadId;
            }

            crossCar.run();
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

311. `[1282]Group the People Given the Group Size They Belong To.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> groupThePeople(int[] groupSizes) {
        List<List<Integer>> res = new ArrayList<>();
        Map<Integer, List<Integer>> map = new HashMap<>();
        for (int i = 0; i < groupSizes.length; i++) {
            int size = groupSizes[i];
            if (!map.containsKey(size)) {
                map.put(size, new ArrayList<>());
            }
            map.get(size).add(i);
            if (map.get(size).size() == size) {
                res.add(map.get(size));
                map.put(size, new ArrayList<>());
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

312. `[1283]Find the Smallest Divisor Given a Threshold.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int smallestDivisor(int[] nums, int threshold) {
        int left = 1;
        int right = 0;
        for (int num : nums) {
            right = Math.max(right, num);
        }
        while (left <= right) {
            int mid = (left + right) >>> 1;
            if (isCheck(nums, mid, threshold)) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }

    private boolean isCheck(int[] nums, int mid, int threshold) {
        int count = 0;
        for (int num : nums) {
            count += (num + (mid - 1)) / mid;
            if (count > threshold) {
                return true;
            }
        }
        return false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

313. `[1287]Element Appearing More Than 25% In Sorted Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findSpecialInteger(int[] arr) {
        int bar = arr.length / 4;
        int count = 1;
        int result = arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] == result) {
                count++;

                if (count > bar) {
                    return result;
                }
            } else {
                count = 1;
                result = arr[i];
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

314. `[1289]Minimum Falling Path Sum II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minFallingPathSum(int[][] grid) {
        int n = grid.length;

        for (int i = 1; i < n; i++) {
            int min1 = Integer.MAX_VALUE;
            int min2 = Integer.MAX_VALUE;

            for (int j = 0; j < n; j++) {
                int value = grid[i - 1][j];

                if (value < min1) {
                    min2 = min1;
                    min1 = value;
                } else if (value < min2) {
                    min2 = value;
                }
            }

            for (int j = 0; j < n; j++) {
                if (grid[i - 1][j] == min1) {
                    grid[i][j] += min2;
                } else {
                    grid[i][j] += min1;
                }
            }
        }

        int minSum = Integer.MAX_VALUE;

        for (int i = 0; i < n; i++) {
            minSum = Math.min(minSum, grid[n - 1][i]);
        }

        return minSum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

315. `[1291]Sequential Digits.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> sequentialDigits(int low, int high) {
        List<Integer> list = new ArrayList<>();

        for (int i = 1; i < 10; i++) {
            int num = 0;
            int j = i;
            while (num <= high && j < 10) {
                num = num * 10 + j;

                if (num >= low && num <= high) {
                    list.add(num);
                }

                j++;
            }
        }

        Collections.sort(list);
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public List<Integer> sequentialDigits(int low, int high) {
        List<Integer> list = new ArrayList<>();
        String digits = "123456789";

        for (int len = 2; len <= digits.length(); len++) {
            for (int start = 0; start <= digits.length() - len; start++) {
                int num = Integer.parseInt(digits.substring(start, start + len));

                if (num >= low && num <= high) {
                    list.add(num);
                }
            }
        }

        return list;
    }
}

class Solution {
    public List<Integer> sequentialDigits(int low, int high) {
        List<Integer> list = new ArrayList<>();

        for (int i = 1; i < 10; i++) {
            int num = 0;
```

```
            int j = i;
            while (num <= high && j < 10) {
                num = num * 10 + j;

                if (num >= low && num <= high) {
                    list.add(num);
                }

                j++;
            }
        }

        Collections.sort(list);
        return list;
    }
}
 */
```

316. [1295]Find Numbers with Even Number of Digits.java

```
1295
        Find Numbers with Even Number of Digits
        2023-01-16 01:41:58

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findNumbers(int[] nums) {
        int count = 0;
        for (int num : nums) {
            if ((num > 9 && num < 99) || (num > 999 && num < 9999) || num == 100000) {
                count++;
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

317. `[1296]Divide Array in Sets of K Consecutive Numbers.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isPossibleDivide(int[] nums, int k) {
        if (nums.length % k != 0) {
            return false;
        }

        Map<Integer, Integer> countMap = new TreeMap<>();

        for (int num : nums) {
            countMap.put(num, countMap.getOrDefault(num, 0) + 1);
        }

        for (int key : countMap.keySet()) {
            int count = countMap.get(key);

            if (count > 0) {
                for (int i = 1; i < k; i++) {
                    int nextKey = key + i;

                    if (countMap.getOrDefault(nextKey, 0) < count) {
                        return false;
                    }

                    countMap.put(nextKey, countMap.get(nextKey) - count);
                }
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

318. [1299]Replace Elements with Greatest Element on Right Side.java

```
1299
        Replace Elements with Greatest Element on Right Side
        2023-01-18 11:03:19

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] replaceElements(int[] arr) {
        int n = 0;
        for (int num : arr) {
            if (num > 0) {
                n++;
            }
        }
        int[] dp = new int[n];
        dp[n - 1] = -1;
        for (int i = n - 2; i >= 0; i--) {
            dp[i] = Math.max(dp[i + 1], arr[i + 1]);
        }
        return dp;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

319. `[1302]Deepest Leaves Sum.java`

```
1302
        Deepest Leaves Sum
        2022-12-03 15:48:50
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public int deepestLeavesSum(TreeNode root) {
        if (root == null)
            return 0;
        int ans = 0;
        Deque<TreeNode> queue = new ArrayDeque<>();
        queue.add(root);
        while (!queue.isEmpty()) {
            int currLength = queue.size();
            ans = 0;
            for (int i = 0; i < currLength; i++) {
                TreeNode node = queue.remove();
                ans += node.val;
                if (node.left != null)
                    queue.add(node.left);
                if (node.right != null)
                    queue.add(node.right);
            }
        }
        return ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

320. `[1306].java`

```
1306
Jump Game III
2023-03-05 09:58:15
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canReach(int[] arr, int start) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

321. `[1306]Jump Game III.java`

```


//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public boolean canReach(int[] arr, int start) {
    int n = arr.length;
    boolean[] visited = new boolean[n];
    Queue<Integer> queue = new ArrayDeque<>();

    visited[start] = true;
    queue.offer(start);

    while (!queue.isEmpty()) {
      int curr = queue.poll();

      if (arr[curr] == 0) {
        return true;
      }

      int next = curr + arr[curr];
      if (next < n && !visited[next]) {
        visited[next] = true;
        queue.offer(next);
      }

      next = curr - arr[curr];
      if (next >= 0 && !visited[next]) {
        visited[next] = true;
        queue.offer(next);
      }
    }

    return false;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

322. [1310]XOR Queries of a Subarray.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] xorQueries(int[] arr, int[][] queries) {
        int n = arr.length;
        int[] prefixXOR = new int[n];
        prefixXOR[0] = arr[0];

        for (int i = 1; i < n; i++) {
            prefixXOR[i] = prefixXOR[i - 1] ^ arr[i];
        }

        int[] result = new int[queries.length];

        for (int i = 0; i < queries.length; i++) {
            int left = queries[i][0];
            int right = queries[i][1];

            if (left == 0) {
                result[i] = prefixXOR[right];
            } else {
                result[i] = prefixXOR[right] ^ prefixXOR[left - 1];
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

323. `[1312]Minimum Insertion Steps to Make a String`

Palindrome.java

```
/**
 * <p>Given a string <code>s</code>. In one step you can insert any character at any
index of the string.</p>
 *
 * <p>Return <em>the minimum number of steps</em> to make <code>s</code> palindrome.
</p>
 *
 * <p>A <b>Palindrome String</b> is one that reads the same backward as well as
forward.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> s = "zzazz"
 * <strong>Output:</strong> 0
 * <strong>Explanation:</strong> The string "zzazz" is already palindrome we do not need
any insertions.
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> s = "mbadm"
 * <strong>Output:</strong> 2
 * <strong>Explanation:</strong> String can be "mbdadbm" or "mdbabdm".
 * </pre>
 *
 * <p><strong class="example">Example 3:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> s = "leetcode"
 * <strong>Output:</strong> 5
 * <strong>Explanation:</strong> Inserting 5 characters the string becomes
"leetcodocteel".
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>1 &lt;= s.length &lt;= 500</code></li>
 * <li><code>s</code> consists of lowercase English letters.</li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>String</li><li>Dynamic Programming</li></div>
</div><br><div><li>👍 3158</li><li>👎 36</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
```

```
    public int minInsertions(String s) {
        int lps = dyP(s);
        return s.length() - lps;
    }

    public int dyP(String s) {
        char[] c = s.toCharArray();
        int n = c.length;
        int max = 0;
        int[] dp = new int[n];
        for (int j = 0; j < n; j++) {
            dp[j] = 1;
            max = 0;
            for (int i = j - 1; i >= 0; i--) {
                int len = dp[i];
                if (c[i] == c[j]) {
                    dp[i] = 2 + max;
                }
                max = Math.max(max, len);
            }
        }
        for (int len : dp) {
            max = Math.max(max, len);
        }
        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

324. [1318]Minimum Flips to Make a OR b Equal to c.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minFlips(int a, int b, int c) {
        int res = 0;
        while (a != 0 | b != 0 | c != 0) {
            if ((c & 1) == 1) {
                if ((a & 1) == 0 && (b & 1) == 0) {
                    res++;
                }
            } else {
                res += (a & 1) + (b & 1);
            }
            a >>= 1;
            b >>= 1;
            c >>= 1;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

325. [1319]Number of Operations to Make Network Connected.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[] group;
    int count;

    public int makeConnected(int n, int[][] connections) {
        if (connections.length < n - 1) {
            return -1;
        }
        group = new int[n];
        count = n;
        for (int i = 0; i < n; i++) {
            group[i] = i;
        }
        for (int[] arr : connections) {
            union(arr[0], arr[1]);
        }
        return count - 1;
    }

    public int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }

    public void union(int i, int j) {
        int x = find(i);
        int y = find(j);
        if (x != y) {
            group[x] = y;
            count--;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

326. `[1325]Delete Leaves With a Given Value.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public TreeNode removeLeafNodes(TreeNode root, int target) {
        if (root == null) {
            return null;
        }

        root.left = removeLeafNodes(root.left, target);
        root.right = removeLeafNodes(root.right, target);

        if (root.left == null && root.right == null && root.val == target) {
            return null;
        }

        return root;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

327. [1326]Minimum Number of Taps to Open to Water a Garden.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minTaps(int n, int[] ranges) {
        int[] interval = new int[ranges.length];
        for (int i = 0; i < ranges.length; i++) {
            int left = Math.max(0, i - ranges[i]);
            int right = i + ranges[i];
            interval[left] = Math.max(interval[left], right);
        }

        int currEnd = 0;
        int nextEnd = 0;
        int res = 0;
        for (int i = 0; i <= n; i++) {
            if (i > nextEnd) {
                return -1;
            }
            if (i > currEnd) {
                res++;
                currEnd = nextEnd;
            }
            nextEnd = Math.max(interval[i], nextEnd);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

328. `[1328].java`

```
1328
        Break a Palindrome
        2023-03-06 11:28:15

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String breakPalindrome(String palindrome) {
        if (palindrome.length() == 1) {
            return "";
        }
        char[] ch = palindrome.toCharArray();
        int n = ch.length;
        for (int i = 0; i < n / 2; i++) {
            if (ch[i] != 'a') {
                ch[i] = 'a';
                return String.valueOf(ch);
            }
        }
        ch[n - 1] = 'b';
        return String.valueOf(ch);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

329. `[1331]Rank Transform of an Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] arrayRankTransform(int[] arr) {
        if (arr == null || arr.length == 0) {
            return arr;
        }

        TreeSet<Integer> set = new TreeSet();
        int n = arr.length;

        for (int num : arr) {
            set.add(num);
        }

        Map<Integer, Integer> map = new HashMap();
        int rank = 1;

        for (int num : set) {
            map.put(num, rank++);
        }

        int[] result = new int[n];

        for (int i = 0; i < n; i++) {
            result[i] = map.get(arr[i]);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

330. `[1334]Find the City With the Smallest Number of`

Neighbors at a Threshold Distance.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findTheCity(int n, int[][] edges, int distanceThreshold) {
        int[][] dist = new int[n][n];

        for (int i = 0; i < n; i++) {
            Arrays.fill(dist[i], Integer.MAX_VALUE);
            dist[i][i] = 0;
        }

        for (int[] edge : edges) {
            int from = edge[0];
            int to = edge[1];
            int weight = edge[2];
            dist[from][to] = weight;
            dist[to][from] = weight;
        }

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                if (dist[i][k] == Integer.MAX_VALUE) {
                    continue;
                }

                for (int j = 0; j < n; j++) {
                    if (dist[k][j] != Integer.MAX_VALUE && dist[i][k] + dist[k][j] <
dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }

        int result = -1;
        int smallest = n;

        for (int i = 0; i < n; i++) {
            int count = 0;

            for (int j = 0; j < n; j++) {
                if (i != j && dist[i][j] <= distanceThreshold) {
                    count++;
                }
            }

            if (count < smallest || (count == smallest && i > result)) {
                result = i;
                smallest = count;
            }
        }
```

```
        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

331. [1335]Minimum Difficulty of a Job Schedule.java

```
1335
        Minimum Difficulty of a Job Schedule
        2023-02-21 12:05:20

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[][] memo;

    public int minDifficulty(int[] jobDifficulty, int d) {
        int n = jobDifficulty.length;

        if (n < d) {
            return -1;
        }

        memo = new int[n][d + 1];

        for (int i = 0; i < n; i++) {
            Arrays.fill(memo[i], -1);
        }

        return minDifficultyUtil(jobDifficulty, 0, d);
    }

    private int minDifficultyUtil(int[] jobDifficulty, int start, int d) {
        int n = jobDifficulty.length;

        if (d == 0 && start == n) {
            return 0;
        }

        if (d == 0 || start == n) {
            return Integer.MAX_VALUE;
        }

        if (memo[start][d] != -1) {
            return memo[start][d];
        }

        int maxDiff = 0;
        int minDifficulty = Integer.MAX_VALUE;

        for (int i = start; i < n; i++) {
            maxDiff = Math.max(maxDiff, jobDifficulty[i]);
            int temp = minDifficultyUtil(jobDifficulty, i + 1, d - 1);

            if (temp != Integer.MAX_VALUE) {
                minDifficulty = Math.min(minDifficulty, temp + maxDiff);
            }
        }
```

```
        memo[start][d] = minDifficulty;
        return minDifficulty;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

332. [1337]The K Weakest Rows in a Matrix.java

```
1337
        The K Weakest Rows in a Matrix
        2022-12-19 11:01:03

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] kWeakestRows(int[][] mat, int k) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> (a[1] != b[1] ? b[1] -
a[1] : b[0] - a[0]));
        int[] result = new int[k];

        for (int i = 0; i < mat.length; i++) {
            int count = getCount(mat[i]);

            pq.add(new int[]{i, count});

            if (pq.size() > k) {
                pq.poll();
            }
        }

        for (int i = k - 1; i >= 0; i--) {
            int[] curr = pq.poll();
            result[i] = curr[0];
        }

        return result;
    }

    private int getCount(int[] arr) {
        int left = 0;
        int right = arr.length - 1;

        while (left <= right) {
            int mid = (left + right) >> 1;

            if (arr[mid] == 1) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

333. [1338]Reduce Array Size to The Half.java

```
1338
        Reduce Array Size to The Half
        2022-12-11 01:50:03

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minSetSize(int[] arr) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int num : arr) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }
        int n = arr.length;
        int target = n / 2;
        int res = 0;
        int[] freq = new int[map.size()];
        int len = 0;
        for (int num : map.keySet()) {
            freq[len++] = map.get(num);
        }
        Arrays.sort(freq);
        int i = len - 1;
        while (n > target) {
            n -= freq[i];
            res++;
            i--;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

334. [1339]Maximum Product of Splitted Binary Tree.java

```
1339
        Maximum Product of Splitted Binary Tree
        2022-12-10 09:15:13
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    private List<Integer> list = new ArrayList<>();

    public int maxProduct(TreeNode root) {
        long totalSum = treeSum(root);
        long rec = 0;
        final int mod = 1_000_000_007;
        for (long sum : list) {
            rec = Math.max(rec, sum * (totalSum - sum));
        }
        return (int) (rec % mod);
    }

    private int treeSum(TreeNode node) {
        if (node == null)
            return 0;
        int left = treeSum(node.left);
        int right = treeSum(node.right);
        int total = node.val + left + right;
        list.add(total);
        return total;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

335. `[1342]Number of Steps to Reduce a Number to Zero.java`

```
1342
        Number of Steps to Reduce a Number to Zero
        2022-12-05 18:17:53

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfSteps(int num) {
        int count = 0;
        while (num != 0) {
            if (num % 2 == 0)
                num /= 2;
            else
                num -= 1;
            count++;
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

336. `[1345].java`

```
1345
        Jump Game IV
        2023-03-05 09:54:47

//leetcode submit region begin(Prohibit modification and deletion)
/*
還不會，但是DP不能用
 */
class Solution {
    public int minJumps(int[] arr) {
        int n = arr.length;
        if (n <= 1) {
            return 0;
        }
        Map<Integer, List<Integer>> map = new HashMap<>();
        for (int i = 0; i < n; i++) {
            map.putIfAbsent(arr[i], new ArrayList<>());
            map.get(arr[i]).add(i);
        }

        List<Integer> list = new LinkedList<>();
        list.add(0);
        Set<Integer> set = new HashSet<>();
        int step = 0;
        while (!list.isEmpty()) {
            List<Integer> next = new LinkedList<>();
            for (int node : list) {
                if (node == n - 1) {
                    return step;
                }
                for (int child : map.get(arr[node])) {
                    if (!set.contains(child)) {
                        set.add(child);
                        next.add(child);
                    }
                }
                map.get(arr[node]).clear();
                if (node + 1 < n && !set.contains(node + 1)) {
                    set.add(node + 1);
                    next.add(node + 1);
                }
                if (node - 1 >= 0 && !set.contains(node - 1)) {
                    set.add(node - 1);
                    next.add(node - 1);
                }
            }
            list = next;
            step++;
        }
        return -1;
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

337. [1345]Jump Game IV.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minJumps(int[] arr) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

338. [1346]Check If N and Its Double Exist.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean checkIfExist(int[] arr) {
        Set<Integer> seen = new HashSet<>();

        for (int x : arr) {
            if (seen.contains(2 * x)) {
                return true;
            }

            if (x % 2 == 0 && seen.contains(x / 2)) {
                return true;
            }

            seen.add(x);
        }

        return false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

339. `[1347]Minimum Number of Steps to Make Two Strings Anagram.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minSteps(String s, String t) {
        int[] countS = getCount(s);
        int[] countT = getCount(t);
        int count = 0;

        for (int i = 0; i < countS.length; i++) {
            if (countT[i] < countS[i]) {
                count += countS[i] - countT[i];
            }
        }

        return count;
    }

    private int[] getCount(String s) {
        int[] arr = new int[26];
        for (char c : s.toCharArray()) {
            arr[c - 'a']++;
        }

        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

340. `[1349]Maximum Students Taking Exam.java`

```
1349
        Maximum Students Taking Exam
        2023-01-27 14:15:56

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxStudents(char[][] seats) {
        int m = seats.length;
        int n = seats[0].length;
        int[] rows = new int[m];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                rows[i] = (rows[i] << 1) + (seats[i][j] == '.' ? 1 : 0);
            }
        }
        int stateSize = 1 << n;
        int[][] dp = new int[m][stateSize];
        for (int i = 0; i < m; i++) {
            Arrays.fill(dp[i], -1);
        }
        int ans = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < stateSize; j++) {
                if (((j & rows[i]) == j) && ((j & (j >> 1)) == 0)) {
                    if (i == 0) {
                        dp[i][j] = Integer.bitCount(j);
                    } else {
                        for (int k = 0; k < stateSize; k++) {
                            if ((j & (k >> 1)) == 0 && ((j >> 1) & k) == 0 && dp[i - 1]
[k] != -1) {
                                dp[i][j] = Math.max(dp[i][j], dp[i - 1][k] +
Integer.bitCount(j));
                            }
                        }
                    }
                    ans = Math.max(ans, dp[i][j]);
                }
            }
        }
        return ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

341. `[1351]Count Negative Numbers in a Sorted Matrix.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countNegatives(int[][] grid) {
        int count = 0;
        for (int[] arr : grid) {
            for (int num : arr) {
                if (num < 0) {
                    count++;
                }
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int countNegatives(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int i = m - 1;
        int j = 0;
        int count = 0;
        while (i >= 0 && j < n) {
            if (grid[i][j] < 0) {
                i--;
                count += n - j;
            } else {
                j++;
            }
        }
        return count;
    }
}
 */
```

342. `[1352]Product of the Last K Numbers.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class ProductOfNumbers {
  private List<Integer> prefix;

  public ProductOfNumbers() {
    prefix = new ArrayList<>();
    prefix.add(1);
  }

  public void add(int num) {
    if (num == 0) {
      prefix.clear();
      prefix.add(1);
    } else {
      int lastProduct = prefix.get(prefix.size() - 1);
      prefix.add(lastProduct * num);
    }
  }

  public int getProduct(int k) {
    int size = prefix.size();
    if (k >= size) {
      return 0;
    }
    return prefix.get(size - 1) / prefix.get(size - 1 - k);
  }
}

/**
 * Your ProductOfNumbers object will be instantiated and called as such: ProductOfNumbers
obj = new
 * ProductOfNumbers(); obj.add(num); int param_2 = obj.getProduct(k);
 */
// leetcode submit region end(Prohibit modification and deletion)
```

343. `[1353]Maximum Number of Events That Can Be Attended.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxEvents(int[][] events) {
        Arrays.sort(events, (a, b) -> a[0] - b[0]);
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        int n = events.length;
        int res = 0;
        int i = 0;
        int d = 0;
        while (i < n || !pq.isEmpty()) {
            if (pq.isEmpty()) {
                d = events[i][0];
            }
            while (i < n && events[i][0] == d) {
                pq.add(events[i++][1]);
            }
            pq.poll();
            res++;
            d++;
            while (!pq.isEmpty() && pq.peek() < d) {
                pq.poll();
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

344. [1358]Number of Substrings Containing All Three Characters.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfSubstrings(String s) {
        char[] c = s.toCharArray();
        return getCount(c, 3) - getCount(c, 2);
    }

    private int getCount(char[] c, int k) {
        int n = c.length;
        int[] arr = new int[3];
        int count = 0;
        int res = 0;
        for (int left = 0, right = 0; right < n; right++) {
            if (arr[c[right] - 'a'] == 0) {
                count++;
            }
            arr[c[right] - 'a']++;
            while (count > k) {
                arr[c[left] - 'a']--;
                if (arr[c[left] - 'a'] == 0) {
                    count--;
                }
                left++;
            }
            res += right - left + 1;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

345. `[1359]Count All Valid Pickup and Delivery`

Options.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countOrders(int n) {
        final int MOD = (int) 1e9 + 7;
        long res = 1;
        for (int i = 1; i <= n; i++) {
            res = (res * i * (2 * i - 1)) % MOD;
        }
        return (int) res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
- 12ms
class Solution {
    final int MOD = (int) 1e9 + 7;
    long[][] dp;

    public int countOrders(int n) {
        dp = new long[n + 1][n + 1];
        return (int) dyP(n, n);
    }

    private long dyP(int unP, int unD) {
        if (unD == 0 && unP == 0) {
            return 1;
        }
        if (unD < 0 || unP < 0 || unD < unP) {
            return 0;
        }
        if (dp[unP][unD] != 0) {
            return dp[unP][unD];
        }
        long res = 0;
        res += (unP * dyP(unP - 1, unD) + (unD - unP) * dyP(unP, unD - 1));
        res %= MOD;
        return dp[unP][unD] = res;
    }
}

- 15 ms
class Solution {
    public int countOrders(int n) {
        final int MOD = (int) 1e9 + 7;
        long[][] dp = new long[n + 1][n + 1];
        dp[0][0] = 1;
        for (int i = 0; i <= n; i++) {
            for (int j = i; j <= n; j++) {
                if (i == 0 && j == 0) {
                    dp[i][j] = 1;
                }
```

```
                if (i > 0) {
                    dp[i][j] += i * dp[i - 1][j];
                }
                dp[i][j] %= MOD;
                if (j > i) {
                    dp[i][j] += (j - i) * dp[i][j - 1];
                }
                dp[i][j] %= MOD;
            }
        }
        return (int) dp[n][n];
    }
}
 */
```

346. `[1361]Validate Binary Tree Nodes.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class UnionFind {
    private final int n;
    private final int[] group;
    public int count;

    UnionFind(int n) {
        this.n = n;
        group = new int[n];
        count = n;

        for (int i = 0; i < n; i++) {
            group[i] = i;
        }
    }

    public boolean union(int i, int j) {
        int x = find(i);
        int y = find(j);

        if (y != j || x == y) {
            return false;
        }

        count--;
        group[y] = x;
        return true;
    }

    private int find(int x) {
        if (group[x] != x) {
            group[x] = find(group[x]);
        }

        return group[x];
    }
}

class Solution {
    public boolean validateBinaryTreeNodes(int n, int[] leftChild, int[] rightChild) {
        UnionFind uf = new UnionFind(n);
        for (int node = 0; node < n; node++) {
            int[] children = {leftChild[node], rightChild[node]};
            for (int j : children) {
                if (j == -1) {
                    continue;
                }

                if (!uf.union(node, j)) {
```

```
                    return false;
                }
            }
        }

        return uf.count == 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
BFS

class Solution {
    public boolean validateBinaryTreeNodes(int n, int[] leftChild, int[] rightChild) {
        int[] inDegree = new int[n];
        int root = -1;
        boolean[] isVisited = new boolean[n];
        Deque<Integer> deque = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            if (leftChild[i] != -1) {
                inDegree[leftChild[i]]++;
            }

            if (rightChild[i] != -1) {
                inDegree[rightChild[i]]++;
            }
        }

        for (int i = 0; i < n; i++) {
            if (inDegree[i] == 0) {
                if (root == -1) {
                    root = i;
                } else {
                    return false;
                }
            }
        }

        if (root == -1) {
            return false;
        }

        deque.offer(root);

        while (!deque.isEmpty()) {
            int curr = deque.poll();

            if (isVisited[curr]) {
                return false;
            }

            isVisited[curr] = true;
```

```
            if (leftChild[curr] != -1) {
                deque.offer(leftChild[curr]);
            }

            if (rightChild[curr] != -1) {
                deque.offer(rightChild[curr]);
            }
        }

        int count = 0;

        for (boolean item : isVisited) {
            if (item) {
                count++;
            }
        }

        return count == n;
    }
}
 */
```

347. [1367]Linked List in Binary Tree.java

```
1367
        Linked List in Binary Tree
        2022-11-25 11:38:53
//leetcode submit region begin(Prohibit modification and deletion)
/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode() {}
 * ListNode(int val) { this.val = val; }
 * ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    public boolean isSubPath(ListNode head, TreeNode root) {
        if (head == null) {
            return true;
        }
        if (root == null) {
            return false;
        }
        if (dfs(head, root)) {
            return true;
        }
        return isSubPath(head, root.left) || isSubPath(head, root.right);
    }

    public boolean dfs(ListNode head, TreeNode root) {
        if (head == null) {
            return true;
        }
        if (root == null) {
            return false;
        }
```

```
        if (head.val != root.val) {
            return false;
        }
        return dfs(head.next, root.left) || dfs(head.next, root.right);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

348. `[1368]Minimum Cost to Make at Least One Valid Path`

in a Grid.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minCost(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int[][] dist = new int[m][n];
        for(int[] row: dist){
            Arrays.fill(row, Integer.MAX_VALUE);
        }
        dist[0][0] = 0;

        Deque<int[]> deque = new ArrayDeque<>();
        deque.offerFirst(new int[]{0, 0});

        int[][] dirs = { {0,1}, {0,-1}, {1,0}, {-1,0} };

        while(!deque.isEmpty()){
            int[] current = deque.pollFirst();
            int i = current[0];
            int j = current[1];

            int dir = grid[i][j];
            int[] d = dirs[dir - 1];
            int ni = i + d[0];
            int nj = j + d[1];

            if(ni >= 0 && ni < m && nj >= 0 && nj < n){
                if(dist[ni][nj] > dist[i][j]){
                    dist[ni][nj] = dist[i][j];
                    deque.offerFirst(new int[]{ni, nj});
                }
            }

            for(int k = 0; k < 4; k++){
                if(k == (dir - 1)) continue;
                int[] dk = dirs[k];
                int ci = i + dk[0];
                int cj = j + dk[1];
                if(ci >= 0 && ci < m && cj >= 0 && cj < n){
                    if(dist[ci][cj] > dist[i][j] + 1){
                        dist[ci][cj] = dist[i][j] + 1;
                        deque.offerLast(new int[]{ci, cj});
                    }
                }
            }
        }

        return dist[m-1][n-1];
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

349. [1371]Find the Longest Substring Containing Vowels in Even Counts.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private static final int[] VOWEL_MASK = new int[128];

    static {
        VOWEL_MASK['a'] = 1;
        VOWEL_MASK['e'] = 2;
        VOWEL_MASK['i'] = 4;
        VOWEL_MASK['o'] = 8;
        VOWEL_MASK['u'] = 16;
    }

    public int findTheLongestSubstring(String s) {
        int[] firstOccurrence = new int[32];
        Arrays.fill(firstOccurrence, -1);
        firstOccurrence[0] = 0;
        int maxLength = 0;
        int mask = 0;
        int n = s.length();

        for (int i = 0; i < n; i++) {
            char c = s.charAt(i);

            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
                mask ^= VOWEL_MASK[c];
            }

            if (firstOccurrence[mask] != -1) {
                maxLength = Math.max(maxLength, i + 1 - firstOccurrence[mask]);
            } else {
                firstOccurrence[mask] = i + 1;
            }
        }

        return maxLength;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

350. `[1372]Longest ZigZag Path in a Binary Tree.java`

```
/**
 * <p>You are given the <code>root</code> of a binary tree.</p>
 *
 * <p>A ZigZag path for a binary tree is defined as follow:</p>
 *
 * <ul>
 * <li>Choose <strong>any </strong>node in the binary tree and a direction (right or
left).</li>
 * <li>If the current direction is right, move to the right child of the current node;
otherwise, move to the left child.</li>
 * <li>Change the direction from right to left or from left to right.</li>
 * <li>Repeat the second and third steps until you can't move in the tree.</li>
 * </ul>
 *
 * <p>Zigzag length is defined as the number of nodes visited - 1. (A single node has a
length of 0).</p>
 *
 * <p>Return <em>the longest <strong>ZigZag</strong> path contained in that tree</em>.
</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/01/22/sample_1_1702.png"
style="width: 221px; height: 383px;" />
 * <pre>
 * <strong>Input:</strong> root =
[1,null,1,1,1,null,null,1,1,null,1,null,null,null,1,null,1]
 * <strong>Output:</strong> 3
 * <strong>Explanation:</strong> Longest ZigZag path in blue nodes (right -&gt; left -
&gt; right).
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/01/22/sample_2_1702.png"
style="width: 157px; height: 329px;" />
 * <pre>
 * <strong>Input:</strong> root = [1,1,1,null,1,null,null,1,1,null,1]
 * <strong>Output:</strong> 4
 * <strong>Explanation:</strong> Longest ZigZag path in blue nodes (left -&gt; right -
&gt; left -&gt; right).
 * </pre>
 *
 * <p><strong class="example">Example 3:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> root = [1]
 * <strong>Output:</strong> 0
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
```

```
 *
 * <ul>
 * <li>The number of nodes in the tree is in the range <code>[1, 5 * 10<sup>4</sup>]
</code>.</li>
 * <li><code>1 &lt;= Node.val &lt;= 100</code></li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>Dynamic Programming</li><li>Tree</li><li>Depth-
First Search</li><li>Binary Tree</li></div></div><br><div><li>👍 1500</li><li>👎 26</li>
</div>
 */

//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    int max = 0;

    public int longestZigZag(TreeNode root) {
        dfs(root.left, true, 1);
        dfs(root.right, false, 1);
        return max;
    }

    public void dfs(TreeNode node, boolean goLeft, int level) {
        if (node == null) {
            return;
        }
        if (level > max) {
            max = level;
        }
        if (goLeft) {
            dfs(node.left, true, 1);
            dfs(node.right, false, level + 1);
        } else {
            dfs(node.left, true, level + 1);
            dfs(node.right, false, 1);
        }
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

351. [1373]Maximum Sum BST in Binary Tree.java

```
1373
Maximum Sum BST in Binary Tree
2023-02-17 11:51:58
//leetcode submit region begin(Prohibit modification and deletion)
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int maxSumBST(TreeNode root) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

352. [1376]Time Needed to Inform All Employees.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numOfMinutes(int n, int headID, int[] manager, int[] informTime) {
        int res = 0;
        for (int i = 0; i < n; i++) {
            res = Math.max(res, dfs(i, manager, informTime));
        }
        return res;
    }

    public int dfs(int curr, int[] manager, int[] informTime) {
        if (manager[curr] != -1) {
            informTime[curr] += dfs(manager[curr], manager, informTime);
            manager[curr] = -1;
        }
        return informTime[curr];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int numOfMinutes(int n, int headID, int[] manager, int[] informTime) {
        int res = 0;
        for (int i = 0; i < n; i++) {
            res = Math.max(res, dfs(i, manager, informTime));
        }
        return res;
    }

    public int dfs(int i, int[] manager, int[] informTime) {
        if (manager[i] != -1) {
            informTime[i] += dfs(manager[i], manager, informTime);
            manager[i] = -1;
        }
        return informTime[i];
    }
}


DFS:
class Solution {
    int res = 0;

    public int numOfMinutes(int n, int headID, int[] manager, int[] informTime) {
        List<List<Integer>> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (int i = 0; i < n; i++) {
            if (manager[i] != -1) {
```

```
                list.get(manager[i]).add(i);
            }
        }
        dfs(list, informTime, headID, 0);
        return res;
    }

    public void dfs(List<List<Integer>> list, int[] informTime, int curr, int time) {
        res = Math.max(res, time);
        for (int num : list.get(curr)) {
            dfs(list, informTime, num, time + informTime[curr]);
        }
    }
}


BFS:
class Solution {
    public int numOfMinutes(int n, int headID, int[] manager, int[] informTime) {
        int[] arr = new int[n];
        List<List<Integer>> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (int i = 0; i < n; i++) {
            if (manager[i] != -1) {
                list.get(manager[i]).add(i);
            }
        }
        int res = 0;
        Deque<Integer> dq = new ArrayDeque<>();
        dq.add(headID);
        while (!dq.isEmpty()) {
            int curr = dq.poll();
            for (int num : list.get(curr)) {
                arr[num] = arr[curr] + informTime[curr];
                res = Math.max(res, arr[num]);
                dq.add(num);
            }
        }
        return res;
    }
}
 */
```

353. [1380]Lucky Numbers in a Matrix.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> luckyNumbers(int[][] matrix) {
        List<Integer> result = new ArrayList<>();

        for (int i = 0; i < matrix.length; i++) {
            int col = findMin(matrix, i);
            int maxCol = matrix[i][col];

            if (maxInCol(matrix, maxCol, col)) {
                result.add(maxCol);
            }
        }

        return result;
    }

    private int findMin(int[][] matrix, int row) {
        int val = matrix[row][0];
        int col = 0;

        for (int i = 1; i < matrix[0].length; i++) {
            if (matrix[row][i] < val) {
                val = matrix[row][i];
                col = i;
            }
        }

        return col;
    }

    private boolean maxInCol(int[][] matrix, int val, int col) {
        for (int i = 0; i < matrix.length; i++) {
            if (matrix[i][col] > val) {
                return false;
            }
        }

        return true;
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

354. [1381]Design a Stack With Increment Operation.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class CustomStack {
    private int[] stack;
    private int[] inc;
    private int maxSize;
    private int stackPointer;

    public CustomStack(int maxSize) {
        this.maxSize = maxSize;
        this.stack = new int[maxSize];
        this.inc = new int[maxSize];
        this.stackPointer = 0;
    }

    public void push(int x) {
        if (stackPointer < maxSize) {
            stack[stackPointer] = x;
            inc[stackPointer] = 0;
            stackPointer++;
        }
    }

    public int pop() {
        if (stackPointer == 0) {
            return -1;
        }

        stackPointer--;

        if (stackPointer > 0) {
            inc[stackPointer - 1] += inc[stackPointer];
        }

        int res = stack[stackPointer] + inc[stackPointer];
        inc[stackPointer] = 0;
        return res;
    }

    public void increment(int k, int val) {
        int idx = Math.min(k, stackPointer) - 1;

        if (idx >= 0) {
            inc[idx] += val;
        }
    }
}

/**
 * Your CustomStack object will be instantiated and called as such:
 * CustomStack obj = new CustomStack(maxSize);
```

```
 * obj.push(x);
 * int param_2 = obj.pop();
 * obj.increment(k,val);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

355. `[1382]Balance a Binary Search Tree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public TreeNode balanceBST(TreeNode root) {
        List<Integer> sortedNode = new ArrayList<>();
        inOrder(root, sortedNode);
        return buildBalancedBST(sortedNode, 0, sortedNode.size() - 1);
    }

    private void inOrder(TreeNode node, List<Integer> list) {
        if (node == null) {
            return;
        }

        inOrder(node.left, list);
        list.add(node.val);
        inOrder(node.right, list);
    }

    private TreeNode buildBalancedBST(List<Integer> list, int left, int right) {
        if (left > right) {
            return null;
        }
        int mid = left + (right - left) / 2;
        TreeNode node = new TreeNode(list.get(mid));
        node.left = buildBalancedBST(list, left, mid - 1);
        node.right = buildBalancedBST(list, mid + 1, right);
        return node;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

356. [1383]Maximum Performance of a Team.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxPerformance(int n, int[] speed, int[] efficiency, int k) {
        int[][] arrs = new int[n][2];
        for (int i = 0; i < n; i++) {
            arrs[i] = new int[]{speed[i], efficiency[i]};
        }
        Arrays.sort(arrs, (a, b) -> b[1] - a[1]);
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        long MOD = (long) 1e9 + 7;
        long res = 0;
        long sum = 0;
        for (int[] arr : arrs) {
            pq.add(arr[0]);
            sum += arr[0];
            if (pq.size() > k) {
                sum -= pq.poll();
            }
            res = Math.max(res, sum * arr[1]);
        }
        return (int) (res % MOD);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

357. [1386]Cinema Seat Allocation.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxNumberOfFamilies(int n, int[][] reservedSeats) {
        Map<Integer, List<Integer>> map = new HashMap<>();
        for (int[] arr : reservedSeats) {
            if (!map.containsKey(arr[0])) {
                map.put(arr[0], new ArrayList<>());
            }
            map.get(arr[0]).add(arr[1]);
        }
        int res = 2 * (n - map.size());
        for (Integer key : map.keySet()) {
            List<Integer> list = map.get(key);
            boolean isLeft = true;
            boolean isMiddle = true;
            boolean isRight = true;
            for (int col : list) {
                if (col >= 2 && col <= 5) {
                    isLeft = false;
                }
                if (col >= 4 && col <= 7) {
                    isMiddle = false;
                }
                if (col >= 6 && col <= 9) {
                    isRight = false;
                }
                if (!isLeft && !isMiddle && !isRight) {
                    break;
                }
            }
            res += Math.max((isLeft ? 1 : 0) + (isRight ? 1 : 0), (isMiddle ? 1 : 0));
        }
        return res;
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

358. `[1394]Find Lucky Integer in an Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findLucky(int[] arr) {
        Map<Integer, Integer> map = new HashMap<>();

        for (int num : arr) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        int max = -1;

        for (Map.Entry<Integer,Integer> entry : map.entrySet()) {
            if (entry.getKey().equals(entry.getValue())) {
                max = Math.max(max,entry.getKey());
            }
        }

        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

359. `[1395]Count Number of Teams.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numTeams(int[] rating) {
        int n = rating.length;
        int count = 0;

        for (int j = 1; j < n - 1; j++) {
            int leftSmaller = 0;
            int leftLarger = 0;
            int rightSmaller = 0;
            int rightLarger = 0;

            for (int i = 0; i < j; i++) {
                if (rating[i] < rating[j]) {
                    leftSmaller++;
                } else if (rating[i] > rating[j]) {
                    leftLarger++;
                }
            }

            for (int k = j + 1; k < n; k++) {
                if (rating[k] < rating[j]) {
                    rightSmaller++;
                } else if (rating[k] > rating[j]) {
                    rightLarger++;
                }
            }

            count += leftSmaller * rightLarger + leftLarger * rightSmaller;
        }

        return count;
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

360. `[1396]Design Underground System.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class UndergroundSystem {
    Map<Integer, Pair<String, Integer>> custs;
    Map<String, int[]> dists;

    public UndergroundSystem() {
        custs = new HashMap<>();
        dists = new HashMap<>();
    }

    public void checkIn(int id, String stationName, int t) {
        custs.put(id, new Pair(stationName, t));
    }

    public void checkOut(int id, String stationName, int t) {
        Pair<String, Integer> cust = custs.get(id);
        dists.putIfAbsent(cust.getKey() + "-" + stationName, new int[2]);
        int[] sum = dists.get(cust.getKey() + "-" + stationName);
        sum[0] += t - cust.getValue();
        sum[1]++;
        custs.remove(id);
    }

    public double getAverageTime(String startStation, String endStation) {
        int[] sum = dists.get(startStation + "-" + endStation);
        return (double) sum[0] / sum[1];
    }
}

/**
 * Your UndergroundSystem object will be instantiated and called as such:
 * UndergroundSystem obj = new UndergroundSystem();
 * obj.checkIn(id,stationName,t);
 * obj.checkOut(id,stationName,t);
 * double param_3 = obj.getAverageTime(startStation,endStation);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

361. `[1400]Construct K Palindrome Strings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canConstruct(String s, int k) {
        if (k > s.length()) {
            return false;
        }

        HashMap<Character, Integer> charCount = new HashMap<>();
        for (char c : s.toCharArray()) {
            charCount.put(c, charCount.getOrDefault(c, 0) + 1);
        }

        int oddCount = 0;
        for (int count : charCount.values()) {
            if (count % 2 != 0) {
                oddCount++;
            }
        }

        return oddCount <= k;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

362. `[1402]Reducing Dishes.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxSatisfaction(int[] satisfaction) {
        Arrays.sort(satisfaction);
        int res = 0;
        int suf = 0;
        int t = 0;
        for (int i = satisfaction.length - 1; i >= 0; i--) {
            suf += satisfaction[i];
            t += suf;
            res = Math.max(res, t);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

363. `[1403]Minimum Subsequence in Non-Increasing Order.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> minSubsequence(int[] nums) {
        int n = nums.length;
        int sum = 0;
        int prefixSum = 0;
        List<Integer> list = new ArrayList<>();

        Arrays.sort(nums);

        for (int i = 0; i < n; i++) {
            prefixSum += nums[i];
        }

        while (sum <= prefixSum) {
            sum += nums[n - 1];
            prefixSum-=nums[n-1];
            list.add(nums[n - 1]);
            n--;
        }

        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

364. `[1404]Number of Steps to Reduce a Number in Binary Representation to One.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numSteps(String s) {
        int steps = 0;
        int carry = 0;
        int n = s.length();

        for (int i = n - 1; i > 0; i--) {
            int bit = s.charAt(i) - '0';

            if (bit == 0 && carry == 0) {
                steps++;
            } else if (bit == 1 && carry == 1) {
                steps++;
                carry = 1;
            } else {
                steps += 2;
                carry = 1;
            }
        }

        if (carry == 1) {
            steps++;
        }

        return steps;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

365. `[1405]Longest Happy String.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String longestDiverseString(int a, int b, int c) {
        PriorityQueue<Element> pq = new PriorityQueue<>((e1, e2) -> e2.count - e1.count);

        if (a > 0) {
            pq.offer(new Element('a', a));
        }

        if (b > 0) {
            pq.offer(new Element('b', b));
        }

        if (c > 0) {
            pq.offer(new Element('c', c));
        }

        StringBuilder result = new StringBuilder();

        while (!pq.isEmpty()) {
            Element first = pq.poll();
            int len = result.length();

            if (len >= 2 && result.charAt(len - 1) == first.ch && result.charAt(len - 2)
== first.ch) {
                if (pq.isEmpty()) {
                    break;
                }

                Element second = pq.poll();
                result.append(second.ch);
                second.count--;

                if (second.count > 0) {
                    pq.offer(second);
                }

                pq.offer(first);
            } else {
                result.append(first.ch);
                first.count--;

                if (first.count > 0) {
                    pq.offer(first);
                }
            }
        }

        return result.toString();
    }
```

```
    private static class Element {
        char ch;
        int count;

        Element(char ch, int count) {
            this.ch = ch;
            this.count = count;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

366. `[1406]Stone Game III.java`

```

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String stoneGameIII(int[] stoneValue) {
        int n = stoneValue.length;
        int[] dp = new int[n + 1];
        for (int i = n - 1; i >= 0; --i) {
            dp[i] = Integer.MIN_VALUE;
            for (int k = 0, take = 0; k < 3 && i + k < n; k++) {
                take += stoneValue[i + k];
                dp[i] = Math.max(dp[i], take - dp[i + k + 1]);
            }
        }
        if (dp[0] == 0) {
            return "Tie";
        }
        return dp[0] > 0 ? "Alice" : "Bob";
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

367. [1415]The k-th Lexicographical String of All Happy Strings of Length n.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  private int count = 0;
  private String ans = "";

  public String getHappyString(int n, int k) {
    backtrack(n, k, new StringBuilder());
    return ans;
  }

  private void backtrack(int n, int k, StringBuilder sb) {
    if (sb.length() == n) {
      count++;

      if (count == k) {
        ans = sb.toString();
      }

      return;
    }

    for (char c : new char[] {'a', 'b', 'c'}) {
      if (sb.length() == 0 || sb.charAt(sb.length() - 1) != c) {
        sb.append(c);
        backtrack(n, k, sb);

        if (!ans.isEmpty()) {
          return;
        }

        sb.deleteCharAt(sb.length() - 1);
      }
    }
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

368. [1416]Restore The Array.java

```
/**
 * <p>A program was supposed to print an array of integers. The program forgot to print
whitespaces and the array is printed as a string of digits <code>s</code> and all we know
is that all integers in the array were in the range <code>[1, k]</code> and there are no
leading zeros in the array.</p>
 *
 * <p>Given the string <code>s</code> and the integer <code>k</code>, return <em>the
number of the possible arrays that can be printed as </em><code>s</code><em> using the
mentioned program</em>. Since the answer may be very large, return it
<strong>modulo</strong> <code>10<sup>9</sup> + 7</code>.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> s = "1000", k = 10000
 * <strong>Output:</strong> 1
 * <strong>Explanation:</strong> The only possible array is [1000]
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> s = "1000", k = 10
 * <strong>Output:</strong> 0
 * <strong>Explanation:</strong> There cannot be an array that was printed this way and
has all integer &gt;= 1 and &lt;= 10.
 * </pre>
 *
 * <p><strong class="example">Example 3:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> s = "1317", k = 2000
 * <strong>Output:</strong> 8
 * <strong>Explanation:</strong> Possible arrays are [1317],[131,7],[13,17],[1,317],
[13,1,7],[1,31,7],[1,3,17],[1,3,1,7]
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>1 &lt;= s.length &lt;= 10<sup>5</sup></code></li>
 * <li><code>s</code> consists of only digits and does not contain leading zeros.</li>
 * <li><code>1 &lt;= k &lt;= 10<sup>9</sup></code></li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>String</li><li>Dynamic Programming</li></div>
</div><br><div><li>👍 482</li><li>👎 21</li></div>
 */
```

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int mod = (int) 1e9 + 7;

    public int numberOfArrays(String s, int k) {
        char[] c = s.toCharArray();
        int n = c.length;
        int[] dp = new int[n + 1];
        dp[n] = 1;
        for (int i = n - 1; i >= 0; i--) {
            if (c[i] == '0') {
                dp[i] = 0;
                continue;
            }
            long count = 0;
            for (int j = i; j < n; j++) {
                count = count * 10 + (c[j] - '0');
                if (count > k) {
                    break;
                }
                dp[i] = (dp[i] + dp[j + 1]) % mod;
            }
        }
        return dp[0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

369. [1420]Build Array Where You Can Find The Maximum

Exactly K Comparisons.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    Integer[][][] memo;
    final int MOD = (int) 1e9 + 7;

    public int numOfArrays(int n, int m, int k) {
        memo = new Integer[n + 1][k + 1][m + 1];
        return dp(n, m, k, 0);
    }

    private int dp(int n, int m, int k, int max) {
        if (k < 0 || n < k) {
            return 0;
        }

        if (n == 0) {
            return 1;
        }

        if (memo[n][k][max] != null) {
            return memo[n][k][max];
        }

        int sum = 0;

        for (int i = 1; i <= m; i++) {
            sum += dp(n - 1, m, i > max ? k - 1 : k, Math.max(i, max));
            sum %= MOD;
        }
        memo[n][k][max] = sum;
        return sum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int numOfArrays(int n, int m, int k) {
        long[][][] dp = new long[n][k][m];
        final int MOD = (int)1e9+7;
        long result = 0;

        Arrays.fill(dp[0][0], 1);

        for (int i = 1; i < n; i++) {
            for (int cost = 0; cost < Math.min(i + 1, k); cost++) {
                for (int max = 0; max < m; max++) {
                    long sum = 0;
                    sum += dp[i - 1][cost][max] * (max + 1);

                    if (cost != 0) {
```

```
                        long[] arr = dp[i - 1][cost - 1];
                        for (int prevMax = 0; prevMax < max; prevMax++) {
                            sum += arr[prevMax];
                        }
                    }

                    dp[i][cost][max] = sum % MOD;
                }
            }
        }

        for (int max = 0; max < m; max++) {
            result += dp[n - 1][k - 1][max];
            result %= MOD;
        }
        return (int) result;
    }
}
 */
```

370. [1422]Maximum Score After Splitting a String.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxScore(String s) {
        int zeros = 0;
        int ones = 0;
        int maxScore = 0;

        for (char c : s.toCharArray()) {
            if (c == '1') {
                ones++;
            }
        }

        for (int i = 0; i < s.length() - 1; i++) {
            if (s.charAt(i) == '0') {
                zeros++;
            } else {
                ones--;
            }

            maxScore = Math.max(maxScore, zeros + ones);
        }

        return maxScore;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

371. `[1423]Maximum Points You Can Obtain from Cards.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int maxScore(int[] cardPoints, int k) {
    int n = cardPoints.length;
    int total = 0;

    for (int point : cardPoints) {
      total += point;
    }

    if (k == n) {
      return total;
    }

    int windowSize = n - k;
    int left = 0;
    int currSum = 0;
    int maxSum = 0;

    for (int right = 0; right < n; right++) {
      currSum += cardPoints[right];

      if (right - left + 1 == n - k) {
        maxSum = Math.max(maxSum, total - currSum);
        currSum -= cardPoints[left];
        left++;
      }
    }

    return maxSum;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

372. `[1424]Diagonal Traverse II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] findDiagonalOrder(List<List<Integer>> nums) {
        Deque<int[]> deque = new ArrayDeque<>();
        deque.offer(new int[]{0, 0});
        List<Integer> list = new ArrayList();

        while (!deque.isEmpty()) {
            int[] curr = deque.poll();
            int row = curr[0];
            int col = curr[1];
            list.add(nums.get(row).get(col));

            if (col == 0 && row + 1 < nums.size()) {
                deque.offer(new int[]{row + 1, col});
            }

            if (col + 1 < nums.get(row).size()) {
                deque.offer(new int[]{row, col + 1});
            }
        }

        int[] result = new int[list.size()];

        for (int i = 0; i < list.size(); i++) {
            result[i] = list.get(i);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
BFS - 19 ms
class Solution {
    public int[] findDiagonalOrder(List<List<Integer>> nums) {
        Deque<int[]> deque = new ArrayDeque<>();
        deque.offer(new int[]{0, 0});
        List<Integer> list = new ArrayList();

        while (!deque.isEmpty()) {
            int[] curr = deque.poll();
            int row = curr[0];
            int col = curr[1];
            list.add(nums.get(row).get(col));

            if (col == 0 && row + 1 < nums.size()) {
                deque.offer(new int[]{row + 1, col});
            }
```

```
            if (col + 1 < nums.get(row).size()) {
                deque.offer(new int[]{row, col + 1});
            }
        }

        int[] result = new int[list.size()];

        for (int i = 0; i < list.size(); i++) {
            result[i] = list.get(i);
        }

        return result;
    }
}

HashMap - 34 ms
class Solution {
    public int[] findDiagonalOrder(List<List<Integer>> nums) {
        Map<Integer, List<Integer>> map = new HashMap<>();
        int size = 0;

        for (int i = 0; i < nums.size(); i++) {
            for (int j = 0; j < nums.get(i).size(); j++) {
                map.putIfAbsent(i + j, new ArrayList<>());
                map.get(i + j).add(nums.get(i).get(j));
                size++;
            }
        }

        int[] result = new int[size];
        int index = 0;
        int curr = 0;

        while (map.containsKey(curr)) {
            List<Integer> list = map.get(curr);

            for (int i = list.size() - 1; i >= 0; i--) {
                result[index++] = list.get(i);
            }

            curr++;
        }

        return result;
    }
}
 */
```

373. `[1425]Constrained Subsequence Sum.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int constrainedSubsetSum(int[] nums, int k) {
        int result = nums[0];
        Deque<Integer> deque = new ArrayDeque<>();

        for (int i = 0; i < nums.length; i++) {
            nums[i] += !deque.isEmpty() ? deque.peek() : 0;
            result = Math.max(result, nums[i]);

            while (!deque.isEmpty() && nums[i] > deque.peekLast()) {
                deque.pollLast();
            }

            if (nums[i] > 0) {
                deque.offer(nums[i]);
            }

            if (i >= k && !deque.isEmpty() && deque.peek() == nums[i - k]) {
                deque.poll();
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int constrainedSubsetSum(int[] nums, int k) {
        int n = nums.length;
        Deque<Integer> deque = new ArrayDeque<>();
        int[] dp = new int[n];
        int result = Integer.MIN_VALUE;

        for (int i = 0; i < n; i++) {
            if (!deque.isEmpty() && i - deque.peek() > k) {
                deque.poll();
            }

            dp[i] = (!deque.isEmpty() ? dp[deque.peek()] : 0) + nums[i];

            while (!deque.isEmpty() && dp[deque.peekLast()] < dp[i]) {
                deque.pollLast();
            }

            if (dp[i] > 0) {
                deque.offer(i);
            }
        }
```

```
        for (int num : dp) {
            result = Math.max(result, num);
        }

        return result;
    }
}
class Solution {
    public int constrainedSubsetSum(int[] nums, int k) {
        PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) -> b[0] - a[0]);
        int result = nums[0];

        heap.add(new int[]{nums[0], 0});

        for (int i = 1; i < nums.length; i++) {
            while (i - heap.peek()[1] > k) {
                heap.remove();
            }

            int curr = Math.max(0, heap.peek()[0]) + nums[i];
            result = Math.max(result, curr);
            heap.add(new int[]{curr, i});
        }

        return result;
    }
}
 */
```

374. [1426]Counting Elements.java

```
1426
        Counting Elements
        2022-11-26 00:55:50

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countElements(int[] arr) {
        Set<Integer> set = new HashSet<>();
        int ans = 0;
        for (int num : arr) {
            set.add(num);
        }
        for (int i = 0; i < arr.length; i++) {
            if (set.contains(arr[i] + 1))
                ans++;
        }
        return ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

375. [1428]Leftmost Column with at Least a One.java

```


//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int leftMostColumnWithOne(BinaryMatrix binaryMatrix) {
        List<Integer> dimensions = binaryMatrix.dimensions();
        int row = dimensions.get(0);
        int col = dimensions.get(1);
        int currRow = 0;
        int currCol = col - 1;
        int result = -1;

        while (currRow < row && currCol >= 0) {
            if (binaryMatrix.get(currRow, currCol) == 1) {
                result = currCol;
                currCol--;
            } else {
                currRow++;
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

376. `[1431]Kids With the Greatest Number of Candies.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Boolean> kidsWithCandies(int[] candies, int extraCandies) {
        int max = 0;
        for (int num : candies) {
            max = Math.max(max, num);
        }
        List<Boolean> list = new ArrayList<>();
        max -= extraCandies;
        for (int num : candies) {
            list.add(num >= max);
        }
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

377. [1432]Max Difference You Can Get From Changing an

Integer.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxDiff(int num) {
        String original = Integer.toString(num);
        String maxStr = buildMaxString(original);
        String minStr = buildMinString(original);
        int maxValue = Integer.parseInt(maxStr);
        int minValue = Integer.parseInt(minStr);
        return maxValue - minValue;
    }

    private String buildMaxString(String s) {
        char[] chars = s.toCharArray();
        char targetChar = 0;
        for (char c : chars) {
            if (c != '9') {
                targetChar = c;
                break;
            }
        }
        if (targetChar == 0) {
            return s;
        }
        for (int i = 0; i < chars.length; i++) {
            if (chars[i] == targetChar) {
                chars[i] = '9';
            }
        }
        return new String(chars);
    }

    private String buildMinString(String s) {
        char[] chars = s.toCharArray();
        char firstChar = chars[0];
        if (firstChar != '1') {
            for (int i = 0; i < chars.length; i++) {
                if (chars[i] == firstChar) {
                    chars[i] = '1';
                }
            }
            return new String(chars);
        }
        char targetChar = 0;
        for (int i = 1; i < chars.length; i++) {
            char c = chars[i];
            if (c != '0' && c != '1') {
                targetChar = c;
                break;
            }
```

```
        }
        if (targetChar == 0) {
            return s;
        }
        for (int i = 1; i < chars.length; i++) {
            if (chars[i] == targetChar) {
                chars[i] = '0';
            }
        }
        return new String(chars);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

378. [1434]Number of Ways to Wear Different Hats to Each Other.java

```
// Java
class Solution {
    static final int MOD = 1_000_000_007;

    public int numberWays(List<List<Integer>> hats) {
        int n = hats.size();
        List<Integer>[] hatToPeople = new ArrayList[41];
        for (int i = 1; i <= 40; i++) hatToPeople[i] = new ArrayList<>();
        for (int person = 0; person < n; person++) {
            for (int hat : hats.get(person)) {
                hatToPeople[hat].add(person);
            }
        }

        int[][] dp = new int[41][1 << n];
        dp[0][0] = 1;

        for (int hat = 1; hat <= 40; hat++) {
            for (int mask = 0; mask < (1 << n); mask++) {
                dp[hat][mask] = dp[hat - 1][mask];

                for (int person : hatToPeople[hat]) {
                    if ((mask & (1 << person)) != 0) {
                        dp[hat][mask] = (dp[hat][mask] + dp[hat - 1][mask ^ (1 <<
person)]) % MOD;
                    }
                }
            }
        }
        return dp[40][(1 << n) - 1];
    }
}
```

379. `[1436]Destination City.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String destCity(List<List<String>> paths) {
        Set<String> set = new HashSet<>();

        for (List<String> name : paths) {
            set.add(name.get(0));
        }

        for (List<String> name : paths) {
            if (!set.contains(name.get(1))) {
                return name.get(1);
            }
        }

        return "";
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

380. `[1438]Longest Continuous Subarray With Absolute`

Diff Less Than or Equal to Limit.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestSubarray(int[] nums, int limit) {
        Deque<Integer> maxDeque = new ArrayDeque<>();
        Deque<Integer> minDeque = new ArrayDeque<>();
        int left = 0;
        int result = 0;
        int n = nums.length;

        for (int right = 0; right < n; right++) {
            while (!maxDeque.isEmpty() && nums[maxDeque.peekLast()] <= nums[right]) {
                maxDeque.pollLast();
            }

            maxDeque.offerLast(right);

            while (!minDeque.isEmpty() && nums[minDeque.peekLast()] >= nums[right]) {
                minDeque.pollLast();
            }

            minDeque.offerLast(right);

            while (nums[maxDeque.peekFirst()] - nums[minDeque.peekFirst()] > limit) {
                left++;

                if (maxDeque.peekFirst() < left) {
                    maxDeque.pollFirst();
                }

                if (minDeque.peekFirst() < left) {
                    minDeque.pollFirst();
                }
            }

            result = Math.max(result, right - left + 1);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int longestSubarray(int[] nums, int limit) {
        TreeMap<Integer, Integer> map = new TreeMap<>();
        int left = 0;
        int right;
        int n = nums.length;

        for (right = 0; right < n; right++) {
            map.put(nums[right], map.getOrDefault(nums[right], 0) + 1);
```

```
            if (map.lastEntry().getKey() - map.firstEntry().getKey() > limit) {
                map.put(nums[left], map.get(nums[left]) - 1);

                if (map.get(nums[left]) == 0) {
                    map.remove(nums[left]);
                }

                left++;
            }
        }

        return right - left;
    }
}
 */
```

381. [1439]Find the Kth Smallest Sum of a Matrix With Sorted Rows.java

```
1439
        Find the Kth Smallest Sum of a Matrix With Sorted Rows
        2022-12-19 11:42:42

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private int count;
    private int m;
    private int n;

    public int kthSmallest(int[][] mat, int k) {
        this.m = mat.length;
        this.n = mat[0].length;
        int left = 0;
        int right = 0;
        for (int i = 0; i < this.m; i++) {
            left += mat[i][0];
            right += mat[i][this.n - 1];
        }
        int base = left;
        while (left < right) {
            int mid = (left + right) >> 1;
            this.count = 0;
            countLessMiddle(mat, k, mid, base, 0);
            if (this.count < k) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        return left;
    }

    private void countLessMiddle(int[][] mat, int k, int mid, int base, int row) {
        if (row == this.m) {
            this.count++;
            return;
        }
        for (int i = 0; i < this.n; i++) {
            int currSum = base + mat[row][i] - mat[row][0];
            if (currSum > mid || this.count >= k) {
                break;
            }
            countLessMiddle(mat, k, mid, currSum, row + 1);
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

382. [1441]Build an Array With Stack Operations.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<String> buildArray(int[] target, int n) {
        List<String> list = new ArrayList<>();
        int index = 0;
        int count = 0;

        while (index < target.length && count++ < n) {
            list.add("Push");

            if (target[index] == count) {
                index++;
            } else {
                list.add("Pop");
            }
        }

        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

383. [1442]Count Triplets That Can Form Two Arrays of Equal XOR.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countTriplets(int[] arr) {
        int n = arr.length;
        int[] prefix = new int[n + 1];
        int count = 0;

        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] ^ arr[i];
        }

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (prefix[i] == prefix[j + 1]) {
                    count += (j - i);
                }
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int countTriplets(int[] arr) {
        int n = arr.length;
        int[] prefix = new int[n + 1];
        int count = 0;

        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] ^ arr[i];
        }

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (prefix[i] == prefix[j + 1]) {
                    count += (j - i);
                }
            }
        }

        return count;
    }
}
 */
```

384. `[1443]Minimum Time to Collect All Apples in a Tree.java`

```
1443
        Minimum Time to Collect All Apples in a Tree
        2023-01-11 09:27:18

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minTime(int n, int[][] edges, List<Boolean> hasApple) {
        List<List<Integer>> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (int[] edge : edges) {
            int x = edge[0];
            int y = edge[1];
            list.get(x).add(y);
            list.get(y).add(x);
        }
        boolean[] isVisited = new boolean[n];
        return dfs(0, list, hasApple, isVisited);
    }

    private int dfs(int node, List<List<Integer>> list, List<Boolean> hasApple, boolean[]
isVisited) {
        isVisited[node] = true;
        int result = 0;
        for (int child : list.get(node)) {
            if (!isVisited[child]) {
                result += dfs(child, list, hasApple, isVisited);
            }
        }
        if ((result > 0 || hasApple.get(node)) && node != 0) {
            result += 2;
        }
        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

385. `[1444].java`

```
1444
        Number of count of Cutting a Pizza
        2023-03-06 15:01:20

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int ways(String[] pizza, int k) {
        int m = pizza.length;
        int n = pizza[0].length();
        int[][] arr = new int[m + 1][n + 1];
        long[][][] dp = new long[k][m][n];
        int MOD = (int) 1e9 + 7;
        for (int i = m - 1; i >= 0; i--) {
            for (int j = n - 1; j >= 0; j--) {
                arr[i][j] = arr[i + 1][j] + arr[i][j + 1] - arr[i + 1][j + 1] +
(pizza[i].charAt(j) == 'A' ? 1 : 0);
                dp[0][i][j] = arr[i][j] > 0 ? 1 : 0;
            }
        }
        for (int remain = 1; remain < k; remain++) {
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < m; j++) {
                    for (int nextI = i + 1; nextI < m; nextI++) {
                        if (arr[i][j] - arr[nextI][j] > 0) {
                            dp[remain][i][j] += dp[remain - 1][nextI][j];
                        }
                    }
                    for (int nextJ = j + 1; nextJ < n; nextJ++) {
                        if (arr[i][j] - arr[i][nextJ] > 0) {
                            dp[remain][i][j] += dp[remain - 1][i][nextJ];
                        }
                    }
                }
            }
        }
        return (int) (dp[k - 1][0][0] % MOD);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

386. `[1446]Consecutive Characters.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxPower(String s) {
        char[] c = s.toCharArray();
        int count = 1;
        int max = 0;

        for (int i = 1; i < c.length; i++) {
            max = Math.max(max, count);

            if (c[i] == c[i - 1]) {
                count++;
            } else {
                count = 1;
            }
        }

        return Math.max(max, count);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

387. `[1448]Count Good Nodes in Binary Tree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  int count = 0;

  public int goodNodes(TreeNode root) {
    dfs(root, Integer.MIN_VALUE);
    return count;
  }

  private void dfs(TreeNode node, int currMax) {
    if (node == null) {
      return;
    }

    if (node.val >= currMax) {
      count++;
      currMax = node.val;
    }

    dfs(node.left, currMax);
    dfs(node.right, currMax);
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

388. [1455]Check If a Word Occurs As a Prefix of Any Word in a Sentence.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int isPrefixOfWord(String sentence, String searchWord) {
        String[] words = sentence.split(" ");
        int n = words.length;

        for (int i = 0; i < n; i++) {
            String word = words[i];

            if (word.startsWith(searchWord)) {
                return i + 1;
            }
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

389. [1456]Maximum Number of Vowels in a Substring of Given Length.java

```
1456
        TMaximum Number of Vowels in a Substring of Given Length
        2023-05-05 09:36:14

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxVowels(String s, int k) {
        char[] c = s.toCharArray();
        int n = c.length;
        int[] arr = new int[n + 1];
        int count = 0;
        int max = 0;
        for (int i = 0; i < n; i++) {
            if (c[i] == 'a' || c[i] == 'e' || c[i] == 'i' || c[i] == 'o' || c[i] == 'u')
{
                count++;
            }
            arr[i + 1] = count;
        }
        for (int i = k; i < n + 1; i++) {
            max = Math.max(max, arr[i] - arr[i - k]);
        }
        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

390. `[1457]Pseudo-Palindromic Paths in a Binary Tree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int result = 0;

    public int pseudoPalindromicPaths(TreeNode root) {
        dfs(root, 0);
        return result;
    }

    private void dfs(TreeNode root, int count) {
        if (root == null) {
            return;
        }

        count ^= (1 << root.val);

        if (root.left == null && root.right == null) {
            if ((count & (count - 1)) == 0) {
                result++;
            }
        } else {
            dfs(root.left, count);
            dfs(root.right, count);
        }

        count ^= (1 << root.val);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

391. `[1458]Max Dot Product of Two Subsequences.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxDotProduct(int[] nums1, int[] nums2) {
        int firstMax = Integer.MIN_VALUE;
        int secondMax = Integer.MIN_VALUE;
        int firstMin = Integer.MAX_VALUE;
        int secondMin = Integer.MAX_VALUE;
        int m = nums1.length;
        int n = nums2.length;
        int[][] dp = new int[m + 1][n + 1];

        for (int num : nums1) {
            firstMax = Math.max(firstMax, num);
            firstMin = Math.min(firstMin, num);
        }

        for (int num : nums2) {
            secondMax = Math.max(secondMax, num);
            secondMin = Math.min(secondMin, num);
        }

        if (firstMax < 0 && secondMin > 0) {
            return firstMax * secondMin;
        }

        if (secondMax < 0 && firstMin > 0) {
            return secondMax * firstMin;
        }

        for (int i = m - 1; i >= 0; i--) {
            for (int j = n - 1; j >= 0; j--) {
                int product = nums1[i] * nums2[j] + dp[i + 1][j + 1];

                dp[i][j] = Math.max(product, Math.max(dp[i + 1][j], dp[i][j + 1]));
            }
        }

        return dp[0][0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    int[][] memo;

    public int maxDotProduct(int[] nums1, int[] nums2) {
        int firstMax = Integer.MIN_VALUE;
        int secondMax = Integer.MIN_VALUE;
        int firstMin = Integer.MAX_VALUE;
        int secondMin = Integer.MAX_VALUE;
```

```
        memo = new int[nums1.length][nums2.length];

        for (int num : nums1) {
            firstMax = Math.max(firstMax, num);
            firstMin = Math.min(firstMin, num);
        }

        for (int num : nums2) {
            secondMax = Math.max(secondMax, num);
            secondMin = Math.min(secondMin, num);
        }

        if (firstMax < 0 && secondMin > 0) {
            return firstMax * secondMin;
        }

        if (secondMax < 0 && firstMin > 0) {
            return secondMax * firstMin;
        }

        return dp(0, 0, nums1, nums2);
    }

    private int dp(int i, int j, int[] nums1, int[] nums2) {
        if (i == nums1.length || j == nums2.length) {
            return 0;
        }

        if (memo[i][j] != 0) {
            return memo[i][j];
        }

        int product = nums1[i] * nums2[j] + dp(i + 1, j + 1, nums1, nums2);
        memo[i][j] = Math.max(product, Math.max(dp(i + 1, j, nums1, nums2), dp(i, j + 1,
nums1, nums2)));
        return memo[i][j];
    }
}
 */
```

392. `[1460]Make Two Arrays Equal by Reversing Subarrays.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canBeEqual(int[] target, int[] arr) {
        Map<Integer, Integer> targetMap = new HashMap<>();
        Map<Integer, Integer> arrMap = new HashMap<>();
        IntStream.of(target).forEach(num -> targetMap.merge(num, 1, Integer::sum));
        IntStream.of(arr).forEach(num -> arrMap.merge(num, 1, Integer::sum));
        return targetMap.equals(arrMap);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public boolean canBeEqual(int[] target, int[] arr) {
        Map<Integer, Integer> targetMap = new HashMap<>();
        Map<Integer, Integer> arrMap = new HashMap<>();

        for (int num : target) {
            targetMap.merge(num, 1, Integer::sum);
        }

        for (int num : arr) {
            arrMap.merge(num, 1, Integer::sum);
        }

        return targetMap.equals(arrMap);
    }
}
 */
```

393. `[1462]Course Schedule IV.java`

```
1462
        Course Schedule IV
        2023-02-15 16:17:27

        https://www.youtube.com/watch?v=mXncR8of7Ns&t=736s

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Boolean> checkIfPrerequisite(int numCourses, int[][] prerequisites, int[]
[] queries) {
        Map<Integer, Set<Integer>> map = new HashMap();
        Map<Integer, Set<Integer>> preSet = new HashMap();
        for (int i = 0; i < numCourses; i++) {
            map.put(i, new HashSet<>());
            preSet.put(i, new HashSet<>());
        }
        int[] inDegree = new int[numCourses];
        for (int[] pre : prerequisites) {
            Set<Integer> set = map.get(pre[0]);
            set.add(pre[1]);
            map.put(pre[0], set);
            inDegree[pre[1]]++;
        }

        Deque<Integer> dq = new ArrayDeque();
        for (int i = 0; i < numCourses; i++) {
            Set<Integer> set = preSet.get(i);
            set.add(i);
            preSet.put(i, set);
            if (inDegree[i] == 0) {
                dq.add(i);
            }
        }
        while (!dq.isEmpty()) {
            int curr = dq.poll();
            if (map.containsKey(curr)) {
                for (int course : map.get(curr)) {
                    for (int preCourse : preSet.get(curr)) {
                        Set<Integer> set = preSet.get(course);
                        set.add(preCourse);
                        preSet.put(course, set);
                    }
                    inDegree[course]--;
                    if (inDegree[course] == 0) {
                        dq.add(course);
                    }
                }
            }
        }
        List<Boolean> list = new ArrayList();
        for (int[] query : queries) {
```

```
            if (preSet.get(query[1]).contains(query[0])) {
                list.add(true);
            } else {
                list.add(false);
            }
        }
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

394. [1463]Cherry Pickup II.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int cherryPickup(int[][] grid) {
        int n = grid.length;
        int m = grid[0].length;
        int dp[][][] = new int[n][m][m];

        for (int[][] layer : dp) {
            for (int[] a : layer) {
                Arrays.fill(a, Integer.MIN_VALUE);
            }
        }

        return Math.max(0, dyP(0, 0, m - 1, grid, dp));
    }

    public int dyP(int r, int c1, int c2, int[][] grid, int[][][] dp) {
        if (r == grid.length) {
            return 0;
        }

        if (dp[r][c1][c2] != Integer.MIN_VALUE) {
            return dp[r][c1][c2];
        }

        int ans = grid[r][c1];

        if (c1 != c2) {
            ans += grid[r][c2];
        }

        int temp = 0;

        for (int newc1 = c1 - 1; newc1 <= c1 + 1; newc1++) {
            for (int newc2 = c2 - 1; newc2 <= c2 + 1; newc2++) {
                if (newc1 >= 0 && newc1 < grid[0].length && newc2 >= 0 && newc2 <
grid[0].length) {
                    temp = Math.max(temp, dyP(r + 1, newc1, newc2, grid, dp));
                }
            }
        }

        ans += temp;
        dp[r][c1][c2] = ans;
        return ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

395. `[1464]Maximum Product of Two Elements in an Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxProduct(int[] nums) {
        int first = 0;
        int second = 0;

        for (int num : nums) {
            if (num > first) {
                second = first;
                first = num;
            } else if (num > second) {
                second = num;
            }
        }

        return (first - 1) * (second - 1);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

396. `[1466]Reorder Routes to Make All Paths Lead to the`

City Zero.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    List<List<int[]>> list = new ArrayList<>();
    boolean[] isVisited;
    int res = 0;

    public int minReorder(int n, int[][] connections) {
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (int[] edge : connections) {
            list.get(edge[0]).add(new int[]{edge[1], 1});
            list.get(edge[1]).add(new int[]{edge[0], 0});
        }
        isVisited = new boolean[n];
        dfs(0);
        return res;
    }

    public void dfs(int curr) {
        isVisited[curr] = true;
        for (int[] num : list.get(curr)) {
            if (!isVisited[num[0]]) {
                res += num[1];
                dfs(num[0]);
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
DFS:
class Solution {
    List<List<int[]>> list = new ArrayList<>();
    boolean[] isVisited;
    int res = 0;

    public int minReorder(int n, int[][] connections) {
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (int[] edge : connections) {
            list.get(edge[0]).add(new int[]{edge[1], 1});
            list.get(edge[1]).add(new int[]{edge[0], 0});
        }
        isVisited = new boolean[n];
        dfs(0);
        return res;
    }

    public void dfs(int curr) {
```

```
        isVisited[curr] = true;
        for (int[] num : list.get(curr)) {
            if (!isVisited[num[0]]) {
                res += num[1];
                dfs(num[0]);
            }
        }
    }
}


BFS:
class Solution {
    public int minReorder(int n, int[][] connections) {
        List<List<int[]>> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (int[] edge : connections) {
            list.get(edge[0]).add(new int[]{edge[1], 1});
            list.get(edge[1]).add(new int[]{edge[0], 0});
        }
        Deque<Integer> dq = new ArrayDeque<>();
        boolean[] isVisited = new boolean[n];
        int res = 0;
        dq.add(0);
        while (!dq.isEmpty()) {
            int curr = dq.poll();
            isVisited[curr] = true;
            for (int[] num : list.get(curr)) {
                if (!isVisited[num[0]]) {
                    res += num[1];
                    dq.add(num[0]);
                }
            }
        }
        return res;
    }
}
 */
```

397. [1469]Find All The Lonely Nodes.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> getLonelyNodes(TreeNode root) {
        List<Integer> lonelyNodes = new ArrayList<>();
        dfs(root, lonelyNodes);
        return lonelyNodes;
    }

    private void dfs(TreeNode node, List<Integer> lonelyNodes) {
        if (node == null) {
            return;
        }

        if (node.left != null && node.right == null) {
            lonelyNodes.add(node.left.val);
        }

        if (node.left == null && node.right != null) {
            lonelyNodes.add(node.right.val);
        }

        dfs(node.left, lonelyNodes);
        dfs(node.right, lonelyNodes);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

398. [1470]Shuffle the Array.java

```
1470
        Shuffle the Array
        2023-02-06 09:53:33

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] shuffle(int[] nums, int n) {
        int[] arr = new int[2 * n];
        for (int i = 0; i < n; i++) {
            arr[2 * i] = nums[i];
            arr[2 * i + 1] = nums[n + i];
        }
        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

399. `[1472]Design Browser History.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class BrowserHistory {
    List<String> list;
    int curr;
    int last;

    public BrowserHistory(String homepage) {
        list = new ArrayList<>(Arrays.asList(homepage));
        curr = 0;
        last = 0;
    }

    public void visit(String url) {
        curr++;
        if (list.size() > curr) {
            list.set(curr, url);
        } else {
            list.add(url);
        }
        last = curr;
    }

    public String back(int steps) {
        curr = Math.max(0, curr - steps);
        return list.get(curr);
    }

    public String forward(int steps) {
        curr = Math.min(last, curr + steps);
        return list.get(curr);
    }
}

/**
 * Your BrowserHistory object will be instantiated and called as such:
 * BrowserHistory obj = new BrowserHistory(homepage);
 * obj.visit(url);
 * String param_2 = obj.back(steps);
 * String param_3 = obj.forward(steps);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

400. `[1473]Paint House III.java`

```
1473
        Paint House III
        2023-02-25 16:50:42

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    final int MAX = 1000001;

    public int minCost(int[] houses, int[][] cost, int m, int n, int target) {
        int[][] prevDp = new int[target + 1][n];
        for (int i = 0; i <= target; i++) {
            Arrays.fill(prevDp[i], MAX);
        }
        for (int i = 1; i <= n; i++) {
            if (houses[0] == i) {
                prevDp[1][i - 1] = 0;
            } else if (houses[0] == 0) {
                prevDp[1][i - 1] = cost[0][i - 1];
            }
        }
        for (int house = 1; house < m; house++) {
            int[][] dp = new int[target + 1][n];
            for (int i = 0; i <= target; i++) {
                Arrays.fill(dp[i], MAX);
            }
            for (int nei = 1; nei <= Math.min(target, house + 1); nei++) {
                for (int i = 1; i <= n; i++) {
                    if (houses[house] == 0 || i == houses[house]) {
                        int curr = MAX;
                        for (int prev = 1; prev <= n; prev++) {
                            if (prev != i) {
                                curr = Math.min(curr, prevDp[nei - 1][prev - 1]);
                            } else {
                                curr = Math.min(curr, prevDp[nei][i - 1]);
                            }
                        }
                        int costToPaint = houses[house] != 0 ? 0 : cost[house][i - 1];
                        dp[nei][i - 1] = curr + costToPaint;
                    }
                }
            }
            prevDp = dp;
        }
        int res = MAX;
        for (int i = 1; i <= n; i++) {
            res = Math.min(res, prevDp[target][i - 1]);
        }
        return res == MAX ? -1 : res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

401. `[1475]Final Prices With a Special Discount in a Shop.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] finalPrices(int[] prices) {
        int n = prices.length;
        int[] arr = new int[n];
        Deque<Integer> dq = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            while (!dq.isEmpty() && prices[dq.peek()] >= prices[i]) {
                int index = dq.pop();
                arr[index] = prices[index] - prices[i];
            }

            dq.push(i);
            arr[i] = prices[i];
        }

        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

402. `[1480]Running Sum of 1d Array.java`

```
1480
        Running Sum of 1dArray
        2022-12-05 18:04:35

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] runningSum(int[] nums) {
        int n = nums.length;
        int[] preSum = new int[n + 1];
        int[] arr = new int[n];
        for (int i = 1; i <= n; i++) {
            preSum[i] = preSum[i - 1] + nums[i - 1];
            arr[i - 1] = preSum[i];
        }
        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

403. [1481]Least Number of Unique Integers after K Removals.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findLeastNumOfUniqueInts(int[] arr, int k) {
        Map<Integer, Integer> map = new HashMap<>();

        for (int num : arr) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        Queue<Integer> heap = new PriorityQueue<>((a, b) -> map.get(a) - map.get(b));
        heap.addAll(map.keySet());

        while (k > 0) {
            k -= map.get(heap.poll());
        }

        return k < 0 ? heap.size() + 1 : heap.size();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

404. `[1482]Minimum Number of Days to Make m Bouquets.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minDays(int[] bloomDay, int m, int k) {
        if (bloomDay.length / k < m) {
            return -1;
        }

        int left = Integer.MAX_VALUE;
        int right = Integer.MIN_VALUE;

        for (int day : bloomDay) {
            left = Math.min(left, day);
            right = Math.max(right, day);
        }

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (canMakeBouquets(bloomDay, m, k, mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }

    private boolean canMakeBouquets(int[] bloomDay, int m, int k, int days) {
        int bouquets = 0;
        int flowers = 0;

        for (int day : bloomDay) {
            if (day <= days) {
                flowers++;

                if (flowers == k) {
                    bouquets++;
                    flowers = 0;
                }
            } else {
                flowers = 0;
            }
        }

        return bouquets >= m;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

405. `[1485]Clone Binary Tree With Random Pointer.java`

```
/**
 * <p>A binary tree is given such that each node contains an additional random pointer
which could point to any node in the tree or null.</p>
 *
 * <p>Return a <a href="https://en.wikipedia.org/wiki/Object_copying#Deep_copy"
target="_blank"><strong>deep copy</strong></a> of the tree.</p>
 *
 * <p>The tree is represented in the same input/output way as normal binary trees where
each node is represented as a pair of <code>[val, random_index]</code> where:</p>
 *
 * <ul>
 * <li><code>val</code>: an integer representing <code>Node.val</code></li>
 * <li><code>random_index</code>: the index of the node (in the input) where the random
pointer points to, or <code>null</code> if it does not point to any node.</li>
 * </ul>
 *
 * <p>You will be given the tree in class <code>Node</code> and you should return the
cloned tree in class <code>NodeCopy</code>. <code>NodeCopy</code> class is just a clone
of <code>Node</code> class with the same attributes and constructors.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/06/17/clone_1.png"
style="width: 500px; height: 473px;" />
 * <pre>
 * <strong>Input:</strong> root = [[1,null],null,[4,3],[7,0]]
 * <strong>Output:</strong> [[1,null],null,[4,3],[7,0]]
 * <strong>Explanation:</strong> The original binary tree is [1,null,4,7].
 * The random pointer of node one is null, so it is represented as [1, null].
 * The random pointer of node 4 is node 7, so it is represented as [4, 3] where 3 is the
index of node 7 in the array representing the tree.
 * The random pointer of node 7 is node 1, so it is represented as [7, 0] where 0 is the
index of node 1 in the array representing the tree.
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/06/17/clone_2.png"
style="width: 500px; height: 540px;" />
 * <pre>
 * <strong>Input:</strong> root = [[1,4],null,[1,0],null,[1,5],[1,5]]
 * <strong>Output:</strong> [[1,4],null,[1,0],null,[1,5],[1,5]]
 * <strong>Explanation:</strong> The random pointer of a node can be the node itself.
 * </pre>
 *
 * <p><strong class="example">Example 3:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2020/06/17/clone_3.png"
style="width: 500px; height: 426px;" />
 * <pre>
 * <strong>Input:</strong> root = [[1,6],[2,5],[3,4],[4,3],[5,2],[6,1],[7,0]]
 * <strong>Output:</strong> [[1,6],[2,5],[3,4],[4,3],[5,2],[6,1],[7,0]]
 * </pre>
```

```
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li>The number of nodes in the <code>tree</code> is in the range <code>[0, 1000].
</code></li>
 * <li><code>1 &lt;= Node.val &lt;= 10<sup>6</sup></code></li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>Hash Table</li><li>Tree</li><li>Depth-First
Search</li><li>Breadth-First Search</li><li>Binary Tree</li></div></div><br><div><li>👍
347</li><li>👎 67</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for Node.
 * public class Node {
 * int val;
 * Node left;
 * Node right;
 * Node random;
 * Node() {}
 * Node(int val) { this.val = val; }
 * Node(int val, Node left, Node right, Node random) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * this.random = random;
 * }
 * }
 */

class Solution {
    Map<Node, NodeCopy> map = new HashMap<>();

    public NodeCopy copyRandomBinaryTree(Node root) {
        if (root == null) {
            return null;
        }
        if (map.containsKey(root)) {
            return map.get(root);
        }
        NodeCopy rootCopy = new NodeCopy(root.val);
        map.put(root, rootCopy);
        rootCopy.left = copyRandomBinaryTree(root.left);
        rootCopy.right = copyRandomBinaryTree(root.right);
        rootCopy.random = copyRandomBinaryTree(root.random);
        return rootCopy;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

406. `[1486]XOR Operation in an Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int xorOperation(int n, int start) {
        int res = 0;
        for (int i = 0; i < n; i++) {
            res ^= start + 2 * i;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

407. [1489]Find Critical and Pseudo-Critical Edges in

Minimum Spanning Tree.java

```
/*
還不會
 */
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> findCriticalAndPseudoCriticalEdges(int n, int[][] edges) {
        int row = edges.length;
        int col = edges[0].length;
        int[][] edgeWithIndex = new int[row][col + 1];
        List<Integer> criticals = new ArrayList<>();
        List<Integer> pseduos = new ArrayList<>();
        for (int i = 0; i < row; i++) {
            edgeWithIndex[i][0] = edges[i][0];
            edgeWithIndex[i][1] = edges[i][1];
            edgeWithIndex[i][2] = edges[i][2];
            edgeWithIndex[i][3] = i;
        }
        Arrays.sort(edgeWithIndex, (a, b) -> a[2] - b[2]);
        int minCost = buildMST(n, edgeWithIndex, null, null);
        for (int[] edge : edgeWithIndex) {
            int index = edge[3];
            int costWithout = buildMST(n, edgeWithIndex, null, edge);
            if (costWithout > minCost) {
                criticals.add(index);
            } else {
                int costWith = buildMST(n, edgeWithIndex, edge, null);
                if (costWith == minCost) {
                    pseduos.add(index);
                }
            }
        }
        return Arrays.asList(criticals, pseduos);
    }

    private int buildMST(int n, int[][] edges, int[] pick, int[] skip) {
        UnionFind uf = new UnionFind(n);
        int cost = 0;
        if (pick != null) {
            uf.union(pick[0], pick[1]);
            cost += pick[2];
        }
        for (int[] edge : edges) {
            if (edge == skip) {
                continue;
            }
            if (uf.union(edge[0], edge[1])) {
                cost += edge[2];
            }
        }
        return uf.getCount() == 1 ? cost : Integer.MAX_VALUE;
    }
```

```
}

class UnionFind {
    int[] group;

    public UnionFind(int n) {
        group = new int[n];
        for (int i = 0; i < n; i++) {
            group[i] = i;
        }
    }

    public int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }

    public boolean union(int i, int j) {
        int x = find(i);
        int y = find(j);
        if (x == y) {
            return false;
        }
        group[x] = y;
        return true;
    }

    public int getCount() {
        int count = 0;
        for (int i = 0; i < group.length; i++) {
            if (group[i] == i) {
                count++;
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public List<List<Integer>> findCriticalAndPseudoCriticalEdges(int n, int[][] edges) {
        int[][][] g = new int[n][n][2]; //{w,ind}
        for (int i = 0; i < edges.length; i++) {
            int[] e = edges[i];
            int f = e[0];
            int t = e[1];
            int w = e[2];
            g[f][t][0] = w;
            g[t][f][0] = w;
            g[f][t][1] = i;
            g[t][f][1] = i;
        }
        List<Integer>[] mst = new List[n];
        for (int i = 0; i < n; i++) {
            mst[i] = new ArrayList<>();
```

```
        }
        boolean[] mstSet = new boolean[edges.length];
        Arrays.sort(edges, (a, b) -> a[2] - b[2]);
        buildMST(n, edges, mstSet, mst, g);
        List<List<Integer>> ans = new ArrayList<>(2);
        Set<Integer> pce = new HashSet<>();
        List<Integer> ce = new LinkedList<>();

        for (int i = 0; i < edges.length; i++) {
            int f = edges[i][0], t = edges[i][1];
            int w = edges[i][2];
            int ind = g[f][t][1];
            if (!mstSet[ind]) {
                Set<Integer> cur = new HashSet<>();
                boolean p = path(f, t, w, -1, mst, g, cur);
                if (p && cur.size() > 0) {
                    pce.addAll(cur);
                    pce.add(ind);
                }
                if (!p) {
                    System.out.println("Should not reach here");
                }
            }

        }

        for (int i = 0; i < edges.length; i++) {
            int f = edges[i][0], t = edges[i][1];
            int w = edges[i][2];
            int ind = g[f][t][1];
            if (mstSet[ind] && !pce.contains(ind)) {
                ce.add(ind);
            }
        }
        ans.add(ce);
        ans.add(new LinkedList<>(pce));
        return ans;
    }

    boolean path(int f, int t, int w, int p, List<Integer>[] mst, int[][][] g,
Set<Integer> ind) {
        if (f == t) {
            return true;
        }
        for (int nbr : mst[f]) {
            if (p != nbr) {
                if (path(nbr, t, w, f, mst, g, ind)) {
                    if (g[f][nbr][0] == w) {
                        ind.add(g[f][nbr][1]);
                    }
                    return true;
                }
            }
        }
```

```
        return false;
    }

    private void buildMST(int n, int[][] edges, boolean[] mste, List<Integer>[] mstg,
int[][][] g) {
        int weight = 0;
        UnionFind uf = new UnionFind(n);
        for (int i = 0; i < edges.length; i++) {

            if (uf.union(edges[i][0], edges[i][1])) {
                weight += edges[i][2];
                int[] edge = edges[i];
                mstg[edge[0]].add(edge[1]);
                mstg[edge[1]].add(edge[0]);
                mste[g[edge[0]][edge[1]][1]] = true;
            }
        }
    }
}


class UnionFind {
    int[] group;

    public UnionFind(int n) {
        group = new int[n];
        for (int i = 0; i < n; i++) {
            group[i] = i;
        }
    }

    public int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }

    public boolean union(int i, int j) {
        int x = find(i);
        int y = find(j);
        if (x == y) {
            return false;
        }
        group[x] = y;
        return true;
    }
}
 */
```

408. `[1490]Clone N-ary Tree.java`

```
1490
        Clone N-ary Tree
        2023-02-04 14:31:10
//leetcode submit region begin(Prohibit modification and deletion)
/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> children;


    public Node() {
        children = new ArrayList<Node>();
    }

    public Node(int _val) {
        val = _val;
        children = new ArrayList<Node>();
    }

    public Node(int _val,ArrayList<Node> _children) {
        val = _val;
        children = _children;
    }
};
*/

class Solution {
    Map<Node, Node> map = new HashMap<>();

    public Node cloneTree(Node root) {
        if (root == null) {
            return null;
        }
        return dfs(root);
    }

    Node dfs(Node node) {
        if (map.containsKey(node)) {
            return map.get(node);
        }
        Node clone = new Node(node.val);
        map.put(node, clone);
        for (Node item : node.children) {
            clone.children.add(dfs(item));
        }
        return clone;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

409. [1491]Average Salary Excluding the Minimum and Maximum Salary.java

```
1491
        Average Salary Excluding the Minimum and Maximum Salary
        2023-02-26 15:22:20

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public double average(int[] salary) {
        int n = salary.length;
        int sum = 0;
        int max = Integer.MIN_VALUE;
        int min = Integer.MAX_VALUE;
        for (int num : salary) {
            sum += num;
            max = Math.max(max, num);
            min = Math.min(min, num);
        }
        return (double) (sum - max - min) / (n - 2);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

410. [1492]The kth Factor of n.java

```
1492
        The kth Factor of n
        2023-02-26 15:33:00

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int kthFactor(int n, int k) {
        int count = 0;
        for (int i = 1; i <= n; i++) {
            if (n % i == 0) {
                count++;
            }
            if (count == k) {
                return i;
            }
        }
        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

411. [1493]Longest Subarray of 1's After Deleting One Element.java

```
1493
        Longest Subarray of 1's After Deleting One Element
        2023-02-26 16:54:06

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestSubarray(int[] nums) {
        int left = 0;
        int right;
        int buffer = 1;
        int result = 0;

        for (right = 0; right < nums.length; right++) {
            if (nums[right] == 0) {
                buffer--;
            }

            while (buffer < 0) {
                if (nums[left] == 0) {
                    buffer++;
                }

                left++;
            }

            result = Math.max(result, right - left);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

412. [1497]Check If Array Pairs Are Divisible by k.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canArrange(int[] arr, int k) {
        int[] count = new int[k];

        for (int num : arr) {
            int remainder = ((num % k) + k) % k;
            count[remainder]++;
        }

        if (count[0] % 2 != 0) {
            return false;
        }

        for (int i = 1; i <= k / 2; i++) {
            if (i == k - i) {
                if (count[i] % 2 != 0) {
                    return false;
                }
            } else {
                if (count[i] != count[k - i]) {
                    return false;
                }
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

413. `[1498]Number of Subsequences That Satisfy the Given Sum Condition.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numSubseq(int[] nums, int target) {
        final int MOD = 1_000_000_007;
        Arrays.sort(nums);
        int n = nums.length;
        int[] arr = new int[n];
        arr[0] = 1;

        for (int i = 1; i < n; i++) {
            arr[i] = arr[i - 1] * 2 % MOD;
        }

        int count = 0;
        int left = 0;
        int right = n - 1;

        while (left <= right) {
            if (nums[left] + nums[right] <= target) {
                count = (count + arr[right - left]) % MOD;
                left++;
            } else {
                right--;
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

414. [1502]Can Make Arithmetic Progression From Sequence.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canMakeArithmeticProgression(int[] arr) {
        Arrays.sort(arr);
        int diff = arr[1] - arr[0];

        for (int i = 2; i < arr.length; i++) {
            if (arr[i] - arr[i - 1] != diff) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

415. [1503]Last Moment Before All Ants Fall Out of a Plank.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int getLastMoment(int n, int[] left, int[] right) {
        int result = 0;

        for (int num : left) {
            result = Math.max(result, num);
        }

        for (int num : right) {
            result = Math.max(result, n - num);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

416. `[1507]Reformat Date.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String reformatDate(String date) {
        StringBuilder sb = new StringBuilder();
        sb.append(date.substring(date.length() - 4)).append("-");
        String month = date.length() == 13 ? date.substring(5, 8) : date.substring(4, 7);

        switch (month) {
            case "Jan" -> sb.append("01-");
            case "Feb" -> sb.append("02-");
            case "Mar" -> sb.append("03-");
            case "Apr" -> sb.append("04-");
            case "May" -> sb.append("05-");
            case "Jun" -> sb.append("06-");
            case "Jul" -> sb.append("07-");
            case "Aug" -> sb.append("08-");
            case "Sep" -> sb.append("09-");
            case "Oct" -> sb.append("10-");
            case "Nov" -> sb.append("11-");
            case "Dec" -> sb.append("12-");
        }

        if (date.length() < 13) {
            sb.append("0");
            sb.append(date.substring(0, 1));
        } else {
            sb.append(date.substring(0, 2));
        }

        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

417. `[1508]Range Sum of Sorted Subarray Sums.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int rangeSum(int[] nums, int n, int left, int right) {
        final int MOD = (int) 1e9 + 7;
        int index = 0;
        int result = 0;
        int[] arr = new int[n * (n + 1) / 2];

        for (int i = 0; i < n; i++) {
            int sum = 0;

            for (int j = i; j < n; j++) {
                sum += nums[j];
                arr[index++] = sum;
            }
        }

        Arrays.sort(arr);

        for (int i = left - 1; i < right; i++) {
            result = (result + arr[i]) % MOD;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

418. [1509]Minimum Difference Between Largest and Smallest Value in Three Moves.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minDifference(int[] nums) {
        if (nums.length <= 4) {
            return 0;
        }

        Arrays.sort(nums);
        int n = nums.length;
        return Arrays.stream(new int[]{
                        nums[n - 4] - nums[0],
                        nums[n - 3] - nums[1],
                        nums[n - 2] - nums[2],
                        nums[n - 1] - nums[3]})
                .min()
                .getAsInt();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

419. [1510]Stone Game IV.java

```
/**
<p>Alice and Bob take turns playing a game, with Alice starting first.</p>

<p>Initially, there are <code>n</code> stones in a pile. On each player's turn, that
player makes a <em>move</em> consisting of removing <strong>any</strong> non-zero
<strong>square number</strong> of stones in the pile.</p>

<p>Also, if a player cannot make a move, he/she loses the game.</p>

<p>Given a positive integer <code>n</code>, return <code>true</code> if and only if Alice
wins the game otherwise return <code>false</code>, assuming both players play optimally.
</p>

<p> </p>
<p><strong class="example">Example 1:</strong></p>

<pre>
<strong>Input:</strong> n = 1
<strong>Output:</strong> true
<strong>Explanation: </strong>Alice can remove 1 stone winning the game because Bob
doesn't have any moves.</pre>

<p><strong class="example">Example 2:</strong></p>

<pre>
<strong>Input:</strong> n = 2
<strong>Output:</strong> false
<strong>Explanation: </strong>Alice can only remove 1 stone, after that Bob removes the
last one winning the game (2 -&gt; 1 -&gt; 0).
</pre>

<p><strong class="example">Example 3:</strong></p>

<pre>
<strong>Input:</strong> n = 4
<strong>Output:</strong> true
<strong>Explanation:</strong> n is already a perfect square, Alice can win with one move,
removing 4 stones (4 -&gt; 0).
</pre>

<p> </p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>1 &lt;= n &lt;= 10<sup>5</sup></code></li>
</ul>

<div><div>Related Topics</div><div><li>Math</li><li>Dynamic Programming</li><li>Game
Theory</li></div></div><br><div><li>👍 1438</li><li>👎 64</li></div>
*/
```

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean winnerSquareGame(int n) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

420. `[1512]Number of Good Pairs.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numIdenticalPairs(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        int count = 0;

        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        for (int num : map.values()) {
            count += num * (num - 1) / 2;
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
 - 0ms
class Solution {
    public int numIdenticalPairs(int[] nums) {
        int[] arr = new int[101];
        int count = 0;
        for (int num : nums) {
            arr[num]++;
        }
        for (int num : arr) {
            if (num > 1) {
                count += num * (num - 1) / 2;
            }
        }
        return count;
    }
}

- 1ms;
class Solution {
    public int numIdenticalPairs(int[] nums) {
        int n = nums.length;
        int count = 0;
        for (int i = 0; i < n - 1; i++) {
            for (int j = i + 1; j < n; j++) {
                if (nums[i] == nums[j]) {
                    count++;
                }
            }
        }
        return count;
    }
```

```
}
 */
```

421. [1513]Number of Substrings With Only 1s.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numSub(String s) {
        final int MOD = (int) 1e9 + 7;
        int result = 0;
        int count = 0;

        for (char c : s.toCharArray()) {
            if (c == '1') {
                count++;
            } else {
                count = 0;
            }

            result = (result + count) % MOD;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

422. [1514]Path with Maximum Probability.java

```
1514
        Path with Maximum Probability
        2023-02-14 15:43:20

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public double maxProbability(int n, int[][] edges, double[] succProb, int start, int
end) {
        double[] maxProb = new double[n];
        maxProb[start] = 1.0;
        for (int i = 0; i < n; i++) {
            boolean isUpdated = false;
            for (int j = 0; j < edges.length; j++) {
                int x = edges[j][0];
                int y = edges[j][1];
                double pathProb = succProb[j];
                if (maxProb[x] * pathProb > maxProb[y]) {
                    maxProb[y] = maxProb[x] * pathProb;
                    isUpdated = true;
                }
                if (maxProb[y] * pathProb > maxProb[x]) {
                    maxProb[x] = maxProb[y] * pathProb;
                    isUpdated = true;
                }
            }
            if (!isUpdated) {
                break;
            }
        }
        return maxProb[end];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
Dijkstra's Algorithm: -54ms
class Solution {
    public double maxProbability(int n, int[][] edges, double[] succProb, int start, int
end) {
        Map<Integer, List<Pair<Integer, Double>>> map = new HashMap<>();
        for (int i = 0; i < edges.length; i++) {
            int x = edges[i][0];
            int y = edges[i][1];
            double z = succProb[i];
            map.computeIfAbsent(x, k -> new ArrayList<>()).add(new Pair<>(y, z));
            map.computeIfAbsent(y, k -> new ArrayList<>()).add(new Pair<>(x, z));
        }
        double[] maxProb = new double[n];
        maxProb[start] = 1.0;
        PriorityQueue<Pair<Double, Integer>> pq = new PriorityQueue<>((a, b) -> -
Double.compare(a.getKey(), b.getKey()));
        pq.add(new Pair<>(1.0, start));
```

```
        while (!pq.isEmpty()) {
            Pair<Double, Integer> curr = pq.poll();
            double currProb = curr.getKey();
            int currNode = curr.getValue();
            if (currNode == end) {
                return currProb;
            }
            for (Pair<Integer, Double> next : map.getOrDefault(currNode, new ArrayList<>
())) {
                int nextNode = next.getKey();
                double pathProb = next.getValue();
                if (currProb * pathProb > maxProb[nextNode]) {
                    maxProb[nextNode] = currProb * pathProb;
                    pq.add(new Pair<>(maxProb[nextNode], nextNode));
                }
            }
        }
        return 0.0;
    }
}


Bellman-Ford Algorithm: -9ms
class Solution {
    public double maxProbability(int n, int[][] edges, double[] succProb, int start, int
end) {
        double[] maxProb = new double[n];
        maxProb[start] = 1.0;
        for (int i = 0; i < n; i++) {
            boolean updated = false;
            for (int j = 0; j < edges.length; j++) {
                int x = edges[j][0];
                int y = edges[j][1];
                double pathProb = succProb[j];
                if (maxProb[x] * pathProb > maxProb[y]) {
                    maxProb[y] = maxProb[x] * pathProb;
                    updated = true;
                }
                if (maxProb[y] * pathProb > maxProb[x]) {
                    maxProb[x] = maxProb[y] * pathProb;
                    updated = true;
                }
            }
            if (!updated) {
                break;
            }
        }
        return maxProb[end];
    }
}
 */
```

423. `[1518]Water Bottles.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numWaterBottles(int numBottles, int numExchange) {
        int result = numBottles;
        int emptyBottle = numBottles;

        while (emptyBottle >= numExchange) {
            int newBottle = emptyBottle / numExchange;
            result += newBottle;
            emptyBottle = newBottle + (emptyBottle % numExchange);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

424. [1519]Number of Nodes in the Sub-Tree With the Same Label.java

```
1519
        Number of Nodes in the Sub-Tree With the Same Label
        2023-01-12 09:53:16

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] countSubTrees(int n, int[][] edges, String labels) {
        List<List<Integer>> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (int[] edge : edges) {
            int x = edge[0];
            int y = edge[1];
            list.get(x).add(y);
            list.get(y).add(x);
        }
        int[] arr = new int[n];
        char[] label = labels.toCharArray();
        dfs(-1, 0, list, label, arr);
        return arr;
    }

    private int[] dfs(int prev, int node, List<List<Integer>> list, char[] label, int[]
arr) {
        int[] nodeCount = new int[26];
        nodeCount[label[node] - 'a'] = 1;
        for (int child : list.get(node)) {
            if (child != prev) {
                int[] childCount = dfs(node, child, list, label, arr);
                for (int i = 0; i < 26; i++) {
                    nodeCount[i] += childCount[i];
                }
            }
        }
        arr[node] = nodeCount[label[node] - 'a'];
        return nodeCount;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

425. [1522]Diameter of N-Ary Tree.java

```
class Solution {
    int result = 0;

    public int diameter(Node root) {
        dfs(root);
        return result;
    }

    private int dfs(Node node) {
        if (node == null) {
            return 0;
        }

        int max1 = 0;
        int max2 = 0;

        for (Node child : node.children) {
            int depth = dfs(child);

            if (depth > max1) {
                max2 = max1;
                max1 = depth;
            } else if (depth > max2) {
                max2 = depth;
            }
        }

        result = Math.max(result, max1 + max2);
        return max1 + 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

426. [1523]Count Odd Numbers in an Interval Range.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countOdds(int low, int high) {
        return (high + 1) / 2 - low / 2;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

427. [1524]Number of Sub-arrays With Odd Sum.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int numOfSubarrays(int[] arr) {
    int mod = 1_000_000_007;
    int odd = 0;
    int even = 1;
    int curr = 0;
    int result = 0;

    for (int num : arr) {
      curr = (curr + num) % 2;

      if (curr == 0) {
        result = (result + odd) % mod;
        even++;
      } else {
        result = (result + even) % mod;
        odd++;
      }
    }

    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

428. [1530]Number of Good Leaf Nodes Pairs.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countPairs(TreeNode root, int distance) {
        AtomicInteger result = new AtomicInteger(0);
        dfs(root, distance, result);
        return result.get();
    }

    private List<Integer> dfs(TreeNode node, int distance, AtomicInteger result) {
        if (node == null) {
            return new ArrayList<>();
        }

        if (node.left == null && node.right == null) {
            List<Integer> leafDistances = new ArrayList<>();
            leafDistances.add(0);
            return leafDistances;
        }

        List<Integer> leftDistances = dfs(node.left, distance, result);
        List<Integer> rightDistances = dfs(node.right, distance, result);

        for (int leftDist : leftDistances) {
            for (int rightDist : rightDistances) {
                if (leftDist + rightDist + 2 <= distance) {
                    result.incrementAndGet();
                }
            }
        }

        List<Integer> currentDistances = new ArrayList<>();

        for (int leftDist : leftDistances) {
            if (leftDist + 1 < distance) {
                currentDistances.add(leftDist + 1);
            }
        }

        for (int rightDist : rightDistances) {
            if (rightDist + 1 < distance) {
                currentDistances.add(rightDist + 1);
            }
        }

        return currentDistances;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
```

```
    int result = 0;

    public int countPairs(TreeNode root, int distance) {
        dfs(root, distance);
        return result;
    }

    private int[] dfs(TreeNode root, int distance) {
        if (root == null) {
            return new int[distance + 1];
        }

        if (root.left == null && root.right == null) {
            int res[] = new int[distance + 1];
            res[1]++;
            return res;
        }

        int[] left = dfs(root.left, distance);
        int[] right = dfs(root.right, distance);

        for (int l = 1; l < left.length; l++) {
            for (int r = distance - 1; r >= 0; r--) {
                if (l + r <= distance)
                    result += left[l] * right[r];
            }
        }

        int res[] = new int[distance + 1];

        for (int i = res.length - 2; i >= 1; i--) {
            res[i + 1] = left[i] + right[i];
        }

        return res;
    }
}
 */
```

429. `[1531]String Compression II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int getLengthOfOptimalCompression(String s, int k) {
        int[][] memo = new int[s.length()][k + 1];
        for (int[] row : memo)
            Arrays.fill(row, -1);
        return dp(s, k, memo, 0);
    }

    private int dp(String s, int k, int[][] memo, int i) {
        if (k < 0) {
            return s.length();
        }

        if (i + k >= s.length()) {
            return 0;
        }

        if (memo[i][k] != -1) {
            return memo[i][k];
        }

        int result = dp(s, k - 1, memo, i + 1);
        int len = 0;
        int same = 0;
        int diff = 0;

        for (int j = i; j < s.length() && diff <= k; j++) {
            if (s.charAt(j) == s.charAt(i)) {
                same++;

                if (same <= 2 || same == 10 || same == 100) {
                    len++;
                }
            } else {
                diff++;
            }
            result = Math.min(result, len + dp(s, k - diff, memo, j + 1));
        }

        memo[i][k] = result;
        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

430. [1533]Find the Index of the Large Integer.java

```
1533
        Find the Index of the Large Integer
        2023-01-16 23:22:19
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * // This is ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface ArrayReader {
 * // Compares the sum of arr[l..r] with the sum of arr[x..y]
 * // return 1 if sum(arr[l..r]) > sum(arr[x..y])
 * // return 0 if sum(arr[l..r]) == sum(arr[x..y])
 * // return -1 if sum(arr[l..r]) < sum(arr[x..y])
 * public int compareSub(int l, int r, int x, int y) {}
 * <p>
 * // Returns the length of the array
 * public int length() {}
 * }
 */

class Solution {
    public int getIndex(ArrayReader reader) {
        int left = 0;
        int right = reader.length() - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            int val = reader.compareSub(left, mid, mid + (right - left) % 2, right);
            if (val == 0) {
                return mid;
            } else if (val < 0) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

431. `[1535]Find the Winner of an Array Game.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int getWinner(int[] arr, int k) {
        int count = 0;
        int winner = arr[0];

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > winner) {
                winner = arr[i];
                count = 0;
            }

            if (++count == k) {
                break;
            }
        }

        return winner;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

432. [1538]Guess the Majority in a Hidden Array.java

```
/**
 * // This is the ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface ArrayReader {
 * public:
 * // Compares 4 different elements in the array
 * // return 4 if the values of the 4 elements are the same (0 or 1).
 * // return 2 if three elements have a value equal to 0 and one element has value equal
to 1 or vice versa.
 * // return 0 : if two element have a value equal to 0 and two elements have a value
equal to 1.
 * public int query(int a, int b, int c, int d);
 * <p>
 * // Returns the length of the array
 * public int length();
 * };
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int guessMajority(ArrayReader reader) {
        int n = reader.length(), config = 0, one = 0, zero = 0;
        int[] g = new int[5];
        for (int i = 0; i < 5; i++) {
            int[] s = new int[]{(i + 1) % 5, (i + 2) % 5, (i + 3) % 5, (i + 4) % 5};
            Arrays.sort(s);
            g[i] = reader.query(s[0], s[1], s[2], s[3]);
        }
        for (int i = 0; i < 5 && ++config < 32; ) {
            one = Integer.bitCount(config);
            zero = 5 - one;
            for (i = 0; i < 5; i++) {
                if (g[i] != Math.abs(zero - one + ((config & (1 << (4 - i))) == 0 ? -1 :
1)))
                    break;
            }
        }
        int score = Math.abs(one - zero);
        int target = zero > one ? 0 : 1;
        int[] majority = new int[3];
        for (int i = 0, j = 0; j < 3; i++) {
            int cur = ((config & (1 << (4 - i))) > 0 ? 1 : 0);
            if (cur == target) {
                majority[j++] = i;
            }
        }
        int minority = -1;
        for (int i = 5; i < n - 1; i += 2) {
            int m = reader.query(majority[0], majority[1], i, i + 1);
            if (m == 0) {
```

```
                score -= 2;
                minority = i;
            } else if (m == 4) {
                score += 2;
            }
        }
        if (n % 2 == 0) {
            int m = reader.query(majority[0], majority[1], majority[2], n - 1);
            score += m == 4 ? 1 : -1;
        }
        if (score == 0)
            return -1;
        return score > 0 ? majority[0] : minority;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

433. `[1539].java`

```
1539
        Kth Missing Positive Number
        2023-03-06 10:34:21

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findKthPositive(int[] arr, int k) {
        int left = 0;
        int right = arr.length - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (arr[mid] - mid - 1 < k) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left + k;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

434. `[1539]Kth Missing Positive Number.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findKthPositive(int[] arr, int k) {
        int left = 0;
        int right = arr.length - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (arr[mid] - mid - 1 < k) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left + k;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int findKthPositive(int[] arr, int k) {
        Set<Integer> set = new HashSet<>();
        for (int num : arr) {
            set.add(num);
        }
        int count = 0;
        int key = 1;
        while (count < k) {
            if (!set.contains(key)) {
                count++;
            }
            key++;
        }
        return key - 1;
    }
}
 */
```

435. `[1544]Make The String Great.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String makeGood(String s) {
        Deque<Character> dq = new ArrayDeque<>();

        for (char c : s.toCharArray()) {
            if (!dq.isEmpty() && Math.abs(dq.peek() - c) == 32) {
                dq.pop();
            } else {
                dq.push(c);
            }
        }

        StringBuilder sb = new StringBuilder();

        while (!dq.isEmpty()) {
            sb.append(dq.pop());
        }

        return sb.reverse().toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

436. [1547]Minimum Cost to Cut a Stick.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minCost(int n, int[] cuts) {
        List<Integer> list = new ArrayList<>();
        for (int num : cuts) {
            list.add(num);
        }
        list.add(0);
        list.add(n);
        Collections.sort(list);
        int k = list.size();
        int[][] dp = new int[k][k];
        for (int d = 2; d < k; d++) {
            for (int i = 0; i < k - d; i++) {
                dp[i][i + d] = Integer.MAX_VALUE;
                for (int m = i + 1; m < i + d; m++) {
                    dp[i][i + d] = Math.min(dp[i][i + d], dp[i][m] + dp[m][i + d] +
list.get(i + d) - list.get(i));
                }
            }
        }
        return dp[0][k - 1];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
Bottom-up
class Solution {
    public int minCost(int n, int[] cuts) {
        List<Integer> list = new ArrayList<>();
        for (int num : cuts) {
            list.add(num);
        }
        list.add(0);
        list.add(n);
        Collections.sort(list);
        int k = list.size();
        int[][] dp = new int[k][k];
        for (int d = 2; d < k; d++) {
            for (int i = 0; i < k - d; i++) {
                dp[i][i + d] = Integer.MAX_VALUE;
                for (int m = i + 1; m < i + d; m++) {
                    dp[i][i + d] = Math.min(dp[i][i + d], dp[i][m] + dp[m][i + d] +
list.get(i + d) - list.get(i));
                }
            }
        }
        return dp[0][k - 1];
    }
}
```

```
Top-Down
class Solution {
    int[][] dp;
    List<Integer> list;

    public int minCost(int n, int[] cuts) {
        list = new ArrayList<>();
        for (int num : cuts) {
            list.add(num);
        }
        list.add(0);
        list.add(n);
        Collections.sort(list);
        int k = list.size();
        dp = new int[k][k];
        for (int d = 2; d < k; d++) {
            for (int i = 0; i < k - d; i++) {
                dp[i][i + d] = dyP(i, i + d);
            }
        }
        return dp[0][k - 1];
    }

    public int dyP(int i, int j) {
        if(dp[i][j]!=0){
            return dp[i][j];
        }
        dp[i][j] = Integer.MAX_VALUE;
        for (int m = i + 1; m < j; m++) {
            dp[i][j] = Math.min(dp[i][j], dp[i][m] + dp[m][j] + list.get(j) -
list.get(i));
        }
        return dp[i][j];
    }
}
 */
```

437. [1548]The Most Similar Path in a Graph.java

```
/*
還不會
 */
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> mostSimilar(int n, int[][] roads, String[] names, String[]
targetPath) {
        boolean[][] graph = new boolean[n][n];
        for (int[] r : roads) {
            graph[r[0]][r[1]] = true;
            graph[r[1]][r[0]] = true;
        }
        int m = targetPath.length;
        int[][] dp = new int[m][n];
        int[][] prev = new int[m][n];
        for (int j = 0; j < n; j++) dp[0][j] = names[j].equals(targetPath[0]) ? 0 : 1;
        for (int i = 1; i < m; i++) {
            for (int j = 0; j < n; j++) {
                int min = m + 1;
                for (int k = 0; k < n; k++) {
                    if (graph[k][j]) {
                        if (dp[i - 1][k] < min) {
                            min = dp[i - 1][k];
                            prev[i][j] = k;
                        }
                    }
                }
                dp[i][j] = min + (names[j].equals(targetPath[i]) ? 0 : 1);
            }
        }
        List<Integer> res = new ArrayList<>();
        int min = m + 1, candidate = 0;
        for (int j = 0; j < n; j++) {
            if (dp[m - 1][j] < min) {
                min = dp[m - 1][j];
                candidate = j;
            }
        }
        res.add(candidate);
        for (int i = m - 1; i > 0; i--) {
            candidate = prev[i][candidate];
            res.add(candidate);
        }
        Collections.reverse(res);
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

438. `[1550]Three Consecutive Odds.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public boolean threeConsecutiveOdds(int[] arr) {
    int oddCount = 0;

    for (int num : arr) {
      if (num % 2 == 1) {
        oddCount++;
      } else {
        oddCount = 0;
      }

      if (oddCount == 3) {
        return true;
      }
    }

    return false;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

439. [1552]Magnetic Force Between Two Balls.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxDistance(int[] position, int m) {
        Arrays.sort(position);
        int left = 1;
        int right = position[position.length - 1] - position[0];
        int result = 0;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (canPlaceBall(position, m, mid)) {
                result = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return result;
    }

    private boolean canPlaceBall(int[] position, int m, int minDist) {
        int count = 1;
        int lastPos = position[0];
        int n = position.length;

        for (int i = 1; i < n; i++) {
            if (position[i] - lastPos >= minDist) {
                count++;
                lastPos = position[i];

                if (count == m) {
                    return true;
                }
            }
        }

        return false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

440. [1557]Minimum Number of Vertices to Reach All Nodes.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> findSmallestSetOfVertices(int n, List<List<Integer>> edges) {
        int[] inDegree = new int[n];
        for (List<Integer> edge : edges) {
            inDegree[edge.get(1)]++;
        }
        List<Integer> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if (inDegree[i] == 0) {
                list.add(i);
            }
        }
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

441. [1561]Maximum Number of Coins You Can Get.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxCoins(int[] piles) {
        int n = piles.length;
        int sum = 0;
        Arrays.sort(piles);

        for (int i = (n / 3); i < n; i += 2) {
            sum += piles[i];
        }

        return sum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

442. `[1567]Maximum Length of Subarray With Positive Product.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int getMaxLen(int[] nums) {
        int pos = 0;
        int neg = 0;
        int res = 0;
        for (int num : nums) {
            if (num == 0) {
                pos = 0;
                neg = 0;
            } else if (num > 0) {
                pos++;
                neg = neg == 0 ? 0 : neg + 1;
            } else {
                int temp = pos;
                pos = neg == 0 ? 0 : neg + 1;
                neg = temp + 1;
            }
            res = Math.max(res, pos);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

443. [1569]Number of Ways to Reorder Array to Get Same

BST.java

```
/*
還不會
 */
//leetcode submit region begin(Prohibit modification and deletion)

import java.math.BigInteger;

class Solution {
    private long MODULUS_LONG = (long) 1e9 + 7;
    private BigInteger MODULUS_BIG = BigInteger.valueOf(MODULUS_LONG);
    public int numOfWays(int[] nums) {
        Bst bst = new Bst(nums[0]);
        for (int i = 1; i < nums.length; ++i) {
            bst.add(nums[i]);
        }
        return countWays(bst) - 1;
    }

    private int countWays(Bst bst) {
        if (bst == null) {
            return 1;
        } else if (bst.left == null) {
            return countWays(bst.right);
        } else if (bst.right == null) {
            return countWays(bst.left);
        } else {
            long a = countWays(bst.left);
            long b = countWays(bst.right);
            long c = nCr(bst.size - 1, bst.left.size);
            return (int) ((a * b) % MODULUS_LONG * c % MODULUS_LONG);
        }
    }

    private int nCr(int n, int r) {
        if (n < 2 * r) {
            return nCr(n, n - r);
        }
        BigInteger result = BigInteger.ONE;
        for (int i = 0; i < r; i++) {
            result = result.multiply(BigInteger.valueOf(n - i));
            result = result.divide(BigInteger.valueOf(i + 1));
        }
        return result.mod(MODULUS_BIG).intValue();
    }

    private class Bst {
        private int rootVal;
        private Bst left;
        private Bst right;
        private int size;
        public Bst(int val) {
```

```
            this.rootVal = val;
            this.size = 1;
        }
        public void add(int val) {
            if (val < this.rootVal) {
                if (this.left == null) {
                    this.left = new Bst(val);
                } else {
                    this.left.add(val);
                }
            } else {
                if (this.right == null) {
                    this.right = new Bst(val);
                } else {
                    this.right.add(val);
                }
            }
            this.size++;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

444. [1572]Matrix Diagonal Sum.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int diagonalSum(int[][] mat) {
        int n = mat.length;
        int res = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j || i + j == n - 1) {
                    res += mat[i][j];
                }
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

445. `[1574]Shortest Subarray to be Removed to Make Array Sorted.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findLengthOfShortestSubarray(int[] arr) {
        int len = arr.length;
        int left = 0;
        int right = len - 1;

        while (left + 1 < len && arr[left] <= arr[left + 1]) {
            left++;
        }

        while (right - 1 >= left && arr[right] >= arr[right - 1]) {
            right--;
        }

        if (left == right) {
            return 0;
        }

        int ans = Math.min(len - left - 1, right);
        int i = 0;
        int j = right;

        while (i <= left && j < len) {
            if (arr[i] <= arr[j]) {
                ans = Math.min(ans, j - i - 1);
                i++;
            } else {
                j++;
            }
        }

        return ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

446. [1575]Count All Possible Routes.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private final int MOD = (int) 1e9 + 7;
    int[][] dp;
    int[] locations;
    int n;

    public int countRoutes(int[] locations, int start, int finish, int fuel) {
        this.locations = locations;
        n = locations.length;
        dp = new int[n][fuel + 1];
        for (int[] arr : dp) {
            Arrays.fill(arr, -1);
        }
        return dyP(start, finish, fuel);
    }

    private int dyP(int curr, int finish, int fuel) {
        if (dp[curr][fuel] != -1) {
            return dp[curr][fuel];
        }
        if (fuel == 0 && curr != finish) {
            dp[curr][fuel] = 0;
            return 0;
        }
        if (fuel > 0 && Math.abs(locations[curr] - locations[finish]) > fuel) {
            dp[curr][fuel] = 0;
            return 0;
        }
        int count = curr == finish ? 1 : 0;
        for (int next = 0; next < n; next++) {
            if (next != curr) {
                int cost = Math.abs(locations[curr] - locations[next]);
                if (cost <= fuel) {
                    count = (count + dyP(next, finish, fuel - cost)) % MOD;
                }
            }
        }
        dp[curr][fuel] = count;
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    private final int MOD = (int) 1e9 + 7;
    int[][] dp;
    int[] locations;
    int n;
```

```
    public int countRoutes(int[] locations, int start, int finish, int fuel) {
        this.locations = locations;
        n = locations.length;
        dp = new int[n][fuel + 1];
        for (int[] arr : dp) {
            Arrays.fill(arr, -1);
        }
        return dyP(start, finish, fuel);
    }

    private int dyP(int curr, int finish, int fuel) {
        if (fuel < 0) {
            return 0;
        }
        if (dp[curr][fuel] != -1) {
            return dp[curr][fuel];
        }
        int count = curr == finish ? 1 : 0;
        for (int next = 0; next < n; next++) {
            if (next != curr) {
                int cost = Math.abs(locations[curr] - locations[next]);
                count = (count + dyP(next, finish, fuel - cost)) % MOD;
            }
        }
        dp[curr][fuel] = count;
        return count;
    }
}
 */
```

447. [1578]Minimum Time to Make Rope Colorful.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minCost(String colors, int[] neededTime) {
        int totalTime = 0;

        for (int i = 1; i < neededTime.length; i++) {
            if (colors.charAt(i - 1) == colors.charAt(i)) {
                if (neededTime[i - 1] <= neededTime[i]) {
                    totalTime += neededTime[i - 1];
                } else {
                    totalTime += neededTime[i];
                    neededTime[i] = neededTime[i - 1];
                }
            }
        }

        return totalTime;
    }
}
//leetcode submit region end(Prohibit modification and deletion
/*
class Solution {
    public int minCost(String colors, int[] neededTime) {
        int n = neededTime.length;
        int left = 0;
        int right = 1;
        int sum = 0;

        while (right < n) {
            int tempSum = 0;

            if (colors.charAt(right) == colors.charAt(left)) {
                int max = neededTime[left];

                while (right < n && colors.charAt(right) == colors.charAt(left)) {
                    if (max < neededTime[right]) {
                        tempSum += max;
                        max = neededTime[right];
                    } else {
                        tempSum += neededTime[right];
                    }

                    right++;
                }
            }

            sum += tempSum;
            left = right;
            right++;
        }
```

```
        return sum;
    }
}
 */
```

448. [1579]Remove Max Number of Edges to Keep Graph

Fully Traversable.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxNumEdgesToRemove(int n, int[][] edges) {
        if (edges.length < n - 1) {
            return -1;
        }

        int[] parentAlice = new int[n + 1];

        for (int i = 0; i < parentAlice.length; i++) {
            parentAlice[i] = i;
        }

        int numType3Edges = 0;

        for (int[] edge : edges) {
            if (edge[0] == 3) {
                if (union(parentAlice, edge[1], edge[2])) {
                    numType3Edges++;
                }
            }
        }

        int[] parentBob = parentAlice.clone();
        int numType1Edges = 0;

        for (int[] edge : edges) {
            if (edge[0] == 1) {
                if (union(parentAlice, edge[1], edge[2])) {
                    numType1Edges++;
                }
            }
        }

        if (numType1Edges + numType3Edges + 1 != n) {
            return -1;
        }

        int numType2Edges = 0;

        for (int[] edge : edges) {
            if (edge[0] == 2) {
                if (union(parentBob, edge[1], edge[2])) {
                    numType2Edges++;
                }
            }
        }

        if (numType2Edges + numType3Edges + 1 != n) {
            return -1;
```

```
        }

        return edges.length - numType1Edges - numType2Edges - numType3Edges;
    }

    int find(int[] parent, int x) {
        return parent[x] == x ? parent[x] : (parent[x] = find(parent, parent[x]));
    }

    boolean union(int[] parent, int i, int j) {
        int x = find(parent, i);
        int y = find(parent, j);

        if (x == y) {
            return false;
        }

        parent[y] = x;
        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

449. [1580]Put Boxes Into the Warehouse II.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxBoxesInWarehouse(int[] boxes, int[] warehouse) {
        Arrays.sort(boxes);

        int left = 0;
        int right = warehouse.length - 1;
        int count = 0;
        int index = boxes.length - 1;

        while (left <= right && index >= 0) {
            if (boxes[index] <= warehouse[left]) {
                count++;
                left++;
            } else if (boxes[index] <= warehouse[right]) {
                count++;
                right--;
            }

            index--;
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

450. [1582]Special Positions in a Binary Matrix.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numSpecial(int[][] mat) {
        int count = 0;

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[0].length; j++) {
                if (mat[i][j] == 1) {
                    if (checkCol(mat, i, j) && checkRow(mat, i, j)) {
                        count++;
                    }
                }
            }
        }

        return count;
    }

    private boolean checkRow(int[][] mat, int r, int c) {
        for (int i = 0; i < mat.length; i++) {
            if (i == r) {
                continue;
            }

            if (mat[i][c] == 1) {
                return false;
            }
        }

        return true;
    }

    private boolean checkCol(int[][] mat, int r, int c) {
        for (int i = 0; i < mat[0].length; i++) {
            if (i == c) {
                continue;
            }

            if (mat[r][i] == 1) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

451. [1584]Min Cost to Connect All Points.java

```
1584
        Min Cost to Connect All Points
        2023-02-10 09:55:56

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minCostConnectPoints(int[][] points) {
        int n = points.length;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[2] - b[2]);
        for (int i = 0; i < n - 1; i++) {
            for (int j = i + 1; j < n; j++) {
                int len = getDist(points[i], points[j]);
                pq.add(new int[]{i, j, len});
            }
        }
        UnionFind uf = new UnionFind(n);
        int count = 1;
        int res = 0;
        while (count < n) {
            int[] curr = pq.poll();
            if (uf.union(curr[0], curr[1])) {
                res += curr[2];
                count++;
            }
        }
        return res;
    }

    public int getDist(int[] x, int[] y) {
        return Math.abs(x[0] - y[0]) + Math.abs(x[1] - y[1]);
    }
}

class UnionFind {
    private int[] group;

    public UnionFind(int n) {
        group = new int[n];
        for (int i = 0; i < n; i++) {
            group[i] = i;
        }
    }

    private int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }

    public boolean union(int i, int j) {
        int x = find(i);
        int y = find(j);
        if (x == y) {
```

```
            return false;
        }
        group[x] = y;
        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)

/*
class Solution {
    public int minCostConnectPoints(int[][] points) {
        int n = points.length;
        int res = 0;
        int count = 0;
        boolean[] isVisited = new boolean[n];
        int[] arr = new int[n];
        arr[0] = 0;
        for (int i = 1; i < n; i++) {
            arr[i] = Integer.MAX_VALUE;
        }
        while (count < n) {
            int num = Integer.MAX_VALUE;
            int vertex = -1;
            for (int i = 0; i < n; i++) {
                if (!isVisited[i] && num > arr[i]) {
                    num = arr[i];
                    vertex = i;
                }
            }
            res += num;
            count++;
            isVisited[vertex] = true;
            for (int i = 0; i < n; i++) {
                int weight = Math.abs(points[vertex][0] - points[i][0]) +
Math.abs(points[vertex][1] - points[i][1]);
                if (!isVisited[i] && arr[i] > weight) {
                    arr[i] = weight;
                }
            }
        }
        return res;
    }
}
 */
```

452. [1590]Make Sum Divisible by P.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minSubarray(int[] nums, int p) {
        long total = 0;

        for (int num : nums) {
            total += num;
        }

        long r = total % p;

        if (r == 0) {
            return 0;
        }

        Map<Long, Integer> map = new HashMap<>();
        map.put(0L, -1);
        long prefix = 0;
        int n = nums.length;
        int min = n;

        for (int i = 0; i < n; i++) {
            prefix = (prefix + nums[i]) % p;
            long need = (prefix - r + p) % p;

            if (map.containsKey(need)) {
                min = Math.min(min, i - map.get(need));

                if (min == 1) {
                    return 1;
                }
            }

            map.put(prefix, i);
        }

        return min == n ? -1 : min;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

453. [1593]Split a String Into the Max Number of Unique Substrings.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxUniqueSplit(String s) {
        Set<String> used = new HashSet<>();
        int[] maxCount = {0};
        backtrack(s, 0, used, 0, maxCount);
        return maxCount[0];
    }

    private void backtrack(String s, int start, Set<String> used, int count, int[]
maxCount) {
        if (start == s.length()) {
            maxCount[0] = Math.max(maxCount[0], count);
            return;
        }

        for (int end = start + 1; end <= s.length(); end++) {
            String substring = s.substring(start, end);

            if (!used.contains(substring)) {
                used.add(substring);
                backtrack(s, end, used, count + 1, maxCount);
                used.remove(substring);
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

454. `[1598]Crawler Log Folder.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minOperations(String[] logs) {
        int depth = 0;

        for (String log : logs) {
            if ("../".equals(log)) {
                if (depth > 0) {
                    depth--;
                }
            } else if (!"./".equals(log)) {
                depth++;
            }
        }

        return depth;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

455. [1601]Maximum Number of Achievable Transfer Requests.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int res = 0;

    public int maximumRequests(int n, int[][] requests) {
        backtrack(requests, new int[n], 0, 0);
        return res;
    }

    private void backtrack(int[][] requests, int[] inDegree, int index, int count) {
        if (index == requests.length) {
            for (int num : inDegree) {
                if (num != 0) {
                    return;
                }
            }
            res = Math.max(res, count);
            return;
        }
        int x = requests[index][0];
        int y = requests[index][1];
        inDegree[x]--;
        inDegree[y]++;
        backtrack(requests, inDegree, index + 1, count + 1);
        inDegree[x]++;
        inDegree[y]--;
        backtrack(requests, inDegree, index + 1, count);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

456. `[1603]Design Parking System.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class ParkingSystem {
    int[] arr;
    int[] park;

    public ParkingSystem(int big, int medium, int small) {
        arr = new int[4];
        park = new int[4];
        park[1] = big;
        park[2] = medium;
        park[3] = small;
    }

    public boolean addCar(int carType) {
        arr[carType]++;
        return arr[carType] > park[carType] ? false : true;
    }
}

/**
 * Your ParkingSystem object will be instantiated and called as such:
 * ParkingSystem obj = new ParkingSystem(big, medium, small);
 * boolean param_1 = obj.addCar(carType);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

457. [1605]Find Valid Matrix Given Row and Column Sums.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[][] restoreMatrix(int[] rowSum, int[] colSum) {
        int rows = rowSum.length;
        int cols = colSum.length;
        int[][] matrix = new int[rows][cols];
        int i = rows - 1;
        int j = cols - 1;

        while (i >= 0 && j >= 0) {
            if (rowSum[i] <= colSum[j]) {
                matrix[i][j] = rowSum[i];
                colSum[j] -= rowSum[i];
                i--;
            } else {
                matrix[i][j] = colSum[j];
                rowSum[i] -= colSum[j];
                j--;
            }
        }

        return matrix;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

458. [1608]Special Array With X Elements Greater Than or Equal X.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int specialArray(int[] nums) {
        Arrays.sort(nums);
        int n = nums.length;

        if (nums[0] >= n) {
            return n;
        }
su
        for (int i = 1; i < n; i++) {
            if (nums[i] >= n - i) {
                if (nums[i - 1] >= n - i) {
                    return -1;
                }

                return n - i;
            }
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

459. `[1609]Even Odd Tree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isEvenOddTree(TreeNode root) {
        Deque<TreeNode> deque = new ArrayDeque<>();
        deque.offer(root);
        boolean isEvenLevel = true;

        while (!deque.isEmpty()) {
            int size = deque.size();
            int preValue = isEvenLevel ? Integer.MIN_VALUE : Integer.MAX_VALUE;

            for (int i = 0; i < size; i++) {
                TreeNode node = deque.poll();

                if (isEvenLevel) {
                    if (node.val % 2 == 0 || node.val <= preValue) {
                        return false;
                    }
                } else {
                    if (node.val % 2 != 0 || node.val >= preValue) {
                        return false;
                    }
                }

                preValue = node.val;

                if (node.left != null) {
                    deque.offer(node.left);
                }

                if (node.right != null) {
                    deque.offer(node.right);
                }
            }

            isEvenLevel = !isEvenLevel;
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

460. `[1611]Minimum One Bit Operations to Make Integers Zero.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumOneBitOperations(int n) {
        int sign = 1;
        int res = 0;

        while (n > 0) {
            res += n ^ (n - 1) * sign;
            n &= n - 1;
            sign = -sign;
        }

        return Math.abs(res);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int minimumOneBitOperations(int n) {
        return getCount(n, 0);
    }

    private int getCount(int n, int res) {
        if (n == 0) {
            return res;
        }

        int b = 1;

        while ((b << 1) <= n) {
            b = b << 1;
        }

        return getCount((b >> 1) ^ b ^ n, res + b);
    }
}
 */
```

461. [1614]Maximum Nesting Depth of the Parentheses.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxDepth(String s) {
        int current = 0;
        int max = 0;

        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);

            if (c == '(') {
                current++;
                max = Math.max(max, current);
            } else if (c == ')') {
                current--;
            }
        }

        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

462. `[1615]Maximal Network Rank.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximalNetworkRank(int n, int[][] roads) {
        if (roads.length == 0) {
            return 0;
        }
        int[] edge = new int[n];
        int[][] adj = new int[n][n];
        for (int[] road : roads) {
            int x = road[0];
            int y = road[1];
            edge[x]++;
            edge[y]++;
            adj[x][y] = 1;
            adj[y][x] = 1;
        }
        int res = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                res = Math.max(res, edge[i] + edge[j] - adj[i][j]);
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

463. `[1624]Largest Substring Between Two Equal`

Characters.java

```
/**
 * Given a string s, return the length of the longest substring between two equal
 * characters, excluding the two characters. If there is no such substring return -1
 * .
 * <p>
 * A substring is a contiguous sequence of characters within a string.
 * <p>
 * <p>
 * Example 1:
 * <p>
 * <p>
 * Input: s = "aa"
 * Output: 0
 * Explanation: The optimal substring here is an empty substring between the two
 * 'a's.
 * <p>
 * Example 2:
 * <p>
 * <p>
 * Input: s = "abca"
 * Output: 2
 * Explanation: The optimal substring here is "bc".
 * <p>
 * <p>
 * Example 3:
 * <p>
 * <p>
 * Input: s = "cbzxy"
 * Output: -1
 * Explanation: There are no characters that appear twice in s.
 * <p>
 * <p>
 * <p>
 * Constraints:
 * <p>
 * <p>
 * 1 <= s.length <= 300
 * s contains only lowercase English letters.
 * <p>
 * <p>
 * Related Topics Hash Table String 👍 645 👎 38
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxLengthBetweenEqualCharacters(String s) {
        int max = -1;
        int[] arr = new int[26];
        Arrays.fill(arr, -1);

        for (int i = 0; i < s.length(); i++) {
```

```
            int index = s.charAt(i) - 'a';

            if (arr[index] == -1) {
                arr[index] = i;
            } else {
                max = Math.max(max, i - arr[index] - 1);
            }
        }

        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

464. [1626]Best Team With No Conflicts.java

```
1626
        Best Team With No Conflicts
        2023-01-31 09:32:08

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int bestTeamScore(int[] scores, int[] ages) {
        int n = ages.length;
        int[][] arr = new int[n][2];
        for (int i = 0; i < n; i++) {
            arr[i][0] = ages[i];
            arr[i][1] = scores[i];
        }
        Arrays.sort(arr, (a, b) -> (a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]));
        return findMax(arr);
    }

    int findMax(int[][] arr) {
        int n = arr.length;
        int res = 0;
        int[] dp = new int[n];
        for (int i = 0; i < n; i++) {
            dp[i] = arr[i][1];
            res = Math.max(res, dp[i]);
        }
        for (int i = 0; i < n; i++) {
            for (int j = i - 1; j >= 0; j--) {
                if (arr[i][1] >= arr[j][1]) {
                    dp[i] = Math.max(dp[i], arr[i][1] + dp[j]);
                }
            }
            res = Math.max(res, dp[i]);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

465. `[1630]Arithmetic Subarrays.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Boolean> checkArithmeticSubarrays(int[] nums, int[] l, int[] r) {
        List<Boolean> list = new ArrayList<>();
        int n = l.length;

        for (int i = 0; i < n; i++) {
            list.add(isArithmetic(Arrays.copyOfRange(nums, l[i], r[i] + 1)));
        }

        return list;
    }

    private boolean isArithmetic(int[] arr) {
        if (arr.length <= 1) {
            return true;
        }

        Arrays.sort(arr);
        int diff = arr[1] - arr[0];

        for (int i = 2; i < arr.length; i++) {
            if (arr[i] - arr[i - 1] != diff) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public List<Boolean> checkArithmeticSubarrays(int[] nums, int[] l, int[] r) {
        List<Boolean> list = new ArrayList<>();
        int n = l.length;

        for (int i = 0; i < n; i++) {
            list.add(isArithmetic(Arrays.copyOfRange(nums, l[i], r[i] + 1)));
        }

        return list;
    }

    private boolean isArithmetic(int[] arr) {
        if (arr.length <= 1) {
            return true;
        }

        Arrays.sort(arr);
```

```
        int diff = arr[1] - arr[0];
        for (int i = 2; i < arr.length; i++) {
            if (arr[i] - arr[i - 1] != diff) {
                return false;
            }
        }

        return true;
    }
}

class Solution {
    public List<Boolean> checkArithmeticSubarrays(int[] nums, int[] l, int[] r) {
        List<Boolean> list = new ArrayList<>();
        int n = l.length;
        for (int i = 0; i < n; i++) {
            list.add(check(nums, l[i], r[i]));
        }
        return list;
    }

    public boolean check(int[] nums, int left, int right) {
        int n = right - left;
        if (n == 0) {
            return true;
        }
        int max = Integer.MIN_VALUE;
        int min = Integer.MAX_VALUE;
        for (int i = left; i <= right; i++) {
            max = Math.max(max, nums[i]);
            min = Math.min(min, nums[i]);
        }
        if ((max - min) % n != 0) {
            return false;
        }
        int diff = (max - min) / n;
        if (diff == 0) {
            return true;
        }
        boolean[] isCheck = new boolean[max - min + 1];
        for (int i = left; i <= right; i++) {
            int currDiff = nums[i] - min;
            if (isCheck[currDiff] || currDiff % diff != 0) {
                return false;
            }
            isCheck[currDiff] = true;
        }
        return true;
    }
}
 */
```

466. [1631]Path With Minimum Effort.java

```
1631
        Path With Minimum Effort
        2022-12-12 17:29:57

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumEffortPath(int[][] heights) {
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int m = heights.length;
        int n = heights[0].length;
        boolean[][] isVisited = new boolean[m][n];
        int[][] arrs = new int[m][n];
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[2] - b[2]);

        for (int[] arr : arrs) {
            Arrays.fill(arr, Integer.MAX_VALUE);
        }

        arrs[0][0] = 0;
        pq.add(new int[]{0, 0, 0});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int i = curr[0];
            int j = curr[1];
            isVisited[i][j] = true;

            if (i == m - 1 && j == n - 1) {
                return curr[2];
            }

            for (int[] dir : dirs) {
                int x = i + dir[0];
                int y = j + dir[1];

                if (x < 0 || x >= m || y < 0 || y >= n || isVisited[x][y]) {
                    continue;
                }

                int diff = Math.abs(heights[x][y] - heights[i][j]);
                int max = Math.max(diff, curr[2]);

                if (max < arrs[x][y]) {
                    arrs[x][y] = max;
                    pq.add(new int[]{x, y, max});
                }
            }
        }

        return -1;
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

467. [1634]Add Two Polynomials Represented as Linked Lists.java

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
    public PolyNode addPoly(PolyNode poly1, PolyNode poly2) {
        PolyNode dummy = new PolyNode(0, 0);
        PolyNode curr = dummy;

        while (poly1 != null && poly2 != null) {
            if (poly1.power == poly2.power) {
                int sumCoeff = poly1.coefficient + poly2.coefficient;

                if (sumCoeff != 0) {
                    curr.next = new PolyNode(sumCoeff, poly1.power);
                    curr = curr.next;
                }

                poly1 = poly1.next;
                poly2 = poly2.next;
            } else if (poly1.power > poly2.power) {
                curr.next = new PolyNode(poly1.coefficient, poly1.power);
                curr = curr.next;
                poly1 = poly1.next;
            } else {
                curr.next = new PolyNode(poly2.coefficient, poly2.power);
                curr = curr.next;
                poly2 = poly2.next;
            }
        }

        if (poly1 != null) {
            curr.next = poly1;
        }

        if (poly2 != null) {
            curr.next = poly2;
        }

        return dummy.next;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

468. `[1636]Sort Array by Increasing Frequency.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] frequencySort(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();

        for (int num : nums) {
            map.merge(num, 1, Integer::sum);
        }

        return Arrays.stream(nums)
                .boxed()
                .sorted((a, b) -> {
                    int compare = map.get(a).compareTo(map.get(b));
                    return compare == 0 ? b.compareTo(a) : compare;
                })
                .mapToInt(Integer::intValue)
                .toArray();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

469. [1637]Widest Vertical Area Between Two Points Containing No Points.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxWidthOfVerticalArea(int[][] points) {
        int n = points.length;
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = points[i][0];
        }

        Arrays.sort(arr);
        int max = 0;

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] - arr[i - 1] > max) {
                max = arr[i] - arr[i - 1];
            }
        }

        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

470. [1639]Number of Ways to Form a Target String Given a Dictionary.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    final int mod = (int) 1e9 + 7;

    public int numWays(String[] words, String target) {
        char[] t = target.toCharArray();
        int m = words[0].length();
        int n = t.length;
        int[][] count = new int[m][26];
        for (String word : words) {
            for (int i = 0; i < m; i++) {
                count[i][word.charAt(i) - 'a']++;
            }
        }
        long[][] dp = new long[m][n];
        for (int i = 0; i < m; i++) {
            Arrays.fill(dp[i], -1);
        }
        return (int) dyP(dp, count, t, 0, 0, m, n);
    }

    public long dyP(long[][] dp, int[][] count, char[] t, int i, int j, int m, int n) {
        if (j == n) {
            return 1;
        }
        if (m - i < n - j) {
            return 0;
        }
        if (dp[i][j] != -1) {
            return dp[i][j];
        }
        long res = count[i][t[j] - 'a'] * dyP(dp, count, t, i + 1, j + 1, m, n);
        res += dyP(dp, count, t, i + 1, j, m, n);
        res %= mod;
        dp[i][j] = res;
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

471. `[1642]Furthest Building You Can Reach.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int furthestBuilding(int[] heights, int bricks, int ladders) {
        Queue<Integer> pq = new PriorityQueue<>();

        for (int i = 0; i < heights.length - 1; i++) {
            int diff = heights[i + 1] - heights[i];

            if (diff > 0) {
                pq.offer(diff);

                if (pq.size() > ladders) {
                    bricks -= pq.poll();
                }

                if (bricks < 0) {
                    return i;
                }
            }
        }

        return heights.length - 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

472. [1644]Lowest Common Ancestor of a Binary Tree II.java

```
1644
        Lowest Common Ancestor of a Binary Tree II
        2022-12-27 16:53:19
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    int count = 0;

    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode node = dfs(root, p, q);
        return count == 2 ? node : null;
    }

    private TreeNode dfs(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null) {
            return root;
        }
        TreeNode left = dfs(root.left, p, q);
        TreeNode right = dfs(root.right, p, q);
        if (root == p || root == q) {
            count++;
            return root;
        }
        if (left != null && right != null) {
            return root;
        }
        return left == null ? right : left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

473. `[1647]Minimum Deletions to Make Character Frequencies Unique.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minDeletions(String s) {
        int[] freq = new int[26];

        for (char c : s.toCharArray()) {
            freq[c - 'a']++;
        }

        Set<Integer> set = new HashSet<>();
        int res = 0;

        for (int num : freq) {
            int curr = num;

            while (curr > 0) {
                if (!set.contains(curr)) {
                    set.add(curr);
                    break;
                }

                curr--;
                res++;
            }
        }

        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

474. [1650]Lowest Common Ancestor of a Binary Tree III.java

```
1650
        Lowest Common Ancestor of a Binary Tree III
        2022-12-27 16:53:21
//leetcode submit region begin(Prohibit modification and deletion)
/*
// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node parent;
};
*/

class Solution {
    public Node lowestCommonAncestor(Node p, Node q) {
        Node x = p;
        Node y = q;
        while (x != y) {
            x = x == null ? q : x.parent;
            y = y == null ? p : y.parent;
        }
        return x;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

475. `[1652]Defuse the Bomb.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] decrypt(int[] code, int k) {
        int n = code.length;
        int[] result = new int[n];

        if (k == 0) {
            return result;
        }

        int determint = Math.abs(k);
        boolean isPositive = k > 0;

        for (int i = 0; i < n; i++) {
            int sum = 0;

            for (int j = 1; j <= determint; j++) {
                int index;

                if (isPositive) {
                    index = (i + j) % n;
                } else {
                    index = (i - j + n) % n;
                }

                sum += code[index];
            }

            result[i] = sum;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

476. `[1653]Minimum Deletions to Make String Balanced.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumDeletions(String s) {
        int count = 0;
        int result = 0;

        for (char c : s.toCharArray()) {
            if (c == 'b') {
                count++;
            } else if (count != 0) {
                result++;
                count--;
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

477. [1654]Minimum Jumps to Reach Home.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumJumps(int[] forbidden, int a, int b, int x) {
        Set<Integer> set = new HashSet<>();
        for (int num : forbidden) {
            set.add(num);
        }
        Deque<Node> dq = new ArrayDeque<>();
        dq.add(new Node(0, false));
        set.add(0);
        int count = -1;
        int max = 6000;
        while (!dq.isEmpty()) {
            int size = dq.size();
            count++;
            for (int i = 0; i < size; i++) {
                Node curr = dq.poll();
                if (curr.val == x) {
                    return count;
                }
                int next = 0;
                if (!curr.isBack) {
                    next = curr.val - b;
                    if (next > 0 && !set.contains(next)) {
                        dq.add(new Node(next, true));
                        set.add(next);
                    }
                }
                next = curr.val + a;
                if (next < max && !set.contains(next)) {
                    dq.add(new Node(next, false));
                    set.add(next);
                }
            }
        }
        return -1;
    }
}

class Node {
    int val;
    boolean isBack;

    public Node(int val, boolean isBack) {
        this.val = val;
        this.isBack = isBack;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

478. `[1656].java`

```
1656
        Design an Ordered Stream
        2023-03-07 20:26:21

//leetcode submit region begin(Prohibit modification and deletion)
class OrderedStream {
    String[] data;
    int index;
    int n;

    public OrderedStream(int n) {
        index = 0;
        this.n = n;
        data = new String[n];
    }

    public List<String> insert(int idKey, String value) {
        List<String> list = new ArrayList<>();
        data[idKey - 1] = value;
        while (index < n && data[index] != null) {
            list.add(data[index]);
            index++;
        }
        return list;
    }
}

/**
 * Your OrderedStream object will be instantiated and called as such:
 * OrderedStream obj = new OrderedStream(n);
 * List<String> param_1 = obj.insert(idKey,value);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

479. [1657]Determine if Two Strings Are Close.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean closeStrings(String word1, String word2) {
        if (word1.length() != word2.length())
            return false;
        int[] arr1 = getCount(word1);
        int[] arr2 = getCount(word2);

        for (int i = 0; i < 26; i++) {
            if (arr1[i] == 0 && arr2[i] > 0 || ar1657r2[i] == 0 && arr1[i] > 0) {
                return false;
            }
        }

        Arrays.sort(arr1);
        Arrays.sort(arr2);
        return Arrays.equals(arr1, arr2);
    }

    private int[] getCount(String s) {
        int[] arr = new int[26];

        for (char c : s.toCharArray()) {
            arr[c - 'a']++;
        }

        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

480. [1658]Minimum Operations to Reduce X to Zero.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minOperations(int[] nums, int x) {
        int total = Arrays.stream(nums).sum();
        int n = nums.length;
        int target = total - x;
        int left = 0;
        int max = -1;
        int current = 0;

        for (int right = 0; right < n; right++) {
            current += nums[right];

            while (current > target && left <= right) {
                current -= nums[left];
                left++;
            }

            if (current == target) {
                max = Math.max(max, right - left + 1);
            }
        }

        return max == -1 ? max : n - max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

481. `[1660]Correct a Binary Tree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    Set<Integer> visited = new HashSet<>();

    public TreeNode correctBinaryTree(TreeNode root) {
        if (root == null) {
            return null;
        }

        if (root.right != null && visited.contains(root.right.val)) {
            return null;
        }

        visited.add(root.val);
        root.right = correctBinaryTree(root.right);
        root.left = correctBinaryTree(root.left);

        return root;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public TreeNode correctBinaryTree(TreeNode root) {
        Deque<TreeNode[]> deque = new ArrayDeque<>();
        deque.add(new TreeNode[]{root, null});

        while (!deque.isEmpty()) {
            int n = deque.size();
            Set<TreeNode> visited = new HashSet<>();

            for (int i = 0; i < n; i++) {
                TreeNode[] curr = deque.poll();
                TreeNode node = curr[0];
```

```
                TreeNode parent = curr[1];

                if (visited.contains(node.right)) {
                    if (parent.left == node) {
                        parent.left = null;
                    } else {
                        parent.right = null;
                    }

                    return root;
                }

                visited.add(node);

                if (node.right != null) {
                    deque.add(new TreeNode[]{node.right, node});
                }
                if (node.left != null) {
                    deque.add(new TreeNode[]{node.left, node});
                }
            }
        }

        return root;
    }
}

class Solution {
    Set<Integer> visited = new HashSet<>();

    public TreeNode correctBinaryTree(TreeNode root) {
        if (root == null) {
            return null;
        }

        if (root.right != null && visited.contains(root.right.val)) {
            return null;
        }

        visited.add(root.val);
        root.right = correctBinaryTree(root.right);
        root.left = correctBinaryTree(root.left);

        return root;
    }
}
 */
```

482. `[1662]Check If Two String Arrays are Equivalent.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean arrayStringsAreEqual(String[] word1, String[] word2) {
        StringBuilder sb1 = new StringBuilder();
        StringBuilder sb2 = new StringBuilder();

        for (String word : word1) {
            sb1.append(word);
        }

        for (String word : word2) {
            sb2.append(word);
        }

        return sb1.toString().equals(sb2.toString());
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

483. `[1669]Merge In Between Linked Lists.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public ListNode mergeInBetween(ListNode list1, int a, int b, ListNode list2) {
        ListNode dummy = new ListNode(0);
        dummy.next = list1;
        ListNode prev = dummy;

        for (int i = 0; i < a; i++) {
            prev = prev.next;
        }

        ListNode end = prev;

        for (int i = a; i <= b; i++) {
            end = end.next;
        }

        prev.next = list2;

        while (list2.next != null) {
            list2 = list2.next;
        }

        list2.next = end.next;
        return dummy.next;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

484. [1672]Richest Customer Wealth.java

```
1672
        Richest Customer Wealth
        2022-12-05 18:07:26

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumWealth(int[][] accounts) {
        int max = Integer.MIN_VALUE;
        for (int[] arr : accounts) {
            int sum = 0;
            for (int num : arr) {
                sum += num;
            }
            max = Math.max(max, sum);
        }
        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

485. [1675]Minimize Deviation in Array.java

```
1675
        Minimize Deviation in Array
        2023-02-24 09:32:00

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumDeviation(int[] nums) {
        TreeSet<Integer> set = new TreeSet<>();
        for (int num : nums) {
            if (num % 2 == 1) {
                set.add(num * 2);
            } else {
                set.add(num);
            }
        }
        int minDiff = set.last() - set.first();
        while (set.last() % 2 == 0) {
            set.add(set.last() / 2);
            set.remove(set.last());
            minDiff = Math.min(minDiff, set.last() - set.first());
        }
        return minDiff;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

486. [1676]Lowest Common Ancestor of a Binary Tree IV.java

```
1676
        Lowest Common Ancestor of a Binary Tree IV
        2022-12-27 16:53:24
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode[] nodes) {
        TreeNode node = dfs(root, nodes);
        return node;
    }

    public TreeNode dfs(TreeNode root, TreeNode[] nodes) {
        if (root == null) {
            return root;
        }
        for (TreeNode node : nodes) {
            if (root == node) {
                return root;
            }
        }
        TreeNode left = dfs(root.left, nodes);
        TreeNode right = dfs(root.right, nodes);
        if (left != null && right != null) {
            return root;
        }
        return left == null ? right : left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

487. [1682]Longest Palindromic Subsequence II.java

```
/**
 * <p>A subsequence of a string <code>s</code> is considered a <strong>good palindromic
subsequence</strong> if:</p>
 *
 * <ul>
 * <li>It is a subsequence of <code>s</code>.</li>
 * <li>It is a palindrome (has the same value if reversed).</li>
 * <li>It has an <strong>even</strong> length.</li>
 * <li>No two consecutive characters are equal, except the two middle ones.</li>
 * </ul>
 *
 * <p>For example, if <code>s = "abcabcabb"</code>, then <code>"abba"</code> is
considered a <strong>good palindromic subsequence</strong>, while <code>"bcb"</code> (not
even length) and <code>"bbbb"</code> (has equal consecutive characters) are not.</p>
 *
 * <p>Given a string <code>s</code>, return <em>the <strong>length</strong> of the
<strong>longest good palindromic subsequence</strong> in </em><code>s</code>.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> s = "bbabab"
 * <strong>Output:</strong> 4
 * <strong>Explanation:</strong> The longest good palindromic subsequence of s is "baab".
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> s = "dcbccacdb"
 * <strong>Output:</strong> 4
 * <strong>Explanation:</strong> The longest good palindromic subsequence of s is "dccd".
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>1 &lt;= s.length &lt;= 250</code></li>
 * <li><code>s</code> consists of lowercase English letters.</li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>String</li><li>Dynamic Programming</li></div>
</div><br><div><li>👍 134</li><li>👎 28</li></div>
 */
/*
還不會
 */
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
```

```
    public int longestPalindromeSubseq(String s) {
        char[] c = s.toCharArray();
        int n = c.length;
        int[] dp = new int[n];
        int res = 0;
        for (int i = 1; i < n; i++) {
            int max = 0;
            for (int j = i - 1; j >= 0; j--) {
                if (c[i] == c[j]) {
                    dp[j] = max + 2;
                    res = Math.max(res, dp[j]);
                } else {
                    max = Math.max(max, dp[j]);
                }
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int longestPalindromeSubseq(String s) {
        char[] c = s.toCharArray();
        int n = c.length;
        int[] dp = new int[n];
        int res = 0;
        for (int i = 1; i < n; i++) {
            int max = 0;
            for (int j = i - 1; j >= 0; j--) {
                if (c[i] == c[j]) {
                    res = Math.max(res, dp[j] = max + 2);
                } else {
                    max = Math.max(max, dp[j]);
                }
            }
        }
        return res;
    }
}
 */
```

488. [1684]Count the Number of Consistent Strings.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countConsistentStrings(String allowed, String[] words) {
        boolean[] charSet = new boolean[26];

        for (char c : allowed.toCharArray()) {
            charSet[c - 'a'] = true;
        }

        int count = 0;

        for (String word : words) {
            boolean isConsistent = true;
            int n = word.length();

            for (int i = 0; i < n; i++) {
                if (!charSet[word.charAt(i) - 'a']) {
                    isConsistent = false;
                    break;
                }
            }

            if (isConsistent) {
                count++;
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

489. `[1685]Sum of Absolute Differences in a Sorted`

Array.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] getSumAbsoluteDifferences(int[] nums) {
        int n = nums.length;
        int[] arr = new int[n];
        int leftSum = 0;
        int rightSum = 0;

        for (int num : nums) {
            rightSum += num;
        }

        for (int i = 0; i < n; i++) {
            rightSum -= nums[i];
            arr[i] = (nums[i] * i - leftSum) + (rightSum - nums[i] * (n - i - 1));
            leftSum += nums[i];
        }

        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*

public class Solution {
    public int[] getSumAbsoluteDifferences(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        int leftSum = 0, rightSum = 0;

        // Calculate the initial total sum (rightSum)
        for (int num : nums) {
            rightSum += num;
        }

        // Iterate over the array
        for (int i = 0; i < n; i++) {
            // Update the right sum by subtracting the current element
            rightSum -= nums[i];

            // Calculate the sum of absolute differences for nums[i]
            // Left part: nums[i] * i - leftSum
            // Right part: rightSum - nums[i] * (n - i - 1)
            result[i] = nums[i] * i - leftSum + rightSum - nums[i] * (n - i - 1);

            // Update the left sum by adding the current element
            leftSum += nums[i];
        }

        return result;
```

```
    }
}

 */
```

490. [1688]Count of Matches in Tournament.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfMatches(int n) {
        return n - 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

491. `[1694]Reformat Phone Number.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String reformatNumber(String number) {
        StringBuilder sb = new StringBuilder();
        int count = 0;
        for (char c : number.toCharArray()) {
            if (c == ' ' || c == '-') {
                continue;
            }
            if (count == 3) {
                count = 0;
                sb.append("-");
            }
            count++;
            sb.append(c);
        }
        if (count == 1) {
            int n = sb.length();
            char c = sb.charAt(n - 3);
            sb.setCharAt(n - 2, c);
            sb.setCharAt(n - 3, '-');
        }
        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
public String reformatNumber(String number) {
    return number.replaceAll("\\D", "").replaceAll("...?(?=..)", "$0-");
}
 */
```

492. [1695]Maximum Erasure Value.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumUniqueSubarray(int[] nums) {
        Set<Integer> set = new HashSet<>();
        int left = 0;
        int maxSum = 0;
        int windowSum = 0;

        for (int right = 0; right < nums.length; right++) {
            int current = nums[right];
            while (set.contains(current)) {
                int leftVal = nums[left++];
                set.remove(leftVal);
                windowSum -= leftVal;
            }

            set.add(current);
            windowSum += current;
            maxSum = Math.max(maxSum, windowSum);
        }

        return maxSum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

493. [1696]Jump Game VI.java

```
1696
Jump Game VI
2023-02-08 22:16:16
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxResult(int[] nums, int k) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

494. `[1697]Checking Existence of Edge Length Limited`

Paths.java

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
    public boolean[] distanceLimitedPathsExist(int n, int[][] edgeList, int[][] queries)
{
        UnionFind uf = new UnionFind(n);
        int count = queries.length;
        boolean[] res = new boolean[count];
        int[][] arrs = new int[count][4];
        for (int i = 0; i < count; ++i) {
            arrs[i][0] = queries[i][0];
            arrs[i][1] = queries[i][1];
            arrs[i][2] = queries[i][2];
            arrs[i][3] = i;
        }
        Arrays.sort(edgeList, (a, b) -> a[2] - b[2]);
        Arrays.sort(arrs, (a, b) -> a[2] - b[2]);
        int edgesIndex = 0;
        for (int queryIndex = 0; queryIndex < count; queryIndex += 1) {
            int p = arrs[queryIndex][0];
            int q = arrs[queryIndex][1];
            int limit = arrs[queryIndex][2];
            int queryOriginalIndex = arrs[queryIndex][3];
            while (edgesIndex < edgeList.length && edgeList[edgesIndex][2] < limit) {
                int x = edgeList[edgesIndex][0];
                int y = edgeList[edgesIndex][1];
                uf.join(x, y);
                edgesIndex += 1;
            }
            res[queryOriginalIndex] = uf.areConnected(p, q);
        }
        return res;
    }
}

class UnionFind {
    private int[] group;
    private int[] rank;

    UnionFind(int size) {
        group = new int[size];
        rank = new int[size];
        for (int i = 0; i < size; ++i) {
            group[i] = i;
        }
    }

    public int find(int x) {
        return group[x] == x ? group[x] : (group[x] = find(group[x]));
    }
```

```
    public void join(int i, int j) {
        int x = find(i);
        int y = find(j);
        if (x == y) {
            return;
        }
        if (rank[x] > rank[y]) {
            group[y] = x;
        } else if (rank[x] < rank[y]) {
            group[x] = y;
        } else {
            group[x] = y;
            rank[y]++;
        }
    }

    public boolean areConnected(int i, int j) {
        int x = find(i);
        int y = find(j);
        return x == y;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

495. [1700]Number of Students Unable to Eat Lunch.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countStudents(int[] students, int[] sandwiches) {
        int[] count = new int[2];

        for (int student : students) {
            count[student]++;
        }

        for (int sandwich : sandwiches) {
            if (count[sandwich] == 0) {
                break;
            }
            count[sandwich]--;
        }

        return count[0] + count[1];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

496. `[1701]Average Waiting Time.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public double averageWaitingTime(int[][] customers) {
        int currentTime = 0;
        long totalWaitingTime = 0;

        for (int[] customer : customers) {
            int arrivalTime = customer[0];
            int preparationTime = customer[1];

            if (currentTime < arrivalTime) {
                currentTime = arrivalTime;
            }

            currentTime += preparationTime;
            totalWaitingTime += (currentTime - arrivalTime);
        }

        return (double) totalWaitingTime / customers.length;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

497. [1704]Determine if String Halves Are Alike.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean halvesAreAlike(String s) {
        Set<Character> vowels = new HashSet<>(Arrays.asList('a', 'e', 'i', 'o', 'u', 'A',
'E', 'I', 'O', 'U'));
        int mid = s.length() / 2;
        int countFirst = getCount(s.substring(0, mid), vowels);
        int countSecond = getCount(s.substring(mid), vowels);
        return countFirst == countSecond;
    }

    private int getCount(String s, Set<Character> set) {
        int count = 0;

        for (char c : s.toCharArray()) {
            if (set.contains(c)) {
                count++;
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

498. [1706]Where Will the Ball Fall.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] findBall(int[][] grid) {
        int[] arr = new int[grid[0].length];
        for (int i = 0; i < grid[0].length; i++) {
            arr[i] = dfs(grid, 0, i);
        }
        return arr;
    }

    public int dfs(int[][] grid, int i, int j) {
        if (i == grid.length) {
            return j;
        }
        int y = j + grid[i][j];
        if (y < 0 || y >= grid[0].length || grid[i][j] != grid[i][y]) {
            return -1;
        }
        return dfs(grid, i + 1, y);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

499. [1708]Largest Subarray Length K.java

```
1708
        Largest Subarray Length K
        2023-02-27 11:00:29

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] largestSubarray(int[] nums, int k) {
        int n = nums.length;
        int index = 0;
        for (int i = 1; i <= n - k; i++) {
            if (nums[i] > nums[index]) {
                index = i;
            }
        }
        return Arrays.copyOfRange(nums, index, index + k);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

500. `[1710]Maximum Units on a Truck.java`

```
1710
        Maximum Units on a Truck
        2022-12-11 01:29:31

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumUnits(int[][] boxTypes, int truckSize) {
        Arrays.sort(boxTypes, (a, b) -> b[1] - a[1]);
        int res = 0;
        for (int[] boxType : boxTypes) {
            int box = Math.min(truckSize, boxType[0]);
            res += box * boxType[1];
            truckSize -= box;
            if (truckSize == 0)
                break;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

501. `[1716]Calculate Money in Leetcode Bank.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int totalMoney(int n) {
        int sum = 0;
        int count = 0;

        while (n >= 7) {
            sum += (28 + count * 7);
            count++;
            n -= 7;
        }

        for (int i = 0; i < n; i++) {
            sum += (count + i + 1);
        }

        return sum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int totalMoney(int n) {
        int weeks = n / 7;
        int remainingDays = n % 7;
        int totalForWeeks = weeks * (2 * 28 + (weeks - 1) * 7) / 2;
        int totalForDays = remainingDays * (2 * (weeks + 1) + (remainingDays - 1)) / 2;
        return totalForWeeks + totalForDays;
    }
}
 */
```

502. [1717]Maximum Score From Removing Substrings.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumGain(String s, int x, int y) {
        StringBuilder remainingString = new StringBuilder();
        int result = 0;

        if (x > y) {
            result += processString(s, 'a', 'b', x, remainingString);
            result += processString(remainingString.toString(), 'b', 'a', y, new
StringBuilder());
        } else {
            result += processString(s, 'b', 'a', y, remainingString);
            result += processString(remainingString.toString(), 'a', 'b', x, new
StringBuilder());
        }

        return result;
    }

    private int processString(String s, char first, char second, int points,
StringBuilder remainingString) {
        int score = 0;
        Deque<Character> stack = new ArrayDeque<>();

        for (char c : s.toCharArray()) {
            if (!stack.isEmpty() && stack.peek() == first && c == second) {
                stack.pop();
                score += points;
            } else {
                stack.push(c);
            }
        }

        while (!stack.isEmpty()) {
            remainingString.append(stack.pollLast());
        }

        return score;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

503. [1718]Construct the Lexicographically Largest

Valid Sequence.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  private int[] ans;
  private boolean[] used;
  private int n;

  public int[] constructDistancedSequence(int n) {
    this.n = n;
    int size = 2 * n - 1;
    ans = new int[size];
    used = new boolean[n + 1];
    backtrack(0);
    return ans;
  }

  private boolean backtrack(int index) {
    if (index == ans.length) {
      return true;
    }

    if (ans[index] != 0) {
      return backtrack(index + 1);
    }

    for (int num = n; num >= 1; num--) {
      if (used[num]) {
        continue;
      }

      if (num == 1) {
        ans[index] = 1;
        used[1] = true;

        if (backtrack(index + 1)) {
          return true;
        }

        ans[index] = 0;
        used[1] = false;
      } else {
        int secondIndex = index + num;

        if (secondIndex < ans.length && ans[secondIndex] == 0) {
          ans[index] = num;
          ans[secondIndex] = num;
          used[num] = true;

          if (backtrack(index + 1)) {
            return true;
          }
```

```
          ans[index] = 0;
          ans[secondIndex] = 0;
          used[num] = false;
        }
      }
    }

    return false;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

504. [1720]Decode XORed Array.java

```
1720
        Decode XORed Array
        2023-02-23 18:19:42

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] decode(int[] encoded, int first) {
        int n = encoded.length;
        int[] arr = new int[n + 1];
        arr[0] = first;
        for (int i = 0; i < n; i++) {
            arr[i + 1] = arr[i] ^ encoded[i];
        }
        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
The inverse is XOR!

If you have:

c = a^b;
You can get a or b back if you have the other value available:

a = c^b; // or b^c (order is not important)
b = c^a; // or a^c

encoded[i] = arr[i] ^ arr[i+1]

Here we have an encoded array, and we want a resultant array, so we can interchange

res[0] = first
res[i+1] = res[i] ^ encoded[i]
 */
```

505. [1721]Swapping Nodes in a Linked List.java

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode() {}
 * ListNode(int val) { this.val = val; }
 * ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode swapNodes(ListNode head, int k) {
        ListNode temp = head;
        ListNode left = head;
        ListNode right = head;
        int count = 1;
        while (count < k) {
            left = left.next;
            temp = temp.next;
            count++;
        }
        while (temp.next != null) {
            temp = temp.next;
            right = right.next;
        }
        int value = left.val;
        left.val = right.val;
        right.val = value;
        return head;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

506. `[1726]Tuple with Same Product.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int tupleSameProduct(int[] nums) {
        Map<Integer, Integer> productCount = new HashMap<>();
        int n = nums.length;

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int product = nums[i] * nums[j];
                productCount.put(product, productCount.getOrDefault(product, 0) + 1);
            }
        }

        int result = 0;

        for (int count : productCount.values()) {
            if (count > 1) {
                result += 8 * (count * (count - 1) / 2);
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

507. [1727]Largest Submatrix With Rearrangements.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int largestSubmatrix(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        int max = 0;

        for (int i = 1; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 1) {
                    matrix[i][j] = matrix[i - 1][j] + 1;
                }
            }
        }

        for (int[] arr : matrix) {
            Arrays.sort(arr);

            for (int j = n - 1; j >= 0; j--) {
                int height = arr[j];
                max = Math.max(max, height * (n - j));
            }
        }

        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

508. [1732]Find the Highest Altitude.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int largestAltitude(int[] gain) {
        int prev = 0;
        int highest = Integer.MIN_VALUE;

        for (int num : gain) {
            highest = Math.max(highest, prev);
            prev += num;
        }

        return highest = Math.max(highest, prev);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

509. `[1740]Find Distance in a Binary Tree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findDistance(TreeNode root, int p, int q) {
        TreeNode lca = findLCA(root, p, q);
        int distanceP = findLevel(lca, p, 0);
        int distanceQ = findLevel(lca, q, 0);
        return distanceP + distanceQ;
    }

    private TreeNode findLCA(TreeNode root, int p, int q) {
        if (root == null || root.val == p || root.val == q) {
            return root;
        }

        TreeNode left = findLCA(root.left, p, q);
        TreeNode right = findLCA(root.right, p, q);

        if (left != null && right != null) {
            return root;
        }

        return left != null ? left : right;
    }

    private int findLevel(TreeNode root, int value, int level) {
        if (root == null) {
            return -1;
        }

        if (root.val == value) {
            return level;
        }

        int leftLevel = findLevel(root.left, value, level + 1);

        if (leftLevel == -1) {
            return findLevel(root.right, value, level + 1);
        }

        return leftLevel;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

510. [1743]Restore the Array From Adjacent Pairs.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] restoreArray(int[][] adjacentPairs) {
        Map<Integer, List<Integer>> map = new HashMap<>();
        int root = 0;

        for (int[] edge : adjacentPairs) {
            int x = edge[0];
            int y = edge[1];

            if (!map.containsKey(x)) {
                map.put(x, new ArrayList<>());
            }

            if (!map.containsKey(y)) {
                map.put(y, new ArrayList<>());
            }

            map.get(x).add(y);
            map.get(y).add(x);
        }

        for (int key : map.keySet()) {
            if (map.get(key).size() == 1) {
                root = key;
                break;
            }
        }

        int curr = root;
        int[] result = new int[map.size()];
        result[0] = root;
        int prev = Integer.MAX_VALUE;

        for (int i = 1; i < map.size(); i++) {
            for (int neighbor : map.get(curr)) {
                if (neighbor != prev) {
                    result[i] = neighbor;
                    prev = curr;
                    curr = neighbor;
                    break;
                }
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

511. [1749]Maximum Absolute Sum of Any Subarray.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int maxAbsoluteSum(int[] nums) {
    int sum = 0;
    int maxSum = 0;
    int minSum = 0;

    for (int num : nums) {
      sum += num;
      maxSum = Math.max(maxSum, sum);
      minSum = Math.min(minSum, sum);
    }

    return maxSum - minSum;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

512. [1750]Minimum Length of String After Deleting Similar Ends.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumLength(String s) {
        int left = 0;
        int right = s.length() - 1;

        while (left < right && s.charAt(left) == s.charAt(right)) {
            char currChar = s.charAt(left);

            while (left <= right && s.charAt(left) == currChar) {
                left++;
            }

            while (left <= right && s.charAt(right) == currChar) {
                right--;
            }
        }

        return right - left + 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

513. [1751]Maximum Number of Events That Can Be Attended II.java

```
//leetcode submit region begin(Prorightbit modification and deletion)
class Solution {
    public int maxValue(int[][] events, int k) {
        int max = 0;
        if (k == 1) {
            for (int[] event : events)
                max = Math.max(max, event[2]);
            return max;
        }
        Arrays.sort(events, (a, b) -> a[0] - b[0]);
        int n = events.length;
        return dyP(events, new Integer[k + 1][events.length], 0, k, 0);
    }

    private int dyP(int[][] events, Integer[][] dp, int index, int remain, int lastDay) {
        if (index >= events.length || remain == 0) {
            return 0;
        }
        if (lastDay >= events[index][0]) {
            return dyP(events, dp, index + 1, remain, lastDay);
        }
        if (dp[remain][index] != null) {
            return dp[remain][index];
        }
        return dp[remain][index] = Math.max(dyP(events, dp, index + 1, remain, lastDay),
dyP(events, dp, index + 1, remain - 1, events[index][1]) + events[index][2]);
    }
}
//leetcode submit region end(Prorightbit modification and deletion)
```

514. [1752]Check if Array Is Sorted and Rotated.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean check(int[] nums) {
        int count = 0;
        int n = nums.length;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] > nums[(i + 1) % n]) {
                count++;
            }

            if (count > 1) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

515. [1759]Count Number of Homogenous Substrings.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countHomogenous(String s) {
        final int MOD = (int) 1e9 + 7;
        char curr = s.charAt(0);
        int result = 0;
        int count = 0;

        for (char c : s.toCharArray()) {
            if (c == curr) {
                count++;
            } else {
                count = 1;
            }

            curr = c;
            result = (result + count) % MOD;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

516. [1760]Minimum Limit of Balls in a Bag.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumSize(int[] nums, int maxOperations) {
        int maxNum = 0;

        for (int num : nums) {
            if (num > maxNum) {
                maxNum = num;
            }
        }

        int low = 1;
        int high = maxNum;

        while (low < high) {
            int mid = low + (high - low) / 2;
            long operations = 0;

            for (int num : nums) {
                operations += (long)(num - 1) / mid;
                if (operations > maxOperations) {
                    break;
                }
            }

            if (operations <= maxOperations) {
                high = mid;
            } else {
                low = mid + 1;
            }
        }

        return low;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

517. `[1762]Buildings With an Ocean View.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] findBuildings(int[] heights) {
        List<Integer> list = new ArrayList<>();
        int maxHeight = 0;

        for (int i = heights.length - 1; i >= 0; i--) {
            if (heights[i] > maxHeight) {
                maxHeight = heights[i];
                list.add(i);
            }
        }

        int[] result = new int[list.size()];

        for (int i = 0; i < list.size(); i++) {
            result[i] = list.get(list.size() - 1 - i);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

518. `[1765]Map of Highest Peak.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[][] highestPeak(int[][] isWater) {
        int m = isWater.length;
        int n = isWater[0].length;
        int[][] height = new int[m][n];

        for(int[] row : height){
            Arrays.fill(row, -1);
        }

        Queue<int[]> queue = new LinkedList<>();

        for(int i = 0; i < m; i++){
            for(int j = 0; j < n; j++){
                if(isWater[i][j] == 1){
                    height[i][j] = 0;
                    queue.offer(new int[]{i, j});
                }
            }
        }

        int[][] directions = { { -1, 0 }, { 0, 1 }, { 1, 0 }, { 0, -1 } };

        while(!queue.isEmpty()){
            int[] cell = queue.poll();
            int currentHeight = height[cell[0]][cell[1]];

            for(int[] dir : directions){
                int ni = cell[0] + dir[0];
                int nj = cell[1] + dir[1];

                if(ni >=0 && ni < m && nj >=0 && nj < n && height[ni][nj] == -1){
                    height[ni][nj] = currentHeight + 1;
                    queue.offer(new int[]{ni, nj});
                }
            }
        }

        return height;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

519. `[1768]Merge Strings Alternately.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String mergeAlternately(String word1, String word2) {
        int len1 = word1.length();
        int len2 = word2.length();
        int i = 0;
        int j = 0;
        StringBuilder sb = new StringBuilder();
        while (i < len1 && j < len2) {
            sb.append(word1.charAt(i++));
            sb.append(word2.charAt(j++));
        }
        if (i < len1) {
            sb.append(word1.substring(i));
        }
        if (j < len2) {
            sb.append(word2.substring(j));
        }
        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

520. [1769]Minimum Number of Operations to Move All Balls to Each Box.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] minOperations(String boxes) {
        int n = boxes.length();
        int[] answer = new int[n];

        int leftCount = 0;
        int leftDistance = 0;
        for(int i = 0; i < n; i++) {
            answer[i] += leftDistance;
            if(boxes.charAt(i) == '1') {
                leftCount += 1;
            }
            leftDistance += leftCount;
        }

        int rightCount = 0;
        int rightDistance = 0;
        for(int i = n - 1; i >= 0; i--) {
            answer[i] += rightDistance;
            if(boxes.charAt(i) == '1') {
                rightCount += 1;
            }
            rightDistance += rightCount;
        }

        return answer;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

521. [1770]Maximum Score from Performing Multiplication Operations.java

```
1770
        Maximum Score from Performing Multiplication Operations
        2023-02-20 15:33:22

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumScore(int[] nums, int[] multipliers) {
        int m = multipliers.length;
        int n = nums.length;
        int[][] dp = new int[m + 1][m + 1];
        for (int i = m - 1; i >= 0; i--) {
            for (int left = i; left >= 0; left--) {
                int right = n - 1 - (i - left);
                dp[i][left] = Math.max(multipliers[i] * nums[left] + dp[i + 1][left + 1],
multipliers[i] * nums[right] + dp[i + 1][left]);
            }
        }
        return dp[0][0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

522. [1780]Check if Number is a Sum of Powers of Three.java

```


//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean checkPowersOfThree(int n) {
        while (n > 0) {
            if (n % 3 == 2) {
                return false;
            }

            n /= 3;
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

523. `[1790]Check if One String Swap Can Make Strings Equal.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public boolean areAlmostEqual(String s1, String s2) {
    if (s1.equals(s2)) {
      return true;
    }

    int n = s1.length();
    int first = -1;
    int second = -1;

    for (int i = 0; i < n; i++) {
      if (s1.charAt(i) != s2.charAt(i)) {
        if (first == -1) {
          first = i;
        } else if (second == -1) {
          second = i;
        } else {
          return false;
        }
      }
    }

    return second != -1
        && s1.charAt(first) == s2.charAt(second)
        && s1.charAt(second) == s2.charAt(first);
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

524. `[1791]Find Center of Star Graph.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findCenter(int[][] edges) {
        int u1 = edges[0][0];
        int v1 = edges[0][1];
        int u2 = edges[1][0];
        int v2 = edges[1][1];

        if (u1 == u2 || u1 == v2) {
            return u1;
        } else {
            return v1;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

525. [1792]Maximum Average Pass Ratio.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public double maxAverageRatio(int[][] classes, int extraStudents) {
        PriorityQueue<ClassInfo> maxHeap = new PriorityQueue<>((a, b) ->
Double.compare(b.getGain(), a.getGain()));

        for (int[] cls : classes) {
            int passed = cls[0];
            int total = cls[1];

            ClassInfo classInfo = new ClassInfo(passed, total);
            maxHeap.offer(classInfo);
        }

        for (int i = 0; i < extraStudents; i++) {
            ClassInfo bestClass = maxHeap.poll();
            if (bestClass == null) break; // Just in case, though it shouldn't happen

            bestClass.passed += 1;
            bestClass.total += 1;

            maxHeap.offer(bestClass);
        }

        double sum = 0.0;
        while (!maxHeap.isEmpty()) {
            ClassInfo cls = maxHeap.poll();
            sum += (double)cls.passed / cls.total;
        }

        return sum / classes.length;
    }

    static class ClassInfo {
        int passed;
        int total;

        public ClassInfo(int passed, int total) {
            this.passed = passed;
            this.total = total;
        }

        public double getGain() {
            return (double)(passed + 1) / (total + 1) - (double)passed / total;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

526. `[1793]Maximum Score of a Good Subarray.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumScore(int[] nums, int k) {
        int n = nums.length;
        int left = k;
        int right = k;
        int min = nums[k];
        int result = nums[k];

        while (left > 0 || right < n - 1) {
            if (left == 0) {
                right++;
            } else if (right == n - 1) {
                left--;
            } else if (nums[left - 1] < nums[right + 1]) {
                right++;
            } else {
                left--;
            }

            min = Math.min(min, Math.min(nums[left], nums[right]));
            result = Math.max(result, min * (right - left + 1));
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

527. [1796]Second Largest Digit in a String.java

```
1796
        Second Largest Digit in a String
        2022-12-27 16:22:53

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int secondHighest(String s) {
        int first = -1;
        int second = -1;
        char[] chars = s.toCharArray();
        for (char ch : chars) {
            if (ch >= '0' && ch <= '9') {
                int digit = ch - '0';
                if (digit > first) {
                    second = first;
                    first = digit;
                } else if (digit > second && digit < first) {
                    second = digit;
                }
            }
        }
        return second;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

528. [1797]Design Authentication Manager.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class AuthenticationManager {
    int timeToLive;
    Map<String, Integer> map;
    TreeSet<Integer> set;

    public AuthenticationManager(int timeToLive) {
        this.timeToLive = timeToLive;
        map = new HashMap<>();
        set = new TreeSet<>();
    }

    public void generate(String tokenId, int currentTime) {
        deleteExpiredToken(currentTime);
        map.put(tokenId, currentTime + timeToLive);
        set.add(currentTime + timeToLive);
    }

    public void renew(String tokenId, int currentTime) {
        deleteExpiredToken(currentTime);
        if (map.containsKey(tokenId) && map.get(tokenId) > currentTime) {
            set.remove(map.get(tokenId));
            map.put(tokenId, currentTime + timeToLive);
            set.add(currentTime + timeToLive);
        }
    }

    public int countUnexpiredTokens(int currentTime) {
        deleteExpiredToken(currentTime);
        return set.size();
    }

    private void deleteExpiredToken(int currentTime) {
        while (!set.isEmpty() && set.first() <= currentTime) {
            set.remove(set.first());
        }
    }
}

/**
 * Your AuthenticationManager object will be instantiated and called as such:
 * AuthenticationManager obj = new AuthenticationManager(timeToLive);
 * obj.generate(tokenId,currentTime);
 * obj.renew(tokenId,currentTime);
 * int param_3 = obj.countUnexpiredTokens(currentTime);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

529. `[1799]Maximize Score After N Operations.java`

```
/*
還不會
 */
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxScore(int[] nums) {
        int n = nums.length;
        int[][] gcd = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                gcd[i][j] = gcd(nums[i], nums[j]);
            }
        }
        int[] dp = new int[1 << n];
        for (int i = 0; i < (1 << n); i++) {
            int cnt = Integer.bitCount(i);
            if (cnt % 2 != 0) {
                continue;
            }
            for (int x = 0; x < n; x++) {
                if ((i & (1 << x)) > 0) {
                    continue;
                }
                for (int y = x + 1; y < n; y++) {
                    if ((i & (1 << y)) > 0) {
                        continue;
                    }
                    dp[i | (1 << x) | (1 << y)] = Math.max(dp[i] + gcd[x][y] * (cnt / 2 +
1), dp[i | (1 << x) | (1 << y)]);
                }
            }
        }
        return dp[(1 << n) - 1];
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

530. `[1800]Maximum Ascending Subarray Sum.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int maxAscendingSum(int[] nums) {
    int n = nums.length;
    int maxSum = nums[0];
    int currentSum = nums[0];

    for (int i = 1; i < n; i++) {
      if (nums[i] > nums[i - 1]) {
        currentSum += nums[i];
      } else {
        maxSum = Math.max(maxSum, currentSum);
        currentSum = nums[i];
      }
    }

    return Math.max(maxSum, currentSum);
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

531. `[1801]Number of Orders in the Backlog.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int getNumberOfBacklogOrders(int[][] orders) {
        final int MOD = (int) 1e9 + 7;
        PriorityQueue<int[]> pqBuy = new PriorityQueue<>((a, b) -> b[0] - a[0]);
        PriorityQueue<int[]> pqSell = new PriorityQueue<>((a, b) -> a[0] - b[0]);
        for (int[] order : orders) {
            if (order[2] == 0) {
                pqBuy.add(order);
            } else {
                pqSell.add(order);
            }
            while (!pqBuy.isEmpty() && !pqSell.isEmpty() && pqBuy.peek()[0] >=
pqSell.peek()[0]) {
                int min = Math.min(pqBuy.peek()[1], pqSell.peek()[1]);
                pqBuy.peek()[1] -= min;
                pqSell.peek()[1] -= min;
                if (pqBuy.peek()[1] == 0) {
                    pqBuy.poll();
                }
                if (pqSell.peek()[1] == 0) {
                    pqSell.poll();
                }
            }
        }
        int res = 0;
        for (int[] arr : pqBuy) {
            res = (res + arr[1]) % MOD;
        }
        for (int[] arr : pqSell) {
            res = (res + arr[1]) % MOD;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

532. `[1802]Maximum Value at a Given Index in a Bounded Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxValue(int n, int index, int maxSum) {
        int left = 1;
        int right = maxSum - n;
        while (left <= right) {
            int mid = (left + right) >>> 1;
            if (getSum(n, index, maxSum - n, mid)) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }

    private boolean getSum(int n, int index, int maxSum, int val) {
        int leftVal = Math.max(val - index, 0);
        int rightVal = Math.max(val - ((n - 1) - index), 0);
        long sumBefore = (long) (val + leftVal) * (val - leftVal + 1) / 2;
        long sumAfter = (long) (val + rightVal) * (val - rightVal + 1) / 2;
        return sumBefore + sumAfter - val <= maxSum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

533. [1804]Implement Trie II (Prefix Tree).java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Trie {
    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {
        TrieNode node = root;

        for (char c : word.toCharArray()) {
            int index = c - 'a';

            if (node.children[index] == null) {
                node.children[index] = new TrieNode();
            }

            node = node.children[index];
            node.prefixCount++;
        }

        node.count++;
    }

    public int countWordsEqualTo(String word) {
        TrieNode node = root;

        for (char c : word.toCharArray()) {
            int index = c - 'a';

            if (node.children[index] == null) {
                return 0;
            }

            node = node.children[index];
        }

        return node.count;
    }

    public int countWordsStartingWith(String prefix) {
        TrieNode node = root;

        for (char c : prefix.toCharArray()) {
            int index = c - 'a';

            if (node.children[index] == null) {
                return 0;
            }
```

```
            node = node.children[index];
        }

        return node.prefixCount;
    }

    public void erase(String word) {
        TrieNode node = root;

        for (char c : word.toCharArray()) {
            int index = c - 'a';
            node = node.children[index];
            node.prefixCount--;
        }

        node.count--;
    }
}

class TrieNode {
    public int count;
    public int prefixCount;
    public TrieNode[] children;

    public TrieNode() {
        count = 0;
        prefixCount = 0;
        children = new TrieNode[26];
    }
}

/**
 * Your Trie object will be instantiated and called as such:
 * Trie obj = new Trie();
 * obj.insert(word);
 * int param_2 = obj.countWordsEqualTo(word);
 * int param_3 = obj.countWordsStartingWith(prefix);
 * obj.erase(word);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

534. [1813]Sentence Similarity III.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean areSentencesSimilar(String sentence1, String sentence2) {
        String[] words1 = sentence1.trim().split(" ");
        String[] words2 = sentence2.trim().split(" ");

        if (words1.length < words2.length) {
            return areSentencesSimilar(sentence2, sentence1);
        }

        int m = words1.length;
        int n = words2.length;
        int i = 0;

        while (i < n && words1[i].equals(words2[i])) {
            i++;
        }

        if (i == n) {
            return true;
        }

        int j = 0;

        while (j < n - i && words1[m - 1 - j].equals(words2[n - 1 - j])) {
            j++;
        }

        return i + j == n;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

535. [1814]Count Nice Pairs in an Array.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    final int MOD = (int) 1e9 + 7;

    public int countNicePairs(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        long count = 0;

        for (int num : nums) {
            int key = num - rev(num);
            map.put(key, map.getOrDefault(key, 0) + 1);
        }

        for (int val : map.values()) {
            count = (count + (long) val * (val - 1) / 2) % MOD;
        }

        return (int) count;
    }

    private int rev(int x) {
        int result = 0;

        while (x > 0) {
            result = result * 10 + (x % 10);
            x /= 10;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

536. [1822]Sign of the Product of an Array.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int arraySign(int[] nums) {
        int neg = 0;
        for (int num : nums) {
            if (num == 0) {
                return 0;
            } else if (num < 0) {
                neg++;
            }
        }
        return neg % 2 == 0 ? 1 : -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

537. [1823]Find the Winner of the Circular Game.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findTheWinner(int n, int k) {
        int winner = 0;

        for (int i = 2; i <= n; i++) {
            winner = (winner + k) % i;
        }

        return winner + 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

538. [1829]Maximum XOR for Each Query.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] getMaximumXor(int[] nums, int maximumBit) {
        int n = nums.length;
        int[] answer = new int[n];
        int mask = (1 << maximumBit) - 1;
        int totalXor = 0;

        for (int num : nums) {
            totalXor ^= num;
        }

        for (int i = 0; i < n; i++) {
            int k = totalXor ^ mask;
            answer[i] = k;
            totalXor ^= nums[n - 1 - i];
        }

        return answer;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

539. [1832]Check if the Sentence Is Pangram.java

```
1832
        Check if the Sentence Is Pangram
        2022-11-26 00:37:52

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean checkIfPangram(String sentence) {
        char[] chars = sentence.toCharArray();
        Set<Character> set = new HashSet<>();
        for (char ch : chars) {
            set.add(ch);
        }
        return set.size() == 26 ? true : false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

540. `[1833]Maximum Ice Cream Bars.java`

```
1833
        Maximum Ice Cream Bars
        2023-01-06 09:25:20

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxIceCream(int[] costs, int coins) {
        Arrays.sort(costs);
        int count = 0;
        for (int cost : costs) {
            if (coins >= cost) {
                coins -= cost;
                count++;
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

541. `[1834]Single-Threaded CPU.java`

```
1834
        Single-Threaded CPU 2022-12-29 10:12:59

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] getOrder(int[][] tasks) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] != b[1] ? a[1] -
b[1] : a[2] - b[2]);
        int n = tasks.length;
        int[][] sortedTasks = new int[n][3];
        for (int i = 0; i < n; i++) {
            sortedTasks[i] = new int[]{tasks[i][0], tasks[i][1], i};
        }
        Arrays.sort(sortedTasks, (a, b) -> a[0] - b[0]);
        int[] taskOrder = new int[n];
        long currTime = 0;
        int taskIndex = 0;
        int resIndex = 0;
        while (taskIndex < n || !pq.isEmpty()) {
            if (pq.isEmpty() && currTime < sortedTasks[taskIndex][0]) {
                currTime = sortedTasks[taskIndex][0];
            }
            while (taskIndex < n && currTime >= sortedTasks[taskIndex][0]) {
                pq.add(sortedTasks[taskIndex]);
                taskIndex++;
            }
            int[] currTask = pq.remove();
            int processTime = currTask[1];
            int index = currTask[2];
            currTime += processTime;
            taskOrder[resIndex] = index;
            resIndex++;
        }
        return taskOrder;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

542. `[1836]Remove Duplicates From an Unsorted Linked List.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode() {}
 * ListNode(int val) { this.val = val; }
 * ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode deleteDuplicatesUnsorted(ListNode head) {
        Map<Integer, Integer> map = new HashMap<>();
        ListNode curr = head;

        while (curr != null) {
            map.merge(curr.val, 1, Integer::sum);
            curr = curr.next;
        }

        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode prev = dummy;
        curr = head;

        while (curr != null) {
            if (map.get(curr.val) > 1) {
                prev.next = curr.next;
            } else {
                prev = curr;
            }

            curr = curr.next;
        }

        return dummy.next;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

543. [1838]Frequency of the Most Frequent Element.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxFrequency(int[] nums, int k) {
        int left = 0;
        int right;
        int result = 1;
        long sum = 0;
        Arrays.sort(nums);

        for (right = 0; right < nums.length; right++) {
            sum += nums[right];

            while ((long) (right - left + 1) * nums[right] - sum > k) {
                sum -= nums[left++];
            }

            result = Math.max(result, (right - left + 1));
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
the key is to find out the valid condition:
k + sum >= size * max
 */
```

544. `[1839]Longest Substring Of All Vowels in Order.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestBeautifulSubstring(String word) {
        char[] c = word.toCharArray();
        int count = 1;
        int index = 1;
        int res = 0;
        for (int i = 1; i < c.length; i++) {
            if (c[i] == c[i - 1]) {
                count++;
            } else if (c[i] > c[i - 1]) {
                count++;
                index++;
            } else {
                count = 1;
                index = 1;
            }
            if (index == 5) {
                res = Math.max(res, count);
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

545. `[1845]Seat Reservation Manager.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class SeatManager {
    PriorityQueue<Integer> pq;
    int index;

    public SeatManager(int n) {
        pq = new PriorityQueue<>();
        index = 1;
    }

    public int reserve() {
        if (!pq.isEmpty()) {
            return pq.poll();
        }

        return index++;
    }

    public void unreserve(int seatNumber) {
        pq.offer(seatNumber);
    }
}

/**
 * Your SeatManager object will be instantiated and called as such:
 * SeatManager obj = new SeatManager(n);
 * int param_1 = obj.reserve();
 * obj.unreserve(seatNumber);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

546. `[1846]Maximum Element After Decreasing and Rearranging.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumElementAfterDecrementingAndRearranging(int[] arr) {
        int result = 0;
        Arrays.sort(arr);

        for (int num : arr) {
            result = Math.min(result + 1, num);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

547. [1857]Largest Color Value in a Directed Graph.java

```
// leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
  public int largestPathValue(String colors, int[][] edges) {
    int n = colors.length();
    List<List<Integer>> adj = new ArrayList<>(n);
    int[] inDegree = new int[n];
    for (int i = 0; i < n; i++) {
      adj.add(new ArrayList<>());
    }
    for (int[] e : edges) {
      adj.get(e[0]).add(e[1]);
      inDegree[e[1]]++;
    }

    int[][] dp = new int[n][26];
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
      if (inDegree[i] == 0) {
        q.offer(i);
      }
      dp[i][colors.charAt(i) - 'a'] = 1;
    }

    int processed = 0;
    int result = 0;

    while (!q.isEmpty()) {
      int u = q.poll();
      processed++;
      for (int c = 0; c < 26; c++) {
        result = Math.max(result, dp[u][c]);
      }
      for (int v : adj.get(u)) {
        for (int c = 0; c < 26; c++) {
          int candidate = dp[u][c] + (colors.charAt(v) - 'a' == c ? 1 : 0);
          if (candidate > dp[v][c]) {
            dp[v][c] = candidate;
          }
        }
        if (--inDegree[v] == 0) {
          q.offer(v);
        }
      }
    }

    return processed == n ? result : -1;
  }
}
//read the official solution
//leetcode submit region end(Prohibit modification and deletion)
```

548. `[1858]Longest Word With All Prefixes.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String longestWord(String[] words) {
        Arrays.sort(words);
        Set<String> validWords = new HashSet<>();
        String result = "";

        for (String word : words) {
            if (word.length() == 1 || validWords.contains(word.substring(0, word.length()
- 1))) {
                validWords.add(word);

                if (word.length() > result.length()) {
                    result = word;
                }
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

549. `[1861]Rotating the Box.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public char[][] rotateTheBox(char[][] box) {
        int m = box.length;
        int n = box[0].length;
        char[][] rotated = new char[n][m];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                rotated[j][m - 1 - i] = box[i][j];
            }
        }

        for (int j = 0; j < m; j++) {
            int pointer = n - 1;

            for (int i = n - 1; i >= 0; i--) {
                if (rotated[i][j] == '*') {
                    pointer = i - 1;
                } else if (rotated[i][j] == '#') {
                    rotated[i][j] = '.';
                    rotated[pointer][j] = '#';
                    pointer--;
                }
            }
        }

        return rotated;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

550. `[1863]Sum of All Subset XOR Totals.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int subsetXORSum(int[] nums) {
        return subsetXOR(nums, 0, 0);
    }

    private int subsetXOR(int[] nums, int index, int xor) {
        if (index == nums.length) {
            return xor;
        }

        int totalXOR = subsetXOR(nums, index + 1, xor ^ nums[index]);
        totalXOR += subsetXOR(nums, index + 1, xor);
        return totalXOR;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

551. [1865]Finding Pairs With a Certain Sum.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class FindSumPairs {
    private int[] nums1;
    private int[] nums2;
    private Map<Integer, Integer> freq2;

    public FindSumPairs(int[] nums1, int[] nums2) {
        this.nums1 = nums1;
        this.nums2 = nums2;
        this.freq2 = new HashMap<>();

        for (int num : nums2) {
            freq2.put(num, freq2.getOrDefault(num, 0) + 1);
        }
    }

    public void add(int index, int val) {
        int oldVal = nums2[index];
        int newVal = oldVal + val;

        freq2.put(oldVal, freq2.get(oldVal) - 1);

        if (freq2.get(oldVal) == 0) {
            freq2.remove(oldVal);
        }

        freq2.put(newVal, freq2.getOrDefault(newVal, 0) + 1);
        nums2[index] = newVal;
    }

    public int count(int tot) {
        int count = 0;

        for (int num1 : nums1) {
            int target = tot - num1;
            count += freq2.getOrDefault(target, 0);
        }

        return count;
    }
}

/**
 * Your FindSumPairs object will be instantiated and called as such:
 * FindSumPairs obj = new FindSumPairs(nums1, nums2);
 * obj.add(index,val);
 * int param_2 = obj.count(tot);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

552. [1870]Minimum Speed to Arrive on Time.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minSpeedOnTime(int[] dist, double hour) {
        if (dist.length - 1 + 1e-7 > hour) {
            return -1;
        }
        int left = 1;
        int right = (int) 1e7;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (isCheck(dist, mid, hour)) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }

    private boolean isCheck(int[] dist, int mid, double hour) {
        int count = 0;
        int n = dist.length;
        for (int i = 0; i < n - 1; i++) {
            count += (dist[i] + (mid - 1)) / mid;
        }
        double dCount = count + (1d * dist[n - 1]) / mid;
        return dCount > hour;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

553. `[1873]Calculate Special Bonus.java`

```
/**
<p>Table: <code>Employees</code></p>

<pre>
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| employee_id | int     |
| name        | varchar |
| salary      | int     |
+-------------+---------+
employee_id is the primary key for this table.
Each row of this table indicates the employee ID, employee name, and salary.
</pre>

<p> </p>

<p>Calculate the bonus of each employee. The bonus of an employee is <code>100%</code> of their salary if the ID of the employee is <strong>an odd number</strong> and <strong>the employee name does not start with the character </strong><code>'M'</code>. The bonus of an employee is <code>0</code> otherwise.</p>

<p>Return the result table ordered by <code>employee_id</code>.</p>

<p>The result format is in the following example.</p>

<p> </p>
<p><strong class="example">Example 1:</strong></p>

<pre>
<strong>Input:</strong>
Employees table:
+-------------+---------+--------+
| employee_id | name    | salary |
+-------------+---------+--------+
| 2           | Meir    | 3000   |
| 3           | Michael | 3800   |
| 7           | Addilyn | 7400   |
| 8           | Juan    | 6100   |
| 9           | Kannon  | 7700   |
+-------------+---------+--------+
<strong>Output:</strong>
+-------------+-------+
| employee_id | bonus |
+-------------+-------+
| 2           | 0     |
| 3           | 0     |
| 7           | 7400  |
| 8           | 0     |
| 9           | 7700  |
+-------------+-------+
```

```
<strong>Explanation:</strong>
The employees with IDs 2 and 8 get 0 bonus because they have an even employee_id.
The employee with ID 3 gets 0 bonus because their name starts with 'M'.
The rest of the employees get a 100% bonus.
</pre>

<div><div>Related Topics</div><div><li>Database</li></div></div><br><div><li>👍 907</li>
<li>👎 62</li></div>
*/

//There is no code of Java type for this problem
```

554. [1877]Minimize Maximum Pair Sum in Array.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minPairSum(int[] nums) {
        int left = 0;
        int right = nums.length - 1;
        int max = 0;

        Arrays.sort(nums);

        while (left < right) {
            max = Math.max(max, nums[left++] + nums[right--]);
        }

        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

555. [1885]Count Pairs in Two Arrays.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long countPairs(int[] nums1, int[] nums2) {
        int[] nums = caluculatePairs(nums1, nums2);
        Arrays.sort(nums);
        return countPositivePairs(nums);
    }

    private int[] caluculatePairs(int[] nums1, int[] nums2) {
        int n = nums1.length;
        int[] nums = new int[n];

        for (int i = 0; i < n; i++) {
            nums[i] = nums1[i] - nums2[i];
        }

        return nums;
    }

    private long countPositivePairs(int[] nums) {
        int n = nums.length;
        long result = 0;
        int left = 0;
        int right = n - 1;

        while (left < right) {
            if (nums[left] + nums[right] > 0) {
                result += right - left;
                right--;
            } else {
                left++;
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

556. [1886]Determine Whether Matrix Can Be Obtained By Rotation.java

```
1886
        Determine Whether Matrix Can Be Obtained By Rotation
        2022-11-28 09:29:35

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean findRotation(int[][] mat, int[][] target) {
        int n = mat.length;
        int r0 = 0;
        int r1 = 0;
        int r2 = 0;
        int r3 = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                int x = n - 1 - i;
                int y = n - 1 - j;
                if (mat[i][j] == target[i][j]) {
                    r0++;
                }
                if (mat[i][j] == target[y][i]) {
                    r1++;
                }
                if (mat[i][j] == target[x][y]) {
                    r2++;
                }
                if (mat[i][j] == target[j][x]) {
                    r3++;
                }
            }
        }
        int total = n * n;
        return r0 == total || r1 == total || r2 == total || r3 == total;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

557. `[1887]Reduction Operations to Make the Array Elements Equal.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int reductionOperations(int[] nums) {
        int count = 0;
        int result = 0;
        Arrays.sort(nums);

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[i - 1]) {
                count++;
            }

            result += count;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
自己想的
class Solution {
    public int reductionOperations(int[] nums) {
        Map<Integer, Integer> map = new TreeMap<>();
        int index = 0;
        int result = 0;

        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        for (int key : map.keySet()) {
            result += map.get(key) * index;
            index++;
        }

        return result;
    }
}
 */
```

558. `[1894]Find the Student that Will Replace the`

Chalk.java

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
    public int chalkReplacer(int[] chalk, int k) {
        int n = chalk.length;
        long totalChalk = 0;

        for (int num : chalk) {
            totalChalk += num;
        }

        k %= totalChalk;

        for (int i = 0; i < n; i++) {
            if (k < chalk[i]) {
                return i;
            }

            k -= chalk[i];
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int chalkReplacer(int[] chalk, int k) {
        int n = chalk.length;
        long[] prefixSum = new long[n];
        prefixSum[0] = chalk[0];

        for (int i = 1; i < n; i++) {
            prefixSum[i] = prefixSum[i - 1] + chalk[i];
        }

        k = (int) (k % prefixSum[n - 1]);
        int left = 0;
        int right = n - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (prefixSum[mid] <= k) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }

        return left;
```

```
    }
}
 */
```

559. [1897]Redistribute Characters to Make All Strings Equal.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean makeEqual(String[] words) {
        final int LETTERS = 26;
        int[] arr = new int[LETTERS];
        int n = words.length;

        for (String word : words) {
            for (char c : word.toCharArray()) {
                arr[c - 'a']++;
            }
        }

        for (int num : arr) {
            if (num % n != 0) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

560. [1898]Maximum Number of Removable Characters.java

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    public int maximumRemovals(String s, String p, int[] removable) {
        int left = 0;
        int right = removable.length;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (isSubsequence(s, p, removable, mid)) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return left - 1;
    }

    private boolean isSubsequence(String s, String p, int[] removable, int k) {
        boolean[] removed = new boolean[s.length()];

        for (int i = 0; i < k; i++) {
            removed[removable[i]] = true;
        }

        int j = 0;

        for (int i = 0; i < s.length(); i++) {
            if (j < p.length() && !removed[i] && s.charAt(i) == p.charAt(j)) {
                j++;
            }
        }

        return j == p.length();
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

561. `[1900]The Earliest and Latest Rounds Where Players`

Compete.java

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    private final Map<Long, int[]> memo = new HashMap<>();

    public int[] earliestAndLatest(int n, int firstPlayer, int secondPlayer) {
        if (firstPlayer > secondPlayer) {
            int tmp = firstPlayer;
            firstPlayer = secondPlayer;
            secondPlayer = tmp;
        }
        return dfs(n, firstPlayer, secondPlayer);
    }

    private int[] dfs(int k, int a, int b) {
        if (a + b == k + 1) {
            return new int[]{1, 1};
        }

        long key = pack(k, a, b);
        if (memo.containsKey(key)) {
            return memo.get(key);
        }

        int p = k / 2;
        int middle = (k % 2 == 1) ? (p + 1) : -1;

        int forcedBits = 0;
        int maskNeed = 0;

        for (int j = 0; j < p; ++j) {
            int front = j + 1;
            int back = k - j;

            if (front == a || back == a) {
                maskNeed |= (1 << j);
                if (back == a) {
                    forcedBits |= (1 << j);
                }
            } else if (front == b || back == b) {
                maskNeed |= (1 << j);
                if (back == b) {
                    forcedBits |= (1 << j);
                }
            }
        }

        int earliest = Integer.MAX_VALUE;
        int latest = Integer.MIN_VALUE;

        int total = 1 << p;
```

```
        Set<Long> nextStates = new HashSet<>();

        Outer:
        for (int bits = 0; bits < total; ++bits) {
            if ((bits & maskNeed) != forcedBits) {
                continue;
            }

            List<Integer> winners = new ArrayList<>(p + 1);

            for (int j = 0; j < p; ++j) {
                int front = j + 1;
                int back = k - j;

                int winner;
                if (front == a || back == a) {
                    winner = a;
                } else if (front == b || back == b) {
                    winner = b;
                } else {
                    winner = ((bits & (1 << j)) == 0) ? front : back;
                }
                winners.add(winner);
            }
            if (middle != -1) {
                winners.add(middle);
            }

            Collections.sort(winners);

            int kNext = winners.size();
            int aNext = binarySearch(winners, a) + 1;
            int bNext = binarySearch(winners, b) + 1;
            if (aNext > bNext) {
                int tmp = aNext;
                aNext = bNext;
                bNext = tmp;
            }

            nextStates.add(pack(kNext, aNext, bNext));
        }

        for (long st : nextStates) {
            int kNext = (int) (st >> 20);
            int aNext = (int) ((st >> 10) & 0x3FF);
            int bNext = (int) (st & 0x3FF);

            int[] sub = dfs(kNext, aNext, bNext);
            earliest = Math.min(earliest, sub[0] + 1);
            latest = Math.max(latest, sub[1] + 1);
        }

        int[] res = new int[]{earliest, latest};
        memo.put(key, res);
```

```
        return res;
    }


    private static long pack(int k, int a, int b) {
        // k、a、b 均 ≤ 28，只需各 5 bit；這裡用 <<20 / <<10 方便
        return ((long) k << 20) | ((long) a << 10) | b;
    }

    private static int binarySearch(List<Integer> list, int key) {
        int idx = Collections.binarySearch(list, key);
        return idx >= 0 ? idx : -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

562. [1903]Largest Odd Number in String.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String largestOddNumber(String num) {
        for (int i = num.length() - 1; i >= 0; i--) {
            if ((num.charAt(i) - '0') % 2 != 0) {
                return num.substring(0, i + 1);
            }
        }
        return "";
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

563. `[1905]Count Sub Islands.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int m;
    int n;

    public int countSubIslands(int[][] grid1, int[][] grid2) {
        m = grid1.length;
        n = grid1[0].length;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid2[i][j] == 1 && grid1[i][j] == 0) {
                    dfs(grid2, i, j);
                }
            }
        }
        int count = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid2[i][j] == 1) {
                    count++;
                    dfs(grid2, i, j);
                }
            }
        }
        return count;
    }

    public void dfs(int[][] grid, int x, int y) {
        if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] == 0) {
            return;
        }
        grid[x][y] = 0;
        dfs(grid, x + 1, y);
        dfs(grid, x - 1, y);
        dfs(grid, x, y + 1);
        dfs(grid, x, y - 1);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

564. [1909]Remove One Element to Make the Array Strictly Increasing.java

```
1909
Remove One Element to Make the Array Strictly Increasing
2023-01-17 22:42:56
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canBeIncreasing(int[] nums) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

565. [1910]Remove All Occurrences of a Substring.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public String removeOccurrences(String s, String part) {
    StringBuilder res = new StringBuilder();
    int partLen = part.length();

    for (char c : s.toCharArray()) {
      res.append(c);

      if (res.length() >= partLen && res.substring(res.length() - partLen).equals(part))
{
        res.delete(res.length() - partLen, res.length());
      }
    }

    return res.toString();
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

566. [1913]Maximum Product Difference Between Two Pairs.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxProductDifference(int[] nums) {
        int[] max = getMax(nums);
        int[] min = getMin(nums);
        return compute(max, min);
    }

    private int[] getMax(int[] nums) {
        int first = Integer.MIN_VALUE;
        int second = Integer.MIN_VALUE;

        for (int num : nums) {
            if (num > first) {
                second = first;
                first = num;
            } else if (num > second) {
                second = num;
            }
        }

        return new int[]{first, second};
    }

    private int[] getMin(int[] nums) {
        int first = Integer.MAX_VALUE;
        int second = Integer.MAX_VALUE;

        for (int num : nums) {
            if (num < first) {
                second = first;
                first = num;
            } else if (num < second) {
                second = num;
            }
        }

        return new int[]{first, second};
    }

    private int compute(int[] max, int[] min) {
        return (max[0] * max[1]) - (min[0] * min[1]);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

567. `[1915]Number of Wonderful Substrings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    public long wonderfulSubstrings(String word) {
        long res = 0;
        int n = word.length();
        long[] count = new long[1024];
        count[0] = 1;
        int curr = 0;

        for (int i = 0; i < n; i++) {
            curr ^= 1 << (word.charAt(i) - 'a');
            res += count[curr];

            for (int j = 0; j < 10; j++) {
                res += count[curr ^ (1 << j)];
            }

            count[curr]++;
        }

        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

568. `[1921]Eliminate Maximum Number of Monsters.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int eliminateMaximum(int[] dist, int[] speed) {
        int n = dist.length;
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = (dist[i] - 1) / speed[i];
        }

        Arrays.sort(arr);

        for (int i = 0; i < n; i++) {
            if (arr[i] < i) {
                return i;
            }
        }

        return n;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

569. `[1922]Count Good Numbers.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private static final int MOD = 1000000007;

    // Fast modular exponentiation
    private long modPow(long base, long exponent) {
        long result = 1;
        base = base % MOD;
        while (exponent > 0) {
            if ((exponent & 1) == 1) {  // if exponent is odd
                result = (result * base) % MOD;
            }
            base = (base * base) % MOD;
            exponent >>= 1; // divide exponent by 2
        }
        return result;
    }

    public int countGoodNumbers(long n) {
        // evenCount is the number of digits at even indices:
        long evenCount = (n + 1) / 2;  // accounts for 0-index; for odd n, one extra even
index.
        // oddCount is the number of digits at odd indices:
        long oddCount = n / 2;

        // Total good strings = 5^evenCount * 4^oddCount (all modulo MOD)
        long evenPart = modPow(5, evenCount);
        long oddPart = modPow(4, oddCount);
        long result = (evenPart * oddPart) % MOD;

        return (int) result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

570. [1926]Nearest Exit from Entrance in Maze.java

```
1926
        Nearest Exit from Entrance in Maze
        2022-12-07 22:27:39

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int nearestExit(char[][] maze, int[] entrance) {
        int[][] dirs = new int[][]{{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int m = maze.length;
        int n = maze[0].length;
        Deque<int[]> dq = new ArrayDeque<>();
        int count = 0;
        dq.add(entrance);
        maze[entrance[0]][entrance[1]] = '+';
        while (!dq.isEmpty()) {
            int size = dq.size();
            count++;
            for (int i = 0; i < size; i++) {
                int[] curr = dq.poll();
                for (int[] dir : dirs) {
                    int x = curr[0] + dir[0];
                    int y = curr[1] + dir[1];
                    if (x < 0 || x >= m || y < 0 || y >= n || maze[x][y] == '+') {
                        continue;
                    }
                    if (x == 0 || x == m - 1 || y == 0 || y == n - 1) {
                        return count;
                    }
                    dq.add(new int[]{x, y});
                    maze[x][y] = '+';
                }
            }
        }
        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

571. `[1929]Concatenation of Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] getConcatenation(int[] nums) {
        int n = nums.length;
        int[] result = new int[n * 2];

        System.arraycopy(nums, 0, result, 0, n);
        System.arraycopy(nums, 0, result, n, n);

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

572. `[1930]Unique Length-3 Palindromic`

Subsequences.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countPalindromicSubsequence(String s) {
        char[] ch = s.toCharArray();
        Set<Character> set = new HashSet<>();
        int result = 0;

        for (char c : ch) {
            set.add(c);
        }

        for (char c : set) {
            int left = s.indexOf(c);
            int right = s.lastIndexOf(c);

            if (left > -1) {
                boolean[] isVisited = new boolean[26];
                int count = 0;

                for (int i = left + 1; i < right; i++) {
                    if (!isVisited[ch[i] - 'a']) {
                        isVisited[ch[i] - 'a'] = true;
                        count++;
                    }
                }

                result += count;
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
越少Set效率越高

181 ms
class Solution {
    public int countPalindromicSubsequence(String s) {
        char[] ch = s.toCharArray();
        Set<Character> set = new HashSet<>();
        int result = 0;

        for (char c : ch) {
            set.add(c);
        }

        for (char c : set) {
            int left = s.indexOf(c);
```

```
            int right = s.lastIndexOf(c);

            if (left > -1) {
                Set<Character> letters = new HashSet<>();

                for (int i = left + 1; i < right; i++) {
                    letters.add(ch[i]);
                }

                result += letters.size();
            }
        }

        return result;
    }
}
```

28 ms

```
class Solution {
    public int countPalindromicSubsequence(String s) {
        char[] ch = s.toCharArray();
        Set<Character> set = new HashSet<>();
        int result = 0;

        for (char c : ch) {
            set.add(c);
        }

        for (char c : set) {
            int left = s.indexOf(c);
            int right = s.lastIndexOf(c);

            if (left > -1) {
                boolean[] isVisited = new boolean[26];
                int count = 0;

                for (int i = left + 1; i < right; i++) {
                    if (!isVisited[ch[i] - 'a']) {
                        isVisited[ch[i] - 'a'] = true;
                        count++;
                    }
                }

                result += count;
            }
        }

        return result;
    }
}
```

17 ms

```
class Solution {
```

```
    public int countPalindromicSubsequence(String s) {
        char[] ch = s.toCharArray();
        int[] first = new int[26];
        int[] last = new int[26];
        int result = 0;

        Arrays.fill(first, -1);

        for (int i = 0; i < ch.length; i++) {
            int curr = ch[i] - 'a';
            if (first[curr] == -1) {
                first[curr] = i;
            }

            last[curr] = i;
        }

        for (int i = 0; i < 26; i++) {
            if (first[i] == -1) {
                continue;
            }

            boolean[] isVisited = new boolean[26];
            int count = 0;

            for (int j = first[i] + 1; j < last[i]; j++) {
                if (!isVisited[ch[j] - 'a']) {
                    isVisited[ch[j] - 'a'] = true;
                    count++;
                }
            }

            result += count;
        }

        return result;
    }
}
 */
```

573. `[1931]Painting a Grid With Three Different`

Colors.java

```
// leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
  private static final int MOD = 1_000_000_007;

  public int colorTheGrid(int m, int n) {
    List<int[]> states = new ArrayList<>();
    generateStates(m, new int[m], 0, states);
    int S = states.size();

    List<List<Integer>> adj = new ArrayList<>(S);
    for (int i = 0; i < S; i++) {
      adj.add(new ArrayList<>());
    }
    for (int i = 0; i < S; i++) {
      for (int j = 0; j < S; j++) {
        if (canStack(states.get(i), states.get(j), m)) {
          adj.get(i).add(j);
        }
      }
    }

    long[] dpPrev = new long[S];
    long[] dpCurr = new long[S];
    Arrays.fill(dpPrev, 1);

    for (int col = 2; col <= n; col++) {
      Arrays.fill(dpCurr, 0);
      for (int prev = 0; prev < S; prev++) {
        long ways = dpPrev[prev];
        if (ways == 0) continue;
        for (int next : adj.get(prev)) {
          dpCurr[next] = (dpCurr[next] + ways) % MOD;
        }
      }

      long[] tmp = dpPrev;
      dpPrev = dpCurr;
      dpCurr = tmp;
    }

    long ans = 0;
    for (long x : dpPrev) {
      ans = (ans + x) % MOD;
    }
    return (int) ans;
  }

  private void generateStates(int m, int[] arr, int pos, List<int[]> states) {
    if (pos == m) {
      states.add(arr.clone());
      return;
```

```
    }
    for (int c = 0; c < 3; c++) {
      if (pos > 0 && arr[pos - 1] == c) {
        continue;
      }
      arr[pos] = c;
      generateStates(m, arr, pos + 1, states);
    }
  }

  private boolean canStack(int[] a, int[] b, int m) {
    for (int i = 0; i < m; i++) {
      if (a[i] == b[i]) return false;
    }
    return true;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

574. `[1940]Longest Common Subsequence Between Sorted Arrays.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> longestCommonSubsequence(int[][] arrays) {
        int n = arrays.length;
        Map<Integer, Integer> arrMap = new HashMap<>();

        for (int[] arr : arrays) {
            for (int num : arr) {
                arrMap.put(num, arrMap.getOrDefault(num, 0) + 1);
            }
        }

        List<Integer> resultList = new ArrayList<>();

        for (int key : arrMap.keySet()) {
            if (arrMap.get(key) == n) {
                resultList.add(key);
            }
        }

        Collections.sort(resultList);

        return resultList;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

575. [1941]Check if All Characters Have Equal Number of Occurrences.java

```
1941
        Check if All Characters Have Equal Number of Occurrences
        2022-11-27 16:44:46

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean areOccurrencesEqual(String s) {
        Map<Character, Integer> map = new HashMap<>();
        for (char c : s.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }
        Set<Integer> set = new HashSet<>(map.values());
        return set.size() == 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

576. `[1942]The Number of the Smallest Unoccupied Chair.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {

    public int smallestChair(int[][] times, int targetFriend) {
        int targetArrival = times[targetFriend][0];
        Arrays.sort(times, (a, b) -> a[0] - b[0]);

        int nextChair = 0;
        PriorityQueue<int[]> leavingQueue = new PriorityQueue<>((a, b) -> a[0] - b[0]);
        TreeSet<Integer> availableChairs = new TreeSet<>();

        for (int[] time : times) {
            int arrival = time[0];
            int leave = time[1];

            while (!leavingQueue.isEmpty() && leavingQueue.peek()[0] <= arrival) {
                availableChairs.add(leavingQueue.poll()[1]);
            }

            int currentChair;

            if (!availableChairs.isEmpty()) {
                currentChair = availableChairs.first();
                availableChairs.remove(currentChair);
            } else {
                currentChair = nextChair++;
            }

            leavingQueue.offer(new int[]{leave, currentChair});

            if (arrival == targetArrival) {
                return currentChair;
            }
        }

        return 0;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

577. [1945]Sum of Digits of String After Convert.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int getLucky(String s, int k) {
        StringBuilder numString = new StringBuilder();

        for (char c : s.toCharArray()) {
            int num = c - 'a' + 1;
            numString.append(num);
        }

        String transformed = numString.toString();

        for (int i = 0; i < k; i++) {
            int sum = 0;

            for (char c : transformed.toCharArray()) {
                sum += c - '0';
            }

            transformed = Integer.toString(sum);
        }

        return Integer.parseInt(transformed);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

578. [1948]Delete Duplicate Folders in System.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<String>> deleteDuplicateFolder(List<List<String>> paths) {
        Node root = new Node("");

        for (List<String> path : paths) {
            Node cur = root;
            for (String seg : path) {
                cur = cur.children.computeIfAbsent(seg, Node::new);
            }
        }

        Map<String, List<Node>> serialMap = new HashMap<>(1024);
        serialize(root, serialMap);

        for (Map.Entry<String, List<Node>> e : serialMap.entrySet()) {
            String serial = e.getKey();
            if (serial.isEmpty()) {
                continue;
            }
            List<Node> list = e.getValue();
            if (list.size() > 1) {
                for (Node n : list) {
                    n.deleted = true;
                }
            }
        }

        List<List<String>> ans = new ArrayList<>();
        Deque<String> cur = new ArrayDeque<>();
        collect(root, cur, ans);
        return ans;
    }

    private static final class Node {
        final String name;
        final Map<String, Node> children = new HashMap<>();
        boolean deleted;
        String serial;

        Node(String name) {
            this.name = name;
        }
    }

    private String serialize(Node node, Map<String, List<Node>> serialMap) {
        if (node.children.isEmpty()) {
            node.serial = "";
            serialMap.computeIfAbsent(node.serial, k -> new ArrayList<>()).add(node);
            return node.serial;
```

```
        }

        List<String> keys = new ArrayList<>(node.children.keySet());
        Collections.sort(keys);

        StringBuilder sb = new StringBuilder();
        for (String k : keys) {
            Node child = node.children.get(k);
            String childSerial = serialize(child, serialMap);
            sb.append('(').append(k).append(childSerial).append(')');
        }

        node.serial = sb.toString();
        serialMap.computeIfAbsent(node.serial, k -> new ArrayList<>()).add(node);
        return node.serial;
    }

    private void collect(Node node, Deque<String> path, List<List<String>> ans) {
        for (Node child : node.children.values()) {
            if (child.deleted) {
                continue;
            }
            path.addLast(child.name);
            ans.add(new ArrayList<>(path));
            collect(child, path, ans);
            path.removeLast();
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

579. [1957]Delete Characters to Make Fancy String.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String makeFancyString(String s) {
        if (s.length() < 3) {
            return s;
        }

        StringBuilder sb = new StringBuilder();
        sb.append(s.charAt(0));
        int count = 1;

        for(int i = 1; i < s.length(); i++) {
            char current = s.charAt(i);

            if(current == sb.charAt(sb.length() - 1)) {
                count++;
            } else {
                count = 1;
            }

            if(count <= 2) {
                sb.append(current);
            }
        }

        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

580. [1960]Maximum Product of the Length of Two Palindromic Substrings.java

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    private static int[] manacherOdd(String str) {
        char[] s = str.toCharArray();
        int n = s.length;
        int[] ans = new int[n];
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = i > r ? 1 : Math.min(ans[l + r - i], r - i + 1);
            int maxK = Math.min(i, n - i - 1);
            int i1 = i - k, i2 = i + k;
            while (k <= maxK && s[i1--] == s[i2++]) k++;
            ans[i] = k--;
            if (i + k > r) {
                l = i - k;
                r = i + k;
            }
        }
        return ans;
    }

    public long maxProduct(String s) {
        int n = s.length();
        int[] d = manacherOdd(s);
        int[] left = new int[n], right = new int[n];
        for (int i = 0; i < n; i++) {
            left[i + d[i] - 1] = Math.max(left[i + d[i] - 1], 2 * d[i] - 1);
            right[i - d[i] + 1] = 2 * d[i] - 1;
        }
        for (int i = n - 2, j = n - 1; i >= 0; i--, j--) left[i] = Math.max(left[i],
left[j] - 2);
        for (int i = 1, j = 0; i < n; i++, j++) right[i] = Math.max(right[i], right[j] -
2);

        for (int i = 1, j = 0; i < n; i++, j++) left[i] = Math.max(left[i], left[j]);
        for (int i = n - 2, j = n - 1; i >= 0; i--, j--) right[i] = Math.max(right[i],
right[j]);

        long ans = 1;
        for (int i = 1; i < n; i++) ans = Math.max(ans, (long) left[i - 1] * right[i]);
        return ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

581. [1962]Remove Stones to Minimize the Total.java

```
1962
        Remove Stones to Minimize the Total
        2022-12-09 14:55:46

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minStoneSum(int[] piles, int k) {
        PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> b - a);
        for (int pile : piles) {
            pq.add(pile);
        }
        for (int i = 0; i < k; i++) {
            int temp = pq.remove();
            temp -= temp / 2;
            pq.add(temp);
        }
        int res = 0;
        for (int num : pq) {
            res += num;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

582. [1963]Minimum Number of Swaps to Make the String Balanced.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minSwaps(String s) {
        int balance = 0;
        int maxImbalance = 0;
        int n = s.length();

        for (int i = 0; i < n; i++) {
            if (s.charAt(i) == '[') {
                balance++;
            } else {
                balance--;
            }

            if (balance < 0) {
                maxImbalance = Math.max(maxImbalance, -balance);
            }
        }

        return (maxImbalance + 1) / 2;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

583. [1964]Find the Longest Valid Obstacle Course at Each Position.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] longestObstacleCourseAtEachPosition(int[] obstacles) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

584. [1970]Last Day Where You Can Still Cross.java

```
//leetcode submit region begin(Prohibit modification and deletion)


class Solution {
    public int latestDayToCross(int row, int col, int[][] cells) {
        UnionFind uf = new UnionFind(row * col + 2);
        int[][] grid = new int[row][col];
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}, {1, 1}, {1, -1}, {-1, 1}, {-1,
-1}};
        for (int i = 0; i < row * col; i++) {
            int r = cells[i][0] - 1;
            int c = cells[i][1] - 1;
            grid[r][c] = 1;
            int index1 = r * col + c + 1;
            for (int[] dir : dirs) {
                int x = r + dir[0];
                int y = c + dir[1];
                int index2 = x * col + y + 1;
                if (x < 0 || x >= row || y < 0 || y >= col || grid[x][y] == 0) {
                    continue;
                }
                uf.union(index1, index2);
            }
            if (c == 0) {
                uf.union(0, index1);
            }
            if (c == col - 1) {
                uf.union(row * col + 1, index1);
            }
            if (uf.find(0) == uf.find(row * col + 1)) {
                return i;
            }
        }
        return -1;
    }
}
class UnionFind {
    int[] group;

    public UnionFind(int n) {
        group = new int[n];
        for (int i = 0; i < n; i++) {
            group[i] = i;
        }
    }

    public int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }
```

```
    public void union(int i, int j) {
        int x = find(i);
        int y = find(j);
        if (x != y) {
            group[x] = y;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

585. `[1971]Find if Path Exists in Graph.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean validPath(int n, int[][] edges, int source, int destination) {
        UnionFind uf = new UnionFind(n);

        for (int[] edge : edges) {
            uf.union(edge[0], edge[1]);
        }

        return uf.isConnected(source, destination);
    }
}

public class UnionFind {
    private int[] parent;
    private int[] rank;

    public UnionFind(int size) {
        parent = new int[size];
        rank = new int[size];

        for (int i = 0; i < size; i++) {
            parent[i] = i;
            rank[i] = 0;
        }
    }

    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);
        }

        return parent[x];
    }

    public void union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX != rootY) {
            if (rank[rootX] < rank[rootY]) {
                parent[rootX] = rootY;
            } else if (rank[rootX] > rank[rootY]) {
                parent[rootY] = rootX;
            } else {
                parent[rootX] = rootY;
                rank[rootY]++;
            }
        }
    }
```

```
    public boolean isConnected(int x, int y) {
        return find(x) == find(y);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public boolean validPath(int n, int[][] edges, int source, int destination) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }
        for (int[] edge : edges) {
            int x = find(arr, edge[0]);
            int y = find(arr, edge[1]);
            if (x != y) {
                arr[x] = y;
            }
        }
        return find(arr, source) == find(arr, destination);
    }

    public int find(int[] arr, int x) {
        return arr[x] == x ? x : (arr[x] = find(arr, arr[x]));
    }
}
 */
```

586. [1973]Count Nodes Equal to Sum of Descendants.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int result = 0;

    public int equalToDescendants(TreeNode root) {
        dfs(root);
        return result;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int sum = dfs(root.left) + dfs(root.right);

        if (root.val == sum) {
            result++;
        }

        return sum + root.val;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

587. `[1975]Maximum Matrix Sum.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long maxMatrixSum(int[][] matrix) {
        long totalSum = 0;
        int negativeCount = 0;
        int smallestAbs = Integer.MAX_VALUE;

        for(int i = 0; i < matrix.length; i++){
            for(int j = 0; j < matrix[0].length; j++){
                if(matrix[i][j] < 0) {
                    negativeCount++;
                }

                totalSum += Math.abs(matrix[i][j]);
                smallestAbs = Math.min(smallestAbs, Math.abs(matrix[i][j]));
            }
        }

        if(negativeCount % 2 != 0) {
            totalSum -= 2 * smallestAbs;
        }

        return totalSum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

588. [1976]Number of Ways to Arrive at Destination.java

```
class Solution {
  public int countPaths(int n, int[][] roads) {
    List<int[]>[] graph = new ArrayList[n];

    for (int i = 0; i < n; i++) {
      graph[i] = new ArrayList<>();
    }

    for (int[] road : roads) {
      int u = road[0], v = road[1], t = road[2];
      graph[u].add(new int[] {v, t});
      graph[v].add(new int[] {u, t});
    }

    long[] dist = new long[n];
    Arrays.fill(dist, Long.MAX_VALUE);
    dist[0] = 0;

    int[] ways = new int[n];
    ways[0] = 1;

    int mod = 1_000_000_007;
    PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
    pq.offer(new long[] {0, 0});

    while (!pq.isEmpty()) {
      long[] cur = pq.poll();
      long curDist = cur[0];
      int node = (int) cur[1];

      if (curDist > dist[node]) {
        continue;
      }

      for (int[] neighbor : graph[node]) {
        int next = neighbor[0];
        int time = neighbor[1];
        long newDist = curDist + time;

        if (newDist < dist[next]) {
          dist[next] = newDist;
          ways[next] = ways[node];
          pq.offer(new long[] {newDist, next});
        } else if (newDist == dist[next]) {
          ways[next] = (ways[next] + ways[node]) % mod;
        }
      }
    }

    return ways[n - 1];
  }
```

```
}
// leetcode submit region end(Prohibit modification and deletion)
```

589. [1980]Find Unique Binary String.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String findDifferentBinaryString(String[] nums) {
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < nums.length; i++) {
            char c = nums[i].charAt(i) == '0' ? '1' : '0';
            sb.append(c);
        }

        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

590. [1985]Find the Kth Largest Integer in the Array.java

```
1985
        Find the Kth Largest Integer in the Array
        2022-12-27 15:09:48

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String kthLargestNumber(String[] nums, int k) {
        Arrays.sort(nums, (a, b) -> a.length() == b.length() ? b.compareTo(a) :
b.length() - a.length());
        return nums[k - 1];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

591. `[1991]Find the Middle Index in Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findMiddleIndex(int[] nums) {
        int n = nums.length;
        int[] prefixSumLeft = new int[n];
        int[] prefixSumRight = new int[n];

        for (int i = 1; i < n; i++) {
            prefixSumLeft[i] = prefixSumLeft[i - 1] + nums[i - 1];
        }

        for (int i = n - 2; i >= 0; i--) {
            prefixSumRight[i] = prefixSumRight[i + 1] + nums[i + 1];
        }

        for (int i = 0; i < n; i++) {
            if (prefixSumLeft[i] == prefixSumRight[i]) {
                return i;
            }
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

592. [1992]Find All Groups of Farmland.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private int minRow;
    private int minCol;
    private int maxRow;
    private int maxCol;

    public int[][] findFarmland(int[][] land) {
        List<int[]> farmlands = new ArrayList<>();
        int m = land.length;
        int n = land[0].length;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (land[i][j] == 1) {
                    minRow = i;
                    maxRow = i;
                    minCol = j;
                    maxCol = j;
                    dfs(land, i, j);
                    farmlands.add(new int[]{minRow, minCol, maxRow, maxCol});
                }
            }
        }

        return farmlands.toArray(new int[0][]);
    }

    private void dfs(int[][] land, int row, int col) {
        if (row < 0 || row >= land.length || col < 0 || col >= land[0].length ||
land[row][col] != 1) {
            return;
        }

        land[row][col] = 0;
        minRow = Math.min(minRow, row);
        maxRow = Math.max(maxRow, row);
        minCol = Math.min(minCol, col);
        maxCol = Math.max(maxCol, col);
        dfs(land, row - 1, col);
        dfs(land, row + 1, col);
        dfs(land, row, col - 1);
        dfs(land, row, col + 1);
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

593. `[1998].java`

```
1998
        GCD Sort of an Array
        2023-03-03 14:18:28

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[] arr;

    public boolean gcdSort(int[] nums) {
        int n = nums.length;
        Set<Integer> set = new HashSet<>();
        int max = Integer.MIN_VALUE;
        for (int num : nums) {
            set.add(num);
            max = Math.max(max, num);
        }
        int[] sorted = nums.clone();
        Arrays.sort(sorted);
        arr = new int[max + 1];
        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }
        boolean[] flag = new boolean[max + 1];
        for (int i = 2; i * 2 <= max; i++) {
            if (!flag[i]) {
                for (int j = 2; i * j <= max; j++) {
                    if (set.contains(i * j)) {
                        union(i, i * j);
                    }
                    flag[i * j] = true;
                }
            }
        }
        for (int i = 0; i < n; i++) {
            if (find(nums[i]) != find(sorted[i])) {
                return false;
            }
        }
        return true;
    }

    private int find(int x) {
        return arr[x] == x ? x : (arr[x] = find(arr[x]));
    }

    private void union(int x, int y) {
        x = find(x);
        y = find(y);
        if (x != y) {
            arr[x] = y;
        }
```

```
    }
}

class UnionFind {


    public UnionFind(int n) {

    }


}
//leetcode submit region end(Prohibit modification and deletion)
```

594. `[2000]Reverse Prefix of Word.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String reversePrefix(String word, char ch) {
        int index = word.indexOf(ch);

        if (index == -1) {
            return word;
        }

        String toReverse = word.substring(0, index + 1);
        String reversed = new StringBuilder(toReverse).reverse().toString();
        return reversed + word.substring(index + 1);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

595. `[2001]Number of Pairs of Interchangeable Rectangles.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long interchangeableRectangles(int[][] rectangles) {
        Map<Double, Long> map = new HashMap<>();
        long count = 0;

        for (int[] rec : rectangles) {
            int x = rec[0];
            int y = rec[1];
            double ratio = (double) x / y;

            map.put(ratio, map.getOrDefault(ratio, 0L) + 1);
        }

        for (long num : map.values()) {
            count += num * (num - 1) / 2;
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

596. `[2009]Minimum Number of Operations to Make Array Continuous.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minOperations(int[] nums) {
        int n = nums.length;
        int m = 1;
        int result = n;

        Arrays.sort(nums);

        for (int i = 1; i < n; i++) {
            if (nums[i] != nums[i - 1]) {
                nums[m++] = nums[i];
            }
        }

        for (int i = 0, j = 0; i < m; i++) {
            while (j < m && nums[j] - nums[i] < n) {
                j++;
            }

            result = Math.min(result, n - (j - i));
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

597. [2016]Maximum Difference Between Increasing Elements.java

```
class Solution {
    public int maximumDifference(int[] nums) {
        int min = nums[0];
        int maxDiff = -1;
        int n = nums.length;

        for (int i = 1; i < n; i++) {
            if (nums[i] > min) {
                maxDiff = Math.max(maxDiff, nums[i] - min);
            } else {
                min = nums[i];
            }
        }
        return maxDiff;
    }
}
```

598. [2017]Grid Game.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long gridGame(int[][] grid) {
        int n = grid[0].length;
        long[] pre1 = new long[n + 1];
        for (int i = 0; i < n; i++) {
            pre1[i + 1] = pre1[i] + grid[1][i];
        }

        long[] suf0 = new long[n + 1];
        suf0[n] = 0;
        for (int i = n - 1; i >= 0; i--) {
            suf0[i] = suf0[i + 1] + grid[0][i];
        }

        long res = Long.MAX_VALUE;
        for (int i = 0; i < n; i++) {
            long maxVal = Math.max(pre1[i], suf0[i + 1]);
            res = Math.min(res, maxVal);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

599. `[2022]Convert 1D Array Into 2D Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[][] construct2DArray(int[] original, int m, int n) {
        if (original.length != m * n) {
            return new int[0][0];
        }

        int[][] result = new int[m][n];
        int index = 0;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                result[i][j] = original[index++];
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

600. `[2024]Maximize the Confusion of an Exam.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxConsecutiveAnswers(String answerKey, int k) {
        int countF = 0;
        int countT = 0;
        int left = 0;
        int right = 0;
        int res = 0;
        char[] c = answerKey.toCharArray();
        while (right < c.length) {
            if (c[right] == 'F') {
                countF++;
            } else {
                countT++;
            }
            while (Math.min(countT, countF) > k) {
                if (c[left] == 'F') {
                    countF--;
                } else {
                    countT--;
                }
                left++;
            }
            res = Math.max(res, countF + countT);
            right++;
        }
        return res;
    }

}
//leetcode submit region end(Prohibit modification and deletion)
```

601. `[2028]Find Missing Observations.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] missingRolls(int[] rolls, int mean, int n) {
        int m = rolls.length;
        int totalSum = mean * (m + n);
        int observedSum = 0;

        for (int roll : rolls) {
            observedSum += roll;
        }

        int missingSum = totalSum - observedSum;

        if (missingSum < n || missingSum > 6 * n) {
            return new int[0];
        }

        int[] result = new int[n];
        int baseValue = missingSum / n;
        int remainder = missingSum % n;

        for (int i = 0; i < n; i++) {
            result[i] = baseValue + (i < remainder ? 1 : 0);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

602. `[2037]Minimum Number of Moves to Seat Everyone.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minMovesToSeat(int[] seats, int[] students) {
        Arrays.sort(seats);
        Arrays.sort(students);
        return IntStream.range(0, seats.length)
                .map(i -> Math.abs(students[i] - seats[i]))
                .sum();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int minMovesToSeat(int[] seats, int[] students) {
        Arrays.sort(seats);
        Arrays.sort(students);
        return IntStream.range(0, seats.length)
                .map(i -> Math.abs(students[i] - seats[i]))
                .sum();
    }
}

class Solution {
    public int minMovesToSeat(int[] seats, int[] students) {
        Arrays.sort(seats);
        Arrays.sort(students);
        int diff = 0;
        int n = seats.length;

        for (int i = 0; i < n; i++) {
            diff += Math.abs(students[i] - seats[i]);
        }

        return diff;
    }
}
 */
```

603. [2038]Remove Colored Pieces if Both Neighbors are the Same Color.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean winnerOfGame(String colors) {
        int countA = 0;
        int countB = 0;
        char[] c = colors.toCharArray();

        for (int i = 2; i < c.length; i++) {
            if (c[i] == 'A' && c[i - 1] == 'A' && c[i - 2] == 'A') {
                countA++;
            }

            if (c[i] == 'B' && c[i - 1] == 'B' && c[i - 2] == 'B') {
                countB++;
            }
        }

        return countA > countB;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

604. `[2040]Kth Smallest Product of Two Sorted`

Arrays.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long kthSmallestProduct(int[] nums1, int[] nums2, long k) {
        int n1 = nums1.length;
        int n2 = nums2.length;
        long minProd = Long.MAX_VALUE;
        long maxProd = Long.MIN_VALUE;
        long[] candidates = new long[]{
                (long) nums1[0] * nums2[0],
                (long) nums1[0] * nums2[n2 - 1],
                (long) nums1[n1 - 1] * nums2[0],
                (long) nums1[n1 - 1] * nums2[n2 - 1]
        };
        for (long v : candidates) {
            minProd = Math.min(minProd, v);
            maxProd = Math.max(maxProd, v);
        }

        long left = minProd;
        long right = maxProd;
        while (left < right) {
            long mid = left + ((right - left) >> 1);
            long cnt = countLessEqual(nums1, nums2, mid);
            if (cnt >= k) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }

    private long countLessEqual(int[] nums1, int[] nums2, long target) {
        long count = 0;
        int n2 = nums2.length;
        for (int x : nums1) {
            if (x == 0) {
                if (target >= 0) {
                    count += n2;
                }
            } else if (x > 0) {
                long t = target / x;
                if (target < 0 && target % x != 0) {
                    t -= 1;
                }
                int cntLe = upperBound(nums2, t);
                count += cntLe;
            } else {
                long t = target / x;
                if (target < 0 && x < 0 && target % x != 0) {
                    t += 1;
```

```
                }
                int idx = lowerBound(nums2, t);
                count += (n2 - idx);
            }
        }
        return count;
    }

    private int upperBound(int[] arr, long value) {
        int lo = 0, hi = arr.length;
        while (lo < hi) {
            int mid = lo + ((hi - lo) >> 1);
            if ((long) arr[mid] <= value) {
                lo = mid + 1;
            } else {
                hi = mid;
            }
        }
        return lo;
    }

    private int lowerBound(int[] arr, long value) {
        int lo = 0, hi = arr.length;
        while (lo < hi) {
            int mid = lo + ((hi - lo) >> 1);
            if ((long) arr[mid] < value) {
                lo = mid + 1;
            } else {
                hi = mid;
            }
        }
        return lo;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

605. `[2045]Second Minimum Time to Reach`

Destination.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int secondMinimum(int n, int[][] edges, int time, int change) {
        List<Integer>[] graph = new ArrayList[n + 1];

        for (int i = 0; i <= n; i++) {
            graph[i] = new ArrayList<>();
        }

        for (int[] edge : edges) {
            graph[edge[0]].add(edge[1]);
            graph[edge[1]].add(edge[0]);
        }

        Deque<int[]> deque = new ArrayDeque<>();
        int[][] timeArr = new int[n + 1][2];

        for (int i = 1; i <= n; i++) {
            Arrays.fill(timeArr[i], Integer.MAX_VALUE);
        }

        deque.offer(new int[]{1, 0});
        timeArr[1][0] = 0;

        while (!deque.isEmpty()) {
            int[] current = deque.poll();
            int node = current[0];
            int currentTime = current[1];
            int nextTime = calculateNextTime(currentTime, time, change);

            for (int nextNode : graph[node]) {
                if (timeArr[nextNode][0] > nextTime) {
                    timeArr[nextNode][1] = timeArr[nextNode][0];
                    timeArr[nextNode][0] = nextTime;
                    deque.offer(new int[]{nextNode, nextTime});
                } else if (timeArr[nextNode][0] < nextTime && timeArr[nextNode][1] >
nextTime) {
                    timeArr[nextNode][1] = nextTime;
                    deque.offer(new int[]{nextNode, nextTime});
                }

                if (nextNode == n && timeArr[nextNode][1] < Integer.MAX_VALUE) {
                    return timeArr[nextNode][1];
                }
            }
        }

        return -1;
    }

    private int calculateNextTime(int currentTime, int travelTime, int change) {
```

```
        int currentCycle = currentTime / change;

        if (currentCycle % 2 == 0) {
            return currentTime + travelTime;
        } else {
            return (currentCycle + 1) * change + travelTime;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

606. `[2050]Parallel Courses III.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumTime(int n, int[][] relations, int[] time) {
        List<Integer>[] graph = new ArrayList[n];
        int[] inDegree = new int[n];
        Deque<Integer> deque = new ArrayDeque<>();
        int[] maxTime = new int[n];
        int result = 0;

        for (int i = 0; i < n; i++) {
            graph[i] = new ArrayList<>();
        }

        for (int[] arr : relations) {
            int x = arr[0] - 1;
            int y = arr[1] - 1;
            graph[x].add(y);
            inDegree[y]++;
        }

        for (int i = 0; i < n; i++) {
            if (inDegree[i] == 0) {
                deque.offer(i);
                maxTime[i] = time[i];
            }
        }

        while (!deque.isEmpty()) {
            int curr = deque.poll();

            for (int neighbor : graph[curr]) {
                maxTime[neighbor] = Math.max(maxTime[neighbor], maxTime[curr] +
time[neighbor]);
                inDegree[neighbor]--;

                if (inDegree[neighbor] == 0) {
                    deque.offer(neighbor);
                }
            }
        }

        for (int i = 0; i < n; i++) {
            result = Math.max(result, maxTime[i]);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
```

```
class Solution {
    public int minimumTime(int n, int[][] relations, int[] time) {
        Map<Integer, List<Integer>> graph = new HashMap<>();
        int[] inDegree = new int[n];
        Deque<Integer> deque = new ArrayDeque<>();
        int[] maxTime = new int[n];
        int result = 0;

        for (int i = 0; i < n; i++) {
            graph.put(i, new ArrayList<>());
        }

        for (int[] arr : relations) {
            int x = arr[0] - 1;
            int y = arr[1] - 1;
            graph.get(x).add(y);
            inDegree[y]++;
        }

        for (int i = 0; i < n; i++) {
            if (inDegree[i] == 0) {
                deque.offer(i);
                maxTime[i] = time[i];
            }
        }

        while (!deque.isEmpty()) {
            int curr = deque.poll();

            for (int neighbor : graph.get(curr)) {
                maxTime[neighbor] = Math.max(maxTime[neighbor], maxTime[curr] +
time[neighbor]);
                inDegree[neighbor]--;

                if (inDegree[neighbor] == 0) {
                    deque.offer(neighbor);
                }
            }
        }

        for (int i = 0; i < n; i++) {
            result = Math.max(result, maxTime[i]);
        }

        return result;
    }
}

class Solution {
    Map<Integer, List<Integer>> graph = new HashMap<>();
    Map<Integer, Integer> memo = new HashMap<>();

    public int minimumTime(int n, int[][] relations, int[] time) {
        int result = 0;
```

```
        for (int i = 0; i < n; i++) {
            graph.put(i, new ArrayList<>());
        }

        for (int[] arr : relations) {
            int x = arr[0] - 1;
            int y = arr[1] - 1;
            graph.get(x).add(y);
        }

        for (int i = 0; i < n; i++) {
            result = Math.max(result, dfs(i, time));
        }

        return result;
    }

    private int dfs(int i, int[] time) {
        int result = 0;

        if (memo.containsKey(i)) {
            return memo.get(i);
        }

        if (graph.get(i).size() == 0) {
            return time[i];
        }

        for (int neighbor : graph.get(i)) {
            result = Math.max(result, dfs(neighbor, time));
        }

        memo.put(i, time[i] + result);
        return time[i] + result;
    }
}
 */
```

607. [2053]Kth Distinct String in an Array.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String kthDistinct(String[] arr, int k) {
        Map<String, Integer> map = new HashMap<>();

        for (String str : arr) {
            map.merge(str, 1, Integer::sum);
        }

        int count = 0;

        for (String str : arr) {
            if (map.get(str) == 1) {
                count++;

                if (count == k) {
                    return str;
                }
            }
        }

        return "";
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

608. `[2054]Two Best Non-Overlapping Events.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxTwoEvents(int[][] events) {
        Arrays.sort(events, (a, b) -> {
            if (a[0] != b[0]) return Integer.compare(a[0], b[0]);
            return Integer.compare(a[1], b[1]);
        });

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[1],
b[1]));
        int max = 0;
        int result = 0;

        for (int i = 0; i < events.length; i++) {
            int currentStart = events[i][0];
            int currentValue = events[i][2];

            while (!pq.isEmpty() && pq.peek()[1] < currentStart) {
                max = Math.max(max, pq.poll()[2]);
            }

            result = Math.max(result, max + currentValue);
            result = Math.max(result, currentValue);
            pq.offer(events[i]);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

609. [2058]Find the Minimum and Maximum Number of Nodes Between Critical Points.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] nodesBetweenCriticalPoints(ListNode head) {
        if (head == null || head.next == null || head.next.next == null) {
            return new int[]{-1, -1};
        }

        int first = -1;
        int last = -1;
        int minDist = Integer.MAX_VALUE;
        ListNode prev = head;
        ListNode curr = head.next;
        int index = 1;

        while (curr.next != null) {
            if ((prev.val < curr.val && curr.val > curr.next.val) || (prev.val > curr.val
&& curr.val < curr.next.val)) {
                if (first == -1) {
                    first = index;
                } else {
                    minDist = Math.min(minDist, index - last);
                }

                last = index;
            }

            prev = curr;
            curr = curr.next;
            index++;
        }

        if (minDist == Integer.MAX_VALUE) {
            return new int[]{-1, -1};
        }

        return new int[]{minDist, last - first};
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

610. `[2061]Number of Spaces Cleaning Robot Cleaned.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private final int[] DIRECTIONS = {0, 1, 0, -1, 0};

    public int numberOfCleanRooms(int[][] room) {
        int rows = room.length;
        int cols = room[0].length;
        int[][] visited = new int[rows][cols];
        int cleaned = 0;

        Deque<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{0, 0, 0});

        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            int row = current[0];
            int col = current[1];
            int direction = current[2];

            if (visited[row][col] == 0) {
                cleaned += 1;
            }

            visited[row][col] |= 1 << direction;

            for (int d = 0; d < 4; d++) {
                int nextDir = (direction + d) % 4;
                int nextRow = row + DIRECTIONS[nextDir];
                int nextCol = col + DIRECTIONS[nextDir + 1];

                if (0 <= nextRow && nextRow < rows && 0 <= nextCol && nextCol < cols &&
room[nextRow][nextCol] == 0) {
                    if ((visited[nextRow][nextCol] >> nextDir & 1) == 1) {
                        return cleaned;
                    } else {
                        queue.offer(new int[]{nextRow, nextCol, nextDir});
                        break;
                    }
                }
            }
        }

        return cleaned;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

611. `[2062]Count Vowel Substrings of a String.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countVowelSubstrings(String word) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

612. `[2063]Vowels of All Substrings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long countVowels(String word) {

    }

}
//leetcode submit region end(Prohibit modification and deletion)
```

613. `[2068].java`

```
2068
        Check Whether Two Strings are Almost Equivalent
        2023-03-03 17:52:15

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean checkAlmostEquivalent(String word1, String word2) {
        int[] arr = new int[26];
        char[] c1 = word1.toCharArray();
        char[] c2 = word2.toCharArray();
        for (int i = 0; i < word1.length(); i++) {
            arr[c1[i] - 'a']++;
            arr[c2[i] - 'a']--;
        }
        for (int num : arr) {
            if (num > 3 || num < -3) {
                return false;
            }
        }
        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

614. [2070]Most Beautiful Item for Each Query.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] maximumBeauty(int[][] items, int[] queries) {
        Arrays.sort(items, (a, b) -> a[0] == b[0] ? b[1] - a[1] : a[0] - b[0]);
        int currMaxBeauty = 0;
        TreeMap<Integer, Integer> map = new TreeMap<>();

        for (int[] item : items) {
            if (item[1] <= currMaxBeauty) {
                continue;
            }

            currMaxBeauty = item[1];
            map.put(item[0], currMaxBeauty);
        }

        int[] res = new int[queries.length];

        for (int i = 0; i < queries.length; i++) {
            Map.Entry<Integer, Integer> entry = map.floorEntry(queries[i]);

            if (entry != null) {
                res[i] = entry.getValue();
            }
        }

        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

615. [2071]Maximum Number of Tasks You Can Assign.java

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    public int maxTaskAssign(int[] tasks, int[] workers, int pills, int strength) {
        Arrays.sort(tasks);
        Arrays.sort(workers);
        int n = tasks.length, m = workers.length;

        int lo = 0, hi = Math.min(n, m), ans = 0;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (canAssign(tasks, workers, mid, pills, strength)) {
                ans = mid;
                lo = mid + 1;
            } else {
                hi = mid - 1;
            }
        }
        return ans;
    }

    private boolean canAssign(int[] tasks, int[] workers, int k, int pills, int strength)
{
        if (k == 0) return true;

        int m = workers.length;
        TreeMap<Integer, Integer> map = new TreeMap<>();
        for (int i = m - k; i < m; i++) {
            map.merge(workers[i], 1, Integer::sum);
        }

        for (int i = k - 1; i >= 0; i--) {
            int t = tasks[i];
            Integer w = map.lastKey();
            if (w >= t) {
                removeOne(map, w);
            } else {
                if (pills == 0) return false;
                Integer candidate = map.ceilingKey(t - strength);
                if (candidate == null) return false;
                removeOne(map, candidate);
                pills--;
            }
        }
        return true;
    }

    private void removeOne(TreeMap<Integer, Integer> map, int key) {
        int cnt = map.get(key);
        if (cnt == 1) map.remove(key);
        else map.put(key, cnt - 1);
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

616. [2073]Time Needed to Buy Tickets.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int timeRequiredToBuy(int[] tickets, int k) {
        int count = 0;

        for (int i = 0; i < tickets.length; i++) {
            if (tickets[i] >= benchMark) {
                if (i <= k) {
                    count += benchMark;
                } else {
                    count += benchMark - 1;
                }
            } else {
                count += tickets[i];
            }
        }

        return count;
    }
}
/*
class Solution {
    public int timeRequiredToBuy(int[] tickets, int k) {
        int count = 0;

        for (int i = 0; i < tickets.length; i++) {
            count += Math.min(tickets[i], tickets[k] - (i > k ? 1 : 0));
        }

        return count;
    }
}
 */
//leetcode submit region end(Prohibit modification and deletion)
```

617. [2083]Substrings That Begin and End With the Same Letter.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long numberOfSubstrings(String s) {
        int[] count = new int[26];
        long result = 0;
        int n = s.length();

        for (int i = 0; i < n; i++) {
            char c = s.charAt(i);
            count[c - 'a']++;
            result += count[c - 'a'];
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

618. [2090]K Radius Subarray Averages.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] getAverages(int[] nums, int k) {
        int n = nums.length;
        int[] arr = new int[n];
        Arrays.fill(arr, -1);
        if (2 * k + 1 > n) {
            return arr;
        }
        long[] prefixSum = new long[n + 1];
        for (int i = 0; i < n; i++) {
            prefixSum[i + 1] = prefixSum[i] + nums[i];
        }
        for (int i = k; i < n - k; i++) {
            int left = i - k;
            int right = i + k;
            arr[i] = (int) ((prefixSum[right + 1] - prefixSum[left]) / (2 * k + 1));
        }
        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

619. [2092]Find All People With Secret.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> findAllPeople(int n, int[][] meetings, int firstPerson) {
        Arrays.sort(meetings, (a, b) -> a[2] - b[2]);
        UnionFind uf = new UnionFind(n);
        uf.union(0, firstPerson);
        processMeetings(meetings, uf);
        return getResult(uf, n);
    }

    private void processMeetings(int[][] meetings, UnionFind uf) {
        for (int i = 0; i < meetings.length; ) {
            Set<Integer> pool = new HashSet<>();
            int time = meetings[i][2];

            while (i < meetings.length && time == meetings[i][2]) {
                int[] currMeeting = meetings[i];
                uf.union(currMeeting[0], currMeeting[1]);
                pool.add(currMeeting[0]);
                pool.add(currMeeting[1]);
                i++;
            }

            resetPeople(uf, pool);
        }
    }

    private void resetPeople(UnionFind uf, Set<Integer> pool) {
        for (int person : pool) {
            if (!uf.isConnected(0, person)) {
                uf.reset(person);
            }
        }
    }

    private List<Integer> getResult(UnionFind uf, int n) {
        List<Integer> result = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            if (uf.isConnected(i, 0)) {
                result.add(i);
            }
        }

        return result;
    }
}

class UnionFind {
    public int[] group;
```

```
    public UnionFind(int n) {
        group = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            group[i] = i;
        }
    }

    private int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }

    public void union(int i, int j) {
        int x = find(i);
        int y = find(j);

        if (x == y) {
            return;
        }

        group[x] = y;
    }

    public boolean isConnected(int i, int j) {
        return find(i) == find(j);
    }

    public void reset(int p) {
        group[p] = p;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

620. `[2093]Minimum Cost to Reach City With`

Discounts.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumCost(int n, int[][] highways, int discounts) {
        List<int[]>[] graph = new ArrayList[n];

        for (int i = 0; i < n; i++) {
            graph[i] = new ArrayList<>();
        }

        for (int[] highway : highways) {
            int u = highway[0];
            int v = highway[1];
            int toll = highway[2];
            graph[u].add(new int[]{v, toll});
            graph[v].add(new int[]{u, toll});
        }

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
        pq.add(new int[]{0, 0, discounts});
        int[][] dist = new int[n][discounts + 1];

        for (int[] row : dist) {
            Arrays.fill(row, Integer.MAX_VALUE);
        }

        dist[0][discounts] = 0;

        while (!pq.isEmpty()) {
            int[] state = pq.poll();
            int currentCost = state[0];
            int u = state[1];
            int remainingDiscounts = state[2];

            if (u == n - 1) {
                return currentCost;
            }

            if (currentCost > dist[u][remainingDiscounts]) {
                continue;
            }

            for (int[] edge : graph[u]) {
                int v = edge[0];
                int toll = edge[1];

                if (currentCost + toll < dist[v][remainingDiscounts]) {
                    dist[v][remainingDiscounts] = currentCost + toll;
                    pq.add(new int[]{currentCost + toll, v, remainingDiscounts});
                }
```

```
                if (remainingDiscounts > 0) {
                    int discountedCost = currentCost + toll / 2;

                    if (discountedCost < dist[v][remainingDiscounts - 1]) {
                        dist[v][remainingDiscounts - 1] = discountedCost;
                        pq.add(new int[]{discountedCost, v, remainingDiscounts - 1});
                    }
                }
            }
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

621. `[2096]Step-By-Step Directions From a Binary Tree`

Node to Another.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String getDirections(TreeNode root, int startValue, int destValue) {
        StringBuilder pathToStart = new StringBuilder();
        StringBuilder pathToDest = new StringBuilder();

        findPath(root, startValue, pathToStart);
        findPath(root, destValue, pathToDest);

        int commonLength = 0;

        while (commonLength < pathToStart.length() && commonLength < pathToDest.length()
                && pathToStart.charAt(commonLength) == pathToDest.charAt(commonLength)) {
            commonLength++;
        }

        StringBuilder directions = new StringBuilder();

        for (int i = commonLength; i < pathToStart.length(); i++) {
            directions.append('U');
        }

        directions.append(pathToDest.substring(commonLength));
        return directions.toString();
    }

    private boolean findPath(TreeNode root, int target, StringBuilder path) {
        if (root == null) {
            return false;
        }

        if (root.val == target) {
            return true;
        }

        path.append('L');

        if (findPath(root.left, target, path)) {
            return true;
        }

        path.deleteCharAt(path.length() - 1);
        path.append('R');

        if (findPath(root.right, target, path)) {
            return true;
        }

        path.deleteCharAt(path.length() - 1);
        return false;
```

```
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public String getDirections(TreeNode root, int startValue, int destValue) {
        List<Character> pathToStart = new ArrayList<>();
        List<Character> pathToDest = new ArrayList<>();

        findPath(root, startValue, pathToStart);
        findPath(root, destValue, pathToDest);

        int commonLength = (int) IntStream.range(0, Math.min(pathToStart.size(),
pathToDest.size()))
                .filter(i -> pathToStart.get(i) != pathToDest.get(i))
                .findFirst()
                .orElse(Math.min(pathToStart.size(), pathToDest.size()));

        String directions = Stream.concat(
                        IntStream.range(commonLength, pathToStart.size()).mapToObj(i ->
'U'),
                        pathToDest.subList(commonLength, pathToDest.size())
                                .stream())
                .map(Objects::toString)
                .collect(Collectors.joining());
        return directions;
    }

    private boolean findPath(TreeNode root, int target, List<Character> path) {
        if (root == null) {
            return false;
        }

        if (root.val == target) {
            return true;
        }

        path.add('L');

        if (findPath(root.left, target, path)) {
            return true;
        }

        path.remove(path.size() - 1);
        path.add('R');

        if (findPath(root.right, target, path)) {
            return true;
        }

        path.remove(path.size() - 1);
        return false;
    }
```

```
}
 */
```

622. `[2097]Valid Arrangement of Pairs.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
import java.util.*;

class Solution {
    public int[][] validArrangement(int[][] pairs) {
        Map<Integer, Deque<Integer>> adj = new HashMap<>();
        Map<Integer, Integer> inDegree = new HashMap<>();
        Map<Integer, Integer> outDegree = new HashMap<>();

        for (int[] pair : pairs) {
            int start = pair[0];
            int end = pair[1];
            adj.computeIfAbsent(start, k -> new ArrayDeque<>()).add(end);
            outDegree.put(start, outDegree.getOrDefault(start, 0) + 1);
            inDegree.put(end, inDegree.getOrDefault(end, 0) + 1);
        }

        int startNode = pairs[0][0];

        for (int node : outDegree.keySet()) {
            if (outDegree.getOrDefault(node, 0) - inDegree.getOrDefault(node, 0) == 1) {
                startNode = node;
                break;
            }
        }

        List<Integer> path = new ArrayList<>();
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(startNode);

        while (!stack.isEmpty()) {
            int current = stack.peek();
            Deque<Integer> neighbors = adj.get(current);

            if (neighbors != null && !neighbors.isEmpty()) {
                int next = neighbors.pollFirst();
                stack.push(next);
            } else {
                path.add(stack.pop());
            }
        }

        Collections.reverse(path);
        int n = path.size();
        int[][] result = new int[n - 1][2];

        for (int i = 0; i < n - 1; i++) {
            result[i][0] = path.get(i);
            result[i][1] = path.get(i + 1);
        }
```

```
        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

623. `[2101]Detonate the Maximum Bombs.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumDetonation(int[][] bombs) {
        List<List<Integer>> list = new ArrayList<>();
        int n = bombs.length;
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
            int x1 = bombs[i][0];
            int y1 = bombs[i][1];
            int r1 = bombs[i][2];
            for (int j = 0; j < n; j++) {
                if (i == j) {
                    continue;
                }
                int x2 = bombs[j][0];
                int y2 = bombs[j][1];
                if ((long) (x2 - x1) * (x2 - x1) + (long) (y2 - y1) * (y2 - y1) <= (long)
r1 * r1) {
                    list.get(i).add(j);
                }
            }
        }
        int res = 0;
        for (int i = 0; i < n; i++) {
            res = Math.max(res, dfs(i, list, new boolean[n]));
            if (res == n) {
                return res;
            }
        }
        return res;
    }

    public int dfs(int i, List<List<Integer>> list, boolean[] isVisited) {
        if (isVisited[i]) {
            return 0;
        }
        isVisited[i] = true;
        int count = 1;
        for (int num : list.get(i)) {
            count += dfs(num, list, isVisited);
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
DFS - 13ms
class Solution {
    public int maximumDetonation(int[][] bombs) {
        List<List<Integer>> list = new ArrayList<>();
```

```
        int n = bombs.length;
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
            int xi = bombs[i][0];
            int yi = bombs[i][1];
            int ri = bombs[i][2];
            for (int j = 0; j < n; j++) {
                if (i == j) {
                    continue;
                }
                int xj = bombs[j][0];
                int yj = bombs[j][1];
                if ((long) ri * ri >= (long) (xi - xj) * (xi - xj) + (long) (yi - yj) *
(yi - yj)) {
                    list.get(i).add(j);
                }
            }
        }
        int res = 0;
        for (int i = 0; i < n; i++) {
            res = Math.max(res, dfs(i, list, new boolean[n]));
            if (res == n) {
                return res;
            }
        }
        return res;
    }

    public int dfs(int i, List<List<Integer>> list, boolean[] isVisited) {
        if (isVisited[i]) {
            return 0;
        }
        isVisited[i] = true;
        int count = 1;
        for (int key : list.get(i)) {
            count += dfs(key, list, isVisited);
        }
        return count;
    }
}


BFS - 19ms
class Solution {
    public int maximumDetonation(int[][] bombs) {
        List<List<Integer>> list = new ArrayList<>();
        int n = bombs.length;
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
            int xi = bombs[i][0];
            int yi = bombs[i][1];
            int ri = bombs[i][2];
            for (int j = 0; j < n; j++) {
                if (i == j) {
```

```
                    continue;
                }
                int xj = bombs[j][0];
                int yj = bombs[j][1];
                if ((long) ri * ri >= (long) (xi - xj) * (xi - xj) + (long) (yi - yj) *
(yi - yj)) {
                    list.get(i).add(j);
                }
            }
        }
        int res = 0;
        for (int i = 0; i < n; i++) {
            res = Math.max(res, bfs(i, list));
            if (res == n) {
                return res;
            }
        }
        return res;
    }

    public int bfs(int i, List<List<Integer>> list) {
        Deque<Integer> dq = new ArrayDeque<>();
        Set<Integer> visit = new HashSet<>();
        dq.add(i);
        visit.add(i);
        while (!dq.isEmpty()) {
            int curr = dq.poll();
            for (int key : list.get(curr)) {
                if (!visit.contains(key)) {
                    dq.add(key);
                    visit.add(key);
                }
            }
        }
        return visit.size();
    }
}
 */
```

624. [2106]Maximum Fruits Harvested After at Most K

Steps.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxTotalFruits(int[][] fruits, int startPos, int k) {
        List<int[]> list = new ArrayList<>();
        int leftBound = startPos - k, rightBound = startPos + k;
        for (int[] f : fruits) {
            int pos = f[0];
            if (pos < leftBound) {
                continue;
            }
            if (pos > rightBound) {
                break;
            }
            list.add(f);
        }
        int n = list.size();
        if (n == 0) {
            return 0;
        }

        int[] pos = new int[n];
        long[] preSum = new long[n + 1];
        for (int i = 0; i < n; i++) {
            int[] cur = list.get(i);
            pos[i] = cur[0];
            preSum[i + 1] = preSum[i] + cur[1];
        }

        long ans = 0;
        for (int l = 0, r = 0; r < n; r++) {
            while (l <= r && !withinSteps(pos[l], pos[r], startPos, k)) {
                l++;
            }
            if (l > r) {
                continue;
            }

            long total = preSum[r + 1] - preSum[l];
            ans = Math.max(ans, total);
        }
        return (int) ans;
    }

    private boolean withinSteps(int leftPos, int rightPos, int startPos, int k) {
        int stepsOption1 = (startPos - leftPos) * 2 + (rightPos - startPos);
        int stepsOption2 = (rightPos - startPos) * 2 + (startPos - leftPos);
        return Math.min(stepsOption1, stepsOption2) <= k;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

625. [2107]Number of Unique Flavors After Sharing K

Candies.java

```
//leetcode submit region begin(Prohibit modification and deletion)
import java.util.*;

class Solution {
    public int shareCandies(int[] candies, int k) {
        int n = candies.length;

        if (k == 0) {
            int totalUnique = 0;
            int[] total_freq = new int[100001];

            for (int flavor : candies) {
                if (total_freq[flavor] == 0) {
                    totalUnique++;
                }

                total_freq[flavor]++;
            }

            return totalUnique;
        }

        if (k > n) {
            return 0;
        }

        int[] total_freq = new int[100001];

        for (int flavor : candies) {
            total_freq[flavor]++;
        }

        int total_unique_flavors = 0;

        for (int i = 0; i < total_freq.length; i++) {
            if (total_freq[i] > 0) {
                total_unique_flavors++;
            }
        }

        int[] window_freq = new int[100001];
        int current_flavors_entirely_in_window = 0;

        for (int i = 0; i < k; i++) {
            int flavor = candies[i];
            window_freq[flavor]++;

            if (window_freq[flavor] == total_freq[flavor]) {
                current_flavors_entirely_in_window++;
            }
```

```
        }

        int min_flavors_entirely_in_window = current_flavors_entirely_in_window;

        for (int i = k; i < n; i++) {
            int left_flavor = candies[i - k];

            if (window_freq[left_flavor] == total_freq[left_flavor]) {
                current_flavors_entirely_in_window--;
            }

            window_freq[left_flavor]--;

            if (window_freq[left_flavor] == total_freq[left_flavor]) {
                current_flavors_entirely_in_window++;
            }

            int right_flavor = candies[i];
            window_freq[right_flavor]++;

            if (window_freq[right_flavor] == total_freq[right_flavor]) {
                current_flavors_entirely_in_window++;
            } else if (window_freq[right_flavor] == total_freq[right_flavor] + 1) {
                current_flavors_entirely_in_window--;
            }

            if (current_flavors_entirely_in_window < min_flavors_entirely_in_window) {
                min_flavors_entirely_in_window = current_flavors_entirely_in_window;
            }
        }

        return total_unique_flavors - min_flavors_entirely_in_window;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

626. [2108]Find First Palindromic String in the Array.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String firstPalindrome(String[] words) {
        for (String word : words) {
            if (isPalindrome(word)) {
                return word;
            }
        }

        return "";
    }

    private boolean isPalindrome(String wrod) {
        int left = 0;
        int right = wrod.length() - 1;

        while (left < right) {
            if (wrod.charAt(left) != wrod.charAt(right)) {
                return false;
            }

            left++;
            right--;
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

627. `[2116]Check if a Parentheses String Can Be Valid.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canBeValid(String s, String locked) {
        int n = s.length();

        if (n % 2 != 0) {
            return false;
        }

        int lower = 0;
        int upper = 0;
        for (int i = 0; i < n; i++) {
            char c = s.charAt(i);
            char lock = locked.charAt(i);

            if (lock == '1') {
                if (c == '(') {
                    lower++;
                    upper++;
                } else {
                    lower--;
                    upper--;
                }
            } else {
                lower--;
                upper++;
            }

            if (lower < 0) {
                lower = 0;
            }

            if (upper < 0) {
                return false;
            }
        }

        return lower == 0;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

628. `[2125]Number of Laser Beams in a Bank.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfBeams(String[] bank) {
        int prev = 0;
        int total = 0;

        for (String row : bank) {
            int curr = getCount(row);

            if (curr == 0) {
                continue;
            }

            total += curr * prev;
            prev = curr;
        }

        return total;
    }

    private int getCount(String s) {
        int count = 0;

        for (char c : s.toCharArray()) {
            count += c - '0';
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

629. `[2126]Destroying Asteroids.java`

```
2126
        Destroying Asteroids
        2022-12-10 14:32:57

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean asteroidsDestroyed(int mass, int[] asteroids) {
        Arrays.sort(asteroids);
        long currMass = mass;
        for (int num : asteroids) {
            if (num > currMass)
                return false;
            currMass += num;
        }
        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

630. [2127]Maximum Employees to Be Invited to a

Meeting.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumInvitations(int[] favorite) {
        int n = favorite.length;

        List<List<Integer>> reverseGraph = new ArrayList<>();
        for(int i=0; i<n; i++) {
            reverseGraph.add(new ArrayList<>());
        }
        for(int i=0; i<n; i++) {
            reverseGraph.get(favorite[i]).add(i);
        }

        boolean[] visited = new boolean[n];
        boolean[] onStack = new boolean[n];
        List<List<Integer>> cycles = new ArrayList<>();
        List<int[]> mutualPairs = new ArrayList<>();

        for(int i=0; i<n; i++) {
            if(!visited[i]) {
                List<Integer> stack = new ArrayList<>();
                dfsCycle(i, favorite, visited, onStack, stack, cycles, mutualPairs);
            }
        }

        boolean[] inCycle = new boolean[n];
        for(List<Integer> cycle : cycles){
            if(cycle.size() > 2){
                for(int node : cycle){
                    inCycle[node] = true;
                }
            }
        }

        int[] maxChain = new int[n];
        Arrays.fill(maxChain, -1);
        int mutualPairsContribution = 0;

        for(int[] pair : mutualPairs){
            int A = pair[0];
            int B = pair[1];
            int chainA = findMaxChain(A, reverseGraph, inCycle, B, maxChain);
            int chainB = findMaxChain(B, reverseGraph, inCycle, A, maxChain);
            mutualPairsContribution += 2 + chainA + chainB;
        }

        int maxCycle = 0;
        for(List<Integer> cycle : cycles){
            if(cycle.size() > 2){
                maxCycle = Math.max(maxCycle, cycle.size());
```

```
            }
        }

        return Math.max(maxCycle, mutualPairsContribution);
    }

    private void dfsCycle(int node, int[] favorite, boolean[] visited, boolean[] onStack,
                          List<Integer> stack, List<List<Integer>> cycles, List<int[]>
mutualPairs){
        visited[node] = true;
        onStack[node] = true;
        stack.add(node);

        int fav = favorite[node];
        if(!visited[fav]){
            dfsCycle(fav, favorite, visited, onStack, stack, cycles, mutualPairs);
        }
        else if(onStack[fav]){
            int index = stack.indexOf(fav);
            if(index != -1){
                List<Integer> cycle = new ArrayList<>(stack.subList(index,
stack.size()));
                if(cycle.size() == 2){
                    mutualPairs.add(new int[]{cycle.get(0), cycle.get(1)});
                }
                else{
                    cycles.add(cycle);
                }
            }
        }

        onStack[node] = false;
        stack.remove(stack.size()-1);
    }

    private int findMaxChain(int node, List<List<Integer>> reverseGraph, boolean[]
inCycle,
                             int excludeNode, int[] maxChain){
        if(maxChain[node] != -1){
            return maxChain[node];
        }

        int max = 0;
        for(int neighbor : reverseGraph.get(node)){
            if(neighbor == excludeNode || inCycle[neighbor]){
                continue;
            }
            max = Math.max(max, 1 + findMaxChain(neighbor, reverseGraph, inCycle,
excludeNode, maxChain));
        }
        maxChain[node] = max;
        return max;
    }
}
```

```
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int maximumInvitations(int[] favorite) {
        int n = favorite.length;
        int[] indegrees = new int[n];
        int[] maxDepth = new int[n];
        for(int i = 0; i < n; ++i) {
            indegrees[favorite[i]]++;
        }

        Queue<Integer> q = new LinkedList<>();
        for(int i = 0; i < n; ++i) {
            if(indegrees[i] == 0) {
                q.add(i);
                maxDepth[i] = 0;
            }
        }
        while(!q.isEmpty()) {
            int top = q.poll();
            int v = favorite[top];
            indegrees[v]--;
            maxDepth[v] = Integer.max(maxDepth[v], 1 + maxDepth[top]);
            if(indegrees[v] == 0) {
                q.add(v);
            }
        }

        int resultWithTwoNodeCycle = 0;
        int resultWithFullCycle = 0;

        int[] visited = new int[n];
        for(int i = 0; i < n; ++i) {
            if(indegrees[i] != 0 && visited[i] == 0) {
                int length = 0;
                int j = i;
                while(visited[j] != 1) {
                    visited[j] = 1;
                    j = favorite[j];
                    length++;
                }
                if(length == 2) {
                    resultWithTwoNodeCycle += maxDepth[i] + maxDepth[favorite[i]] + 2;
                }
                resultWithFullCycle = Integer.max(resultWithFullCycle, length);
            }
        }

        return Integer.max(resultWithFullCycle, resultWithTwoNodeCycle);
    }
}
 */
```

631. `[2128]Remove All Ones With Row and Column Flips.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean removeOnes(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;

        for (int i = 0; i < m; i++) {
            boolean same = true;
            boolean opposite = true;

            for (int j = 0; j < n; j++) {
                if (grid[i][j] != grid[0][j]) {
                    same = false;
                }

                if (grid[i][j] != 1 - grid[0][j]) {
                    opposite = false;
                }
            }

            if (!same && !opposite) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

632. [2130]Maximum Twin Sum of a Linked List.java

```
2130
        Maximum Twin Sum of a Linked List
        2022-11-29 14:29:46
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode() {}
 * ListNode(int val) { this.val = val; }
 * ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public int pairSum(ListNode head) {
        List<Integer> list = new ArrayList<>();
        ListNode curr = head;
        while (curr != null) {
            list.add(curr.val);
            curr = curr.next;
        }
        int n = list.size();
        int max = 0;
        for (int i = 0; i <= n / 2; i++) {
            max = Math.max(max, list.get(i) + list.get(n - 1 - i));
        }
        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int pairSum(ListNode head) {
        ListNode mid = middle(head);
        ListNode second = reverse(mid);
        ListNode first = head;
        return twinSum(first, second);
    }

    public ListNode middle(ListNode head) {
        ListNode fast = head;
        ListNode slow = head;
        while (fast != null && fast.next != null) {
            fast = fast.next.next;
            slow = slow.next;
        }
        return slow;
    }
```

```
    public ListNode reverse(ListNode head) {
        ListNode dummy = new ListNode();
        dummy.next = head;
        ListNode prev = dummy;
        ListNode curr = head;
        while (curr.next != null) {
            ListNode temp = curr.next;
            curr.next = temp.next;
            temp.next = prev.next;
            prev.next = temp;
        }
        return dummy.next;
    }

    public int twinSum(ListNode first, ListNode second) {
        int res = Integer.MIN_VALUE;
        while (first != null && second != null) {
            res = Math.max(res, first.val + second.val);
            first = first.next;
            second = second.next;
        }
        return res;
    }
}
 */
```

633. [2131]Longest Palindrome by Concatenating Two Letter Words.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int longestPalindrome(String[] words) {
    int[][] count = new int[26][26];
    int result = 0;

    for (String word : words) {
      int first = word.charAt(0) - 'a';
      int second = word.charAt(1) - 'a';

      if (count[second][first] > 0) {
        result += 4;
        count[second][first]--;
      } else {
        count[first][second]++;
      }
    }

    for (int i = 0; i < 26; i++) {
      if (count[i][i] > 0) {
        result += 2;
        break;
      }
    }

    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

634. `[2134]Minimum Swaps to Group All 1's Together`

II.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minSwaps(int[] nums) {
        int n = nums.length;
        int countOnes = 0;

        for (int num : nums) {
            countOnes += num;
        }

        if (countOnes == 0 || countOnes == n) {
            return 0;
        }

        int currOnes = 0;

        for (int i = 0; i < countOnes; i++) {
            currOnes += nums[i];
        }

        int maxOnes = currOnes;

        for (int i = countOnes; i < n + countOnes; i++) {
            currOnes += nums[i % n] - nums[(i - countOnes) % n];
            maxOnes = Math.max(maxOnes, currOnes);
        }

        return countOnes - maxOnes;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int minSwaps(int[] nums) {
        int n = nums.length;
        int countOnes = Arrays.stream(nums).sum();

        if (countOnes == 0 || countOnes == n) {
            return 0;
        }

        int currOnes = Arrays.stream(nums, 0, countOnes).sum();
        int maxOnes = currOnes;

        for (int i = countOnes; i < n + countOnes; i++) {
            currOnes += nums[i % n] - nums[(i - countOnes) % n];
            maxOnes = Math.max(maxOnes, currOnes);
        }

        return countOnes - maxOnes;
    }
```

```
}
 */
```

635. [2138]Divide a String Into Groups of Size k.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String[] divideString(String s, int k, char fill) {
        StringBuilder sb = new StringBuilder(s);
        while (sb.length() % k != 0) {
            sb.append(fill);
        }

        int groupCount = sb.length() / k;
        String[] result = new String[groupCount];
        for (int i = 0; i < groupCount; i++) {
            result[i] = sb.substring(i * k, (i + 1) * k);
        }
        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

636. [2140]Solving Questions With Brainpower.java

```
2140
        Solving Questions With Brainpower
        2022-12-14 15:34:33

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long mostPoints(int[][] questions) {
        int n = questions.length;
        long dp[] = new long[n + 1];
        for (int i = n - 1; i >= 0; i--) {
            int j = i + questions[i][1] + 1;
            dp[i] = Math.max(dp[i + 1], questions[i][0] + dp[Math.min(j, n)]);
        }
        return dp[0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

637. [2141]Maximum Running Time of N Computers.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long maxRunTime(int n, int[] batteries) {
        int bLen = batteries.length;
        if (bLen < n) {
            return 0;
        }
        long left = 0;
        long right = 0;
        for (int battery : batteries) {
            right += battery;
        }
        right /= n;
        while (left <= right) {
            long mid = (left + right) >> 1;
            if (isCheck(n, batteries, mid)) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left - 1;
    }

    private boolean isCheck(int n, int[] batteries, long target) {
        long sum = target * n;
        for (int battery : batteries) {
            sum -= Math.min(target, battery);
        }
        return sum <= 0;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

638. [2147]Number of Ways to Divide a Long

Corridor.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfWays(String corridor) {
        final int MOD = (int) 1e9 + 7;
        int a = 1;
        int b = 0;
        int c = 0;

        for (char ch : corridor.toCharArray()) {
            if (ch == 'S') {
                a = (a + c) % MOD;
                c = b;
                b = a;
                a = 0;
            } else {
                a = (a + c) % MOD;
            }
        }

        return c;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
Solution 1: Combination
Assume a[i] is the index of ith seat,
there are a[i+1] - a[i] ways to put a divider between ith and i+1th seats.
We only needs to calculate the produit of all these a[i+1] - a[i].

If the number of seat is odd or is less than 2, then no way to divide the corridor.

class Solution {
    public int numberOfWays(String corridor) {
        final long MOD = (long) 1e9 + 7;
        long result = 1;
        long j = 0;
        long k = 0;

        for (int i = 0; i < corridor.length(); i++) {
            if (corridor.charAt(i) == 'S') {
                if (++k > 2 && k % 2 == 1) {
                    result = result * (i - j) % MOD;
                }

                j = i;
            }
        }

        return k % 2 == 0 && k > 0 ? (int) result : 0;
    }
```

```
}

Solution 2: DP
a the number of 0 seat
b the number of 1 seat
c the number of 2 seats

class Solution {
    public int numberOfWays(String corridor) {
        final int MOD = (int) 1e9 + 7;
        int a = 1;
        int b = 0;
        int c = 0;

        for (char ch : corridor.toCharArray()) {
            if (ch == 'S') {
                a = (a + c) % MOD;
                c = b;
                b = a;
                a = 0;
            } else {
                a = (a + c) % MOD;
            }
        }

        return c;
    }
}
 */
```

639. `[2149]Rearrange Array Elements by Sign.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] rearrangeArray(int[] nums) {
        int[] result = new int[nums.length];
        int posIndex = 0;
        int negIndex = 1;

        for (int num : nums) {
            if (num > 0) {
                result[posIndex] = num;
                posIndex += 2;
            } else {
                result[negIndex] = num;
                negIndex += 2;
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

640. `[2161]Partition Array According to Given Pivot.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int[] pivotArray(int[] nums, int pivot) {
    List<Integer> less = new ArrayList<>();
    List<Integer> equal = new ArrayList<>();
    List<Integer> greater = new ArrayList<>();

    for (int num : nums) {
      if (num < pivot) {
        less.add(num);
      } else if (num > pivot) {
        greater.add(num);
      } else {
        equal.add(num);
      }
    }

    int[] result = new int[nums.length];
    int index = 0;

    for (int num : less) {
      result[index++] = num;
    }

    for (int num : equal) {
      result[index++] = num;
    }

    for (int num : greater) {
      result[index++] = num;
    }

    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

641. `[2163]Minimum Difference in Sums After Removal of Elements.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    public long minimumDifference(int[] nums) {
        final int total = nums.length;
        final int n = total / 3;

        long[] leftMin = new long[total];
        long[] rightMax = new long[total];

        PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
        long sumL = 0;
        for (int i = 0; i < total; i++) {
            maxHeap.offer(nums[i]);
            sumL += nums[i];
            if (maxHeap.size() > n) {
                sumL -= maxHeap.poll();
            }
            leftMin[i] = (maxHeap.size() == n) ? sumL : Long.MAX_VALUE / 4;
        }

        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        long sumR = 0;
        for (int i = total - 1; i >= 0; i--) {
            minHeap.offer(nums[i]);
            sumR += nums[i];
            if (minHeap.size() > n) {
                sumR -= minHeap.poll();
            }
            rightMax[i] = (minHeap.size() == n) ? sumR : Long.MIN_VALUE / 4;
        }

        long ans = Long.MAX_VALUE;
        for (int i = n - 1; i <= 2 * n - 1; i++) {
            long diff = leftMin[i] - rightMax[i + 1];
            ans = Math.min(ans, diff);
        }
        return ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

642. `[2164]Sort Even and Odd Indices Independently.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] sortEvenOdd(int[] nums) {
        int n = nums.length;
        PriorityQueue<Integer> even = new PriorityQueue<>();
        PriorityQueue<Integer> odd = new PriorityQueue<>((o1, o2) -> o2 - o1);
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            if (i % 2 == 0) {
                even.add(nums[i]);
            } else {
                odd.add(nums[i]);
            }
        }

        for (int i = 0; i < nums.length; i++) {
            if (i % 2 == 0) {
                arr[i] = even.poll();
            } else {
                arr[i] = odd.poll();
            }
        }

        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

643. [2181]Merge Nodes in Between Zeros.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public ListNode mergeNodes(ListNode head) {
        ListNode dummy = new ListNode(0);
        ListNode currSum = dummy;
        ListNode curr = head.next;
        int sum = 0;

        while (curr != null) {
            if (curr.val == 0) {
                currSum.next = new ListNode(sum);
                currSum = currSum.next;
                sum = 0;
            } else {
                sum += curr.val;
            }

            curr = curr.next;
        }

        return dummy.next;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

644. [2185]Counting Words With a Given Prefix.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int prefixCount(String[] words, String pref) {
        int count = 0;

        for (String word : words) {
            if (word.startsWith(pref)) {
                count++;
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

645. `[2187].java`

```
2187
        Minimum Time to Complete Trips
        2023-03-07 09:46:10

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long minimumTime(int[] time, int totalTrips) {
        int min = Arrays.stream(time).min().getAsInt();
        long left = 1;
        long right = (long) min * totalTrips;
        while (left < right) {
            long mid = (left + right) >> 1;
            long count = 0;
            for (int num : time) {
                count += mid / num;
                if (count >= totalTrips) {
                    break;
                }
            }
            if (count >= totalTrips) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

646. `[2187]Minimum Time to Complete Trips.java`

```
2187
        Minimum Time to Complete Trips
        2023-03-07 09:46:10

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long minimumTime(int[] time, int totalTrips) {
        int min = Arrays.stream(time).min().getAsInt();
        long left = 1;
        long right = (long) min * totalTrips;
        while (left < right) {
            long mid = (left + right) >> 1;
            long count = 0;
            for (int num : time) {
                count += mid / num;
                if (count >= totalTrips) {
                    break;
                }
            }
            if (count >= totalTrips) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

647. `[2191]Sort the Jumbled Numbers.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] sortJumbled(int[] mapping, int[] nums) {
        return Arrays.stream(nums)
                .boxed()
                .sorted((a, b) -> Comparator.comparingInt(num -> getMappedValue(num,
mapping)))
                .mapToInt(Integer::intValue)
                .toArray();
    }

    private int getMappedValue(int number, int[] mapping) {
        int mappedValue = 0;
        int temp = number;
        int place = 1;

        if (temp == 0) {
            return mapping[0];
        }

        while (temp != 0) {
            mappedValue = place * mapping[temp % 10] + mappedValue;
            place *= 10;
            temp /= 10;
        }

        return mappedValue;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int[] sortJumbled(int[] mapping, int[] nums) {
        int n = nums.length;
        int[][] mappedPairs = new int[n][2];

        for (int i = 0; i < n; i++) {
            mappedPairs[i][0] = nums[i];
            mappedPairs[i][1] = getMappedValue(nums[i], mapping);
        }

        Arrays.sort(mappedPairs, (a, b) -> Integer.compare(a[1], b[1]));
        int[] sortedNums = new int[n];

        for (int i = 0; i < n; i++) {
            sortedNums[i] = mappedPairs[i][0];
        }

        return sortedNums;
    }
```

```
    private int getMappedValue(int number, int[] mapping) {
        StringBuilder mappedValue = new StringBuilder();
        String numStr = String.valueOf(number);

        for (char digit : numStr.toCharArray()) {
            mappedValue.append(mapping[digit - '0']);
        }

        return Integer.parseInt(mappedValue.toString());
    }
}
 */
```

648. `[2192]All Ancestors of a Node in a Directed`

Acyclic Graph.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> getAncestors(int n, int[][] edges) {
        List<List<Integer>> graph = new ArrayList<>();
        List<Set<Integer>> ancestors = new ArrayList<>();
        int[] inDegree = new int[n];

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
            ancestors.add(new TreeSet<>());
        }

        for (int[] edge : edges) {
            int from = edge[0];
            int to = edge[1];
            graph.get(from).add(to);
            inDegree[to]++;
        }

        Queue<Integer> queue = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            if (inDegree[i] == 0) {
                queue.offer(i);
            }
        }

        while (!queue.isEmpty()) {
            int node = queue.poll();

            for (int neighbor : graph.get(node)) {
                ancestors.get(neighbor).add(node);
                ancestors.get(neighbor).addAll(ancestors.get(node));
                inDegree[neighbor]--;

                if (inDegree[neighbor] == 0) {
                    queue.offer(neighbor);
                }
            }
        }

        List<List<Integer>> result = new ArrayList<>();

        for (Set<Integer> set : ancestors) {
            result.add(new ArrayList<>(set));
        }

        return result;
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

649. [2196]Create Binary Tree From Descriptions.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public TreeNode createBinaryTree(int[][] descriptions) {
        Map<Integer, TreeNode> map = new HashMap<>();
        Set<Integer> children = new HashSet<>();

        for (int[] desc : descriptions) {
            int parentVal = desc[0];
            int childVal = desc[1];
            boolean isLeft = desc[2] == 1;

            TreeNode parent = map.getOrDefault(parentVal, new TreeNode(parentVal));
            TreeNode child = map.getOrDefault(childVal, new TreeNode(childVal));

            if (isLeft) {
                parent.left = child;
            } else {
                parent.right = child;
            }

            map.put(parentVal, parent);
            map.put(childVal, child);
            children.add(childVal);
        }

        TreeNode root = new TreeNode(-1);

        for (int val : map.keySet()) {
            if (!children.contains(val)) {
                root = map.get(val);
                break;
            }
        }

        return root;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

650. [2206]Divide Array Into Equal Pairs.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean divideArray(int[] nums) {
    Map<Integer, Integer> map = new HashMap<>();

    for (int num : nums) {
      map.put(num, map.getOrDefault(num, 0) + 1);
    }

    for (int num : map.values()) {
      if (num % 2 != 0) {
        return false;
      }
    }

    return true;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

651. [2208]Minimum Operations to Halve Array Sum.java

```
2208
        Minimum Operations to Halve Array Sum
        2022-12-09 10:02:01

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int halveArray(int[] nums) {
        double target = 0;
        PriorityQueue<Double> heap = new PriorityQueue<>(Comparator.reverseOrder());
        for (double num : nums) {
            target += num;
            heap.add(num);
        }
        target /= 2;
        int res = 0;
        while (target > 0) {
            res++;
            double x = heap.remove();
            target -= (x / 2);
            heap.add(x / 2);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

652. [2210]Count Hills and Valleys in an Array.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countHillValley(int[] nums) {
        int count = 0;
        int left = nums[0];

        for (int i = 1; i < nums.length - 1; i++) {
            if ((nums[i] > left && nums[i] > nums[i + 1])
                    || (nums[i] < left && nums[i] < nums[i + 1])) {
                count++;
                left = nums[i];
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

653. [2214]Minimum Health to Beat Game.java

```
2214
        Minimum Health to Beat Game
        2023-01-08 23:05:19

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long minimumHealth(int[] damage, int armor) {
        int max = 0;
        long sum = 0;
        for (int num : damage) {
            max = Math.max(max, num);
            sum += (long) num;
        }
        return sum - Math.min(armor, max) + 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

654. `[2215]Find the Difference of Two Arrays.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> findDifference(int[] nums1, int[] nums2) {
        return Arrays.asList(getList(nums2, nums1), getList(nums1, nums2));
    }

    private List<Integer> getList(int[] nums1, int[] nums2) {
        Set<Integer> set1 = new HashSet<>();
        Set<Integer> set2 = new HashSet<>();

        for (int num : nums1) {
            set1.add(num);
        }

        for (int num : nums2) {
            if (!set1.contains(num)) {
                set2.add(num);
            }
        }

        return new ArrayList<>(set2);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

655. [2218]Maximum Value of K Coins From Piles.java

```
2218
        Maximum Value of K Coins From Piles
        2022-12-14 18:18:05

//leetcode submit region begin(Prohibit modification and deletion)
/*
還不會
 */
class Solution {
    public int maxValueOfCoins(List<List<Integer>> piles, int k) {
        int[] dp = new int[k + 1];
        int[] preSum = new int[k + 1];
        for (List<Integer> pile : piles) {
            int n = Math.min(k, pile.size());
            int sum = 0;
            for (int i = 1; i <= n; i++) {
                preSum[i] = sum += pile.get(i - 1);
            }
            for (int i = k; i > 0; i--) {
                int max = 0;
                for (int j = Math.min(i, n); j >= 0; j--) {
                    max = Math.max(max, preSum[j] + dp[i - j]);
                }
                dp[i] = max;
            }
        }
        return dp[k];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

656. [2220]Minimum Bit Flips to Convert Number.java

```

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minBitFlips(int start, int goal) {
        int xorNum = start ^ goal;
        return Integer.bitCount(xorNum);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

657. [2221]Find Triangular Sum of an Array.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int triangularSum(int[] nums) {
        for (int i = nums.length - 1; i > 0; i--) {
            for (int j = 0; j < i; j++) {
                nums[j] = (nums[j] + nums[j + 1]) % 10;
            }
        }

        return nums[0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

658. [2222]Number of Ways to Select Buildings.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long numberOfWays(String s) {
        long count0 = 0;
        long count1 = 0;
        long count01 = 0;
        long count10 = 0;
        long result = 0;

        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);

            if (c == '0') {
                count0++;
                result += count01;
                count10 += count1;
            } else {
                count1++;
                result += count10;
                count01 += count0;
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

659. [2223]Sum of Scores of Built Strings.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long sumScores(String s) {
        char[] c = s.toCharArray();
        int n = c.length;
        int[] arr = new int[n];
        int left = 0;
        int right = 0;
        for (int i = 1; i < n; i++) {
            if (i < right) {
                arr[i] = Math.min(arr[i - left], right - i);
            }
            while (i + arr[i] < n && c[arr[i]] == c[i + arr[i]]) {
                arr[i]++;
            }
            if (i + arr[i] > right) {
                left = i;
                right = i + arr[i];
            }
        }
        long res = n;
        for (int num : arr) {
            res += num;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

660. `[2225]Find Players With Zero or One Losses.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> findWinners(int[][] matches) {
        Map<Integer, Integer> loseCount = new HashMap<>();

        for (int[] match : matches) {
            int winner = match[0];
            int loser = match[1];
            loseCount.put(winner, loseCount.getOrDefault(winner, 0));
            loseCount.put(loser, loseCount.getOrDefault(loser, 0) + 1);
        }

        List<Integer> noLose = new ArrayList<>();
        List<Integer> oneLose = new ArrayList<>();

        for (int player : loseCount.keySet()) {
            if (loseCount.get(player) == 0) {
                noLose.add(player);
            } else if (loseCount.get(player) == 1) {
                oneLose.add(player);
            }
        }

        List<List<Integer>> result = new ArrayList<>();
        Collections.sort(noLose);
        Collections.sort(oneLose);
        result.add(noLose);
        result.add(oneLose);
        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

661. [2231]Largest Number After Digit Swaps by Parity.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int largestInteger(int num) {
        char[] digits = Integer.toString(num).toCharArray();
        PriorityQueue<Integer> even = new PriorityQueue<>((a, b) -> b - a);
        PriorityQueue<Integer> odd = new PriorityQueue<>((a, b) -> b - a);

        for (char c : digits) {
            int n = c - '0';

            if (n % 2 == 0) {
                even.add(n);
            } else {
                odd.add(n);
            }
        }

        for (int i = 0; i < digits.length; i++) {
            if ((digits[i] - '0') % 2 == 0) {
                digits[i] = (char) (even.poll() + '0');
            } else {
                digits[i] = (char) (odd.poll() + '0');
            }
        }

        return Integer.parseInt(new String(digits));
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

662. [2235]Add Two Integers.java

```
2235
        Add Two Integers
        2023-01-13 13:26:51

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int sum(int num1, int num2) {
        return num1 + num2;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

663. [2239]Find Closest Number to Zero.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findClosestNumber(int[] nums) {
        int low = Integer.MAX_VALUE;
        int high = Integer.MAX_VALUE;
        for (int num : nums) {
            if (num == 0) {
                return num;
            }
            if (num > 0) {
                high = Math.min(high, num);
            }
            if (num < 0) {
                low = Math.min(low, 0 - num);
            }
        }
        return high > low ? 0 - low : high;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int findClosestNumber(int[] nums) {
        int index = 0;
        int diff = Integer.MAX_VALUE;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == 0) {
                return nums[i];
            }
            int abs = Math.abs(nums[i]);
            if (abs < diff) {
                index = i;
                diff = abs;
            } else if (abs == diff && (nums[i] > 0 && nums[index] < 0)) {
                index = i;
            }
        }
        return nums[index];
    }
}
 */
```

664. [2244]Minimum Rounds to Complete All Tasks.java

```
2244
        Minimum Rounds to Complete All Tasks
        2023-01-04 09:36:45

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumRounds(int[] tasks) {
        Arrays.sort(tasks);
        int res = 0;
        int left = 0;
        while (left < tasks.length) {
            int right = left + 1;
            while (right < tasks.length && tasks[right] == tasks[left]) {
                right++;
            }
            if (right == left + 1) {
                return -1;
            }
            if ((right - left) % 3 == 0) {
                res += (right - left) / 3;
            } else {
                res += (right - left) / 3 + 1;
            }
            left = right;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int minimumRounds(int[] tasks) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int task : tasks) {
            map.put(task, map.getOrDefault(task, 0) + 1);
        }
        int res = 0;
        for (int count : map.values()) {
            if (count == 1) {
                return -1;
            }
            if (count % 3 == 0) {
                res += count / 3;
            } else {
                res += count / 3 + 1;
            }
        }
        return res;
    }
}
 */
```

665. `[2246]Longest Path With Different Adjacent Characters.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int result = 0;

    public int longestPath(int[] parent, String s) {
        Map<Integer, List<Integer>> children = new HashMap<>();

        for (int i = 0; i < parent.length; i++) {
            children.computeIfAbsent(parent[i], k -> new ArrayList<>()).add(i);
        }

        dfs(0, children, s);
        return result;
    }

    private int dfs(int node, Map<Integer, List<Integer>> children, String s) {
        int max1 = 0;
        int max2 = 0;

        if (children.containsKey(node)) {
            for (int child : children.get(node)) {
                int depth = dfs(child, children, s);

                if (s.charAt(node) != s.charAt(child)) {
                    if (depth > max1) {
                        max2 = max1;
                        max1 = depth;
                    } else if (depth > max2) {
                        max2 = depth;
                    }
                }
            }
        }

        result = Math.max(result, max1 + max2 + 1);
        return Math.max(max1, max2) + 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

666. [2247]Maximum Cost of Trip With K Highways.java

```
2247
Maximum Cost of Trip With K Highways
2022-11-27 16:27:00
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumCost(int n, int[][] highways, int k) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

667. [2248]Intersection of Multiple Arrays.java

```
2248
        Intersection of Multiple Arrays
        2022-11-27 16:28:08

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> intersection(int[][] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        List<Integer> list = new ArrayList<>();
        for (int[] arr : nums) {
            for (int num : arr) {
                map.put(num, map.getOrDefault(num, 0) + 1);
            }
        }
        int n = nums.length;
        for (int key : map.keySet()) {
            if (map.get(key) == n) {
                list.add(key);
            }
        }
        Collections.sort(list);
        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

668. [2251]Number of Flowers in Full Bloom.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] fullBloomFlowers(int[][] flowers, int[] persons) {
        int m = persons.length;
        int n = flowers.length;
        int[] result = new int[m];
        int[] starts = new int[n];
        int[] ends = new int[n];

        for (int i = 0; i < n; i++) {
            starts[i] = flowers[i][0];
            ends[i] = flowers[i][1];
        }

        Arrays.sort(starts);
        Arrays.sort(ends);

        for (int i = 0; i < m; i++) {
            int x = binarySearch(starts, persons[i] + 1);
            int y = binarySearch(ends, persons[i]);
            result[i] = x - y;
        }

        return result;
    }

    private int binarySearch(int[] arr, int target) {
        int res = arr.length;
        int left = 0;
        int right = arr.length - 1;

        while (left <= right) {
            int mid = (left + right) >> 1;

            if (arr[mid] >= target) {
                res = mid;
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }

        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

669. `[2256]Minimum Average Difference.java`

```
2256
        Minimum Average Difference
        2022-12-04 16:21:00

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumAverageDifference(int[] nums) {
        int n = nums.length;
        int ans = -1;
        int min = Integer.MAX_VALUE;
        long prefixSum = 0;
        long total = 0;
        for (int i = 0; i < n; i++) {
            total += nums[i];
        }
        for (int i = 0; i < n; i++) {
            prefixSum += nums[i];
            long leftPart = prefixSum / (i + 1);
            long rightPart = total - prefixSum;
            if (i != n - 1) {
                rightPart /= (n - (i + 1));
            }
            int currDiff = (int) Math.abs(leftPart - rightPart);
            if (currDiff < min) {
                min = currDiff;
                ans = i;
            }
        }
        return ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

670. [2257]Count Unguarded Cells in the Grid.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countUnguarded(int m, int n, int[][] guards, int[][] walls) {
        int[][] grid = new int[m][n];

        for (int[] guard : guards) {
            grid[guard[0]][guard[1]] = 1;
        }

        for (int[] wall : walls) {
            grid[wall[0]][wall[1]] = 1;
        }

        int guardedCount = 0;
        int[][] directions = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

        for (int[] guard : guards) {
            int r = guard[0];
            int c = guard[1];

            for (int[] dir : directions) {
                int x = r + dir[0];
                int y = c + dir[1];

                while (x >= 0 && x < m && y >= 0 && y < n && grid[x][y] != 1) {
                    if (grid[x][y] == 0) {
                        guardedCount++;
                        grid[x][y] = -1;
                    }

                    x += dir[0];
                    y += dir[1];
                }
            }
        }

        return m * n - guards.length - walls.length - guardedCount;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

671. [2260]Minimum Consecutive Cards to Pick Up.java

```
2260
        Minimum Consecutive Cards to Pick Up
        2022-11-28 13:49:46

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumCardPickup(int[] cards) {
        Map<Integer, Integer> map = new HashMap<>();
        int ans = Integer.MAX_VALUE;
        for (int i = 0; i < cards.length; i++) {
            int num = cards[i];
            if (map.containsKey(num))
                ans = Math.min(ans, i - map.get(num) + 1);
            map.put(num, i);
        }
        return ans == Integer.MAX_VALUE ? -1 : ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

672. [2264]Largest 3-Same-Digit Number in String.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String largestGoodInteger(String num) {
        int max = -1;
        StringBuilder sb = new StringBuilder();
        char[] charsNum = num.toCharArray();

        for (int i = 0; i < num.length() - 2; i++) {
            if (charsNum[i] == charsNum[i + 1] && charsNum[i + 1] == charsNum[i + 2]) {
                max = Math.max(max, charsNum[i] - '0');
            }
        }

        if (max != -1) {
            for (int i = 0; i < 3; i++) {
                sb.append(String.valueOf(max));
            }
        }

        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public String largestGoodInteger(String num) {
        int max = Integer.MIN_VALUE;
        StringBuilder sb = new StringBuilder();
        char[] charsNum = num.toCharArray();

        for (int i = 0; i < num.length() - 2; i++) {
            if (charsNum[i] == charsNum[i + 1] && charsNum[i + 1] == charsNum[i + 2]) {
                int sum = Integer.valueOf(charsNum[i] * 100 + charsNum[i + 1] * 10 +
charsNum[i + 2]);

                if (sum > max) {
                    sb.setLength(0);
                    max = sum;
                    sb.append(charsNum[i + 2]).append(charsNum[i +
1]).append(charsNum[i]);
                }
            }
        }

        return sb.toString();
    }
}
 */
```

673. [2265]Count Nodes Equal to Average of Subtree.java

```
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 * int val;
 * TreeNode left;
 * TreeNode right;
 * TreeNode() {}
 * TreeNode(int val) { this.val = val; }
 * TreeNode(int val, TreeNode left, TreeNode right) {
 * this.val = val;
 * this.left = left;
 * this.right = right;
 * }
 * }
 */
class Solution {
    int result = 0;

    public int averageOfSubtree(TreeNode root) {
        postOrder(root);
        return result;
    }

    private int[] postOrder(TreeNode root) {
        if (root == null) {
            return new int[2];
        }

        int[] left = postOrder(root.left);
        int[] right = postOrder(root.right);

        int count = left[0] + right[0] + 1;
        int sum = left[1] + right[1] + root.val;

        if (root.val == sum / count) {
            result++;
        }

        return new int[]{count, sum};
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

674. `[2272].java`

```
2272
        Substring With Largest Variance
        2023-03-06 18:11:20

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int largestVariance(String s) {
        HashMap<Character, List<Integer>> map = new HashMap<>();
        for (int i = 0; i < s.length(); i++) {
            char cur = s.charAt(i);
            map.putIfAbsent(cur, new ArrayList<>());
            map.get(cur).add(i);
        }
        int max = 0;
        for (char c1 : map.keySet()) {
            for (char c2 : map.keySet()) {
                if (c1 != c2) {
                    int i = 0;
                    int j = 0;
                    int f = 0;
                    int g = Integer.MIN_VALUE;
                    List<Integer> map1 = map.get(c1);
                    List<Integer> map2 = map.get(c2);
                    while (i < map1.size() || j < map2.size()) {
                        if (j == map2.size() || (i < map1.size() && map1.get(i) <
map2.get(j))) {
                            f = Math.max(f, 0) + 1;
                            g++;
                            i++;
                        } else {
                            g = Math.max(f, Math.max(g, 0)) - 1;
                            f = Math.max(f, 0) - 1;
                            j++;
                        }
                        max = Math.max(max, g);
                    }
                }
            }
        }
        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

675. `[2272]Substring With Largest Variance.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int largestVariance(String s) {
        int res = 0;
        int[][] diffA = new int[26][26];
        int[][] diffB = new int[26][26];
        char[] c = s.toCharArray();
        for (int[] arr : diffB) {
            Arrays.fill(arr, -c.length);
        }
        for (int i = 0; i < c.length; i++) {
            int ch = c[i] - 'a';
            for (int j = 0; j < 26; j++) {
                if (j == ch) {
                    continue;
                }
                diffA[ch][j]++;
                diffB[ch][j]++;
                diffA[j][ch]--;
                diffB[j][ch] = diffA[j][ch];
                diffA[j][ch] = Math.max(diffA[j][ch], 0);
                res = Math.max(res, Math.max(diffB[ch][j], diffB[j][ch]));
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

676. [2279]Maximum Bags With Full Capacity of Rocks.java

```
2279
        Maximum Bags With Full Capacity of Rocks
        2022-12-27 09:38:14

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumBags(int[] capacity, int[] rocks, int additionalRocks) {
        int n = rocks.length;
        for (int i = 0; i < n; i++) {
            capacity[i] -= rocks[i];
        }
        Arrays.sort(capacity);
        int res = 0;
        while (res < n) {
            if (additionalRocks >= capacity[res]) {
                additionalRocks -= capacity[res];
                res++;
            } else {
                break;
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

677. `[2281].java`

```
2281
        Sum of Total Strength of Wizards
        2023-03-03 16:28:23

//leetcode submit region begin(Prohibit modification and deletion)
/*
Amazon 近半年很常考

Key words
Subarray + sum -> prefix sum
Subarray + minimum -> mono stack

 */
class Solution {
    public int totalStrength(int[] strength) {
        final int MOD = (int) 1e9 + 7;
        int n = strength.length;
        Deque<Integer> dq = new ArrayDeque<>();
        dq.push(-1);
        int[] leftIndex = new int[n];
        int[] rightIndex = new int[n];
        Arrays.fill(rightIndex, n);

        for (int i = 0; i < n; i++) {
            while (dq.size() > 1 && strength[dq.peek()] >= strength[i]) {
                rightIndex[dq.pop()] = i;
            }
            leftIndex[i] = dq.peek();
            dq.push(i);
        }
        long sum = 0;
        long[] preSum = new long[n + 2];
        for (int i = 1; i <= n; i++) {
            sum += strength[i - 1];
            preSum[i + 1] = (int) ((preSum[i] + sum) % MOD);
        }
        long res = 0;
        for (int i = 0; i < n; i++) {
            int leftBound = leftIndex[i] + 1;
            int rightBound = rightIndex[i] - 1;
            long ledtCount = i - leftBound + 1;
            long rightCount = rightBound - i + 1;
            long total = (ledtCount * (preSum[rightBound + 2] - preSum[i + 1]) -
(rightCount) * (preSum[i + 1] - preSum[leftBound])) % MOD;
            res = (res + strength[i] * total) % MOD;
        }
        return (int) (res + MOD) % MOD;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

678. `[2285]Maximum Total Importance of Roads.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long maximumImportance(int n, int[][] roads) {
        long[] degree = new long[n];

        for (int[] road : roads) {
            degree[road[0]]++;
            degree[road[1]]++;
        }

        Arrays.sort(degree);
        long value = 1;
        long result = 0;

        for (long num : degree) {
            result += (value * num);
            value++;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

679. [2287]Rearrange Characters to Make Target String.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int rearrangeCharacters(String s, String target) {
        int[] countS = new int[26];
        int[] countT = new int[26];
        int count = Integer.MAX_VALUE;

        for (char c : s.toCharArray()) {
            countS[c - 'a']++;
        }

        for (char c : target.toCharArray()) {
            countT[c - 'a']++;
        }

        for (int i = 0; i < 26; i++) {
            if (countT[i] > 0) {
                count = Math.min(count, countS[i] / countT[i]);
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

680. `[2290]Minimum Obstacle Removal to Reach`

Corner.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumObstacles(int[][] grid) {
        int m = grid.length;
        if (m == 0) return -1;
        int n = grid[0].length;
        if (n == 0) return -1;

        int[][] directions = {
                {-1, 0},
                {1, 0},
                {0, -1},
                {0, 1}
        };

        int[][] dist = new int[m][n];

        for (int[] row : dist) {
            java.util.Arrays.fill(row, Integer.MAX_VALUE);
        }
        dist[0][0] = 0;

        Deque<int[]> deque = new LinkedList<>();
        deque.offerFirst(new int[]{0, 0});

        while (!deque.isEmpty()) {
            int[] current = deque.pollFirst();
            int x = current[0];
            int y = current[1];

            if (x == m - 1 && y == n - 1) {
                return dist[x][y];
            }

            for (int[] dir : directions) {
                int newX = x + dir[0];
                int newY = y + dir[1];

                if (newX >= 0 && newX < m && newY >= 0 && newY < n) {
                    int newCost = dist[x][y] + grid[newX][newY];

                    if (newCost < dist[newX][newY]) {
                        dist[newX][newY] = newCost;
                        if (grid[newX][newY] == 0) {
                            deque.offerFirst(new int[]{newX, newY});
                        } else {
                            deque.offerLast(new int[]{newX, newY});
                        }
                    }
                }
```

```
            }
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

681. [2294]Partition Array Such That Maximum Difference Is K.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int partitionArray(int[] nums, int k) {
        Arrays.sort(nums);
        int res = 1;
        int x = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] - x > k) {
                x = nums[i];
                res++;
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

682. `[2297]Jump Game VIII.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long minCost(int[] nums, int[] costs) {
        int n = nums.length;
        Deque<Integer> one = new ArrayDeque<>();
        Deque<Integer> two = new ArrayDeque<>();
        long[] dp = new long[nums.length];
        Arrays.fill(dp, Long.MAX_VALUE);
        dp[0] = 0;

        for (int i = 0; i < n; i++) {
            while (!one.isEmpty() && nums[i] >= nums[one.peek()]) {
                dp[i] = Math.min(dp[i], dp[one.pop()] + costs[i]);
            }

            while (!two.isEmpty() && nums[i] < nums[two.peek()]) {
                dp[i] = Math.min(dp[i], dp[two.pop()] + costs[i]);
            }

            one.push(i);
            two.push(i);
        }

        return dp[n - 1];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

683. [2300]Successful Pairs of Spells and Potions.java

```
2300
        Successful Pairs of Spells and Potions
        2022-12-12 00:46:43

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[] potions;

    public int[] successfulPairs(int[] spells, int[] potions, long success) {
        this.potions = potions;
        int m = potions.length;
        int n = spells.length;
        int[] arr = new int[n];
        Arrays.sort(potions);
        for (int i = 0; i < n; i++) {
            double target = success / (double) spells[i];
            int index = binarySearch(target);
            arr[i] = m - index;
        }
        return arr;
    }

    private int binarySearch(double target) {
        int left = 0;
        int right = potions.length - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (potions[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

684. [2302]Count Subarrays With Score Less Than K.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long countSubarrays(int[] nums, long k) {
    long count = 0L;
    long sum = 0L;
    int left = 0;

    for (int right = 0; right < nums.length; right++) {
      sum += nums[right];

      while (left <= right && sum * (right - left + 1L) >= k) {
        sum -= nums[left++];
      }

      count += (right - left + 1);
    }

    return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

685. `[2305]Fair Distribution of Cookies.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int res = Integer.MAX_VALUE;

    public int distributeCookies(int[] cookies, int k) {
        backtrack(cookies, k, new int[k], 0);
        return res;
    }

    private void backtrack(int[] cookies, int target, int[] arr, int index) {
        if (index == cookies.length) {
            int max = 0;
            for (int num : arr) {
                max = Math.max(max, num);
            }
            res = Math.min(res, max);
            return;
        }
        for (int i = 0; i < target; i++) {
            arr[i] += cookies[index];
            backtrack(cookies, target, arr, index + 1);
            arr[i] -= cookies[index];
            if (arr[i] == 0) {
                break;
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

686. `[2306]Naming a Company.java`

```
2306
        Naming a Company
        2023-02-09 09:07:01

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long distinctNames(String[] ideas) {
        Set<String>[] set = new Set[26];
        for (int i = 0; i < 26; i++) {
            set[i] = new HashSet<>();
        }
        for (String idea : ideas) {
            set[idea.charAt(0) - 'a'].add(idea.substring(1));
        }
        int[][] same = new int[26][26];
        for (int i = 0; i < 26; i++) {
            for (String str : set[i]) {
                for (int j = i + 1; j < 26; j++) {
                    if (set[j].contains(str)) {
                        same[i][j]++;
                    }
                }
            }
        }
        long res = 0;
        for (int i = 0; i < 26; i++) {
            for (int j = i + 1; j < 26; j++) {
                res += (set[i].size() - same[i][j]) * (set[j].size() - same[i][j]) * 2;
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

687. [2311]Longest Binary Subsequence Less Than or Equal to K.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestSubsequence(String s, int k) {
        long value = 0;
        int bitsSelected = 0;
        int length = 0;

        for (int i = s.length() - 1; i >= 0; i--) {
            char ch = s.charAt(i);

            if (ch == '0') {
                length++;
                bitsSelected++;
            } else {
                if (bitsSelected < 31) {
                    long add = 1L << bitsSelected;
                    if (value + add <= k) {
                        value += add;
                        length++;
                        bitsSelected++;
                    }
                }
            }
        }
        return length;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

688. [2316]Count Unreachable Pairs of Nodes in an

Undirected Graph.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class UnionFind {
    int[] parent;
    int[] rank;
    public UnionFind(int size) {
        parent = new int[size];
        for (int i = 0; i < size; i++)
            parent[i] = i;
        rank = new int[size];
    }
    public int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }
    public void union_set(int x, int y) {
        int xset = find(x), yset = find(y);
        if (xset == yset) {
            return;
        } else if (rank[xset] < rank[yset]) {
            parent[xset] = yset;
        } else if (rank[xset] > rank[yset]) {
            parent[yset] = xset;
        } else {
            parent[yset] = xset;
            rank[xset]++;
        }
    }
}

class Solution {
    public long countPairs(int n, int[][] edges) {
        UnionFind dsu = new UnionFind(n);
        for(int[] edge: edges) {
            dsu.union_set(edge[0], edge[1]);
        }

        Map<Integer, Integer> componentSize = new HashMap<>();
        for(int i= 0; i<n; i++) {
            int parent = dsu.find(i);
            if(componentSize.containsKey(parent)) {
                componentSize.put(parent, componentSize.get(parent) + 1);
            } else {
                componentSize.put(parent, 1);
            }
        }

        long numberOfPaths = 0;
        long remainingNodes = n;
        for (int component : componentSize.values()) {
```

```
            numberOfPaths += component * (remainingNodes - component);
            remainingNodes -= component;
        }
        return numberOfPaths;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

689. [2322]Minimum Score After Removals on a Tree.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private List<Integer>[] graph;
    private int[] nums;
    private int[] subXor;
    private int[] tin;
    private int[] tout;
    private int timer;
    private int[] parent;

    public int minimumScore(int[] nums, int[][] edges) {
        int n = nums.length;
        this.nums = nums;
        graph = new ArrayList[n];
        for (int i = 0; i < n; i++) {
            graph[i] = new ArrayList<>();
        }
        for (int[] e : edges) {
            graph[e[0]].add(e[1]);
            graph[e[1]].add(e[0]);
        }

        subXor = new int[n];
        tin = new int[n];
        tout = new int[n];
        parent = new int[n];
        parent[0] = -1;
        timer = 0;

        dfs(0, -1);

        int totalXor = subXor[0];

        List<Integer> cutChild = new ArrayList<>();
        for (int[] e : edges) {
            int u = e[0], v = e[1];
            if (parent[u] == v) {
                cutChild.add(u);
            } else {
                cutChild.add(v);
            }
        }

        int ans = Integer.MAX_VALUE;
        int m = cutChild.size();

        for (int i = 0; i < m; i++) {
            int x = cutChild.get(i);
            for (int j = i + 1; j < m; j++) {
                int y = cutChild.get(j);
```

```
                int a, b, c;
                if (isAncestor(x, y)) {
                    a = subXor[y];
                    b = subXor[x] ^ subXor[y];
                    c = totalXor ^ subXor[x];
                } else if (isAncestor(y, x)) {
                    a = subXor[x];
                    b = subXor[y] ^ subXor[x];
                    c = totalXor ^ subXor[y];
                } else {
                    a = subXor[x];
                    b = subXor[y];
                    c = totalXor ^ subXor[x] ^ subXor[y];
                }

                int max = Math.max(a, Math.max(b, c));
                int min = Math.min(a, Math.min(b, c));
                ans = Math.min(ans, max - min);
            }
        }

        return ans;
    }

    private void dfs(int u, int parent) {
        this.parent[u] = parent;
        tin[u] = ++timer;
        int xor = nums[u];
        for (int v : graph[u]) {
            if (v == parent) {
                continue;
            }
            dfs(v, u);
            xor ^= subXor[v];
        }
        subXor[u] = xor;
        tout[u] = ++timer;
    }

    private boolean isAncestor(int x, int y) {
        return tin[x] <= tin[y] && tout[y] <= tout[x];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

690. [2326]Spiral Matrix IV.java

```
/**
<p>You are given two integers <code>m</code> and <code>n</code>, which represent the
dimensions of a matrix.</p>

<p>You are also given the <code>head</code> of a linked list of integers.</p>

<p>Generate an <code>m x n</code> matrix that contains the integers in the linked list
presented in <strong>spiral</strong> order <strong>(clockwise)</strong>, starting from
the <strong>top-left</strong> of the matrix. If there are remaining empty spaces, fill
them with <code>-1</code>.</p>

<p>Return <em>the generated matrix</em>.</p>

<p> </p>
<p><strong class="example">Example 1:</strong></p>
<img alt="" src="https://assets.leetcode.com/uploads/2022/05/09/ex1new.jpg" style="width:
240px; height: 150px;" />
<pre>
<strong>Input:</strong> m = 3, n = 5, head = [3,0,2,6,8,1,7,9,4,2,5,5,0]
<strong>Output:</strong> [[3,0,2,6,8],[5,0,-1,-1,1],[5,2,4,9,7]]
<strong>Explanation:</strong> The diagram above shows how the values are printed in the
matrix.
Note that the remaining spaces in the matrix are filled with -1.
</pre>

<p><strong class="example">Example 2:</strong></p>
<img alt="" src="https://assets.leetcode.com/uploads/2022/05/11/ex2.jpg" style="width:
221px; height: 60px;" />
<pre>
<strong>Input:</strong> m = 1, n = 4, head = [0,1,2]
<strong>Output:</strong> [[0,1,2,-1]]
<strong>Explanation:</strong> The diagram above shows how the values are printed from
left to right in the matrix.
The last space in the matrix is set to -1.</pre>

<p> </p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>1 &lt;= m, n &lt;= 10<sup>5</sup></code></li>
 <li><code>1 &lt;= m * n &lt;= 10<sup>5</sup></code></li>
 <li>The number of nodes in the list is in the range <code>[1, m * n]</code>.</li>
 <li><code>0 &lt;= Node.val &lt;= 1000</code></li>
</ul>

<div><div>Related Topics</div><div><li>Array</li><li>Linked List</li><li>Matrix</li>
<li>Simulation</li></div></div><br><div><li>👍 471</li><li>👎 19</li></div>
*/

//leetcode submit region begin(Prohibit modification and deletion)
/**
```

```
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public int[][] spiralMatrix(int m, int n, ListNode head) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

691. [2328]Number of Increasing Paths in a Grid.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[][] dp;
    int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    final int MOD = (int) 1e9 + 7;
    int m;
    int n;

    public int countPaths(int[][] grid) {
        m = grid.length;
        n = grid[0].length;
        dp = new int[m][n];
        int res = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                res = (res + dyP(grid, i, j)) % MOD;
            }
        }
        return res;
    }

    private int dyP(int[][] grid, int i, int j) {
        if (dp[i][j] != 0) {
            return dp[i][j];
        }
        int res = 1;
        for (int[] dir : dirs) {
            int x = i + dir[0];
            int y = j + dir[1];
            if (x >= 0 && x < m && y >= 0 && y < n && grid[x][y] < grid[i][j]) {
                res = (res + dyP(grid, x, y)) % MOD;
            }
        }
        dp[i][j] = res;
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

692. `[2330]Valid Palindrome IV.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean makePalindrome(String s) {
        int count = 0;
        int left = 0;
        int right = s.length() - 1;

        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) {
                count++;

                if (count > 2) {
                    return false;
                }
            }

            left++;
            right--;
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

693. `[2331]Evaluate Boolean Binary Tree.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean evaluateTree(TreeNode root) {
        if (root.left == null && root.right == null) {
            return root.val == 1;
        }

        boolean left = evaluateTree(root.left);
        boolean right = evaluateTree(root.right);

        if (root.val == 2) {
            return left || right;
        } else {
            return left && right;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

694. [2336]Smallest Number in Infinite Set.java

```
/**
 * <p>You have a set which contains all positive integers <code>[1, 2, 3, 4, 5, ...]
</code>.</p>
 *
 * <p>Implement the <code>SmallestInfiniteSet</code> class:</p>
 *
 * <ul>
 * <li><code>SmallestInfiniteSet()</code> Initializes the
<strong>SmallestInfiniteSet</strong> object to contain <strong>all</strong> positive
integers.</li>
 * <li><code>int popSmallest()</code> <strong>Removes</strong> and returns the smallest
integer contained in the infinite set.</li>
 * <li><code>void addBack(int num)</code> <strong>Adds</strong> a positive integer
<code>num</code> back into the infinite set, if it is <strong>not</strong> already in the
infinite set.</li>
 * </ul>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input</strong>
 * ["SmallestInfiniteSet", "addBack", "popSmallest", "popSmallest", "popSmallest",
"addBack", "popSmallest", "popSmallest", "popSmallest"]
 * [[], [2], [], [], [], [1], [], [], []]
 * <strong>Output</strong>
 * [null, null, 1, 2, 3, null, 1, 4, 5]
 *
 * <strong>Explanation</strong>
 * SmallestInfiniteSet smallestInfiniteSet = new SmallestInfiniteSet();
 * smallestInfiniteSet.addBack(2);    // 2 is already in the set, so no change is made.
 * smallestInfiniteSet.popSmallest(); // return 1, since 1 is the smallest number, and
remove it from the set.
 * smallestInfiniteSet.popSmallest(); // return 2, and remove it from the set.
 * smallestInfiniteSet.popSmallest(); // return 3, and remove it from the set.
 * smallestInfiniteSet.addBack(1);    // 1 is added back to the set.
 * smallestInfiniteSet.popSmallest(); // return 1, since 1 was added back to the set and
 * // is the smallest number, and remove it from the set.
 * smallestInfiniteSet.popSmallest(); // return 4, and remove it from the set.
 * smallestInfiniteSet.popSmallest(); // return 5, and remove it from the set.
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>1 &lt;= num &lt;= 1000</code></li>
 * <li>At most <code>1000</code> calls will be made <strong>in total</strong> to
<code>popSmallest</code> and <code>addBack</code>.</li>
 * </ul>
 *
```

```
 * <div><div>Related Topics</div><div><li>Hash Table</li><li>Design</li><li>Heap
(Priority Queue)</li></div></div><br><div><li>👍 435</li><li>👎 38</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)
class SmallestInfiniteSet {
    TreeSet<Integer> set;
    int index;

    public SmallestInfiniteSet() {
        set = new TreeSet<>();
        index = 1;
    }

    public int popSmallest() {
        int res = 0;
        if (!set.isEmpty()) {
            res = set.first();
            set.remove(res);
        } else {
            res = index;
            index++;
        }
        return res;
    }

    public void addBack(int num) {
        if (index <= num || set.contains(num)) {
            return;
        }
        set.add(num);
    }
}

/**
 * Your SmallestInfiniteSet object will be instantiated and called as such:
 * SmallestInfiniteSet obj = new SmallestInfiniteSet();
 * int param_1 = obj.popSmallest();
 * obj.addBack(num);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

695. `[2337]Move Pieces to Obtain a String.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canChange(String start, String target) {
        int n = start.length();
        int i = 0;
        int j = 0;

        while (i < n || j < n) {
            while (i < n && start.charAt(i) == '_') {
                i++;
            }

            while (j < n && target.charAt(j) == '_') {
                j++;
            }

            if (i == n && j == n) {
                return true;
            }

            if ((i == n) != (j == n)) {
                return false;
            }

            char s = start.charAt(i);
            char t = target.charAt(j);

            if (s != t) {
                return false;
            }

            if (s == 'L') {
                if (i < j) {
                    return false;
                }
            } else {
                if (i > j) {
                    return false;
                }
            }

            i++;
            j++;
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

696. [2342]Max Sum of a Pair With Equal Sum of Digits.java

```
2342
        Max Sum of a Pair With Equal Sum of Digits
        2022-11-28 14:16:35

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumSum(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        int ans = -1;
        for (int num : nums) {
            int digitSum = getDigitSum(num);
            if (map.containsKey(digitSum))
                ans = Math.max(ans, num + map.get(digitSum));
            map.put(digitSum, Math.max(map.getOrDefault(digitSum, 0), num));
        }
        return ans;
    }

    private int getDigitSum(int num) {
        int digitSum = 0;
        while (num > 0) {
            digitSum += num % 10;
            num /= 10;
        }
        return digitSum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

697. [2343]Query Kth Smallest Trimmed Number.java

```
2343
        Query Kth Smallest Trimmed Number
        2022-12-17 12:47:59

//leetcode submit region begin(Prohibit modification and deletion)
class Pair {
    int index;
    String value;

    public Pair(int index, String value) {
        this.index = index;
        this.value = value;
    }
}

class Solution {
    public int[] smallestTrimmedNumbers(String[] nums, int[][] queries) {
        Pair[] copy = new Pair[nums.length];
        for (int i = 0; i < nums.length; i++) {
            copy[i] = new Pair(i, nums[i]);
        }
        int d = nums[0].length();
        int[][] dp = new int[d][nums.length];
        int digitcount = 0;
        while (digitcount < d) {
            CountSort(copy, d - digitcount - 1);
            for (int i = 0; i < copy.length; i++)
                dp[digitcount][i] = copy[i].index;
            digitcount++;
        }
        int[] result = new int[queries.length];
        for (int i = 0; i < queries.length; i++) {
            result[i] = dp[queries[i][1] - 1][queries[i][0] - 1];
        }
        return result;
    }


    //frequency
    void CountSort(Pair[] nums, int digit) {

        int[] count = new int[10];
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            int value = nums[i].value.charAt(digit) - 48;
            count[value]++;
        }
        for (int i = 1; i <= 9; i++) {
            count[i] += count[i - 1];
        }
        Pair[] temp = new Pair[n];
```

```
        for (int i = n - 1; i >= 0; i--) {
            int value = nums[i].value.charAt(digit) - 48;
            temp[count[value] - 1] = nums[i];
            count[value]--;
        }
        for (int i = 0; i < n; i++) {
            nums[i] = temp[i];
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

698. [2348]Number of Zero-Filled Subarrays.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long zeroFilledSubarray(int[] nums) {
        long count = 0;
        long sum = 0;

        for (int num : nums) {
            if (num == 0) {
                count++;
            } else {
                sum += countZeros(count);
                count = 0;
            }
        }
        return sum += countZeros(count);
    }

    private long countZeros(long n) {
        return n * (n + 1) / 2;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

699. `[2349]Design a Number Container System.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class NumberContainers {
  private Map<Integer, Integer> indexToNumber;
  private Map<Integer, TreeSet<Integer>> numberToIndices;

  public NumberContainers() {
    indexToNumber = new HashMap<>();
    numberToIndices = new HashMap<>();
  }

  public void change(int index, int number) {
    if (indexToNumber.containsKey(index)) {
      int oldNumber = indexToNumber.get(index);

      if (oldNumber == number) {
        return;
      }

      TreeSet<Integer> oldSet = numberToIndices.get(oldNumber);
      oldSet.remove(index);

      if (oldSet.isEmpty()) {
        numberToIndices.remove(oldNumber);
      }
    }

    indexToNumber.put(index, number);
    numberToIndices.putIfAbsent(number, new TreeSet<>());
    numberToIndices.get(number).add(index);
  }

  public int find(int number) {
    if (!numberToIndices.containsKey(number) || numberToIndices.get(number).isEmpty()) {
      return -1;
    }

    return numberToIndices.get(number).first();
  }
}

/**
 * Your NumberContainers object will be instantiated and called as such: NumberContainers
obj = new
 * NumberContainers(); obj.change(index,number); int param_2 = obj.find(number);
 */
// leetcode submit region end(Prohibit modification and deletion)
```

700. `[2351]First Letter to Appear Twice.java`

```
2351
        First Letter to Appear Twice
        2022-11-26 00:22:07

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public char repeatedCharacter(String s) {
        Set<Character> set = new HashSet<>();
        char[] ch = s.toCharArray();
        for (int i = 0; i < ch.length; i++) {
            if (set.contains(ch[i]))
                return ch[i];
            set.add(ch[i]);
        }
        return ' ';
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

701. `[2352]Equal Row and Column Pairs.java`

```
2352
        Equal Row and Column Pairs
        2022-11-28 16:22:06

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int equalPairs(int[][] grid) {
        int n = grid.length;
        int[][] arr = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                arr[i][j] = grid[j][i];
            }
        }
        int count = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (Arrays.equals(arr[i], grid[j])) {
                    count++;
                }
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
HashMap: - 50ms
class Solution {
    public int equalPairs(int[][] grid) {
        Map<String, Integer> mapRow = new HashMap<>();
        for (int[] row : grid) {
            String key = convertToKey(row);
            mapRow.put(key, mapRow.getOrDefault(key, 0) + 1);
        }
        Map<String, Integer> mapCol = new HashMap<>();
        for (int col = 0; col < grid[0].length; col++) {
            int[] currCol = new int[grid.length];
            for (int row = 0; row < grid.length; row++) {
                currCol[row] = grid[row][col];
            }
            String key = convertToKey(currCol);
            mapCol.put(key, mapCol.getOrDefault(key, 0) + 1);
        }
        int ans = 0;
        for (String key : mapRow.keySet()) {
            ans += mapRow.get(key) * mapCol.getOrDefault(key, 0);
        }
        return ans;
    }
```

```
    private String convertToKey(int[] nums) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < nums.length; i++) {
            sb.append(nums[i]);
            sb.append(",");
        }
        return sb.toString();
    }
}
 */
```

702. [2353]Design a Food Rating System.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class FoodRatings {
    class Food {
        String name;
        String cuisine;
        int rating;

        public Food(String name, String cuisine, int rating) {
            this.name = name;
            this.cuisine = cuisine;
            this.rating = rating;
        }
    }

    Map<String, PriorityQueue<Food>> pqMap;
    Map<String, Food> map;

    public FoodRatings(String[] foods, String[] cuisines, int[] ratings) {
        pqMap = new HashMap<>();
        map = new HashMap<>();

        for (int i = 0; i < foods.length; i++) {
            String name = foods[i];
            String cuisine = cuisines[i];
            int rating = ratings[i];

            if (!pqMap.containsKey(cuisine)) {
                pqMap.put(cuisine, createFoodPriorityQueue());
            }

            Food food = new Food(name, cuisine, rating);
            pqMap.get(cuisine).offer(food);
            map.put(name, food);
        }
    }


    public void changeRating(String food, int newRating) {
        Food curr = map.get(food);
        Food updated = new Food(food, curr.cuisine, newRating);
        pqMap.get(curr.cuisine).remove(curr);
        map.put(food, updated);
        pqMap.get(updated.cuisine).offer(updated);
    }

    public String highestRated(String cuisine) {
        PriorityQueue<Food> pq = pqMap.get(cuisine);
        return pq.isEmpty() ? "" : pq.peek().name;
    }
```

```
    private PriorityQueue<Food> createFoodPriorityQueue() {
        return new PriorityQueue<>((a, b) -> a.rating == b.rating ?
a.name.compareTo(b.name) : b.rating - a.rating);
    }
}

/**
 * Your FoodRatings object will be instantiated and called as such:
 * FoodRatings obj = new FoodRatings(foods, cuisines, ratings);
 * obj.changeRating(food,newRating);
 * String param_2 = obj.highestRated(cuisine);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

703. `[2355]Maximum Number of Books You Can Take.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long maximumBooks(int[] books) {
        int n = books.length;
        Deque<Integer> dq = new ArrayDeque<>();
        long[] dp = new long[n];
        long max = Long.MIN_VALUE;

        for (int i = 0; i < n; i++) {
            while (!dq.isEmpty() && books[dq.peek()] - dq.peek() >= books[i] - i) {
                dq.pop();
            }

            if (dq.isEmpty()) {
                dp[i] = getSum(books, 0, i);
            } else {
                int j = dq.peek();
                dp[i] = dp[j] + getSum(books, j + 1, i);
            }

            dq.push(i);
        }

        for (long num : dp) {
            max = Math.max(max, num);
        }
        return max;
    }

    private long getSum(int[] books, int left, int right) {
        long count = Math.min(books[right], right - left + 1);
        return (2 * books[right] - (count - 1)) * count / 2;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

704. [2357]Make Array Zero by Subtracting Equal Amounts.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumOperations(int[] nums) {
        int[] arr = new int[101];
        int count = 0;
        for (int num : nums) {
            if (num > 0 && arr[num] == 0) {
                arr[num]++;
                count++;
            }
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

705. [2359]Find Closest Node to Given Two Nodes.java

```
// leetcode submit region begin(Prohibit modification and deletion)

class Solution {
  public int closestMeetingNode(int[] edges, int node1, int node2) {
    int n = edges.length;
    int[] dist1 = computeDistances(edges, node1, n);
    int[] dist2 = computeDistances(edges, node2, n);
    int answer = -1;
    int bestMaxDist = Integer.MAX_VALUE;

    for (int i = 0; i < n; i++) {
      int d1 = dist1[i];
      int d2 = dist2[i];

      if (d1 >= 0 && d2 >= 0) {
        int maxDist = Math.max(d1, d2);

        if (maxDist < bestMaxDist) {
          bestMaxDist = maxDist;
          answer = i;
        }
      }
    }

    return answer;
  }

  private int[] computeDistances(int[] edges, int start, int n) {
    int[] dist = new int[n];
    Arrays.fill(dist, -1);
    int distance = 0;
    int node = start;

    while (node != -1 && dist[node] == -1) {
      dist[node] = distance++;
      node = edges[node];
    }

    return dist;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

706. `[2360]Longest Cycle in a Graph.java`

```
/**
<p>You are given a <strong>directed</strong> graph of <code>n</code> nodes numbered from
<code>0</code> to <code>n - 1</code>, where each node has <strong>at most one</strong>
outgoing edge.</p>

<p>The graph is represented with a given <strong>0-indexed</strong> array
<code>edges</code> of size <code>n</code>, indicating that there is a directed edge from
node <code>i</code> to node <code>edges[i]</code>. If there is no outgoing edge from node
<code>i</code>, then <code>edges[i] == -1</code>.</p>

<p>Return <em>the length of the <strong>longest</strong> cycle in the graph</em>. If no
cycle exists, return <code>-1</code>.</p>

<p>A cycle is a path that starts and ends at the <strong>same</strong> node.</p>

<p> </p>
<p><strong class="example">Example 1:</strong></p>
<img alt="" src="https://assets.leetcode.com/uploads/2022/06/08/graph4drawio-5.png"
style="width: 335px; height: 191px;" />
<pre>
<strong>Input:</strong> edges = [3,3,4,2,3]
<strong>Output:</strong> 3
<strong>Explanation:</strong> The longest cycle in the graph is the cycle: 2 -&gt; 4 -
&gt; 3 -&gt; 2.
The length of this cycle is 3, so 3 is returned.
</pre>

<p><strong class="example">Example 2:</strong></p>
<img alt="" src="https://assets.leetcode.com/uploads/2022/06/07/graph4drawio-1.png"
style="width: 171px; height: 161px;" />
<pre>
<strong>Input:</strong> edges = [2,-1,3,1]
<strong>Output:</strong> -1
<strong>Explanation:</strong> There are no cycles in this graph.
</pre>

<p> </p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>n == edges.length</code></li>
 <li><code>2 &lt;= n &lt;= 10<sup>5</sup></code></li>
 <li><code>-1 &lt;= edges[i] &lt; n</code></li>
 <li><code>edges[i] != i</code></li>
</ul>

<div><div>Related Topics</div><div><li>Depth-First Search</li><li>Graph</li>
<li>Topological Sort</li></div></div><br><div><li>👍 1953</li><li>👎 38</li></div>
*/

//leetcode submit region begin(Prohibit modification and deletion)
```

```
class Solution {
    int answer = -1;

    public void dfs(int node, int[] edges, Map<Integer, Integer> dist, boolean[] visit) {
        visit[node] = true;
        int neighbor = edges[node];

        if (neighbor != -1 && !visit[neighbor]) {
            dist.put(neighbor, dist.get(node) + 1);
            dfs(neighbor, edges, dist, visit);
        } else if (neighbor != -1 && dist.containsKey(neighbor)) {
            answer = Math.max(answer, dist.get(node) - dist.get(neighbor) + 1);
        }
    }

    public int longestCycle(int[] edges) {
        int n = edges.length;
        boolean[] visit = new boolean[n];

        for (int i = 0; i < n; i++) {
            if (!visit[i]) {
                Map<Integer, Integer> dist = new HashMap<>();
                dist.put(i, 1);
                dfs(i, edges, dist, visit);
            }
        }
        return answer;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

707. [2361]Minimum Costs Using the Train Line.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long[] minimumCosts(int[] regular, int[] express, int expressCost) {
        int n = regular.length;
        long prevRegLane = 0;
        long prevExpLane = expressCost;
        long[] res = new long[n];
        for (int i = 1; i < n + 1; i++) {
            long regCost = regular[i - 1] + Math.min(prevRegLane, prevExpLane);
            long expCost = express[i - 1] + Math.min(expressCost + prevRegLane,
prevExpLane);
            res[i - 1] = Math.min(regCost, expCost);
            prevRegLane = regCost;
            prevExpLane = expCost;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
Top-Down - 43 ms
class Solution {
    Long[][] dp;

    public long[] minimumCosts(int[] regular, int[] express, int expressCost) {
        int n = regular.length;
        dp = new Long[n][2];
        dyP(regular, express, expressCost, 1, n - 1);
        long[] res = new long[n];
        for (int i = 0; i < n; i++) {
            res[i] = dp[i][1];
        }
        return res;
    }

    private long dyP(int[] regular, int[] express, int expressCost, int lane, int i) {
        if (i < 0) {
            return 0;
        }
        if (dp[i][lane] != null) {
            return dp[i][lane];
        }
        long regularLane = regular[i] + dyP(regular, express, expressCost, 1, i - 1);
        long expressLane = (lane == 1 ? expressCost : 0) + express[i] + dyP(regular,
express, expressCost, 0, i - 1);
        dp[i][lane] = Math.min(regularLane, expressLane);
        return dp[i][lane];
    }
}
 */
```

708. [2362]Generate the Invoice.java

```
2362
Generate the Invoice
2023-01-10 13:13:13
//There is no code of Java type for this problem
```

709. [2364]Count Number of Bad Pairs.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public long countBadPairs(int[] nums) {
    Map<Integer, Integer> map = new HashMap<>();
    int n = nums.length;
    long count = 0;

    for (int i = 0; i < n; i++) {
      int key = i - nums[i];
      int value = map.getOrDefault(key, 0);
      map.put(key, value + 1);
      count += i - value;
    }

    return count;
  }
}
//leetcode submit region end(Prohibit modification and deletion)
```

710. `[2366]Minimum Replacements to Sort the Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long minimumReplacement(int[] nums) {
        long res = 0;
        int n = nums.length;
        int prev = nums[n - 1];
        for (int i = n - 2; i >= 0; i--) {
            int element = nums[i] / prev;
            if (nums[i] % prev != 0) {
                element++;
                prev = nums[i] / element;
            }
            res += element - 1;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

711. [2368]Reachable Nodes With Restrictions.java

```
2368
        Reachable Nodes With Restrictions
        2022-12-07 09:32:54

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int reachableNodes(int n, int[][] edges, int[] restricted) {
        List<Integer>[] graph = new ArrayList[n];
        for (int i = 0; i < n; i++) {
            graph[i] = new ArrayList<>();
        }
        for (int[] edge : edges) {
            graph[edge[0]].add(edge[1]);
            graph[edge[1]].add(edge[0]);
        }
        Queue<Integer> queue = new ArrayDeque<>();
        boolean[] isVisited = new boolean[n];
        queue.offer(0);
        isVisited[0] = true;
        for (int node : restricted) {
            isVisited[node] = true;
        }
        int res = 0;
        while (!queue.isEmpty()) {
            int vertex = queue.poll();
            res++;
            for (int neighbour : graph[vertex]) {
                if (!isVisited[neighbour]) {
                    queue.offer(neighbour);
                    isVisited[neighbour] = true;
                }
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

712. `[2369]Check if There is a Valid Partition For The Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean validPartition(int[] nums) {
        int n = nums.length;
        boolean[] dp = new boolean[n + 1];
        dp[0] = true;
        for (int i = 0; i < n; i++) {
            int index = i + 1;
            if (i > 0 && nums[i] == nums[i - 1]) {
                dp[index] |= dp[index - 2];
            }
            if (i > 1 && nums[i] == nums[i - 1] && nums[i] == nums[i - 2]) {
                dp[index] |= dp[index - 3];
            }
            if (i > 1 && nums[i] == nums[i - 1] + 1 && nums[i] == nums[i - 2] + 2) {
                dp[index] |= dp[index - 3];
            }
        }
        return dp[n];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

713. `[2370]Longest Ideal Subsequence.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestIdealString(String s, int k) {
        int[] dp = new int[128];
        int n = s.length();
        int maxLength = 0;

        for (int i = 0; i < n; i++) {
            char c = s.charAt(i);
            int maxLengthForC = 0;
            int start = Math.max('a', c - k);
            int end = Math.min('z', c + k);

            for (int d = start; d <= end; d++) {
                maxLengthForC = Math.max(maxLengthForC, dp[d]);
            }

            dp[c] = maxLengthForC + 1;
            maxLength = Math.max(maxLength, dp[c]);
        }

        return maxLength;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

714. `[2371]Minimize Maximum Value in a Grid.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[][] minScore(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int[][] result = new int[m][n];
        List<Cell> cells = new ArrayList<>();

        for(int r = 0; r < m; r++) {
            for(int c = 0; c < n; c++) {
                cells.add(new Cell(r, c, grid[r][c]));
            }
        }

        Collections.sort(cells, Comparator.comparingInt(a -> a.value));
        int[] rowMax = new int[m];
        int[] colMax = new int[n];

        for(Cell cell : cells) {
            int newVal = Math.max(rowMax[cell.row], colMax[cell.col]) + 1;
            result[cell.row][cell.col] = newVal;
            rowMax[cell.row] = newVal;
            colMax[cell.col] = newVal;
        }

        return result;
    }

    class Cell {
        int row;
        int col;
        int value;

        Cell(int row, int col, int value){
            this.row = row;
            this.col = col;
            this.value = value;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

715. `[2373]Largest Local Values in a Matrix.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[][] largestLocal(int[][] grid) {
        int n = grid.length;
        int[][] maxLocal = new int[n - 2][n - 2];

        for (int i = 0; i < n - 2; i++) {
            for (int j = 0; j < n - 2; j++) {
                maxLocal[i][j] = getMax(grid, i, j);
            }
        }

        return maxLocal;
    }

    private int getMax(int[][] grid, int x, int y) {
        int maxValue = 0;

        for (int i = x; i < x + 3; i++) {
            for (int j = y; j < y + 3; j++) {
                maxValue = Math.max(maxValue, grid[i][j]);
            }
        }

        return maxValue;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

716. `[2375]Construct Smallest Number From DI String.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public String smallestNumber(String pattern) {
    int n = pattern.length();
    StringBuilder result = new StringBuilder();
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i <= n; i++) {
      stack.push(i + 1);

      if (i == n || pattern.charAt(i) == 'I') {
        while (!stack.isEmpty()) {
          result.append(stack.pop());
        }
      }
    }

    return result.toString();
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

717. [2379]Minimum Recolors to Get K Consecutive Black Blocks.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int minimumRecolors(String blocks, int k) {
    int n = blocks.length();
    int count = 0;

    for (int i = 0; i < k; i++) {
      if (blocks.charAt(i) == 'W') {
        count++;
      }
    }

    int minCount = count;

    for (int i = k; i < n; i++) {
      if (blocks.charAt(i) == 'W') {
        count++;
      }

      if (blocks.charAt(i - k) == 'W') {
        count--;
      }

      minCount = Math.min(minCount, count);
    }

    return minCount;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

718. `[2381]Shifting Letters II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String shiftingLetters(String s, int[][] shifts) {
        int n = s.length();
        int[] shiftDiff = new int[n + 1];

        for (int[] shift : shifts) {
            int start = shift[0];
            int end = shift[1];
            int direction = shift[2];

            if (direction == 1) {
                shiftDiff[start]++;
                shiftDiff[end + 1]--;
            } else {
                shiftDiff[start]--;
                shiftDiff[end + 1]++;
            }
        }

        int[] cumulativeShifts = new int[n];
        int shift = 0;
        for (int i = 0; i < n; i++) {
            shift += shiftDiff[i];
            cumulativeShifts[i] = shift;
        }

        char[] result = new char[n];
        for (int i = 0; i < n; i++) {
            int newChar = ((s.charAt(i) - 'a') + cumulativeShifts[i]) % 26;
            if (newChar < 0) {
                newChar += 26;
            }
            result[i] = (char) ('a' + newChar);
        }

        return new String(result);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

719. [2385]Amount of Time for Binary Tree to Be Infected.java

```
//leetcode submit region begin(Prohibit modification and deletion)
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    private int max = 0;
    public int amountOfTime(TreeNode root, int start) {
        dfs(root, start);
        return max;
    }

    public int dfs(TreeNode node, int start) {
        int depth = 0;

        if (node == null) {
            return depth;
        }

        int leftDepth = dfs(node.left, start);
        int rightDepth = dfs(node.right, start);

        if (node.val == start) {
            max = Math.max(leftDepth, rightDepth);
            depth = -1;
        } else if (leftDepth >= 0 && rightDepth >= 0) {
            depth = Math.max(leftDepth, rightDepth) + 1;
        } else {
            int distance = Math.abs(leftDepth) + Math.abs(rightDepth);
            max = Math.max(max, distance);
            depth = Math.min(leftDepth, rightDepth) - 1;
        }

        return depth;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

720. `[2389]Longest Subsequence With Limited Sum.java`

```
2389
        Longest Subsequence With Limited Sum
        2022-12-12 09:57:39

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] answerQueries(int[] nums, int[] queries) {
        Arrays.sort(nums);
        for (int i = 1; i < nums.length; i++) {
            nums[i] += nums[i - 1];
        }
        int n = queries.length;
        int[] res = new int[n];
        for (int i = 0; i < n; i++) {
            int target = queries[i];
            res[i] = binarySearch(nums, target);
        }
        return res;
    }

    private int binarySearch(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (nums[mid] == target) {
                return mid + 1;
            }
            if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

721. `[2390]Removing Stars From a String.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String removeStars(String s) {
        StringBuilder sb = new StringBuilder();
        int count = 0;

        for (int i = s.length() - 1; i >= 0; i--) {
            char c = s.charAt(i);

            if (c == '*') {
                count++;
            } else if (count != 0) {
                count--;
            } else {
                sb.append(c);
            }
        }

        return sb.reverse().toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

722. `[2391]Minimum Amount of Time to Collect Garbage.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int garbageCollection(String[] garbage, int[] travel) {
        int n = garbage.length;
        Map<Character, Integer> map = new HashMap<>();
        int[] prefixSum = new int[n];
        int result = 0;

        for (int i = 1; i < n; i++) {
            prefixSum[i] = prefixSum[i - 1] + travel[i - 1];
        }

        for (int i = 0; i < n; i++) {
            result += garbage[i].length();

            for (char c : garbage[i].toCharArray()) {
                map.put(c, i);
            }
        }

        for (Integer index : map.values()) {
            result += prefixSum[index];
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

723. `[2392]Build a Matrix With Conditions.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[][] buildMatrix(int k, int[][] rowConditions, int[][] colConditions) {
        List<Integer> rowOrder = topologicalSort(k, rowConditions);
        List<Integer> colOrder = topologicalSort(k, colConditions);

        if (rowOrder == null || colOrder == null) {
            return new int[0][0];
        }

        int[][] matrix = new int[k][k];
        int[] rowPosition = new int[k + 1];
        int[] colPosition = new int[k + 1];

        IntStream.range(0, k).forEach(i -> {
            rowPosition[rowOrder.get(i)] = i;
            colPosition[colOrder.get(i)] = i;
        });

        IntStream.rangeClosed(1, k).forEach(i -> {
            matrix[rowPosition[i]][colPosition[i]] = i;
        });

        return matrix;
    }

    private List<Integer> topologicalSort(int k, int[][] conditions) {
        int[] inDegree = new int[k + 1];
        List<List<Integer>> graph = IntStream.rangeClosed(1, k)
                .mapToObj(i -> new ArrayList<Integer>())
                .collect(Collectors.toList());

        Arrays.stream(conditions).forEach(condition -> {
            int u = condition[0];
            int v = condition[1];
            graph.get(u - 1).add(v - 1);
            inDegree[v]++;
        });

        Deque<Integer> queue = IntStream.rangeClosed(1, k)
                .filter(i -> inDegree[i] == 0)
                .boxed()
                .collect(Collectors.toCollection(ArrayDeque::new));

        List<Integer> order = new ArrayList<>();
        while (!queue.isEmpty()) {
            int current = queue.poll();
            order.add(current);
            graph.get(current - 1).forEach(neighbor -> {
                inDegree[neighbor + 1]--;
```

```
                if (inDegree[neighbor + 1] == 0) {
                    queue.add(neighbor + 1);
                }
            });
        }

        if (order.size() != k) {
            return null;
        }

        return order;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

724. `[2393]Count Strictly Increasing Subarrays.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long countSubarrays(int[] nums) {
        long count = 1;
        long length = 1;

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > nums[i - 1]) {
                length++;
            } else {
                length = 1;
            }

            count += length;
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

725. [2402]Meeting Rooms III.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int mostBooked(int n, int[][] meetings) {
        Arrays.sort(meetings, (a, b) -> a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]);
        int[] count = new int[n];
        long[] room = new long[n];

        for (int i = 0; i < meetings.length; i++) {
            int start = meetings[i][0];
            int end = meetings[i][1];
            int nextRoom = getAvailableRoom(start, room);
            count[nextRoom]++;
            room[nextRoom] = Math.max(end, end - start + room[nextRoom]);
        }

        return getMostBookedRoom(count);
    }

    private int getAvailableRoom(int start, long[] room) {
        long earliest = Long.MAX_VALUE;
        int nextAvailable = -1;

        for (int j = 0; j < room.length; j++) {
            if (room[j] <= start) {
                nextAvailable = j;
                break;
            }

            if (room[j] < earliest) {
                nextAvailable = j;
                earliest = room[j];
            }
        }

        return nextAvailable;
    }

    private int getMostBookedRoom(int[] count) {
        int maxBooked = 0;

        for (int i = 1; i < count.length; i++) {
            if (count[i] > count[maxBooked]) {
                maxBooked = i;
            }
        }

        return maxBooked;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

726. [2404]Most Frequent Even Element.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int mostFrequentEven(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        int max = 0;
        int result = Integer.MAX_VALUE;

        for (int num : nums) {
            if (num % 2 == 0) {
                int curr = map.getOrDefault(num, 0) + 1;
                map.put(num, curr);

                if (curr > max || (curr == max && num < result)) {
                    max = curr;
                    result = num;
                }
            }
        }

        return max > 0 ? result : -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

727. [2405]Optimal Partition of String.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int partitionString(String s) {
        int[] arr = new int[26];
        int count = 1;
        for (char c : s.toCharArray()) {
            if (arr[c - 'a'] > 0) {
                count++;
                Arrays.fill(arr, 0);
            }
            arr[c - 'a']++;
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

728. [2406]Divide Intervals Into Minimum Number of Groups.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minGroups(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        PriorityQueue<Integer> pq = new PriorityQueue<>();

        for (int[] interval : intervals) {
            if (!pq.isEmpty() && interval[0] > pq.peek()) {
                pq.poll();
            }

            pq.offer(interval[1]);
        }

        return pq.size();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

729. `[2407]Longest Increasing Subsequence II.java`

```
/**
 * <p>You are given an integer array <code>nums</code> and an integer <code>k</code>.</p>
 *
 * <p>Find the longest subsequence of <code>nums</code> that meets the following
requirements:</p>
 *
 * <ul>
 * <li>The subsequence is <strong>strictly increasing</strong> and</li>
 * <li>The difference between adjacent elements in the subsequence is <strong>at
most</strong> <code>k</code>.</li>
 * </ul>
 *
 * <p>Return<em> the length of the <strong>longest</strong> <strong>subsequence</strong>
that meets the requirements.</em></p>
 *
 * <p>A <strong>subsequence</strong> is an array that can be derived from another array
by deleting some or no elements without changing the order of the remaining elements.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [4,2,1,4,3,4,5,8,15], k = 3
 * <strong>Output:</strong> 5
 * <strong>Explanation:</strong>
 * The longest subsequence that meets the requirements is [1,3,4,5,8].
 * The subsequence has a length of 5, so we return 5.
 * Note that the subsequence [1,3,4,5,8,15] does not meet the requirements because 15 - 8
= 7 is larger than 3.
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [7,4,5,1,8,12,4,7], k = 5
 * <strong>Output:</strong> 4
 * <strong>Explanation:</strong>
 * The longest subsequence that meets the requirements is [4,5,8,12].
 * The subsequence has a length of 4, so we return 4.
 * </pre>
 *
 * <p><strong class="example">Example 3:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [1,5], k = 1
 * <strong>Output:</strong> 1
 * <strong>Explanation:</strong>
 * The longest subsequence that meets the requirements is [1].
 * The subsequence has a length of 1, so we return 1.
 * </pre>
 *
```

```
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>1 &lt;= nums.length &lt;= 10<sup>5</sup></code></li>
 * <li><code>1 &lt;= nums[i], k &lt;= 10<sup>5</sup></code></li>
 * </ul>
 *
 * <div><div>Related Topics</div><div><li>Array</li><li>Divide and Conquer</li>
<li>Dynamic Programming</li><li>Binary Indexed Tree</li><li>Segment Tree</li>
<li>Queue</li><li>Monotonic Queue</li></div></div><br><div><li>👍 557</li><li>👎 22</li>
</div>
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[] seg;
    int len = 0;

    public int lengthOfLIS(int[] nums, int k) {
        int max = 0;
        for (int n : nums) {
            max = Math.max(max, n);
        }
        len = max + 1;
        seg = new int[len * 2];
        int res = 0;
        for (int n : nums) {
            int cur = findMax(Math.max(n - k, 0), n) + 1;
            res = Math.max(res, cur);
            update(n, cur);
        }
        return res;
    }

    public void update(int idx, int val) {
        idx += len;
        while (idx >= 0 && seg[idx] < val) {
            seg[idx] = val;
            idx >>= 1;
        }
    }

    public int findMax(int from, int to) {
        from += len;
        to += len;
        int max = 0;
        while (from < to) {
            if ((from & 1) == 1) {
                max = Math.max(max, seg[from]);
                from++;
            }
            if ((to & 1) == 1) {
                to--;
```

```
                max = Math.max(max, seg[to]);
            }
            from >>= 1;
            to >>= 1;
        }
        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

730. `[2408]Design SQL.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class SQL {

    public SQL(List<String> names, List<Integer> columns) {

    }

    public void insertRow(String name, List<String> row) {

    }

    public void deleteRow(String name, int rowId) {

    }

    public String selectCell(String name, int rowId, int columnId) {

    }
}

/**
 * Your SQL object will be instantiated and called as such:
 * SQL obj = new SQL(names, columns);
 * obj.insertRow(name,row);
 * obj.deleteRow(name,rowId);
 * String param_3 = obj.selectCell(name,rowId,columnId);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

731. `[2410]Maximum Matching of Players With Trainers.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int matchPlayersAndTrainers(int[] players, int[] trainers) {
        Arrays.sort(players);
        Arrays.sort(trainers);
        int playerIndex = 0;
        int trainerIndex = 0;
        int result = 0;
        while (playerIndex < players.length && trainerIndex < trainers.length) {
            if (players[playerIndex] <= trainers[trainerIndex]) {
                playerIndex++;
                result++;
            }
<<<<<<< HEAD
=======

>>>>>>> origin/home
            trainerIndex++;
        }
        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

732. [2411]Smallest Subarrays With Maximum Bitwise OR.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private static final int BITS = 31;

    public int[] smallestSubarrays(int[] nums) {
        final int n = nums.length;
        int[] answer = new int[n];
        int[] lastPos = new int[BITS];
        Arrays.fill(lastPos, -1);

        for (int i = n - 1; i >= 0; i--) {
            for (int b = 0; b < BITS; b++) {
                if ((nums[i] & (1 << b)) != 0) {
                    lastPos[b] = i;
                }
            }

            int farthest = i;

            for (int b = 0; b < BITS; b++) {
                if (lastPos[b] != -1) {
                    farthest = Math.max(farthest, lastPos[b]);
                }
            }

            answer[i] = farthest - i + 1;
        }

        return answer;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

733. [2415]Reverse Odd Levels of Binary Tree.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public TreeNode reverseOddLevels(TreeNode root) {
        if (root == null) {
            return null;
        }

        dfs(root.left, root.right, 0);
        return root;
    }

    private void dfs(TreeNode left, TreeNode right, int depth) {
        if (left == null && right == null) {
            return;
        }

        if (depth % 2 == 0) {
            int temp = left.val;
            left.val = right.val;
            right.val = temp;
        }

        dfs(left.left, right.right, depth + 1);
        dfs(left.right, right.left, depth + 1);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

734. `[2416]Sum of Prefix Scores of Strings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] sumPrefixScores(String[] words) {
        int n = words.length;
        int[] answer = new int[n];
        TrieNode root = buildTrie(words);

        for (int i = 0; i < n; i++) {
            String word = words[i];
            TrieNode current = root;
            int sum = 0;

            for (char ch : word.toCharArray()) {
                int idx = ch - 'a';

                if (current.children[idx] == null) {
                    break;
                }

                current = current.children[idx];
                sum += current.count;
            }

            answer[i] = sum;
        }

        return answer;
    }

    private TrieNode buildTrie(String[] words) {
        TrieNode root = new TrieNode();

        for (String word : words) {
            TrieNode current = root;

            for (char ch : word.toCharArray()) {
                int idx = ch - 'a';

                if (current.children[idx] == null) {
                    current.children[idx] = new TrieNode();
                }

                current = current.children[idx];
                current.count++;
            }
        }

        return root;
    }
}
```

```
class TrieNode {
    TrieNode[] children;
    int count;

    TrieNode() {
        children = new TrieNode[26];
        count = 0;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

735. `[2418]Sort the People.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String[] sortPeople(String[] names, int[] heights) {
        return IntStream.range(0, names.length)
                .boxed()
                .sorted((a, b) -> heights[b] - heights[a])
                .map(i -> names[i])
                .toArray(String[]::new);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

736. [2419]Longest Subarray With Maximum Bitwise AND.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestSubarray(int[] nums) {
        int result = 0;
        int count = 0;
        int max = 0;

        for (int num : nums) {
            max = Math.max(max, num);
        }

        for (int num : nums) {
            if (num == max) {
                count++;
                result = Math.max(result, count);
            } else {
                count = 0;
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

737. [2421]Number of Good Paths.java

```
2421
        Number of Good Paths
        2023-01-15 09:06:18

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfGoodPaths(int[] vals, int[][] edges) {
        int N = vals.length;
        ArrayList<Integer>[] adj = new ArrayList[N];
        TreeMap<Integer, ArrayList<Integer>> sameValues = new TreeMap<>();
        int ans = 0;

        for (int i = 0; i < N; i++) {
            adj[i] = new ArrayList<>();

            if (!sameValues.containsKey(vals[i])) sameValues.put(vals[i], new ArrayList<>
());
            sameValues.get(vals[i]).add(i);
        }

        for (int[] e : edges) {
            int u = e[0];
            int v = e[1];

            if (vals[u] >= vals[v]) {
                adj[u].add(v);
            } else {
                adj[v].add(u);
            }
        }

        UF uf = new UF(N);

        for (int val : sameValues.entrySet()) {
            for (int u : sameValues.get(val)) {
                for (int v : adj[u]) {
                    uf.union(u, v);
                }
            }

            HashMap<Integer, Integer> group = new HashMap<>();

            for (int u : sameValues.get(val)) {
                group.put(uf.find(u), group.getOrDefault(uf.find(u), 0) + 1);
            }

            ans += sameValues.get(val).size();

            for (int key : group.keySet()) {
                int size = group.get(key);
                ans += size * (size - 1) / 2;
```

```
            }
        }

        return ans;

    }

    class UF {

        int[] parent;

        public UF(int len) {
            parent = new int[len];
            Arrays.fill(parent, -1);
        }

        public int find(int a) {
            if (parent[a] >= 0) {
                return parent[a] = find(parent[a]);
            }
            return a;
        }

        public boolean union(int a, int b) {
            int pa = find(a);
            int pb = find(b);

            if (pa == pb) return false;

            if (parent[pa] <= parent[pb]) {
                parent[pa] += parent[pb];
                parent[pb] = pa;
            } else {
                parent[pb] += parent[pa];
                parent[pa] = pb;
            }

            return true;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

738. [2422]Merge Operations to Turn Array Into a Palindrome.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumOperations(int[] nums) {
        int count = 0;
        int left = 0;
        int right = nums.length - 1;

        while (left < right) {
            if (nums[left] == nums[right]) {
                left++;
                right--;
            } else if (nums[left] < nums[right]) {
                nums[left + 1] += nums[left];
                count++;
                left++;
            } else {
                nums[right - 1] += nums[right];

                right--;
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

739. `[2425]Bitwise XOR of All Pairings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int xorAllNums(int[] nums1, int[] nums2) {
        int xorA = 0;
        for (int num : nums1) {
            xorA ^= num;
        }

        int xorB = 0;
        for (int num : nums2) {
            xorB ^= num;
        }

        int n1 = nums1.length;
        int n2 = nums2.length;

        if ((n2 & 1) == 0) {
            if ((n1 & 1) == 1) {
                return xorB;
            } else {
                return 0;
            }
        } else {
            if ((n1 & 1) == 1) {
                return xorA ^ xorB;
            } else {
                return xorA;
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

740. `[2433]Find The Original Array of Prefix Xor.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] findArray(int[] pref) {
        int[] arr = new int[pref.length];

        arr[0] = pref[0];

        for (int i = 1; i < pref.length; i++) {
            arr[i] = pref[i] ^ pref[i - 1];
        }

        return arr;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

741. `[2437]Number of Valid Clock Times.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countTime(String time) {
        int hourCount = 1;
        int minuteCount = 1;
        char c1 = time.charAt(0);
        char c2 = time.charAt(1);
        char c3 = time.charAt(3);
        char c4 = time.charAt(4);

        if (c1 == '?' && c2 == '?') {
            hourCount = 24;
        } else if (c1 == '?') {
            hourCount = c2 < '4' ? 3 : 2;
        } else if (c2 == '?') {
            hourCount = c1 == '2' ? 4 : 10;
        }

        if (c3 == '?') {
            minuteCount *= 6;
        }

        if (c4 == '?') {
            minuteCount *= 10;
        }

        return hourCount * minuteCount;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

742. [2439]Minimize Maximum of Array.java

```
/**
 * <p>You are given a <strong>0-indexed</strong> array <code>nums</code> comprising of
<code>n</code> non-negative integers.</p>
 *
 * <p>In one operation, you must:</p>
 *
 * <ul>
 * <li>Choose an integer <code>i</code> such that <code>1 &lt;= i &lt; n</code> and
<code>nums[i] &gt; 0</code>.</li>
 * <li>Decrease <code>nums[i]</code> by 1.</li>
 * <li>Increase <code>nums[i - 1]</code> by 1.</li>
 * </ul>
 *
 * <p>Return<em> the <strong>minimum</strong> possible value of the
<strong>maximum</strong> integer of </em><code>nums</code><em> after performing
<strong>any</strong> number of operations</em>.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [3,7,1,6]
 * <strong>Output:</strong> 5
 * <strong>Explanation:</strong>
 * One set of optimal operations is as follows:
 * 1. Choose i = 1, and nums becomes [4,6,1,6].
 * 2. Choose i = 3, and nums becomes [4,6,2,5].
 * 3. Choose i = 1, and nums becomes [5,5,2,5].
 * The maximum integer of nums is 5. It can be shown that the maximum number cannot be
less than 5.
 * Therefore, we return 5.
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [10,1]
 * <strong>Output:</strong> 10
 * <strong>Explanation:</strong>
 * It is optimal to leave nums as is, and since 10 is the maximum value, we return 10.
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
 * <ul>
 * <li><code>n == nums.length</code></li>
 * <li><code>2 &lt;= n &lt;= 10<sup>5</sup></code></li>
 * <li><code>0 &lt;= nums[i] &lt;= 10<sup>9</sup></code></li>
 * </ul>
 *
```

```
 * <div><div>Related Topics</div><div><li>Array</li><li>Binary Search</li><li>Dynamic
Programming</li><li>Greedy</li><li>Prefix Sum</li></div></div><br><div><li>👍 529</li>
<li>👎 75</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimizeArrayValue(int[] nums) {
        long max = 0;
        long prefixSum = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] > max) {
                prefixSum += nums[i];
                long temp = (long) Math.ceil((double) prefixSum / (i + 1));
                max = Math.max(max, temp);
            } else {
                prefixSum += nums[i];
            }
        }
        return (int) max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

743. [2441]Largest Positive Integer That Exists With Its Negative.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int findMaxK(int[] nums) {
        Set<Integer> set = new HashSet<>();

        for (int num : nums) {
            set.add(num);
        }

        int max = -1;

        for (int num : nums) {
            if (set.contains(-num) && num > max) {
                max = num;
            }
        }

        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

744. `[2444].java`

```
2444
        Count Subarrays With Fixed Bounds
        2023-03-04 09:05:01

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long countSubarrays(int[] nums, int minK, int maxK) {
        long res = 0;
        int minPos = -1;
        int maxPos = -1;
        int leftBound = -1;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] < minK || nums[i] > maxK) {
                leftBound = i;
            }
            if (nums[i] == minK) {
                minPos = i;
            }
            if (nums[i] == maxK) {
                maxPos = i;
            }
            res += Math.max(0, Math.min(minPos, maxPos) - leftBound);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

745. `[2444]Count Subarrays With Fixed Bounds.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long countSubarrays(int[] nums, int minK, int maxK) {
    long count = 0L;
    int lastMinPosition = -1;
    int lastMaxPosition = -1;
    int lastInvalidPosition = -1;

    for (int i = 0; i < nums.length; i++) {
      int v = nums[i];

      if (v < minK || v > maxK) {
        lastInvalidPosition = i;
            }
      if (v == minK) {
        lastMinPosition = i;
            }
      if (v == maxK) {
        lastMaxPosition = i;
      }

      int validStartCount = Math.min(lastMinPosition, lastMaxPosition) -
lastInvalidPosition;
      if (validStartCount > 0) {
        count += validStartCount;
      }
    }
    return count;
  }
}
//leetcode submit region end(Prohibit modification and deletion)
```

746. `[2448]Minimum Cost to Make Array Equal.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[] nums;
    int[] cost;

    public long minCost(int[] nums, int[] cost) {
        this.nums = nums;
        this.cost = cost;
        int val = binarySearch(0, (int) 1e6 + 1);
        return getCost(val);
    }

    private int binarySearch(int left, int right) {
        while (left <= right) {
            int mid = (left + right) >> 1;
            long cost1 = getCost(mid);
            long cost2 = getCost(mid + 1);
            if (cost1 > cost2) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }

    private long getCost(int val) {
        long res = 0;
        for (int i = 0; i < nums.length; i++) {
            res += 1L * Math.abs(nums[i] - val) * cost[i];
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

747. [2458]Height of Binary Tree After Subtree Removal Queries.java

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
    private Map<Integer, Integer> leftMap = new HashMap<>();
    private Map<Integer, Integer> rightMap = new HashMap<>();
    private Map<Integer, Integer> removed = new HashMap<>();

    public int[] treeQueries(TreeNode root, int[] queries) {
        populateHeights(root, 0);
        calculateRemovedHeights(root, 0);
        int[] output = new int[queries.length];

        for (int i = 0; i < queries.length; i++) {
            output[i] = removed.get(queries[i]);
        }

        return output;
    }

    private void calculateRemovedHeights(TreeNode node, int height) {
        if (node == null) {
            return;
        }

        removed.put(node.val, height);
        calculateRemovedHeights(node.left, Math.max(height, rightMap.get(node.val)));
        calculateRemovedHeights(node.right, Math.max(height, leftMap.get(node.val)));
    }

    private int populateHeights(TreeNode node, int height) {
        if (node == null) {
            return height - 1;
        }

        leftMap.put(node.val, populateHeights(node.left, height + 1));
        rightMap.put(node.val, populateHeights(node.right, height + 1));

        return Math.max(leftMap.get(node.val), rightMap.get(node.val));
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

748. `[2460]Apply Operations to an Array.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int[] applyOperations(int[] nums) {
    int n = nums.length;

    for (int i = 0; i < n - 1; i++) {
      if (nums[i] == nums[i + 1]) {
        nums[i] *= 2;
        nums[i + 1] = 0;
      }
    }

    int[] result = new int[n];
    int index = 0;

    for (int num : nums) {
      if (num != 0) {
        result[index++] = num;
      }
    }

    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

749. `[2461]Maximum Sum of Distinct Subarrays With Length K.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public long maximumSubarraySum(int[] nums, int k) {
    int n = nums.length;
    long currSum = 0;
    long maxSum = 0;
    Map<Integer, Integer> map = new HashMap<>();
    int left = 0;

    for (int right = 0; right < n; right++) {
      map.merge(nums[right], 1, Integer::sum);
      currSum += nums[right];

      if (right - left + 1 == k) {
        if (map.size() == k) {
          maxSum = Math.max(maxSum, currSum);
        }

        int count = map.get(nums[left]);

        if (count == 1) {
          map.remove(nums[left]);
        } else {
          map.put(nums[left], count - 1);
        }

        currSum -= nums[left];
        left++;
      }
    }

    return maxSum;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

750. [2462]Total Cost to Hire K Workers.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long totalCost(int[] costs, int k, int candidates) {
        long res = 0;
        int n = costs.length;
        if (2 * candidates > n - k || n == k) {
            Arrays.sort(costs);
            for (int i = 0; i < k; i++) {
                res += costs[i];
            }
            return res;
        }
        PriorityQueue<Integer> pq1 = new PriorityQueue<>();
        PriorityQueue<Integer> pq2 = new PriorityQueue<>();
        int left = 0;
        int right = n - 1;
        while (k-- > 0) {
            while (pq1.size() < candidates) {
                pq1.add(costs[left++]);
            }
            while (pq2.size() < candidates) {
                pq2.add(costs[right--]);
            }
            if (pq1.peek() <= pq2.peek()) {
                res += pq1.poll();
            } else {
                res += pq2.poll();
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

751. [2463]Minimum Total Distance Traveled.java

```
//leetcode submit region begin(Prohibit modification and deletion)
import java.util.*;

public class Solution {
    class Edge {
        int to, rev;
        long capacity, cost;
        Edge(int to, int rev, long capacity, long cost) {
            this.to = to;
            this.rev = rev;
            this.capacity = capacity;
            this.cost = cost;
        }
    }

   private void addEdge(List<Edge>[] graph, int from, int to, long capacity, long cost) {
        graph[from].add(new Edge(to, graph[to].size(), capacity, cost));
        graph[to].add(new Edge(from, graph[from].size() -1, 0, -cost));
    }

   private long minCostMaxFlow(List<Edge>[] graph, int N, int source, int sink, long
maxFlow) {
        long[] potential = new long[N];
        Arrays.fill(potential, 0);
        long totalCost = 0;
        long flow = 0;

        while (flow < maxFlow) {
            PriorityQueue<long[]> pq = new PriorityQueue<>(Comparator.comparingLong(a ->
a[1]));
            long[] dist = new long[N];
            Arrays.fill(dist, Long.MAX_VALUE);
            dist[source] = 0;
            pq.offer(new long[]{source, 0});
            int[] prevv = new int[N];
            int[] preve = new int[N];
            Arrays.fill(prevv, -1);
            Arrays.fill(preve, -1);

            while (!pq.isEmpty()) {
                long[] current = pq.poll();
                int u = (int)current[0];
                long d = current[1];

                if (d > dist[u]) {
                    continue;
                }

                for (int i = 0; i < graph[u].size(); i++) {
                    Edge e = graph[u].get(i);
```

```
                    if (e.capacity > 0 && dist[e.to] > dist[u] + e.cost + potential[u] -
potential[e.to]) {
                        dist[e.to] = dist[u] + e.cost + potential[u] - potential[e.to];
                        prevv[e.to] = u;
                        preve[e.to] = i;
                        pq.offer(new long[]{e.to, dist[e.to]});
                    }
                }
            }

            if (dist[sink] == Long.MAX_VALUE) {
                break;
            }

            for (int v = 0; v < N; v++) {
                potential[v] += dist[v] < Long.MAX_VALUE ? dist[v] : 0;
            }

            long addFlow = maxFlow - flow;
            int v = sink;

            while (v != source) {
                int u = prevv[v];
                Edge e = graph[u].get(preve[v]);
                addFlow = Math.min(addFlow, e.capacity);
                v = u;
            }

            flow += addFlow;
            totalCost += addFlow * potential[sink];
            v = sink;

            while (v != source) {
                int u = prevv[v];
                Edge e = graph[u].get(preve[v]);
                graph[u].set(preve[v], new Edge(e.to, e.rev, e.capacity - addFlow,
e.cost));
                Edge rev = graph[v].get(e.rev);
                graph[v].set(e.rev, new Edge(rev.to, rev.rev, rev.capacity + addFlow,
rev.cost));
                v = u;
            }
        }

        return totalCost;
    }

    public long minimumTotalDistance(List<Integer> robot, int[][] factory) {
        int numRobots = robot.size();
        int numFactories = factory.length;
        Collections.sort(robot);
        Arrays.sort(factory, Comparator.comparingInt(a -> a[0]));
        int N = 2 + numRobots + numFactories;
```

```
        int source = 0;
        int sink = N -1;

        List<Edge>[] graph = new List[N];

        for(int i=0;i<N;i++) {
            graph[i] = new ArrayList<>();
        }

        for(int i=0;i<numRobots;i++) {
            addEdge(graph, source, 1 + i, 1, 0);
        }

        for(int i=0;i<numRobots;i++) {
            int robotPos = robot.get(i);
            for(int j=0;j<numFactories;j++) {
                int factoryPos = factory[j][0];
                long distance = Math.abs((long)robotPos - (long)factoryPos);
                addEdge(graph, 1 + i, 1 + numRobots + j, 1, distance);
            }
        }

        for(int j=0;j<numFactories;j++) {
            int limit = factory[j][1];
            addEdge(graph, 1 + numRobots + j, sink, limit, 0);
        }

        long totalCost = minCostMaxFlow(graph, N, source, sink, numRobots);
        return totalCost;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

752. [2466]Count Ways To Build Good Strings.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countGoodStrings(int low, int high, int zero, int one) {
        int[] dp = new int[high + 1];
        final int MOD = (int) 1e9 + 7;
        int res = 0;
        dp[0] = 1;
        for (int i = 1; i <= high; i++) {
            if (i >= zero) {
                dp[i] += dp[i - zero];
            }
            if (i >= one) {
                dp[i] += dp[i - one];
            }
            dp[i] %= MOD;
            if (i >= low) {
                res += dp[i];
                res %= MOD;
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

753. [2467]Most Profitable Path in a Tree.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int mostProfitablePath(int[][] edges, int bob, int[] amount) {
    int n = amount.length;
    List<Integer>[] graph = new ArrayList[n];

    for (int i = 0; i < n; i++) {
      graph[i] = new ArrayList<>();
    }

    for (int[] e : edges) {
      int u = e[0], v = e[1];
      graph[u].add(v);
      graph[v].add(u);
    }

    int[] parent = new int[n];
    Arrays.fill(parent, -1);
    dfsParent(0, -1, graph, parent);

    int[] bobTime = new int[n];
    Arrays.fill(bobTime, Integer.MAX_VALUE);
    int time = 0;
    int cur = bob;

    while (cur != -1) {
      bobTime[cur] = time;
      cur = parent[cur];
      time++;
    }

    return dfsAlice(0, -1, 0, graph, amount, bobTime);
  }

  private void dfsParent(int node, int par, List<Integer>[] graph, int[] parent) {
    parent[node] = par;

    for (int nxt : graph[node]) {
      if (nxt == par) {
        continue;
      }

      dfsParent(nxt, node, graph, parent);
    }
  }

  private int dfsAlice(
      int node, int par, int time, List<Integer>[] graph, int[] amount, int[] bobTime) {
    int currIncome = 0;

    if (time < bobTime[node]) {
```

```
      currIncome = amount[node];
    } else if (time == bobTime[node]) {
      currIncome = amount[node] / 2;
    }

    if (node != 0 && graph[node].size() == 1) {
      return currIncome;
    }

    int maxChildIncome = Integer.MIN_VALUE;

    for (int nxt : graph[node]) {
      if (nxt == par) {
        continue;
      }

      int childIncome = dfsAlice(nxt, node, time + 1, graph, amount, bobTime);
      maxChildIncome = Math.max(maxChildIncome, childIncome);
    }

    if (maxChildIncome == Integer.MIN_VALUE) {
      return currIncome;
    }

    return currIncome + maxChildIncome;
  }
}

// leetcode submit region end(Prohibit modification and deletion)
```

754. `[2471]Minimum Number of Operations to Sort a`

Binary Tree by Level.java

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
    public int minimumOperations(TreeNode root) {
        if (root == null) return 0;
        int totalSwaps = 0;
        Queue<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int levelSize = queue.size();
            List<Integer> currentLevel = new ArrayList<>(levelSize);

            for (int i = 0; i < levelSize; i++) {
                TreeNode node = queue.poll();
                currentLevel.add(node.val);
                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }

            totalSwaps += minSwapsToSort(currentLevel);
        }

        return totalSwaps;
    }
    private int minSwapsToSort(List<Integer> list) {
        int n = list.size();

        if (n <= 1) {
            return 0;
        }

        int[] sortedArr = new int[n];

        for (int i = 0; i < n; i++) {
            sortedArr[i] = list.get(i);
        }

        Arrays.sort(sortedArr);

        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = Arrays.binarySearch(sortedArr, list.get(i));
        }

        boolean[] visited = new boolean[n];
        int cycles = 0;

        for (int i = 0; i < n; i++) {
```

```
            if (!visited[i]) {
                cycles++;
                int j = i;

                while (!visited[j]) {
                    visited[j] = true;
                    j = arr[j];
                }
            }
        }

        return n - cycles;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

755. `[2472]Maximum Number of Non-overlapping Palindrome`

Substrings.java

```
/**
<p>You are given a string <code>s</code> and a <strong>positive</strong> integer
<code>k</code>.</p>

<p>Select a set of <strong>non-overlapping</strong> substrings from the string
<code>s</code> that satisfy the following conditions:</p>

<ul>
 <li>The <strong>length</strong> of each substring is <strong>at least</strong>
<code>k</code>.</li>
 <li>Each substring is a <strong>palindrome</strong>.</li>
</ul>

<p>Return <em>the <strong>maximum</strong> number of substrings in an optimal
selection</em>.</p>

<p>A <strong>substring</strong> is a contiguous sequence of characters within a string.
</p>

<p> </p>
<p><strong class="example">Example 1:</strong></p>

<pre>
<strong>Input:</strong> s = "abaccdbbd", k = 3
<strong>Output:</strong> 2
<strong>Explanation:</strong> We can select the substrings underlined in s = "<u>
<strong>aba</strong></u>cc<u><strong>dbbd</strong></u>". Both "aba" and "dbbd" are
palindromes and have a length of at least k = 3.
It can be shown that we cannot find a selection with more than two valid substrings.
</pre>

<p><strong class="example">Example 2:</strong></p>

<pre>
<strong>Input:</strong> s = "adbcda", k = 2
<strong>Output:</strong> 0
<strong>Explanation:</strong> There is no palindrome substring of length at least 2 in
the string.
</pre>

<p> </p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>1 &lt;= k &lt;= s.length &lt;= 2000</code></li>
 <li><code>s</code> consists of lowercase English letters.</li>
</ul>

<div><div>Related Topics</div><div><li>String</li><li>Dynamic Programming</li></div>
</div><br><div><li>👍 322</li><li>👎 8</li></div>
*/
```

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxPalindromes(String s, int k) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

756. [2473]Minimum Cost to Buy Apples.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long[] minCost(int n, int[][] roads, int[] appleCost, int k) {
        List<List<Pair<Integer, Integer>>> graph = new ArrayList<>();

        for (int i = 0; i < n; ++i) {
            graph.add(new ArrayList<>());
        }

        for (int[] road : roads) {
            int cityA = road[0] - 1, cityB = road[1] - 1, cost = road[2];
            graph.get(cityA).add(new Pair<Integer, Integer>(cityB, cost));
            graph.get(cityB).add(new Pair<Integer, Integer>(cityA, cost));
        }

        long[] result = new long[n];

        for (int startCity = 0; startCity < n; startCity++) {
            result[startCity] = appleCost[startCity];
        }

        Queue<Pair<Long, Integer>> heap = new PriorityQueue<>((a, b) ->
Long.compare(a.getKey(), b.getValue()));

        for (int startCity = 0; startCity < n; startCity++) {
            heap.add(new Pair<>((long) appleCost[startCity], startCity));
        }

        while (!heap.isEmpty()) {
            Pair<Long, Integer> current = heap.poll();
            long totalCost = current.getKey();
            int currCity = current.getValue();

            if (result[currCity] < totalCost) {
                continue;
            }

            for (Pair<Integer, Integer> next : graph.get(currCity)) {
                int neighbor = next.getKey(), cost = next.getValue();

                if (result[neighbor] > result[currCity] + (k + 1) * cost) {
                    result[neighbor] = result[currCity] + (k + 1) * cost;
                    heap.add(new Pair<Long, Integer>(result[neighbor], neighbor));
                }
            }
        }

        return result;
    }
```

```
}
//leetcode submit region end(Prohibit modification and deletion)
```

757. [2477]Minimum Fuel Cost to Report to the Capital.java

```
2477
        Minimum Fuel Cost to Report to the Capital
        2023-02-12 10:43:14

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private long res = 0;
    private int seats;

    public long minimumFuelCost(int[][] roads, int seats) {
        this.seats = seats;
        int n = roads.length + 1;
        List<List<Integer>> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            list.add(new ArrayList<>());
        }
        for (int[] road : roads) {
            list.get(road[0]).add(road[1]);
            list.get(road[1]).add(road[0]);
        }
        dfs(0, 0, list);
        return res;
    }

    private int dfs(int city, int prev, List<List<Integer>> list) {
        int count = 1;
        for (Integer nei : list.get(city)) {
            if (nei == prev) {
                continue;
            }
            count += dfs(nei, city, list);
        }
        if (city != 0) {
            res += (count + seats - 1) / seats;
        }
        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

758. [2482]Difference Between Ones and Zeros in Row and Column.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[][] onesMinusZeros(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int[] rowOnes = new int[m];
        int[] colOnes = new int[n];
        int[][] diff = new int[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                rowOnes[i] += grid[i][j];
                colOnes[j] += grid[i][j];
            }
        }

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                diff[i][j] = 2 * (rowOnes[i] + colOnes[j]) - (m + n);
            }
        }

        return diff;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

759. `[2483]Minimum Penalty for a Shop.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int bestClosingTime(String customers) {
        int currPenalty = 0;
        int minPenalty = 0;
        int minTime = 0;
        for (int i = 0; i < customers.length(); i++) {
            if (customers.charAt(i) == 'Y') {
                currPenalty--;
            } else {
                currPenalty++;
            }
            if (currPenalty < minPenalty) {
                minPenalty = currPenalty;
                minTime = i + 1;
            }
        }
        return minTime;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

760. `[2484]Count Palindromic Subsequences.java`

```
2484
        Count Palindromic Subsequences
        2022-11-27 09:18:26

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countPalindromes(String s) {
        int mod = 1000_000_007;
        int size = 5;
        long ans = 0;
        for (char a = '0'; a <= '9'; a++) {
            for (char b = '0'; b <= '9'; b++) {
                char[] pattern = {a, b, '.', b, a};
                long[] dp = new long[size + 1];
                dp[size] = 1;
                for (char c : s.toCharArray()) {
                    for (int i = 0; i < size; i++) {
                        if (pattern[i] == '.' || pattern[i] == c)
                            dp[i] += dp[i + 1];
                    }
                }
                ans += dp[0];
                ans %= mod;
            }
        }
        return (int) ans;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

761. [2485]Find the Pivot Integer.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int pivotInteger(int n) {
        int total = n * (n + 1) / 2;

        for (int i = 1, sum = 0; i <= n; i++) {
            sum += i;

            if (sum * 2 == total + i) {
                return i;
            }
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

762. [2486]Append Characters to String to Make Subsequence.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int appendCharacters(String s, String t) {
        int sIndex = 0, tIndex = 0;
        int sLen = s.length(), tLen = t.length();

        while (sIndex < sLen && tIndex < tLen) {
            if (s.charAt(sIndex) == t.charAt(tIndex)) {
                tIndex++;
            }

            sIndex++;
        }

        return tLen - tIndex;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

763. [2487]Remove Nodes From Linked List.java

```
2487
        Remove Nodes From Linked List
        2023-01-03 11:07:35
//leetcode submit region begin(Prohibit modification and deletion)

/**
 * Definition for singly-linked list.
 * public class ListNode {
 * int val;
 * ListNode next;
 * ListNode() {}
 * ListNode(int val) { this.val = val; }
 * ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode removeNodes(ListNode head) {
        if (head == null) {
            return null;
        }
        head.next = removeNodes(head.next);
        if (head.next != null && head.val < head.next.val) {
            return head.next;
        }
        return head;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

764. `[2490]Circular Sentence.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isCircularSentence(String sentence) {
        String[] words = sentence.split(" ");
        int n = words.length;

        for (int i = 0; i < n; i++) {
            String currentWord = words[i];
            String nextWord = words[(i + 1) % n];
            char lastCurrentChar = currentWord.charAt(currentWord.length() - 1);
            char firstNextChar = nextWord.charAt(0);

            if (lastCurrentChar != firstNextChar) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

765. [2491]Divide Players Into Teams of Equal Skill.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long dividePlayers(int[] skill) {
        int n = skill.length;
        int sumOfSkill = 0;

        for (int s : skill) {
            sumOfSkill += s;
        }

        if (sumOfSkill % (n / 2) != 0) {
            return -1;
        }

        int denominator = sumOfSkill / (n / 2);
        Arrays.sort(skill);
        long result = 0;
        int left = 0;
        int right = n - 1;

        while (left < right) {
            int pairSum = skill[left] + skill[right];

            if (pairSum != denominator) {
                return -1;
            }

            result += (long) skill[left] * (long) skill[right];
            left++;
            right--;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

766. `[2492]Minimum Score of a Path Between Two`

Cities.java

```
/**
 * <p>You are given a positive integer <code>n</code> representing <code>n</code> cities
numbered from <code>1</code> to <code>n</code>. You are also given a <strong>2D</strong>
array <code>roads</code> where <code>roads[i] = [a<sub>i</sub>, b<sub>i</sub>,
distance<sub>i</sub>]</code> indicates that there is a <strong>bidirectional
</strong>road between cities <code>a<sub>i</sub></code> and <code>b<sub>i</sub></code>
with a distance equal to <code>distance<sub>i</sub></code>. The cities graph is not
necessarily connected.</p>
 *
 * <p>The <strong>score</strong> of a path between two cities is defined as the
<strong>minimum </strong>distance of a road in this path.</p>
 *
 * <p>Return <em>the <strong>minimum </strong>possible score of a path between cities
</em><code>1</code><em> and </em><code>n</code>.</p>
 *
 * <p><strong>Note</strong>:</p>
 *
 * <ul>
 * <li>A path is a sequence of roads between two cities.</li>
 * <li>It is allowed for a path to contain the same road <strong>multiple</strong> times,
and you can isVisited cities <code>1</code> and <code>n</code> multiple times along the
path.</li>
 * <li>The test cases are generated such that there is <strong>at least</strong> one path
between <code>1</code> and <code>n</code>.</li>
 * </ul>
 *
 * <p> </p>
 * <p><strong cxss="example">Example 1:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2022/10/12/graph11.png"
style="width: 190px; height: 231px;" />
 * <pre>
 * <strong>Input:</strong> n = 4, roads = [[1,2,9],[2,3,6],[2,4,5],[1,4,7]]
 * <strong>Output:</strong> 5
 * <strong>Expxnation:</strong> The path from city 1 to 4 with the minimum score is: 1 -
&gt; 2 -&gt; 4. The score of this path is min(9,5) = 5.
 * It can be shown that no other path has less score.
 * </pre>
 *
 * <p><strong cxss="example">Example 2:</strong></p>
 * <img alt="" src="https://assets.leetcode.com/uploads/2022/10/12/graph22.png"
style="width: 190px; height: 231px;" />
 * <pre>
 * <strong>Input:</strong> n = 4, roads = [[1,2,2],[1,3,4],[3,4,7]]
 * <strong>Output:</strong> 2
 * <strong>Expxnation:</strong> The path from city 1 to 4 with the minimum score is: 1 -
&gt; 2 -&gt; 1 -&gt; 3 -&gt; 4. The score of this path is min(2,2,4,7) = 2.
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
```

```
 * <ul>
 * <li><code>2 &lt;= n &lt;= 10<sup>5</sup></code></li>
 * <li><code>1 &lt;= roads.length &lt;= 10<sup>5</sup></code></li>
 * <li><code>roads[i].length == 3</code></li>
 * <li><code>1 &lt;= a<sub>i</sub>, b<sub>i</sub> &lt;= n</code></li>
 * <li><code>a<sub>i</sub> != b<sub>i</sub></code></li>
 * <li><code>1 &lt;= distance<sub>i</sub> &lt;= 10<sup>4</sup></code></li>
 * <li>There are no repeated lists.</li>
 * <li>There is at least one path between <code>1</code> and <code>n</code>.</li>
 * </ul>
 *
 * <div><div>Rexted Topics</div><div><li>Depth-First Search</li><li>Breadth-First
Search</li><li>Union Find</li><li>Graph</li></div></div><br><div><li>👍 374</li><li>👎
63</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[] arr;
    int[] values;

    public int minScore(int n, int[][] roads) {
        arr = new int[n + 1];
        values = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = i;
            values[i] = Integer.MAX_VALUE;
        }
        for (int[] road : roads) {
            union(road[0], road[1], road[2]);
        }
        return values[find(n)];
    }

    public int find(int x) {
        return arr[x] == x ? x : (arr[x] = find(arr[x]));
    }

    public void union(int a, int b, int value) {
        int x = find(a);
        int y = find(b);
        values[x] = Math.min(value, Math.min(values[x], values[y]));
        values[y] = Math.min(value, Math.min(values[x], values[y]));
        if (x != y) {
            arr[x] = y;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

767. [2497]Maximum Star Sum of a Graph.java

```
2497
        Maximum Star Sum of a Graph
        2022-12-11 00:10:29

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxStarSum(int[] vals, int[][] edges, int k) {
        int n = vals.length;
        List<Integer>[] graph = new List[n];
        for (int i = 0; i < n; i++) {
            graph[i] = new ArrayList<Integer>();
        }
        for (int[] edge : edges) {
            int a = edge[0];
            int b = edge[1];
            graph[a].add(b);
            graph[b].add(a);
        }
        int res = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
            List<Integer> curr = graph[i];
            curr.sort((a, b) -> vals[b] - vals[a]);
            int max = vals[i];
            int sum = vals[i];
            for (int j = 0; j < k && j < curr.size(); j++) {
                sum += vals[curr.get(j)];
                max = Math.max(max, sum);
            }
            res = Math.max(res, max);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

768. [2498]Frog Jump II.java

```
2498
        Frog Jump II
        2022-12-11 00:13:50

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxJump(int[] stones) {
        if (stones.length == 2)
            return stones[1];
        int res = Integer.MIN_VALUE;
        for (int i = 2; i < stones.length; i++) {
            res = res > (stones[i] - stones[i - 2]) ? res : (stones[i] - stones[i - 2]);
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

769. [2501]Longest Square Streak in an Array.java

```


//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int longestSquareStreak(int[] nums) {
        final int MAX = 100_001;
        int[] dp = new int[MAX];
        int result = 0;
        Arrays.sort(nums);

        for (int num : nums) {
            int root = (int) Math.sqrt(num);

            if (root * root == num && root < MAX) {
                dp[num] = dp[root] + 1;
            } else {
                dp[num] = 1;
            }

            result = Math.max(result, dp[num]);
        }

        return result >= 2 ? result : -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

770. `[2505]Bitwise OR of All Subsequence Sums.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long subsequenceSumOr(int[] nums) {
        long result = 0L;
        long sum = 0L;

        for (int num : nums) {
            sum += num;
            result |= num;
            result |= sum;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

771. [2513]Minimize the Maximum of Two Arrays.java

```
2513
        Minimize the Maximum of Two Arrays
        2022-12-25 02:53:21

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimizeSet(int divisor1, int divisor2, int uniqueCnt1, int uniqueCnt2) {
        long left = 1;
        long right = 1_000_000_000;
        long res = right;
        long lcm = ((long) divisor1 * (long) divisor2) / gcd(divisor1, divisor2);

        while (left <= right) {
            long mid = (left + right) >> 1;
            int x = (int) (mid - mid / divisor1);
            int y = (int) (mid - mid / divisor2);
            int z = (int) (mid - mid / lcm);

            if (x < uniqueCnt1 || y < uniqueCnt2 || z < uniqueCnt1 + uniqueCnt2) {
                left = mid + 1;
            } else {
                res = Math.min(res, mid);
                right = mid - 1;
            }
        }
        return (int) res;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

772. [2514]Count Anagrams.java

```
2514
Count Anagrams
2022-12-25 16:05:55
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countAnagrams(String s) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

773. `[2525]Categorize Box According to Criteria.java`

```
2525
        Categorize Box According to Criteria
        2023-01-08 09:10:47

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String categorizeBox(int length, int width, int height, int mass) {
        long volume = (long) length * width * height;
        boolean bulky = (length >= 1e4 || width >= 1e4 || height >= 1e4 || volume >= 1e9)
? true : false;
        boolean heavy = mass >= 100 ? true : false;

        if (bulky && heavy) {
            return "Both";
        }
        if (bulky) {
            return "Bulky";
        }
        if (heavy) {
            return "Heavy";
        }
        return "Neither";
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

774. [2526]Find Consecutive Integers from a Data Stream.java

```
2526
Find Consecutive Integers from a Data Stream
2023-01-08 09:11:06
//leetcode submit region begin(Prohibit modification and deletion)
class DataStream {
    List<Integer>list;
    int k;
    int value;
    int count = 0;
    public DataStream(int value, int k) {
        this.k = k;
        this.value = value;
    }

    public boolean consec(int num) {
        if(num == value){
            count++;
        }else{
            count = 0;
        }
        return count >= k;
    }
}

/**
 * Your DataStream object will be instantiated and called as such:
 * DataStream obj = new DataStream(value, k);
 * boolean param_1 = obj.consec(num);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

775. [2527]Find Xor-Beauty of Array.java

```
2527
        Find Xor-Beauty of Array
        2023-01-08 09:11:21

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int xorBeauty(int[] nums) {
        int res = 0;
        for (int num : nums) {
            res ^= num;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
Simple reason why taking XOR of all numbers work:

Let's just consider all possible combinations, and simplify the xor operations.

Split all the possible triplets into below 3 types.

Type 1: Considering 1 number a with all 3 indices same
(a | a) & a = a

Type 2: Considering 2 numbers a b with any 2 indices same

Different combinations are:

(a | a) & b = a & b
(b | b) & a = b & a
(a | b) & a = aa | ab = a | ab = a (By Distributive_law and Absorption_law)
(b | a) & a = a
(a | b) & b = b
(b | a) & b = b
XOR of the above values :
(a & b) ^ (b & a) ^ a ^ a ^ b ^ b
= (a&b)^(a&b) ^ a^a ^ b^b
= 0

We can also note that,

1st point and 2nd pt are same.
3rd point and 4th pt are same.
5th point and 6th pt are same.
Type 3: Considering 3 numbers a b c with all 3 indices are different

Whenever we consider (a | b) & c,
we should also consider (b | a) & c.
It is easy to note that, both these have the same values. So, taking XOR of these 2,
equates to 0.
```

```
Similarily, we can do the same for (a|c)&b, along with (c|a)&b. XOR of these is 0.
Also, for (b|c)&a ^ (c|b)&a is 0.
So, the only numbers that remain are the numbers which have all 3 indices same
(a | a) & a.
 */
```

776. [2529]Maximum Count of Positive Integer and Negative Integer.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int maximumCount(int[] nums) {
    int negativeCount = 0;
    int positiveCount = 0;

    for (int num : nums) {
      if (num < 0) {
        negativeCount++;
      } else if (num > 0) {
        positiveCount++;
      }
    }

    return Math.max(negativeCount, positiveCount);
  }
}

// leetcode submit region end(Prohibit modification and deletion)
```

777. [2530]Maximal Score After Applying K Operations.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long maxKelements(int[] nums, int k) {
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);

        for (int num : nums) {
            maxHeap.offer(num);
        }

        long score = 0;

        for (int i = 0; i < k; i++) {
            int currentMax = maxHeap.poll();
            score += currentMax;
            int newVal = (currentMax + 2) / 3;
            maxHeap.offer(newVal);
        }

        return score;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

778. [2540]Minimum Common Value.java

```
class Solution {
    public int getCommon(int[] nums1, int[] nums2) {
        int i = 0;
        int j = 0;

        while (i < nums1.length && j < nums2.length) {
            if (nums1[i] < nums2[j]) {
                i++;
            } else if (nums1[i] > nums2[j]) {
                j++;
            } else {
                return nums1[i];
            }
        }

        return -1;
    }
}
```

779. `[2542]Maximum Subsequence Score.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long maxScore(int[] nums1, int[] nums2, int k) {
        int n = nums1.length;
        int[][] arrs = new int[n][2];
        for (int i = 0; i < n; i++) {
            arrs[i] = new int[]{nums1[i], nums2[i]};
        }
        Arrays.sort(arrs, (a, b) -> b[1] - a[1]);
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        long res = 0;
        long sum = 0;
        for (int[] arr : arrs) {
            pq.add(arr[0]);
            sum += arr[0];
            if (pq.size() > k) {
                sum -= pq.poll();
            }
            if (pq.size() == k) {
                res = Math.max(res, (sum * arr[1]));
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

780. `[2551]Put Marbles in Bags.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long putMarbles(int[] weights, int k) {
        int n = weights.length - 1;
        long[] arr = new long[n];
        long res = 0;
        for (int i = 0; i < n; i++) {
            arr[i] = weights[i] + weights[i + 1];
        }
        for(long num:arr){
            System.out.print(num+"\t");
        }
        Arrays.sort(arr);
        for (int i = 0; i < k-1; i++) {
            res += arr[n - 1 - i] - arr[i];
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
- 38ms
class Solution {
    public long putMarbles(int[] weights, int k) {
        int n = weights.length - 1;
        long[] arr = new long[n];
        long res = 0;
        for (int i = 0; i < n; i++) {
            arr[i] = weights[i] + weights[i + 1];
        }
        Arrays.sort(arr);
        for (int i = 0; i < k-1; i++) {
            res += arr[n - 1 - i] - arr[i];
        }
        return res;
    }
}

- 381ms
class Solution {
    public long putMarbles(int[] weights, int k) {
        int n = weights.length;
        if (k == 1 || k == n) {
            return 0;
        }
        PriorityQueue<Long> pqMax = new PriorityQueue<>();
        PriorityQueue<Long> pqMin = new PriorityQueue<Long>((a, b) -> {
            return Long.compare(b, a);
        });
        for (int i = 0; i < n - 1; i++) {
```

```
            long sum = (long) weights[i] + weights[i + 1];
            pqMin.add(sum);
            pqMax.add(sum);
            if (pqMin.size() == k) {
                pqMin.poll();
            }
            if (pqMax.size() == k) {
                pqMax.poll();
            }
        }
        long res = 0;
        while (!pqMin.isEmpty()) {
            res += pqMax.poll() - pqMin.poll();
        }
        return res;
    }
}
 */
```

781. [2554]Maximum Number of Integers to Choose From a Range I.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxCount(int[] banned, int n, int maxSum) {
        Set<Integer> bannedSet = new HashSet<>();

        for (int num : banned) {
            if (num >= 1 && num <= n) {
                bannedSet.add(num);
            }
        }

        int count = 0;
        long sum = 0;

        for (int i = 1; i <= n; i++) {
            if (bannedSet.contains(i)) {
                continue;
            }

            if (sum + i > maxSum) {
                break;
            }

            sum += i;
            count++;
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

782. [2558]Take Gifts From the Richest Pile.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long pickGifts(int[] gifts, int k) {
        PriorityQueue<Long> maxHeap = new PriorityQueue<>((a, b) -> Long.compare(b, a));

        for (int gift : gifts) {
            maxHeap.offer((long) gift);
        }

        for (int i = 0; i < k; i++) {
            if (maxHeap.isEmpty()) {
                break;
            }

            long currentMax = maxHeap.poll();
            long giftsLeft = (long) Math.sqrt(currentMax);
            maxHeap.offer(giftsLeft);
        }

        long totalGifts = 0;

        for (long gift : maxHeap) {
            totalGifts += gift;
        }

        return totalGifts;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

783. [2559]Count Vowel Strings in Ranges.java

```
//leetcode submit region begin(Prohibit modification and deletion)

import java.util.*;

class Solution {
    public int[] vowelStrings(String[] words, int[][] queries) {
        int n = words.length;
        int[] prefix = new int[n + 1];

        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i] + (isVowelString(words[i]) ? 1 : 0);
        }

        int qLen = queries.length;
        int[] ans = new int[qLen];

        for (int i = 0; i < qLen; i++) {
            int li = queries[i][0];
            int ri = queries[i][1];
            ans[i] = prefix[ri + 1] - prefix[li];
        }

        return ans;
    }

    private boolean isVowelString(String word) {
        if (word == null || word.length() == 0) {
            return false;
        }

        char first = word.charAt(0);
        char last = word.charAt(word.length() - 1);
        return isVowel(first) && isVowel(last);
    }

    private boolean isVowel(char c) {
        return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

784. `[2561]Rearranging Fruits.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long minCost(int[] basket1, int[] basket2) {
        TreeMap<Integer, Integer> count = new TreeMap<>();
        for (int a : basket1) count.merge(a, 1, Integer::sum);
        for (int a : basket2) count.merge(a, -1, Integer::sum);

        List<Integer> fromA = new ArrayList<>();
        List<Integer> fromB = new ArrayList<>();
        int small = Integer.MAX_VALUE;

        for (int x : count.keySet()) {
            int c = count.get(x);
            if (c % 2 != 0) return -1;
            small = Math.min(small, x);
            int times = Math.abs(c) / 2;
            if (c > 0) {
                for (int i = 0; i < times; i++) fromA.add(x);
            } else if (c < 0) {
                for (int i = 0; i < times; i++) fromB.add(x);
            }
        }

        // sort fromA ascending, fromB descending
        Collections.sort(fromA);
        fromB.sort(Collections.reverseOrder());

        long cost = 0;
        for (int i = 0; i < fromA.size(); i++) {
            int a = fromA.get(i);
            int b = fromB.get(i);
            cost += Math.min(Math.min(a, b), 2 * small);
        }

        return cost;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

785. `[2563]Count the Number of Fair Pairs.java`

```java
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public long countFairPairs(int[] nums, int lower, int upper) {
    Arrays.sort(nums);
    return countAtMost(nums, upper) - countAtMost(nums, lower - 1);
  }

  private long countAtMost(int[] nums, int bound) {
    long count = 0;
    int left = 0;
    int right = nums.length - 1;

    while (left < right) {
      if (nums[left] + nums[right] <= bound) {
        count += (right - left);
        left++;
      } else {
        right--;
      }
    }

    return count;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

786. [2566]Maximum Difference by Remapping a Digit.java

```
2566
Maximum Difference by Remapping a Digit
2023-02-19 22:37:42
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minMaxDifference(int num) {
        LinkedList<Integer> list = new LinkedList<>();
        int maxD = -1;
        int minD = -1;
        int min = 0;
        int max = 0;
        while (num > 0) {
            list.addFirst(num % 10);
            num /= 10;
        }
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i) != 9) {
                maxD = list.get(i);
                break;
            }
        }
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i) != 0) {
                minD = list.get(i);
                break;
            }
        }
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i) == maxD) {
                max = max * 10 + 9;
            } else {
                max = max * 10 + list.get(i);
            }
            if (list.get(i) == minD) {
                min = min * 10 + 0;
            } else {
                min = min * 10 + list.get(i);
            }
        }
        return max - min;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

787. [2570]Merge Two 2D Arrays by Summing Values.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int[][] mergeArrays(int[][] nums1, int[][] nums2) {
    List<int[]> mergedList = new ArrayList<>();
    int i = 0;
    int j = 0;

    while (i < nums1.length && j < nums2.length) {
      int id1 = nums1[i][0];
      int val1 = nums1[i][1];
      int id2 = nums2[j][0];
      int val2 = nums2[j][1];

      if (id1 == id2) {
        mergedList.add(new int[] {id1, val1 + val2});
        i++;
        j++;
      } else if (id1 < id2) {
        mergedList.add(new int[] {id1, val1});
        i++;
      } else {
        mergedList.add(new int[] {id2, val2});
        j++;
      }
    }

    while (i < nums1.length) {
      mergedList.add(new int[] {nums1[i][0], nums1[i][1]});
      i++;
    }

    while (j < nums2.length) {
      mergedList.add(new int[] {nums2[j][0], nums2[j][1]});
      j++;
    }

    return mergedList.toArray(new int[mergedList.size()][]);
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

788. [2573]Find the String with LCP.java

```
2573
Find the String with LCP
2023-02-22 14:41:03
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String findTheString(int[][] lcp) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

789. [2577]Minimum Time to Visit a Cell In a Grid.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumTime(int[][] grid) {
        if (grid[0][1] > 1 && grid[1][0] > 1) {
            return -1;
        }

        int m = grid.length, n = grid[0].length;
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

        boolean[][] visited = new boolean[m][n];
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0],
b[0]));
        pq.offer(new int[]{grid[0][0], 0, 0});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int time = curr[0], row = curr[1], col = curr[2];

            if (row == m - 1 && col == n - 1) {
                return time;
            }

            if (visited[row][col]) {
                continue;
            }

            visited[row][col] = true;

            for (int[] dir : dirs) {
                int r = row + dir[0], c = col + dir[1];

                if (r < 0 || r >= m || c < 0 || c >= n || visited[r][c]) {
                    continue;
                }

                int wait = ((grid[r][c] - time) % 2 == 0) ? 1 : 0;
                pq.offer(new int[]{Math.max(grid[r][c] + wait, time + 1), r, c});
            }
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

790. `[2582]Pass the Pillow.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int passThePillow(int n, int time) {
        int effectiveTime = time % (2 * (n - 1));

        if (effectiveTime < n) {
            return effectiveTime + 1;
        } else {
            return 2 * n - 1 - effectiveTime;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

791. `[2593]Find Score of an Array After Marking All`

Elements.java

```
/**
 * <p>You are given an array <code>nums</code> consisting of positive integers.</p>
 *
 * <p>Starting with <code>score = 0</code>, apply the following algorithm:</p>
 *
 * <ul>
 * <li>Choose the smallest integer of the array that is not marked. If there is a tie,
choose the one with the smallest index.</li>
 * <li>Add the value of the chosen integer to <code>score</code>.</li>
 * <li>Mark <strong>the chosen element and its two adjacent elements if they
exist</strong>.</li>
 * <li>Repeat until all the array elements are marked.</li>
 * </ul>
 *
 * <p>Return <em>the score you get after applying the above algorithm</em>.</p>
 *
 * <p> </p>
 * <p><strong class="example">Example 1:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [2,1,3,4,5,2]
 * <strong>Output:</strong> 7
 * <strong>Explanation:</strong> We mark the elements as follows:
 * - 1 is the smallest unmarked element, so we mark it and its two adjacent elements:
[<u>2</u>,<u>1</u>,<u>3</u>,4,5,2].
 * - 2 is the smallest unmarked element, so we mark it and its left adjacent element:
[<u>2</u>,<u>1</u>,<u>3</u>,4,<u>5</u>,<u>2</u>].
 * - 4 is the only remaining unmarked element, so we mark it: [<u>2</u>,<u>1</u>,
<u>3</u>,<u>4</u>,<u>5</u>,<u>2</u>].
 * Our score is 1 + 2 + 4 = 7.
 * </pre>
 *
 * <p><strong class="example">Example 2:</strong></p>
 *
 * <pre>
 * <strong>Input:</strong> nums = [2,3,5,1,3,2]
 * <strong>Output:</strong> 5
 * <strong>Explanation:</strong> We mark the elements as follows:
 * - 1 is the smallest unmarked element, so we mark it and its two adjacent elements:
[2,3,<u>5</u>,<u>1</u>,<u>3</u>,2].
 * - 2 is the smallest unmarked element, since there are two of them, we choose the left-
most one, so we mark the one at index 0 and its right adjacent element: [<u>2</u>,
<u>3</u>,<u>5</u>,<u>1</u>,<u>3</u>,2].
 * - 2 is the only remaining unmarked element, so we mark it: [<u>2</u>,<u>3</u>,
<u>5</u>,<u>1</u>,<u>3</u>,<u>2</u>].
 * Our score is 1 + 2 + 2 = 5.
 * </pre>
 *
 * <p> </p>
 * <p><strong>Constraints:</strong></p>
 *
```

```
 * <ul>
 * <li><code>1 &lt;= nums.length &lt;= 10<sup>5</sup></code></li>
 * <li><code>1 &lt;= nums[i] &lt;= 10<sup>6</sup></code></li>
 * </ul>
 *
 * <div><li>👍 141</li><li>👎 1</li></div>
 */

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long findScore(int[] nums) {
        int n = nums.length;
        if (n == 1) {
            return nums[0];
        }
        int[][] arr = new int[n][2];
        boolean[] isVisited = new boolean[n];
        long score = 0;
        for (int i = 0; i < n; i++) {
            arr[i][0] = nums[i];
            arr[i][1] = i;
        }
        Arrays.sort(arr, (a, b) -> a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]);
        for (int i = 0; i < n; i++) {
            if (!isVisited[arr[i][1]]) {
                score += arr[i][0];
                isVisited[arr[i][1]] = true;
                if (arr[i][1] != n - 1) {
                    isVisited[arr[i][1] + 1] = true;
                }
                if (arr[i][1] != 0) {
                    isVisited[arr[i][1] - 1] = true;
                }
            }
        }
        return score;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

792. `[2597]The Number of Beautiful Subsets.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int beautifulSubsets(int[] nums, int k) {
        Map<Integer, Integer> m = new HashMap<>();

        for (int num : nums) {
            m.put(num, m.getOrDefault(num, 0) + 1);
        }

        int res = 1;
        int prev = 0;
        int prevPrev = 0;

        for (Map.Entry<Integer, Integer> e : m.entrySet()) {
            int cur = e.getKey();

            if (m.containsKey(cur - k)) {
                continue;
            }

            prev = 0;

            while (m.containsKey(cur)) {
                prevPrev = prev;
                prev = ((1 << m.get(cur)) - 1) * res;
                res += prevPrev;
                cur += k;
            }

            res += prev;
        }

        return res - 1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

793. `[2601]Prime Subtraction Operation.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean primeSubOperation(int[] nums) {
        int n = nums.length;
        int prev = 0;

        for (int i = 0; i < n; i++) {
            if (nums[i] <= prev) {
                return false;
            }

            for (int j = nums[i] - prev - 1; j >= 2; j--) {
                if (isPrime(j)) {
                    nums[i] -= j;
                    break;
                }
            }

            prev = nums[i];
        }

        return true;
    }

    private boolean isPrime(int n) {
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                return false;
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

794. [2610]Convert an Array Into a 2D Array With Conditions.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<List<Integer>> findMatrix(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();

        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        List<List<Integer>> result = new ArrayList<>();

        while (!map.isEmpty()) {
            List<Integer> list = new ArrayList<>(map.keySet());
            result.add(list);

            for (int key : list) {
                map.put(key, map.get(key) - 1);

                if (map.get(key) == 0) {
                    map.remove(key);
                }
            }
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

795. `[2611]Mice and Cheese.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int miceAndCheese(int[] reward1, int[] reward2, int k) {
        int n = reward1.length;
        int[] diff = new int[n];
        int res = 0;
        for (int i = 0; i < n; i++) {
            diff[i] = reward1[i] - reward2[i];
            res += reward2[i];
        }
        Arrays.sort(diff);
        for (int i = 0; i < k; i++) {
            res += diff[n - 1 - i];
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

796. [2616]Minimize the Maximum Difference of Pairs.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimizeMax(int[] nums, int p) {
        Arrays.sort(nums);
        int n = nums.length;
        int left = 0;
        int right = nums[n - 1] - nums[0];
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (getCheck(nums, mid, p) < p) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return left;
    }

    private int getCheck(int[] nums, int mid, int p) {
        int k = 0;
        for (int i = 1; i < nums.length && k < p; i++) {
            if (nums[i] - nums[i - 1] <= mid) {
                k++;
                i++;
            }
        }
        return k;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

797. `[2620]Counter.java`

```
/**
<p>Given an integer&amp;nbsp;<code>n</code>,&amp;nbsp;return a <code>counter</code> function.
This <code>counter</code> function initially returns&amp;nbsp;<code>n</code>&amp;nbsp;and then
returns 1 more than the previous value every subsequent time it is called
(<code>n</code>, <code>n + 1</code>, <code>n + 2</code>, etc).</p>

<p>&amp;nbsp;</p>
<p><strong class="example">Example 1:</strong></p>

<pre>
<strong>Input:</strong>
n = 10
["call","call","call"]
<strong>Output:</strong> [10,11,12]
<strong>Explanation:
</strong>counter() = 10 // The first time counter() is called, it returns n.
counter() = 11 // Returns 1 more than the previous time.
counter() = 12 // Returns 1 more than the previous time.
</pre>

<p><strong class="example">Example 2:</strong></p>

<pre>
<strong>Input:</strong>
n = -2
["call","call","call","call","call"]
<strong>Output:</strong> [-2,-1,0,1,2]
<strong>Explanation:</strong> counter() initially returns -2. Then increases after each
sebsequent call.
</pre>

<p>&amp;nbsp;</p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>-1000<sup>&amp;nbsp;</sup>&amp;lt;= n &amp;lt;= 1000</code></li>
 <li><code>At most 1000 calls to counter() will be made</code></li>
</ul>

<div><li>👍 190</li><li>👎 12</li></div>
*/

//There is no code of Java type for this problem
```

798. `[2624]Snail Traversal.java`

```
/**
<p>Write code that enhances all arrays such that you can call the <code>snail(rowsCount, colsCount)</code> method that transforms the 1D&amp;nbsp;array into&amp;nbsp;a 2D array organised in&amp;nbsp;the pattern known as <strong>snail traversal order</strong>. Invalid input values should output an empty array. If&amp;nbsp;<code>rowsCount * colsCount !== nums.length</code>,&amp;nbsp;the input is considered invalid.</p>

<p><strong>Snail traversal order</strong><em>&amp;nbsp;</em>starts at the top left cell with the first value of the current array. It then moves through the entire first column from top to bottom, followed by moving to the next column on the right and traversing it from bottom to top. This pattern continues, alternating the direction of traversal with each column, until the entire current array is covered. For example, when given the input array&amp;nbsp;[19, 10, 3, 7, 9, 8, 5, 2, 1, 17, 16, 14, 12, 18, 6, 13, 11, 20, 4, 15]<code> </code>with <code>rowsCount = 5</code> and <code>colsCount = 4</code>,&amp;nbsp;the desired output matrix is shown below. Note that iterating the matrix following the arrows corresponds to the order of numbers in the original array.</p>

<p>&amp;nbsp;</p>

<p><img alt="Traversal Diagram" src="https://assets.leetcode.com/uploads/2023/04/10/screen-shot-2023-04-10-at-100006-pm.png" style="width: 275px; height: 343px;" /></p>

<p>&amp;nbsp;</p>
<p><strong class="example">Example 1:</strong></p>

<pre>
<strong>Input:</strong>
nums = [19, 10, 3, 7, 9, 8, 5, 2, 1, 17, 16, 14, 12, 18, 6, 13, 11, 20, 4, 15]
rowsCount = 5
colsCount = 4
<strong>Output:</strong>
[
 [19,17,16,15],
&amp;nbsp;[10,1,14,4],
&amp;nbsp;[3,2,12,20],
&amp;nbsp;[7,5,18,11],
&amp;nbsp;[9,8,6,13]
]
</pre>

<p><strong class="example">Example 2:</strong></p>

<pre>
<strong>Input:</strong>
nums = [1,2,3,4]
rowsCount = 1
colsCount = 4
<strong>Output:</strong> [[1, 2, 3, 4]]
</pre>
```

```
<p><strong class="example">Example 3:</strong></p>

<pre>
<strong>Input:</strong>
nums = [1,3]
rowsCount = 2
colsCount = 2
<strong>Output:</strong> []
<strong>Explanation:</strong> 2 multiplied by 2 is 4, and the original array [1,3] has a
length of 2; therefore, the input is invalid.
</pre>

<p> </p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>0 &lt;= nums.length &lt;= 250</code></li>
 <li><code>1 &lt;= nums[i] &lt;= 1000</code></li>
 <li><code>1 &lt;= rowsCount &lt;= 250</code></li>
 <li><code>1 &lt;= colsCount &lt;= 250</code></li>
</ul>

<p> </p>

<div><li>👍 9</li><li>👎 11</li></div>
*/

//There is no code of Java type for this problem
```

799. [2633]Convert Object to JSON String.java

```
/**
<p>Given an object, return a valid JSON string of that object. You may assume the object only includes strings, integers, arrays, objects, booleans, and null. The returned string should not include extra spaces. The order of keys should be the same as the order returned by&amp;nbsp;<code>Object.keys()</code>.</p>

<p>Please solve it without using the built-in <code>JSON.stringify</code> method.</p>

<p>&amp;nbsp;</p>
<p><strong class="example">Example 1:</strong></p>

<pre>
<strong>Input:</strong> object = {"y":1,"x":2}
<strong>Output:</strong> {"y":1,"x":2}
<strong>Explanation:</strong>
Return the JSON representation.
Note that the order of keys should be the same as the order returned by Object.keys().
</pre>

<p><strong class="example">Example 2:</strong></p>

<pre>
<strong>Input:</strong> object = {"a":"str","b":-12,"c":true,"d":null}
<strong>Output:</strong> {"a":"str","b":-12,"c":true,"d":null}
<strong>Explanation:</strong>
The primitives of JSON are strings, numbers, booleans, and null.
</pre>

<p><strong class="example">Example 3:</strong></p>

<pre>
<strong>Input:</strong> object = {"key":{"a":1,"b":[{},null,"Hello"]}}
<strong>Output:</strong> {"key":{"a":1,"b":[{},null,"Hello"]}}
<strong>Explanation:</strong>
Objects and arrays can include other objects and arrays.
</pre>

<p><strong class="example">Example 4:</strong></p>

<pre>
<strong>Input:</strong> object = true
<strong>Output:</strong> true
<strong>Explanation:</strong>
Primitive types are valid inputs.</pre>

<p>&amp;nbsp;</p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>object includes strings, integers, booleans, arrays, objects, and null</code>
</li>
```

```
 <li><code>1 &lt;= JSON.stringify(object).length &lt;= 10<sup>5</sup></code></li>
 <li><code>maxNestingLevel &lt;= 1000</code></li>
 <li><code>all strings will only contain alphanumeric characters</code></li>
</ul>

<div><li>👍 187</li><li>👎 10</li></div>
*/

//There is no code of Java type for this problem
```

800. `[2640]Find the Score of All Prefixes of an`

Array.java

```
/**
<p>We define the <strong>conversion array</strong> <code>conver</code> of an array
<code>arr</code> as follows:</p>

<ul>
 <li><code>conver[i] = arr[i] + max(arr[0..i])</code> where <code>max(arr[0..i])</code>
is the maximum value of <code>arr[j]</code> over <code>0 &amp;lt;= j &amp;lt;= i</code>.</li>
</ul>

<p>We also define the <strong>score</strong> of an array <code>arr</code> as the sum of
the values of the conversion array of <code>arr</code>.</p>

<p>Given a <strong>0-indexed</strong> integer array <code>nums</code> of length
<code>n</code>, return <em>an array </em><code>ans</code><em> of length </em>
<code>n</code><em> where </em><code>ans[i]</code><em> is the score of the prefix</em>
<code>nums[0..i]</code>.</p>

<p>&amp;nbsp;</p>
<p><strong class="example">Example 1:</strong></p>

<pre>
<strong>Input:</strong> nums = [2,3,7,5,10]
<strong>Output:</strong> [4,10,24,36,56]
<strong>Explanation:</strong>
For the prefix [2], the conversion array is [4] hence the score is 4
For the prefix [2, 3], the conversion array is [4, 6] hence the score is 10
For the prefix [2, 3, 7], the conversion array is [4, 6, 14] hence the score is 24
For the prefix [2, 3, 7, 5], the conversion array is [4, 6, 14, 12] hence the score is 36
For the prefix [2, 3, 7, 5, 10], the conversion array is [4, 6, 14, 12, 20] hence the
score is 56
</pre>

<p><strong class="example">Example 2:</strong></p>

<pre>
<strong>Input:</strong> nums = [1,1,2,4,8,16]
<strong>Output:</strong> [2,4,8,16,32,64]
<strong>Explanation:</strong>
For the prefix [1], the conversion array is [2] hence the score is 2
For the prefix [1, 1], the conversion array is [2, 2] hence the score is 4
For the prefix [1, 1, 2], the conversion array is [2, 2, 4] hence the score is 8
For the prefix [1, 1, 2, 4], the conversion array is [2, 2, 4, 8] hence the score is 16
For the prefix [1, 1, 2, 4, 8], the conversion array is [2, 2, 4, 8, 16] hence the score
is 32
For the prefix [1, 1, 2, 4, 8, 16], the conversion array is [2, 2, 4, 8, 16, 32] hence
the score is 64
</pre>

<p>&amp;nbsp;</p>
<p><strong>Constraints:</strong></p>
```

```
<ul>
 <li><code>1 &lt;= nums.length &lt;= 10<sup>5</sup></code></li>
 <li><code>1 &lt;= nums[i] &lt;= 10<sup>9</sup></code></li>
</ul>

<div><div>Related Topics</div><div><li>Array</li><li>Prefix Sum</li></div></div><br><div>
<li>👍 159</li><li>👎 8</li></div>
*/

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long[] findPrefixScore(int[] nums) {

    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

801. `[2641]Cousins in Binary Tree II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
    public TreeNode replaceValueInTree(TreeNode root) {
        int curLvlSum = root.val;
        Deque<TreeNode> queue = new ArrayDeque<>();
        queue.offer(root);
        root.val *= -1;

        while (!queue.isEmpty()) {
            int size = queue.size();
            int newLvlSum = 0;

            for (int i = 0; i < size; ++i) {
                TreeNode cur = queue.poll();
                int childSum = 0;

                if (cur.left != null) {
                    childSum += cur.left.val;
                }

                if (cur.right != null) {
                    childSum += cur.right.val;
                }

                newLvlSum += childSum;

                if (cur.left != null) {
                    cur.left.val = -childSum;
                }

                if (cur.right != null) {
                    cur.right.val = -childSum;
                }

                cur.val += curLvlSum;

                if (cur.left != null) {
                    queue.offer(cur.left);
                }

                if (cur.right != null) {
                    queue.offer(cur.right);
                }
            }

            curLvlSum = newLvlSum;
        }

        return root;
```

```
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

802. `[2642]Design Graph With Shortest Path`

Calculator.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Graph {

    private List<List<int[]>> adjList;

    public Graph(int n, int[][] edges) {
        adjList = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            adjList.add(new ArrayList<>());
        }

        for (int[] edge : edges) {
            addEdge(edge);
        }
    }

    public void addEdge(int[] edge) {
        int from = edge[0];
        int to = edge[1];
        int cost = edge[2];
        adjList.get(from).add(new int[]{to, cost});
    }

    public int shortestPath(int node1, int node2) {
        int n = adjList.size();
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[node1] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.offer(new int[]{node1, 0});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int node = curr[0];
            int cost = curr[1];

            if (node == node2) {
                return cost;
            }

            for (int[] neighbor : adjList.get(node)) {
                int nextNode = neighbor[0], edgeCost = neighbor[1];

                if (cost + edgeCost < dist[nextNode]) {
                    dist[nextNode] = cost + edgeCost;
                    pq.offer(new int[]{nextNode, dist[nextNode]});
                }
            }
```

```
        }

        return dist[node2] == Integer.MAX_VALUE ? -1 : dist[node2];
    }
}

/**
 * Your Graph object will be instantiated and called as such:
 * Graph obj = new Graph(n, edges);
 * obj.addEdge(edge);
 * int param_2 = obj.shortestPath(node1,node2);
 */
//leetcode submit region end(Prohibit modification and deletion)
```

803. `[2650]Design Cancellable Function.java`

```
/**
<p>Sometimes you have a long running task, and you may wish to cancel it before it completes. To help with this goal, write a function&amp;nbsp;<code>cancellable</code> that accepts a generator object and returns an array of two values: a <strong>cancel function</strong> and a <strong>promise</strong>.</p>

<p>You may assume the generator function will only&amp;nbsp;yield promises. It is your function's responsibility to pass the values resolved by the promise back to the generator. If the promise rejects, your function should throw that&amp;nbsp;error back to the generator.</p>

<p>If the cancel callback is called before the generator is done, your function should throw an error back to the generator. That error should be the string&amp;nbsp;<code>"Cancelled"</code>&amp;nbsp;(Not an <code>Error</code>&amp;nbsp;object). If the error was caught, the returned&amp;nbsp;promise should resolve with the next value that was yielded or returned. Otherwise, the promise should reject with the thrown error. No more code should be executed.</p>

<p>When the generator is done, the promise your function returned should resolve the value the generator returned. If, however, the generator throws an error, the returned promise should reject with the error.</p>

<p>An example of how your code would be used:</p>

<pre>
function* tasks() {
  const val = yield new Promise(resolve =&gt; resolve(2 + 2));
  yield new Promise(resolve =&gt; setTimeout(resolve, 100));
  return val + 1; // calculation shouldn't be done.
}
const [cancel, promise] = cancellable(tasks());
setTimeout(cancel, 50);
promise.catch(console.log); // logs "Cancelled" at t=50ms
</pre>

<p>If&amp;nbsp;instead&amp;nbsp;<code>cancel()</code> was not called or was called after <code>t=100ms</code>, the promise would&amp;nbsp;have resolved&amp;nbsp;<code>5</code>.</p>

<p>&amp;nbsp;</p>
<p><strong class="example">Example 1:</strong></p>

<pre>
<strong>Input:</strong>
generatorFunction = function*() {
&amp;nbsp; return 42;
}
cancelledAt = 100
<strong>Output:</strong> {"resolved": 42}
<strong>Explanation:</strong>
const generator = generatorFunction();
const [cancel, promise] = cancellable(generator);
```

```
setTimeout(cancel, 100);
promise.then(console.log); // resolves 42 at t=0ms

The generator immediately yields 42 and finishes. Because of that, the returned promise
immediately resolves 42. Note that cancelling a finished generator does nothing.
</pre>

<p><strong class="example">Example 2:</strong></p>

<pre>
<strong>Input:</strong>
generatorFunction = function*() {
&amp;nbsp; const msg = yield new Promise(res =&amp;gt; res("Hello"));
&amp;nbsp; throw `Error: ${msg}`;
}
cancelledAt = null
<strong>Output:</strong> {"rejected": "Error: Hello"}
<strong>Explanation:</strong>
A promise is yielded. The function handles this by waiting for it to resolve and then
passes the resolved value back to the generator. Then an error is thrown which has the
effect of causing the promise to reject with the same thrown error.
</pre>

<p><strong class="example">Example 3:</strong></p>

<pre>
<strong>Input:</strong>
generatorFunction = function*() {
&amp;nbsp; yield new Promise(res =&amp;gt; setTimeout(res, 200));
&amp;nbsp; return "Success";
}
cancelledAt = 100
<strong>Output:</strong> {"rejected": "Cancelled"}
<strong>Explanation:</strong>
While the function is waiting for the yielded promise to resolve, cancel() is called.
This causes an error message to be sent back to the generator. Since this error is
uncaught, the returned promise rejected with this error.
</pre>

<p><strong class="example">Example 4:</strong></p>

<pre>
<strong>Input:</strong>
generatorFunction = function*() {
&amp;nbsp; let result = 0;
&amp;nbsp; yield new Promise(res =&amp;gt; setTimeout(res, 100));
&amp;nbsp; result += yield new Promise(res =&amp;gt; res(1));
&amp;nbsp; yield new Promise(res =&amp;gt; setTimeout(res, 100));
&amp;nbsp; result += yield new Promise(res =&amp;gt; res(1));
&amp;nbsp; return result;
}
cancelledAt = null
<strong>Output:</strong> {"resolved": 2}
<strong>Explanation:</strong>
```

```
4 promises are yielded. Two of those promises have their values added to the result.
After 200ms, the generator finishes with a value of 2, and that value is resolved by the
returned promise.
</pre>

<p><strong class="example">Example 5:</strong></p>

<pre>
<strong>Input:</strong>
generatorFunction = function*() {
  let result = 0;
  try {
    yield new Promise(res =&gt; setTimeout(res, 100));
    result += yield new Promise(res =&gt; res(1));
    yield new Promise(res =&gt; setTimeout(res, 100));
    result += yield new Promise(res =&gt; res(1));
  } catch(e) {
    return result;
  }
  return result;
}
cancelledAt = 150
<strong>Output:</strong> {"resolved": 1}
<strong>Explanation:</strong>
The first two yielded promises resolve and cause the result to increment. However, at
t=150ms, the generator is cancelled. The error sent to the generator is caught and the
result is returned and finally resolved by the returned promise.
</pre>

<p><strong class="example">Example 6:</strong></p>

<pre>
<strong>Input:</strong>
generatorFunction = function*() {
  try {
    yield new Promise((resolve, reject) =&gt; reject("Promise Rejected"));
  } catch(e) {
    let a = yield new Promise(resolve =&gt; resolve(2));
    let b = yield new Promise(resolve =&gt; resolve(2));
    return a + b;
  };
}
cancelledAt = null
<strong>Output:</strong> {"resolved": 4}
<strong>Explanation:</strong>
The first yielded promise immediately rejects. This error is caught. Because the
generator hasn't been cancelled, execution continues as usual. It ends up resolving 2 + 2
= 4.</pre>

<p> </p>
<p><strong>Constraints:</strong></p>

<ul>
 <li><code>cancelledAt == null or 0 &lt;= cancelledAt &lt;= 1000</code></li>
```

```
 <li><code>generatorFunction returns a generator object</code></li>
</ul>

<div><li>👍 38</li><li>👎 6</li></div>
*/

//There is no code of Java type for this problem
```

804. [2657]Find the Prefix Common Array of Two Arrays.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] findThePrefixCommonArray(int[] A, int[] B) {
        int n = A.length;
        int[] C = new int[n];
        int[] counter = new int[n + 1];
        int count = 0;

        for (int i = 0; i < n; i++) {
            counter[A[i]]++;
            if (counter[A[i]] == 2) {
                count++;
            }

            counter[B[i]]++;
            if (counter[B[i]] == 2) {
                count++;
            }

            C[i] = count;
        }

        return C;
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

805. [2661]First Completely Painted Row or Column.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int firstCompleteIndex(int[] arr, int[][] mat) {
        int m = mat.length;
        int n = mat[0].length;
        int size = m * n + 1;
        int[] rowPos = new int[size];
        int[] colPos = new int[size];

        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                int num = mat[r][c];
                rowPos[num] = r;
                colPos[num] = c;
            }
        }

        int[] rows = new int[m];
        int[] cols = new int[n];
        for (int r = 0; r < m; r++) {
            rows[r] = n;
        }
        for (int c = 0; c < n; c++) {
            cols[c] = m;
        }

        for (int i = 0; i < arr.length; i++) {
            int num = arr[i];
            int r = rowPos[num];
            int c = colPos[num];
            rows[r]--;
            if (rows[r] == 0) {
                return i;
            }
            cols[c]--;
            if (cols[c] == 0) {
                return i;
            }
        }

        return -1;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

806. `[2678]Number of Senior Citizens.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countSeniors(String[] details) {
        int count = 0;

        for (String str : details) {
            int age = Integer.parseInt(str.substring(11, 13));

            if (age > 60) {
                count++;
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int countSeniors(String[] details) {
        return (int) Arrays.stream(details)
                .parallel()
                .map(detail -> Integer.parseInt(detail.substring(11, 13)))
                .filter(age -> age > 60)
                .count();
    }
}
 */
```

807. `[2679]Sum in a Matrix.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int matrixSum(int[][] nums) {
        int m = nums.length;
        int n = nums[0].length;
        int[] arr = new int[n];
        for (int i = 0; i < m; i++) {
            Arrays.sort(nums[i]);
            for (int j = 0; j < n; j++) {
                if (nums[i][j] > arr[j]) {
                    arr[j] = nums[i][j];
                }
            }
        }
        int res = 0;
        for (int num : arr) {
            res += num;
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

808. `[2680]Maximum OR.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long maximumOr(int[] nums, int k) {
        int n = nums.length;
        int[] right = new int[n];
        int left = 0;
        long res = 0;
        for (int i = n - 2; i >= 0; i--) {
            right[i] = right[i + 1] | nums[i + 1];
        }
        for (int i = 0; i < n; i++) {
            res = Math.max(res, left | (long) nums[i] << k | right[i]);
            left |= nums[i];
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

809. `[2681]Power of Heroes.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int sumOfPower(int[] nums) {
        long mod = (long) 1e9 + 7;
        long pre = 0;
        long res = 0;
        Arrays.sort(nums);
        for (long num : nums) {
            res = (res + (num * num % mod) * num % mod + (num * num % mod) * pre % mod) %
mod;
            pre = (pre * 2 + num) % mod;
        }
        return (int) res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

810. `[2683]Neighboring Bitwise XOR.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean doesValidArrayExist(int[] derived) {
        if (isValid(derived, 0)) {
            return true;
        }
        return isValid(derived, 1);
    }

    private boolean isValid(int[] derived, int firstValue) {
        int n = derived.length;
        int[] original = new int[n];
        original[0] = firstValue;

        for (int i = 0; i < n - 1; i++) {
            original[i + 1] = derived[i] ^ original[i];
        }

        int expectedFirst = derived[n - 1] ^ original[n - 1];
        return expectedFirst == original[0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

811. `[2696]Minimum String Length After Removing Substrings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minLength(String s) {
        Deque<Character> stack = new ArrayDeque<>();

        for (char c : s.toCharArray()) {
            if (!stack.isEmpty()) {
                char top = stack.peek();

                if ((top == 'A' && c == 'B') || (top == 'C' && c == 'D')) {
                    stack.pop();
                    continue;
                }
            }

            stack.push(c);
        }

        return stack.size();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

812. `[2698]Find the Punishment Number of an Integer.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public int punishmentNumber(int n) {
    int result = 0;
    for (int i = 0; i <= n; i++) {
      String squareStr = String.valueOf(i * i);

      if (canPartition(squareStr, 0, 0, i)) {
        result += i * i;
      }
    }

    return result;
  }

  private boolean canPartition(String s, int start, int currentSum, int target) {
    if (start == s.length()) {
      return currentSum == target;
    }

    int num = 0;

    for (int end = start; end < s.length(); end++) {
      num = num * 10 + (s.charAt(end) - '0');

      if (currentSum + num > target) {
        break;
      }

      if (canPartition(s, end + 1, currentSum + num, target)) {
        return true;
      }
    }

    return false;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

813. `[2706]Buy Two Chocolates.java`

```
//You are given an integer array prices representing the prices of various
//chocolates in a store. You are also given a single integer money, which represents
//your initial amount of money.
//
// You must buy exactly two chocolates in such a way that you still have some
//non-negative leftover money. You would like to minimize the sum of the prices of
//the two chocolates you buy.
//
// Return the amount of money you will have leftover after buying the two
//chocolates. If there is no way for you to buy two chocolates without ending up in
//debt, return money. Note that the leftover must be non-negative.
//
//
// Example 1:
//
//
//Input: prices = [1,2,2], money = 3
//Output: 0
//Explanation: Purchase the chocolates priced at 1 and 2 units respectively.
//You will have 3 - 3 = 0 units of money afterwards. Thus, we return 0.
//
//
// Example 2:
//
//
//Input: prices = [3,2,3], money = 3
//Output: 3
//Explanation: You cannot buy 2 chocolates without going in debt, so we return 3
//.
//
//
//
// Constraints:
//
//
// 2 <= prices.length <= 50
// 1 <= prices[i] <= 100
// 1 <= money <= 100
//
//
// 👍 308 👎 16


//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int buyChoco(int[] prices, int money) {
        int sum = getMinSum(prices);
        return money < sum ? money : money - sum;
    }

    private int getMinSum(int[] prices) {
```

```
        int first = Integer.MAX_VALUE;
        int second = Integer.MAX_VALUE;

        for (int price : prices) {
            if (price < first) {
                second = first;
                first = price;
            } else if (price < second) {
                second = price;
            }
        }

        return first + second;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

814. [2707]Extra Characters in a String.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minExtraChar(String s, String[] dictionary) {
        int n = s.length();
        int[] dp = new int[n + 1];
        for (int i = n - 1; i >= 0; i--) {
            String str = s.substring(i);
            dp[i] = dp[i + 1] + 1;
            for (int j = 0; j < dictionary.length; j++) {
                if (str.startsWith(dictionary[j])) {
                    dp[i] = Math.min(dp[i], dp[i + dictionary[j].length()]);
                }
            }
        }
        return dp[0];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
Top -Down -42ms
class Solution {
    Integer[] memo;
    Set<String> set;

    public int minExtraChar(String s, String[] dictionary) {
        set = new HashSet<>(Arrays.asList(dictionary));
        int n = s.length();
        memo = new Integer[n];
        return dp(0, n, s);
    }

    private int dp(int start, int n, String s) {
        if (start == n) {
            return 0;
        }
        if (memo[start] != null) {
            return memo[start];
        }
        int res = dp(start + 1, n, s) + 1;
        for (int end = start; end < n; end++) {
            String str = s.substring(start, end + 1);
            if (set.contains(str)) {
                res = Math.min(res, dp(end + 1, n, s));
            }
        }
        return memo[start] = res;
    }
}

Bottom - Up  -13ms
```

```
class Solution {
    public int minExtraChar(String s, String[] dictionary) {
        int n = s.length();
        int[] dp = new int[n + 1];
        for (int i = n - 1; i >= 0; i--) {
            String str = s.substring(i);
            dp[i] = dp[i + 1] + 1;
            for (int j = 0; j < dictionary.length; j++) {
                if (str.startsWith(dictionary[j])) {
                    dp[i] = Math.min(dp[i], dp[i + dictionary[j].length()]);
                }
            }
        }
        return dp[0];
    }
}
 */
```

815. [2709]Greatest Common Divisor Traversal.java

```
//leetcode submit region begin(Prohibit modification and deletion)
public class Solution {
    public boolean canTraverseAllPairs(int[] nums) {
        int maxNum = Arrays.stream(nums).max().getAsInt();
        UnionFind uf = new UnionFind(nums.length);
        Map<Integer, Integer> primeToIndex = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int num = nums[i];

            for (int factor = 2; factor * factor <= num; factor++) {
                if (num % factor == 0) {
                    handlePrimeFactor(factor, i, uf, primeToIndex);

                    while (num % factor == 0) {
                        num /= factor;
                    }
                }
            }

            if (num > 1) {
                handlePrimeFactor(num, i, uf, primeToIndex);
            }
        }

        return isAllConnected(uf, nums.length);
    }

    private void handlePrimeFactor(int factor, int index, UnionFind uf, Map<Integer,
Integer> primeToIndex) {
        if (!primeToIndex.containsKey(factor)) {
            primeToIndex.put(factor, index);
        } else {
            uf.union(index, primeToIndex.get(factor));
        }
    }

    private boolean isAllConnected(UnionFind uf, int length) {
        for (int i = 1; i < length; i++) {
            if (!uf.isConnected(i, 0)) {
                return false;
            }
        }
        return true;
    }
}

class UnionFind {
    public int[] group;
```

```
    public UnionFind(int n) {
        group = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            group[i] = i;
        }
    }

    private int find(int x) {
        return group[x] == x ? x : (group[x] = find(group[x]));
    }

    public void union(int i, int j) {
        int x = find(i);
        int y = find(j);

        if (x == y) {
            return;
        }

        group[x] = y;
    }

    public boolean isConnected(int i, int j) {
        return find(i) == find(j);
    }
}

//leetcode submit region end(Prohibit modification and deletion)
```

816. `[2742]Painting the Walls.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int paintWalls(int[] cost, int[] time) {
        int n = cost.length;
        int[] dp = new int[n + 1];

        Arrays.fill(dp, (int) 1e9);
        dp[0] = 0;
        for (int i = 0; i < n; i++) {
            for (int j = n; j > 0; j--) {
                dp[j] = Math.min(dp[j], dp[Math.max(j - time[i] - 1, 0)] + cost[i]);
            }
        }

        return dp[n];
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

817. [2743]Count Substrings Without Repeating Character.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfSpecialSubstrings(String s) {
        int n = s.length();
        boolean[] contains = new boolean[26];
        int left = 0;
        int result = 0;

        for (int right = 0; right < n; right++) {
            char curr = s.charAt(right);

            while (contains[curr - 'a']) {
                contains[s.charAt(left) - 'a'] = false;
                left++;
            }

            contains[curr - 'a'] = true;
            result += (right - left + 1);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

818. `[2751]Robot Collisions.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> survivedRobotsHealths(int[] positions, int[] healths, String
directions) {
        int n = positions.length;
        Integer[] indices = IntStream.range(0, n).boxed().toArray(Integer[]::new);
        Arrays.sort(indices, (a, b) -> positions[a] - positions[b]);
        Deque<Integer> stack = new ArrayDeque<>();
        List<Integer> survivors = new ArrayList<>();
        int[] finalHealths = new int[n];

        for (int idx : indices) {
            char dir = directions.charAt(idx);
            int health = healths[idx];

            if (dir == 'R') {
                stack.push(idx);
            } else {
                while (!stack.isEmpty() && health > 0) {
                    int topIdx = stack.peek();
                    int topHealth = healths[topIdx];

                    if (topHealth == health) {
                        stack.pop();
                        health = 0;
                    } else if (topHealth > health) {
                        healths[topIdx] -= 1;
                        health = 0;
                    } else {
                        stack.pop();
                        health -= 1;
                    }
                }

                if (health > 0) {
                    finalHealths[idx] = health;
                    survivors.add(idx);
                }
            }
        }

        while (!stack.isEmpty()) {
            int idx = stack.pop();
            finalHealths[idx] = healths[idx];
            survivors.add(idx);
        }

        return survivors.stream()
                .sorted()
                .map(idx -> finalHealths[idx])
                .collect(Collectors.toList());
```

```
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

819. `[2762]Continuous Subarrays.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long continuousSubarrays(int[] nums) {
        Deque<Integer> maxDeque = new LinkedList<>();
        Deque<Integer> minDeque = new LinkedList<>();

        int left = 0;
        long count = 0;

        for (int right = 0; right < nums.length; right++) {
            while (!maxDeque.isEmpty() && nums[right] > maxDeque.peekLast()) {
                maxDeque.pollLast();
            }

            maxDeque.offerLast(nums[right]);

            while (!minDeque.isEmpty() && nums[right] < minDeque.peekLast()) {
                minDeque.pollLast();
            }

            minDeque.offerLast(nums[right]);

            while (!maxDeque.isEmpty() && !minDeque.isEmpty() && maxDeque.peekFirst() -
minDeque.peekFirst() > 2) {
                if (nums[left] == maxDeque.peekFirst()) {
                    maxDeque.pollFirst();
                }

                if (nums[left] == minDeque.peekFirst()) {
                    minDeque.pollFirst();
                }

                left++;
            }

            count += (right - left + 1);
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

820. `[2767]Partition String Into Minimum Beautiful Substrings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    final int MAX_POWER_FIVE = 15625;

    public int minimumBeautifulSubstrings(String s) {
        int n = s.length();
        int[] dp = new int[n + 1];
        Arrays.fill(dp, n + 1);
        dp[0] = 0;

        for (int i = 0; i < n; i++) {
            if (s.charAt(i) == '0') {
                continue;
            }

            updateDP(s, dp, i, n);
        }

        return dp[n] > n ? -1 : dp[n];
    }

    private void updateDP(String s, int[] dp, int i, int n) {
        int curr = 0;

        for (int j = i; j < n; j++) {
            curr = curr * 2 + s.charAt(j) - '0';

            if (curr > MAX_POWER_FIVE) {
                break;
            }

            if (MAX_POWER_FIVE % curr == 0) {
                dp[j + 1] = Math.min(dp[j + 1], dp[i] + 1);
            }
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

821. `[2779]Maximum Beauty of an Array After Applying Operation.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maximumBeauty(int[] nums, int k) {
        if (nums.length == 1) {
            return 1;
        }

        int maxBeauty = 0;
        int maxValue = 0;

        for (int num : nums) {
            maxValue = Math.max(maxValue, num);
        }

        int[] count = new int[maxValue + 1];

        for (int num : nums) {
            count[Math.max(num - k, 0)]++;
            count[Math.min(num + k + 1, maxValue)]--;
        }

        int currentSum = 0;

        for (int val : count) {
            currentSum += val;
            maxBeauty = Math.max(maxBeauty, currentSum);
        }

        return maxBeauty;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

822. `[2785]Sort Vowels in a String.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String sortVowels(String s) {
        char[] chars = s.toCharArray();
        List<Character> list = new ArrayList<>();
        int index = 0;

        for (char c : chars) {
            if (isVowel(c)) {
                list.add(c);
            }
        }

        Collections.sort(list);

        for (int i = 0; i < chars.length; i++) {
            if (isVowel(chars[i])) {
                chars[i] = list.get(index++);
            }
        }

        return new String(chars);
    }

    private boolean isVowel(Character c) {
        return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' ||
                c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U';
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

823. `[2787]Ways to Express an Integer as Sum of`

Powers.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    private static final int MOD = 1_000_000_007;

    public int numberOfWays(int n, int x) {
        List<Integer> powers = new ArrayList<>();
        for (int i = 1; ; i++) {
            long v = intPowBounded(i, x, n);
            if (v > n) {
                break;
            }
            powers.add((int) v);
        }

        int[] dp = new int[n + 1];
        dp[0] = 1;
        for (int p : powers) {
            for (int s = n; s >= p; s--) {
                dp[s] += dp[s - p];
                if (dp[s] >= MOD) {
                    dp[s] -= MOD;
                }
            }
        }
        return dp[n];
    }

    private long intPowBounded(int a, int b, int limit) {
        long res = 1;
        long base = a;
        int exp = b;
        while (exp > 0) {
            if ((exp & 1) == 1) {
                res *= base;
                if (res > limit) {
                    return (long) limit + 1;
                }
            }
            exp >>= 1;
            if (exp > 0) {
                base *= base;
                if (base > limit) {
                    base = (long) limit + 1;
                }
            }
        }
        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

824. `[2802]Find The K-th Lucky Number.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String kthLuckyNumber(int k) {
        StringBuilder luckyNumber = new StringBuilder();
        final char FOUR = '4';
        final char SEVEN = '7';

        while (k-- > 0) {
            if (k % 2 == 0) {
                luckyNumber.append(FOUR);
            } else {
                luckyNumber.append(SEVEN);
            }

            k /= 2;
        }

        return luckyNumber.reverse().toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

825. [2807]Insert Greatest Common Divisors in Linked List.java

```
//leetcode submit region begin(Prohibit modification and deletion)

class Solution {
    public ListNode insertGreatestCommonDivisors(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }

        ListNode curr = head;

        while (curr != null && curr.next != null) {
            int val1 = curr.val;
            int val2 = curr.next.val;
            int gcdValue = gcd(val1, val2);
            ListNode gcdNode = new ListNode(gcdValue);
            gcdNode.next = curr.next;
            curr.next = gcdNode;
            curr = gcdNode.next;
        }

        return head;
    }

    private int gcd(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }

        return a;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

826. `[2810]Faulty Keyboard.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String finalString(String s) {
        StringBuilder sb = new StringBuilder();

        for (char c : s.toCharArray()) {
            if (c == 'i') {
                sb.reverse();
            } else {
                sb.append(c);
            }
        }

        return sb.toString();
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

827. `[2812]Find the Safest Path in a Grid.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    int[][] dirs = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

    public int maximumSafenessFactor(List<List<Integer>> grid) {
        int n = grid.size();
        int[][] distToThief = new int[n][n];

        for (int[] row : distToThief) {
            Arrays.fill(row, Integer.MAX_VALUE);
        }

        Deque<int[]> queue = new ArrayDeque<>();

        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                if (grid.get(r).get(c) == 1) {
                    queue.add(new int[]{r, c});
                    distToThief[r][c] = 0;
                }
            }
        }

        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            int r = current[0];
            int c = current[1];

            for (int[] dir : dirs) {
                int nr = r + dir[0];
                int nc = c + dir[1];

                if (nr >= 0 && nr < n && nc >= 0 && nc < n && distToThief[nr][nc] ==
Integer.MAX_VALUE) {
                    distToThief[nr][nc] = distToThief[r][c] + 1;
                    queue.add(new int[]{nr, nc});
                }
            }
        }

        int left = 0, right = n * 2;

        while (left < right) {
            int mid = (left + right + 1) / 2;

            if (canReachWithSafenessFactor(grid, distToThief, mid)) {
                left = mid;
            } else {
                right = mid - 1;
            }
```

```
        }

        return left;
    }

    private boolean canReachWithSafenessFactor(List<List<Integer>> grid, int[][]
distToThief, int safenessFactor) {
        int n = grid.size();

        if (distToThief[0][0] < safenessFactor || distToThief[n - 1][n - 1] <
safenessFactor) {
            return false;
        }

        boolean[][] visited = new boolean[n][n];
        Deque<int[]> queue = new ArrayDeque<>();
        queue.add(new int[]{0, 0});
        visited[0][0] = true;

        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            int r = current[0], c = current[1];
            if (r == n - 1 && c == n - 1) {
                return true;
            }

            for (int[] dir : dirs) {
                int nr = r + dir[0];
                int nc = c + dir[1];

                if (nr >= 0 && nr < n && nc >= 0 && nc < n && !visited[nr][nc] &&
distToThief[nr][nc] >= safenessFactor) {
                    visited[nr][nc] = true;
                    queue.add(new int[]{nr, nc});
                }
            }
        }

        return false;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

828. `[2816]Double a Number Represented as a Linked List.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode doubleIt(ListNode head) {
        ListNode reversed = reverseList(head);
        ListNode current = reversed;
        int carry = 0;
        ListNode prev = null;

        while (current != null) {
            int sum = current.val * 2 + carry;
            current.val = sum % 10;
            carry = sum / 10;
            prev = current;
            current = current.next;
        }

        if (carry > 0) {
            prev.next = new ListNode(carry);
        }

        return reverseList(reversed);
    }

    private ListNode reverseList(ListNode node) {
        ListNode prev = null;
        ListNode current = node;
        ListNode next = null;

        while (current != null) {
            next = current.next;
            current.next = prev;
            prev = current;
            current = next;
        }

        return prev;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

829. `[2825]Make String a Subsequence Using Cyclic Increments.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean canMakeSubsequence(String str1, String str2) {
        int n = str1.length();
        int m = str2.length();
        int j = 0;

        for (int i = 0; i < n && j < m; i++) {
            char c1 = str1.charAt(i);
            char target = str2.charAt(j);

            if (c1 == target) {
                j++;
            } else {
                char c1Inc = incrementChar(c1);

                if (c1Inc == target) {
                    j++;
                }
            }
        }

        return j == m;
    }

    private char incrementChar(char c) {
        if (c == 'z') {
            return 'a';
        } else {
            return (char) (c + 1);
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

830. `[2833]Furthest Point From Origin.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int furthestDistanceFromOrigin(String moves) {
        int countL = 0;
        int countR = 0;
        int count_ = 0;
        for (char c : moves.toCharArray()) {
            if (c == 'L') {
                countL++;
            } else if (c == 'R') {
                countR++;
            } else {
                count_++;
            }
        }
        int position = countR - countL;
        return Math.abs(position) + count_;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

831. `[2834]Find the Minimum Possible Sum of a Beautiful Array.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long minimumPossibleSum(int n, int target) {
        long k = target / 2;
        if (n <= k) {
            return 1L * n * (n + 1) / 2;
        }
        return k * (k + 1) / 2 + (target + target + n - k - 1) * (n - k) / 2;

    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public long minimumPossibleSum(int n, int target) {
        long sum = 0;
        Set<Long> set = new HashSet<>();
        for (long i = 1; set.size() < n; i++) {
            if (!set.contains(target - i)) {
                set.add(i);
                sum += i;
            }
        }
        return sum;
    }
}
 */
```

832. [2838]Maximum Coins Heroes Can Collect.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long[] maximumCoins(int[] heroes, int[] monsters, int[] coins) {
        int m = monsters.length;
        int[][] sortedMonsters = new int[m][2];

        for (int i = 0; i < m; i++) {
            sortedMonsters[i][0] = monsters[i];
            sortedMonsters[i][1] = coins[i];
        }

        Arrays.sort(sortedMonsters, (a, b) -> Integer.compare(a[0], b[0]));
        long[] prefixSum = new long[m];
        prefixSum[0] = sortedMonsters[0][1];

        for (int i = 1; i < m; i++) {
            prefixSum[i] = prefixSum[i - 1] + sortedMonsters[i][1];
        }

        int n = heroes.length;
        long[] result = new long[n];

        for (int i = 0; i < n; i++) {
            int heroPower = heroes[i];
            int index = findLastIndex(sortedMonsters, heroPower);

            if (index != -1) {
                result[i] = prefixSum[index];
            } else {
                result[i] = 0;
            }
        }

        return result;
    }

    private int findLastIndex(int[][] sortedMonsters, int target) {
        int left = 0;
        int right = sortedMonsters.length - 1;
        int index = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (sortedMonsters[mid][0] <= target) {
                index = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
```

```
        }

        return index;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

833. `[2843] Count Symmetric Integers.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int countSymmetricIntegers(int low, int high) {
        int count = 0;
        for (int i = low; i <= high; i++) {
            if (isSymmetric(i)) {
                count++;
            }
        }
        return count;
    }

    private boolean isSymmetric(int num) {
        String str = Integer.toString(num);
        int n = str.length();
        if (n % 2 != 0) {
            return false;
        }
        int mid = n / 2;
        int firstHalfSum = 0;
        int secondHalfSum = 0;
        for (int i = 0; i < mid; i++) {
            firstHalfSum += str.charAt(i) - '0';
            secondHalfSum += str.charAt(n - 1 - i) - '0';
        }
        return firstHalfSum == secondHalfSum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

834. [2844]Minimum Operations to Make a Special Number.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumOperations(String num) {
        int n = num.length();
        int minOperations = n;  // Initialize with the maximum possible number of
operations

        // Count the occurrences of each digit in the number
        int[] count = new int[10];
        for (char c : num.toCharArray()) {
            count[c - '0']++;
        }

        // Check for each possible last two digits (00, 25, 50, 75)
        for (int tens : new int[]{0, 2, 5, 7}) {
            for (int ones : new int[]{0, 5}) {
                if (tens == ones && count[tens] < 2) {
                    continue;  // Skip if not enough occurrences for the same tens and
ones digit
                }
                if (count[tens] == 0 || count[ones] == 0) {
                    continue;  // Skip if tens or ones digit is not present
                }

                // Count the number of operations needed for this pair of last two digits
                int operations = n - 2;  // We need to keep the last two digits
                for (int i = 0; i <= 9; i++) {
                    operations -= Math.min(count[i], num.lastIndexOf(Integer.toString(i))
- operations);
                }

                // Update the minimum number of operations
                minOperations = Math.min(minOperations, operations);
            }
        }

        return minOperations;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

835. [2845]Count of Interesting Subarrays.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long countInterestingSubarrays(List<Integer> nums, int modulo, int k) {
        int n = nums.size();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = nums.get(i);
        }
        long res = 0;
        int count = 0;
        Map<Long, Long> map = new HashMap<>();
        map.put(0L, 1L);

        for (int num : nums) {
            if (num % modulo == k) {
                count++;
            }
            long remainder = count % modulo;
            if (map.containsKey(remainder)) {
                res += map.get(remainder);
            }
            map.put(remainder, map.getOrDefault(remainder, 0L) + 1);
        }

        return res;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

836. [2849]Determine if a Cell Is Reachable at a Given Time.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public boolean isReachableAtTime(int sx, int sy, int fx, int fy, int t) {
        int need = Math.max(Math.abs(sx - fx), Math.abs(sy - fy));

        if (need == 0 && t == 1) {
            return false;
        }

        return t >= need;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

837. `[2864]Maximum Odd Binary Number.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public String maximumOddBinaryNumber(String s) {
        int oneCount = 0;

        for (char c : s.toCharArray()) {
            if (c == '1') {
                oneCount++;
            }
        }

        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < oneCount - 1; i++) {
            sb.append("1");
        }

        for (int i = 0; i < s.length() - oneCount; i++) {
            sb.append("0");
        }

        sb.append("1");
        return sb.toString();
    }
}
// leetcode submit region end(Prohibit modification and deletion)
```

838. [2869]Minimum Operations to Collect Elements.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minOperations(List<Integer> nums, int k) {
        Set<Integer> set = new HashSet<>();
        int index = nums.size() - 1;
        int count = 0;

        while (set.size() < k) {
            int num = nums.get(index--);
            count++;

            if (num <= k) {
                set.add(num);
            }
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

839. `[2870]Minimum Number of Operations to Make Array`

Empty.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minOperations(int[] nums) {
        Arrays.sort(nums);
        int count = 0;
        int left = 0;

        while (left < nums.length) {
            int right = left;

            while (right < nums.length && nums[left] == nums[right]) {
                right++;
            }

            int num = right - left;

            if (num == 1) {
                return -1;
            }

            count += (int) Math.ceil(num / 3.0);
            left = right;
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
/*
class Solution {
    public int minOperations(int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        int count = 0;

        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        for (int num : map.values()) {
            if (num == 1) {
                return -1;
            }

            count += (int) Math.ceil(num / 3.0);
        }

        return count;
    }
}
 */
```

840. `[2871]Split Array Into Maximum Number of Subarrays.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxSubarrays(int[] nums) {
        int result = 0;
        int init = -1;

        for (int num : nums) {
            init &= num;
            if (init == 0) {
                init = -1;
                result++;
            }
        }

        return Math.max(1, result);
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

841. `[2872]Maximum Number of K-Divisible Components.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxKDivisibleComponents(int n, int[][] edges, int[] values, int k) {
        List<List<Integer>> adj = new ArrayList<>();

        for(int i = 0; i < n; i++) {
            adj.add(new ArrayList<>());
        }

        for(int[] edge : edges) {
            int u = edge[0];
            int v = edge[1];
            adj.get(u).add(v);
            adj.get(v).add(u);
        }

        int[] cuts = new int[1];
        dfs(0, -1, adj, values, k, cuts);
        return cuts[0] + 1;
    }

    private long dfs(int node, int parent, List<List<Integer>> adj, int[] values, int k,
int[] cuts) {
        long currentSum = values[node];

        for(int neighbor : adj.get(node)) {
            if(neighbor != parent) {
                long childSum = dfs(neighbor, node, adj, values, k, cuts);
                currentSum += childSum;
            }
        }

        if(currentSum % k == 0 && parent != -1) {
            cuts[0]++;
            return 0;
        }

        return currentSum;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

842. `[2874]Maximum Value of an Ordered Triplet II.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long maximumTripletValue(int[] nums) {
        long result = 0;
        int maxItem = 0;
        int maxDiff = 0;

        for (int num : nums) {
            result = Math.max(result, (long) maxDiff * num);
            maxItem = Math.max(maxItem, num);
            maxDiff = Math.max(maxDiff, maxItem - num);
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

843. `[2900]Longest Unequal Adjacent Groups Subsequence I.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public List<String> getLongestSubsequence(String[] words, int[] groups) {
    List<String> res = new ArrayList<>();
    int group = -1;
    int n = words.length;

    for (int i = 0; i < n; i++) {
      if (group != groups[i]) {
        res.add(words[i]);
        group = groups[i];
      }
    }

    return res;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

844. [2929]Distribute Candies Among Children II.java

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public long distributeCandies(int n, int limit) {
    if (n > 3L * limit) {
      return 0L;
    }

    long ans = comb2(n + 2);

    if (n > limit) {
      ans -= 3L * comb2(n - limit + 1);
    }
    if (n - 2L >= 2L * limit) {
      ans += 3L * comb2(n - 2 * limit);
    }

    return ans;
  }

  private long comb2(long m) {
    return m * (m - 1) / 2;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

845. [2938]Separate Black and White Balls.java

```

//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public long minimumSteps(String s) {
        long zeroCount = 0;
        long totalSwaps = 0;
        int n = s.length();

        for (int i = n - 1; i >= 0; i--) {
            char current = s.charAt(i);

            if (current == '0') {
                zeroCount++;
            } else if (current == '1') {
                totalSwaps += zeroCount;
            }
        }

        return totalSwaps;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

846. `[2942]Find Words Containing Character.java`

```
// leetcode submit region begin(Prohibit modification and deletion)
class Solution {
  public List<Integer> findWordsContaining(String[] words, char x) {
    List<Integer> result = new ArrayList<>();
    int n = words.length;

    for (int i = 0; i < n; i++) {
      if (words[i].indexOf(x) >= 0) {
        result.add(i);
      }
    }

    return result;
  }
}
// leetcode submit region end(Prohibit modification and deletion)
```

847. [2948]Make Lexicographically Smallest Array by

Swapping Elements.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] lexicographicallySmallestArray(int[] nums, int limit) {
        int n = nums.length;
        Pair[] sortedNums = new Pair[n];
        for(int i = 0; i < n; i++) {
            sortedNums[i] = new Pair(nums[i], i);
        }
        Arrays.sort(sortedNums, new Comparator<Pair>() {
            public int compare(Pair a, Pair b) {
                if(a.value != b.value){
                    return Integer.compare(a.value, b.value);
                } else {
                    return Integer.compare(a.index, b.index);
                }
            }
        });

        int[] result = new int[n];
        System.arraycopy(nums, 0, result, 0, n);

        int start = 0;
        for(int i = 1; i <= n; i++) {
            if(i < n && (long)(sortedNums[i].value) - (long)(sortedNums[i-1].value) <=
(long)limit) {
                continue;
            } else {
                if(start < i) {
                    int size = i - start;
                    int[] indices = new int[size];
                    int[] values = new int[size];
                    for(int j = start; j < i; j++) {
                        indices[j - start] = sortedNums[j].index;
                        values[j - start] = sortedNums[j].value;
                    }
                    Arrays.sort(indices);
                    Arrays.sort(values);
                    for(int j = 0; j < size; j++) {
                        result[indices[j]] = values[j];
                    }
                }
                start = i;
            }
        }
        return result;
    }

    class Pair {
        int value;
        int index;
```

```
        Pair(int v, int i){
            value = v;
            index = i;
        }
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

848. `[2951]Find the Peaks.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public List<Integer> findPeaks(int[] mountain) {
        List<Integer> list = new ArrayList<>();

        for (int i = 1; i < mountain.length - 1; i++) {
            if (mountain[i] > mountain[i - 1] && mountain[i] > mountain[i + 1]) {
                list.add(i);
            }
        }

        return list;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

849. [2952]Minimum Number of Coins to be Added.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int minimumAddedCoins(int[] coins, int target) {
        Arrays.sort(coins);
        int count = 0;
        int curr = 0;

        for (int coin : coins) {
            while (coin > curr + 1 && curr < target) {
                curr += curr + 1;
                count++;
            }

            curr += coin;

            if (curr >= target) {
                break;
            }
        }

        while (curr < target) {
            curr += curr + 1;
            count++;
        }

        return count;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

850. `[2955]Number of Same-End Substrings.java`

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int[] sameEndSubstringCount(String s, int[][] queries) {
        int n = s.length();
        int[][] prefix = new int[26][n + 1];

        for (int i = 0; i < n; i++) {
            for (int c = 0; c < 26; c++) {
                prefix[c][i + 1] = prefix[c][i];
            }

            char currentChar = s.charAt(i);
            prefix[currentChar - 'a'][i + 1]++;
        }

        int q = queries.length;
        int[] result = new int[q];

        for (int i = 0; i < q; i++) {
            int left = queries[i][0];
            int right = queries[i][1];
            long count = 0;

            for (int c = 0; c < 26; c++) {
                int freq = prefix[c][right + 1] - prefix[c][left];

                if (freq > 0) {
                    count += ((long) freq * (freq + 1)) / 2;
                }
            }
            result[i] = (int) count;
        }

        return result;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

851. [2958]Length of Longest Subarray With at Most K Frequency.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int maxSubarrayLength(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        int max = 0;
        int left = 0;

        for (int right = 0; right < nums.length; right++) {
            map.put(nums[right], map.getOrDefault(nums[right], 0) + 1);

            while (map.get(nums[right]) > k) {
                map.put(nums[left], map.get(nums[left]) - 1);
                left++;
            }

            max = Math.max(max, right - left + 1);
        }

        return max;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```

852. `[2959]Number of Possible Sets of Closing`

Branches.java

```
//leetcode submit region begin(Prohibit modification and deletion)
class Solution {
    public int numberOfSets(int n, int maxDistance, int[][] roads) {
        Map<Integer, Map<Integer, Integer>> graph = new HashMap<>();
        for (int[] road : roads) {
            int u = road[0];
            int v = road[1];
            int w = road[2];
            graph.computeIfAbsent(u, k -> new HashMap<>());
            graph.computeIfAbsent(v, k -> new HashMap<>());

            if (!graph.get(u).containsKey(v) || graph.get(u).get(v) > w) {
                graph.get(u).put(v, w);
            }
            if (!graph.get(v).containsKey(u) || graph.get(v).get(u) > w) {
                graph.get(v).put(u, w);
            }
        }

        return countValidCombinations(n, maxDistance, graph, new boolean[n], 0);
    }

    private int countValidCombinations(int n, int maxDistance, Map<Integer, Map<Integer,
Integer>> graph, boolean[] closed, int start) {
        int count = 0;

        if (start == n) {
            if (isValidSubset(closed, n, maxDistance, graph)) {
                count++;
            }
        } else {
            closed[start] = true;
            count += countValidCombinations(n, maxDistance, graph, closed, start + 1);

            closed[start] = false;
            count += countValidCombinations(n, maxDistance, graph, closed, start + 1);
        }

        return count;
    }

    private boolean isValidSubset(boolean[] closed, int n, int maxDistance, Map<Integer,
Map<Integer, Integer>> graph) {
        int[][] distances = new int[n][n];

        for (int[] row : distances) {
            Arrays.fill(row, Integer.MAX_VALUE / 2);
        }
        for (int i = 0; i < n; i++) {
            distances[i][i] = 0;
```

```
            if (!closed[i]) {
                for (int j : graph.getOrDefault(i, new HashMap<>()).keySet()) {
                    if (!closed[j]) {
                        distances[i][j] = distances[j][i] = graph.get(i).get(j);
                    }
                }
            }
        }

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    distances[i][j] = Math.min(distances[i][j], distances[i][k] +
distances[k][j]);
                }
            }
        }

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (!closed[i] && !closed[j] && distances[i][j] > maxDistance) {
                    return false;
                }
            }
        }

        return true;
    }
}
//leetcode submit region end(Prohibit modification and deletion)
```